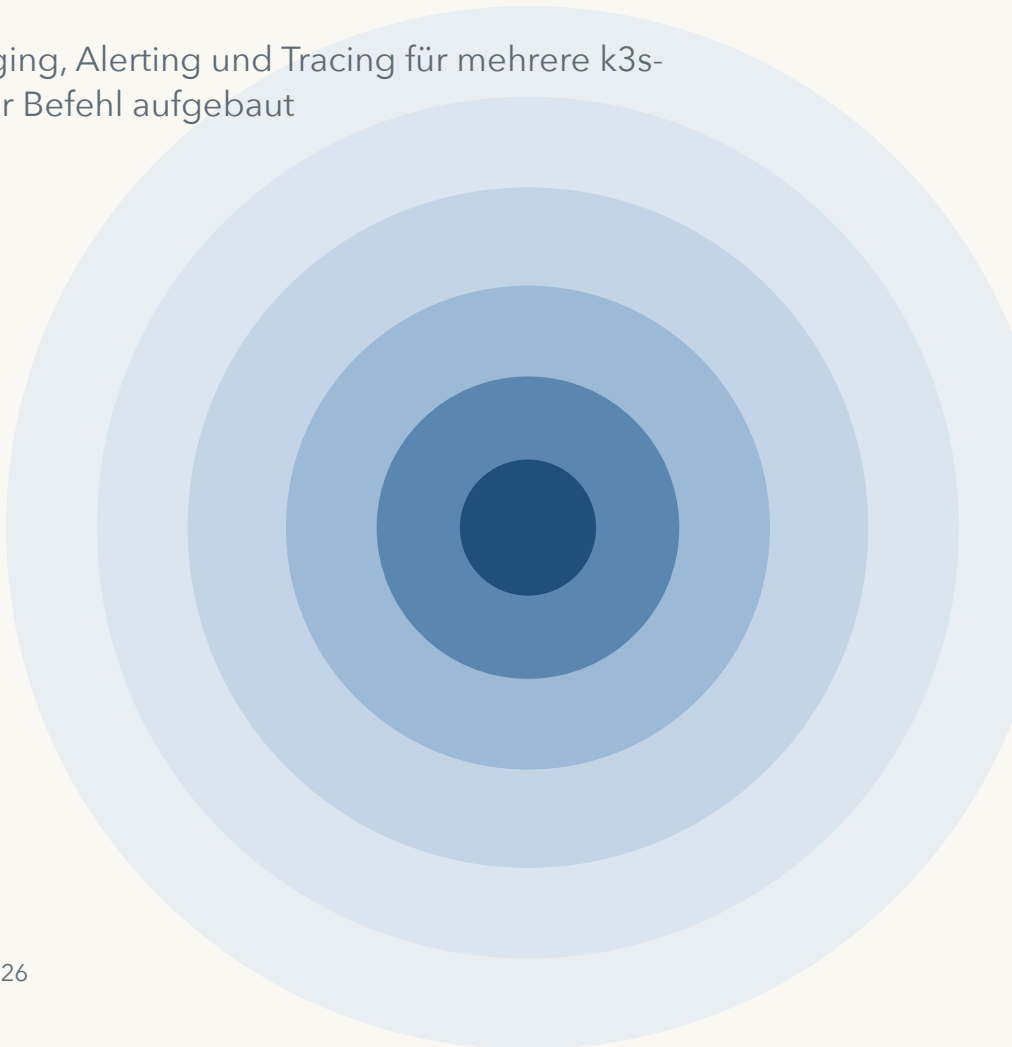
HANDS-ON-HANDBUCH

Kubernetes beobachten

Monitoring, Logging, Alerting und Tracing für mehrere k3s-Cluster, Befehl für Befehl aufgebaut



Thomas Zachmann

Stand: 11. November 2026

Inhalt

So benutzt du dieses Buch 6
Zuerst das Repo klonen 6
Wo du einen Befehl ausführst 6
Die Kästen 7
Was geprüft wurde und was nicht 7
Was du brauchst 8
1 Die Architektur im Überblick 10
1.1 Die Frage hinter allem 10
1.2 Die vier Signale 10
1.3 Zentral oder dezentral 11
1.4 Push statt Pull 12
1.5 Wie die Daten fließen 12
1.6 Was läuft wo 13
1.7 Der Weg durch das Buch 14
1.8 Der Ressourcenplan 15
2 Drei k3s-Cluster als Fundament 18
2.1 Was du aufbaust 18
2.2 Die k3s-Konfiguration 18
2.3 Die VM obs vorbereiten 19
2.4 k3s installieren 20
2.5 Dasselbe auf app1 und app2 21
2.6 Die Kubeconfigs zusammenführen 21
2.7 Der Namespace monitoring 23
2.8 Namen für die Zentrale 24
2.9 Eine eigene CA für TLS 26
2.10 Zertifikat und CA in die Cluster bringen 30
3 S3 im Cluster mit SeaweedFS 34
3.1 Warum S3 schon im Labor 34
3.2 Die Zugangsdaten erzeugen 35
3.3 SeaweedFS starten 36
3.4 Die Buckets anlegen 37
3.5 Die Rechte prüfen 38

3.6 SeaweedFS im Alltag: Dateien mit der aws-CLI 39
3.7 Was die anderen Komponenten wissen müssen 42
4 Die Zentrale: Prometheus und Thanos Receive 44
4.1 Was du aufbaust 44
4.2 Die Werte für kube-prometheus-stack 45
4.3 kube-prometheus-stack installieren 47
4.4 Thanos Receive 48
4.5 Query, Store und Compact 50
4.6 Thanos Query ansehen 52
4.7 Läuft der Remote Write sauber? 53
4.8 Der erste Block in S3 54
5 Metriken aus den App-Clustern 56
5.1 Was du aufbaust 56
5.2 Ein Zugang pro Cluster 56
5.3 Middleware und Ingress 58
5.4 Von außen prüfen 59
5.5 Exporter in app1 und app2 60
5.6 Wie Alloy mehrere Dateien lädt 62
5.7 Die Datei metrics.alloy 63
5.8 Alloy installieren 65
5.9 Die Konfiguration aktualisieren 67
5.10 Alle Cluster in einer Abfrage 68
6 Grafana als Code 72
6.1 Warum keine Klicks 72
6.2 Das Admin-Passwort 73
6.3 Die Helm-Werte 73
6.4 Datenquellen als ConfigMap 75
6.5 Grafana neu ausrollen 76
6.6 Der erste Login 76
6.7 Eine Abfrage über alle Cluster 78
6.8 Ändern, ohne zu klicken 80
7 Alarme, die ankommen 84
7.1 Die Kette 84
7.2 Mailpit als Test-Empfänger 84

7.3 Der Dead Man's Switch ... 85
7.4 Die Alertmanager-Konfiguration ... 86
7.5 Ausrollen ... 89
7.6 Globale Regeln ... 90
7.7 Ein Alarm von Anfang bis Ende ... 91
7.8 Stummschalten ohne amtool ... 95
8 Logs mit Loki und Alloy ... 100
8.1 Was du aufbaust ... 100
8.2 Wenige, stabile Labels ... 100
8.3 Loki auf obs installieren ... 101
8.4 Alloy schickt die Logs ... 103
8.5 Logs in Grafana ... 106
8.6 LogQL in vier Schritten ... 108
8.7 Kubernetes-Events ... 109
8.8 Log-Alarme über den Loki-Ruler ... 110
8.9 Aufbewahrung und Compactor ... 112
9 Traces mit Tempo und OpenTelemetry ... 114
9.1 Was du aufbaust ... 114
9.2 Tempo auf obs installieren ... 114
9.3 Alloy nimmt Traces an ... 116
9.4 Die Demo-App ... 118
9.5 Traces in Grafana ... 120
9.6 TraceQL-Grundlagen ... 122
9.7 Service-Graph und Span-Metriken ... 123
9.8 Von der Metrik zum Trace: Exemplars ... 124
9.9 Von Logs zu Traces und zurück ... 125
10 Fertige Dashboards ... 128
10.1 Warum eigene Dashboards ... 128
10.2 So ist ein Dashboard gebaut ... 129
10.3 Einspielen ... 130
10.4 Die Dashboards im Einzelnen ... 131
10.5 Mitgelieferte und Community-Dashboards ... 136
11 S3 auf AWS ... 140
11.1 Die aws CLI vorbereiten ... 140

11.2 Vier Buckets anlegen ... 141
11.3 Die Riegel vor dem ersten Byte ... 142
11.4 Lebensdauer: Lifecycle-Regeln ... 144
11.5 Ein Benutzer, eine Policy pro Bucket ... 145
11.6 Thanos umziehen ... 147
11.7 Loki und Tempo umschalten ... 149
11.8 Was kostet das? ... 150
11.9 Spiegelung, wenn eine Kopie nicht reicht ... 151
12 Backup und Restore ... 152
12.1 Was gesichert werden muss und was nicht ... 152
12.2 SeaweedFS von außen erreichbar machen ... 153
12.3 etcd-Snapshots nach S3 ... 154
12.4 Die Wiederherstellung üben ... 157
12.5 Velero für Anwendungen ... 158
12.6 Alarm, wenn die Sicherung ausbleibt ... 161
12.7 Variante: Sicherungen auf AWS ... 162
13 Was sonst noch fehlt ... 164
13.1 Von außen prüfen: der Blackbox-Exporter ... 164
13.2 Wer überwacht die Überwachung? ... 167
13.3 Kardinalität: wenn Serien explodieren ... 167
13.4 SLOs mit Burn-Rate-Alarmen ... 169
13.5 Profiling mit Pyroscope (optional) ... 170
13.6 Absicherung und Datenschutz ... 171
13.7 Ausblick: GitOps ... 172
14 Betrieb und Fehlersuche ... 174
14.1 Updates in der richtigen Reihenfolge ... 174
14.2 Ressourcen messen statt schätzen ... 175
14.3 Typische Fehler ... 176
14.4 Aufräumen ... 178
15 Vom Labor zur Enterprise-Architektur: Hochverfügbarkeit und Datenverlust vermeiden ... 180
15.1 Wo das Labor an seine Grenzen stößt ... 180
15.2 Gemessen: obs fällt eine Stunde aus ... 181
15.3 Wie Alloy puffert, und wo die Grenzen liegen ... 184

15.4 Ein hochverfügbarer Kubernetes-Cluster für obs 186
15.5 Thanos Receive mit Replikation 188
15.6 Loki hochverfügbar 189
15.7 Tempo hochverfügbar 190
15.8 S3-Speicher im Vergleich 191
15.9 Zwei Zentralen: Active/Passive und Active/Active 192
15.10 Monitoring des Monitorings 193
15.11 Ausfälle üben: Disaster Recovery 193
15.12 Eine Referenzarchitektur 194
15.13 Aufwand, Ressourcen und Kosten 195
16 Enterprise-Observability: Multi-Tenancy mit Thanos 198
16.1 Plattform-Monitoring und Kunden-Monitoring 199
16.2 Architektur und Datenfluss 199
16.3 Thanos Receive: Mandant und Cluster sind zwei Dinge 200
16.4 Die Sicherheitsgrenze beim Schreiben 202
16.5 Die Sicherheitsgrenze beim Lesen 204
16.6 Grafana für viele Kunden 206
16.7 Speicherung, Hochverfügbarkeit und die Grenzen einer gemeinsamen Plattform . . 207
16.8 Drei Architekturen im Vergleich 208
16.9 GitOps mit Argo CD 209
16.10 Verifikation: Die Trennung beweisen 210
17 Anhang 214
17.1 Versionen 214
17.2 Dein Repo am Ende 215
17.3 Spickzettel PromQL 216
17.4 Spickzettel LogQL 216
17.5 Spickzettel TraceQL 217
17.6 Offizielle Dokumentation 217

So benutzt du dieses Buch

Prometheus ist schnell installiert, Grafana auch. Schwer wird es, sobald mehr als ein Cluster im Spiel ist. Woher weißt du, welche Metrik aus welchem Cluster stammt? Wo liegen die Daten von vor drei Wochen? Und wer merkt es, wenn das Monitoring selbst ausfällt?

Genau diese Fragen beantwortest du in diesem Buch, und zwar an drei eigenen k3s-Clustern. Einer davon ist die Zentrale `obs`. Die beiden anderen, `app1` und `app2`, spielen deine Anwendungs-Cluster. Sie schicken Metriken, Logs und Traces an die Zentrale. Dort landet alles in einem S3-Speicher und ist in Grafana abfragbar.

Jedes Kapitel hat denselben Ablauf. Du erfährst zuerst, welches Problem gelöst wird. Dann tippst du Befehl für Befehl, prüfst das Ergebnis und probierst selbst etwas aus. Lange Skripte zum Abtippen gibt es nicht. Wer den Hintergrund zu den Konzepten vertiefen möchte, findet ihn im Buch *Kubernetes Observability Hands-On*. Hier wird gebaut.

Zuerst das Repo klonen

Alle Dateien, auf die sich das Buch bezieht, liegen in einem öffentlichen Repository auf GitHub: github.com/thomaszachmann/k3s. Alles zu diesem Buch steht im Ordner `observability/`.

MAC Schritt 1

```
git clone https://github.com/thomaszachmann/k3s.git
```

Lädt das Repo mit allen Dateien in den neuen Ordner `k3s`.

MAC Schritt 2

```
cd k3s
```

Wechselt ins Wurzelverzeichnis des Repos. Von hier aus laufen alle Befehle mit dem Etikett **Mac**. Pfade wie `observability/k3s/config-obs.yaml` sind immer relativ zu diesem Ordner.

Wo du einen Befehl ausführst

Jeder Befehl trägt ein farbiges Etikett. Es sagt dir, wo du dich gerade befindest:

Etikett	Bedeutung
MAC	Dein Terminal auf dem Laptop, im Wurzelverzeichnis des Repos. Von hier aus steuerst du alle drei Cluster mit `kubectl --context <name>`.
OBS	Eine SSH-Sitzung auf der VM `k3s-obs` (`172.16.0.126`), der Zentrale.

Etikett	Bedeutung
APP1	Eine SSH-Sitzung auf der VM `k3s-app1` (`172.16.0.228`).
APP2	Eine SSH-Sitzung auf der VM `k3s-app2` (`172.16.0.83`).
POD	Eine Shell in einem Container, die du vorher mit `kubectl exec` geöffnet hast.

Ersetze `<user>` jeweils durch deinen SSH-Login. Die meiste Zeit arbeitest du auf dem Mac. Auf die VMs gehst du nur in Kapitel 2 und beim Backup in Kapitel 12.

WARUM DAS WICHTIG IST

Mit drei Clustern ist der häufigste Fehler der falsche Kontext. Deshalb steht in jedem Befehl ausdrücklich `--context obs`, `--context app1` oder `--context app2`. Den aktiven Standardkontext von `kubectl` nutzt dieses Buch nie.

Die Kästen

WARUM DAS WICHTIG IST

Hier steht der Hintergrund, der dir beim nächsten Problem hilft.

STOLPERFALLE

Hier steht, was typischerweise schiefgeht und woran du es erkennst.

DAS SOLLTEST DU SEHEN

Hier steht das erwartete Ergebnis. Weicht deine Ausgabe ab, geh nicht weiter, sondern kläre zuerst den Unterschied.

PROBIER ES SELBST

Hier bist du dran. Überlege zuerst selbst, bevor du nachschlägst.

Was geprüft wurde und was nicht

Ein Buch voller Befehle ist nur so gut wie seine Prüfung. Deshalb steht hier ehrlich, wie weit sie ging.

Auf echten Clustern durchgespielt am 11. November 2026: Die Kapitel 2 bis 10 und 13 und der Velero-Teil von Kapitel 12 liefen Befehl für Befehl auf drei Ubuntu-VMs mit k3s `v1.36.6+k3s1`, mit genau den Versionen aus dem Anhang. Die Ausgaben in den Kästen „Das solltest du sehen“

stammen aus diesem Lauf, die Bilder der Oberflächen ebenfalls. Wo der Lauf Fehler im Buch aufgedeckt hat, sind sie korrigiert. Ausnahmen:

- Die Installation von k3s und das Zusammenführen der Kubeconfig in Kapitel 2 lagen vor dem Lauf und wurden nicht wiederholt.
- Den Dead Man's Switch bei healthchecks.io ersetzte im Test eine interne Adresse, weil dafür ein eigenes Konto nötig ist.
- Kapitel 11 (AWS) lief nicht gegen ein echtes AWS-Konto. Die etcd-Snapshots in Kapitel 12 brauchen SSH auf die VMs und wurden ebenfalls nicht ausgeführt. Beide Stellen sind im Text markiert.
- Aus Kapitel 14 liefen die Messungen und Prüfbefehle, nicht aber Updates und das Aufräumen.
- Kapitel 15 und 16 sind Lesekapitel. Was dort im Labor gemessen oder getestet wurde, steht ausdrücklich dabei, etwa ein Ausfall der Zentrale über 66 Minuten und die Mandantentrennung mit Thanos. Alles andere sind Architekturbeispiele.

Zusätzlich geprüft am 9. November 2026: Alle Helm-Werte wurden mit `helm template` gegen die Chart-Versionen gerendert, alle Manifeste mit `kubeconform` gegen Kubernetes 1.36 geprüft, die Shell-Skripte bestehen `shellcheck`.

Rechne also damit, dass an einer Stelle etwas abweicht. Das ist kein Grund zur Sorge, sondern Übung: Die Fehlersuche gehört zum Lernen, und Kapitel 14 hilft dir dabei.

Was du brauchst

- Drei frische Ubuntu-24.04-VMs mit festen IP-Adressen im selben Netz:

VM	Hostname	IP	Größe
Zentrale	`k3s-obs`	`172.16.0.126`	4 CPUs, 12 GB RAM (mindestens 8), 80 GB Disk
App-Cluster 1	`k3s-app1`	`172.16.0.228`	2 CPUs, 4 GB RAM, 30 GB Disk
App-Cluster 2	`k3s-app2`	`172.16.0.83`	2 CPUs, 4 GB RAM, 30 GB Disk

- Auf dem Laptop `kubectl`, `helm`, `jq`, `openssl`, `curl` und `git`. Ab Kapitel 11 zusätzlich die `aws` -CLI.
- SSH-Zugang zu allen drei VMs mit einem Benutzer, der `sudo` darf.
- Für Kapitel 11 ein AWS-Konto. Alles andere läuft ohne Cloud.
- Rund drei Wochenenden Zeit, oder ein Kapitel pro Abend.

ANDERE IP-ADRESSEN?

Die drei Adressen stehen in den k3s-Konfigurationen unter `observability/k3s/`, in `observability/k3s/coredns-custom.yaml` und in vielen Befehlen. Hast du andere, ersetze sie überall, bevor du mit Kapitel 2 beginnst. Ein `grep -rnE "172\.16\.0\.(126|228|83)" observability/` zeigt dir jede Stelle.

KAPITEL 1

Die Architektur im Überblick

Du weißt, welche Daten aus welchem Cluster wohin fließen, und warum die App-Cluster ihre Daten selbst zur Zentrale schicken.

Dauer: 20 Minuten Lesen

1.1 Die Frage hinter allem

Ein Cluster allein ist schnell überwacht. Ein Prometheus, ein Grafana, fertig. Mit dem zweiten Cluster beginnen die Probleme. Du hast zwei Grafanas, zwei Sätze Alarme und zwei Orte, an denen du nachsehen musst. Beim dritten Cluster sucht niemand mehr in allen.

Dieses Buch baut deshalb eine **Zentrale**. Sie sammelt die Daten aller Cluster, bewahrt sie günstig in S3 auf und beantwortet Fragen über alle Cluster hinweg. Die App-Cluster bleiben schlank. Sie sammeln nur ein und schicken weiter.

1.2 Die vier Signale

Observability ruht auf vier Arten von Daten. Jede beantwortet eine andere Frage:

Signal	Frage	Beispiel
Metriken	Wie viel, wie oft, wie schnell?	CPU-Last eines Nodes, Anfragen pro Sekunde, Fehlerquote
Logs	Was genau ist passiert?	Eine Fehlermeldung mit Stacktrace, ein Kubernetes-Event
Traces	Wo ist die Zeit geblieben?	Eine Anfrage durch drei Dienste mit der Dauer jedes Schritts
Profile	Welche Codezeile verbraucht die CPU?	Ein Flammendiagramm einer Go-Funktion

Metriken sind billig und schnell. Sie sagen dir, **dass** etwas nicht stimmt. Logs und Traces sagen dir, **was** und **wo**. Profile gehen bis auf die Funktion hinunter. Die ersten drei Signale baust du vollständig auf, Profile streift Kapitel 13 mit Pyroscope.

WARUM DAS WICHTIG IST

Erst die Verbindung macht die Signale wertvoll. Ein Alarm auf eine Metrik führt dich zu den Logs der betroffenen Pods, ein Log-Eintrag mit Trace-ID zum Trace, ein Trace zum langsamen Dienst. Deshalb tragen alle Daten dasselbe Label `cluster`, und Grafana kann zwischen ihnen springen.

1.3 Zentral oder dezentral

Für mehrere Cluster gibt es zwei Grundmuster:

	Dezentral	Zentral
Speicher	Jeder Cluster speichert selbst	Eine Zentrale speichert alles
Abfragen	Pro Cluster getrennt, oder ein Query-Layer, der alle Cluster erreicht	An einer Stelle, über alle Cluster
Last in den App-Clustern	Hoch: Prometheus, Speicher, eventuell Grafana	Gering: nur ein Sammler
Ausfall der Zentrale	Kein Problem	Keine Sicht mehr, Daten puffern kurz im Sammler

Dieses Buch wählt die zentrale Variante. Sie ist einfacher zu betreiben, und die App-Cluster brauchen kaum Ressourcen. Den Nachteil, dass die Zentrale ein einzelner Punkt ist, behandelst du mit Meta-Monitoring und einem Dead Man's Switch in den Kapiteln 7 und 13.

Was passiert, wenn obs ausfällt?

An dieser Frage erkennst du, wie zuverlässig eine Architektur wirklich ist. Für dieses Buch wurde sie im Labor gemessen: Die Zentrale war für die App-Cluster 66 Minuten lang nicht erreichbar. Das Ergebnis in einem Satz: **Metriken kamen vollständig an, Logs zur Hälfte, Traces gar nicht.**

Signal	Wie Alloy puffert	Nach 66 Minuten Ausfall
Metriken	Write-Ahead-Log auf der Platte, bis zu 8 Stunden	nichts verloren
Logs	im Speicher, gibt nach rund 8 Minuten auf	rund 32 Minuten verloren
Traces	im Speicher, gibt nach 5 Minuten auf	alles verloren

Alarme gibt es während des Ausfalls trotzdem: Die Zentrale merkt nach acht Minuten, dass zwei Cluster schweigen. Fällt dagegen die Zentrale selbst aus, kann sie niemanden mehr warnen. Das übernimmt der Dead Man's Switch aus Kapitel 7. Wie die Messung im Detail ablief und wie du die Plattform so baust, dass ein Ausfall möglichst nichts kostet, beschreibt Kapitel 15.

1.4 Push statt Pull

Prometheus holt sich Metriken normalerweise selbst ab (*Pull*). Für die Zentrale drehen wir das um: Die App-Cluster schicken ihre Metriken per *Remote Write* an Thanos Receive (*Push*). Dafür gibt es drei Gründe.

NAT und Firewalls. In der Praxis steht ein App-Cluster oft hinter NAT, in einem anderen Rechenzentrum oder einer anderen Cloud. Die Zentrale kann ihn nicht erreichen, er die Zentrale aber schon. Push braucht nur eine ausgehende HTTPS-Verbindung zu einem einzigen Namen. Im Labor liegen alle VMs im selben Netz, der Aufbau funktioniert aber genauso, wenn sie das nicht tun.

Ein einziges S3-Ziel. Die Alternative wäre ein Thanos Sidecar in jedem Cluster. Dann bräuchte jeder Cluster eigene S3-Zugangsdaten, und die Zentrale müsste per gRPC in jeden Cluster hinein, um aktuelle Daten abzufragen. Mit Push kennt nur die Zentrale den S3-Speicher.

Ein Muster für alles. Loki und Tempo arbeiten ohnehin nach dem Push-Prinzip. Mit Remote Write für Metriken schickt ein einziger Sammler, Grafana Alloy, alle drei Signale auf dieselbe Weise: über HTTPS, mit Basic Auth und mit dem Clusternamen als Mandant.

STOLPERFALLE

Push hat einen Preis. Fällt ein App-Cluster komplett aus, kommen von ihm einfach keine Daten mehr, und das sieht zunächst aus wie Ruhe. Du brauchst deshalb einen Alarm auf *fehlende* Daten. Den baust du in Kapitel 7.

1.5 Wie die Daten fließen

Jedes Signal nimmt seinen eigenen Weg, aber alle enden in S3 und in Grafana:

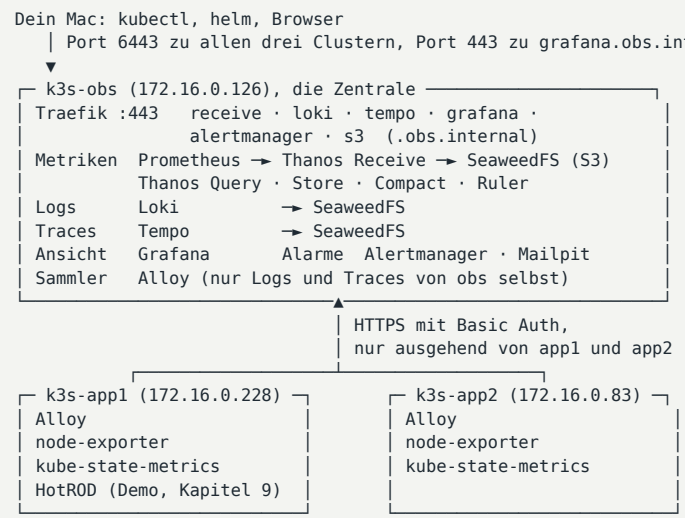
Signal	Sammler im Cluster	Endpunkt auf obs	Speicher	Ansicht
Metriken (obs)	Prometheus aus `kps`	Thanos Receive intern, Port 19291	Bucket `thanos`	Grafana über Thanos Query
Metriken (app1, app2)	Alloy, kube-state-metrics, node-exporter	`https://receive.obs.internal`	Bucket `thanos`	Grafana über Thanos Query
Logs	Alloy	`https://loki.obs.internal`	Bucket `loki`	Grafana mit LogQL
Traces	Alloy (OTLP)	`https://tempo.obs.internal`	Bucket `tempo`	Grafana mit TraceQL

Signal	Sammler im Cluster	Endpunkt auf obs	Speicher	Ansicht
Profile	Pyroscope (optional)	nur auf obs	lokal	Grafana
Alarme	Prometheus, Thanos Ruler, Loki-Ruler	Alertmanager auf obs	–	Mailpit, Watchdog-Dienst

1.6 Was läuft wo

Die Tabelle oben zeigt die Wege der Daten. Die Skizze zeigt, auf welcher VM am Ende was läuft. Sie ist bewusst einfach gehalten: drei Kästen, ein Pfeil.

DATEI `Was läuft auf welcher VM (Endstand nach Kapitel 9)`



Alles Schwere läuft auf obs. In app1 und app2 laufen nur drei kleine Sammler und deine Anwendungen. Prometheus gibt es nur auf obs. In den App-Clustern übernimmt Alloy das Einsammeln und schickt die Daten weiter. SeaweedFS ist nur innerhalb von obs erreichbar. Erst Kapitel 12 öffnet es unter `s3.obs.internal` für die Backups.

Jeder Cluster schreibt als eigener **Mandant**. Thanos Receive erhält den Clusternamen im Header `THANOS-TENANT`, Loki und Tempo im Header `X-Scope-OrgID`. Zusätzlich trägt jede Serie, jeder Log-Stream und jede Span das Label `cluster`. So bleibt die Herkunft auch dann erkennbar, wenn du über alle Mandanten hinweg abfragst.

Die Zentrale ist von außen nur über Traefik erreichbar, mit TLS aus einer eigenen kleinen CA und Basic Auth. Jeder Cluster hat einen eigenen Benutzer. Den S3-Speicher sieht niemand außerhalb von `obs`.

PROBIER ES SELBST

Lies die Tabelle noch einmal und überlege: Welche Verbindungen gehen von `app1` aus, und welche gehen in `app1` hinein? Wenn deine Antwort bei „hinein“ leer ist, hast du das Push-Prinzip verstanden.

1.7 Der Weg durch das Buch

Das Buch baut die Plattform Schicht für Schicht auf. Jede Schicht prüfst du, bevor die nächste darauf aufsetzt. Die Tabelle zeigt die Reihenfolge, wo du arbeitest und woran du erkennst, dass ein Schritt gelungen ist:

Kap.	Was entsteht	Wo	Woran du siehst, dass es läuft
2	Drei k3s-Cluster, Kontexte auf dem Mac, Namen `*.obs.internal`, eigene CA	alle	`kubectl get nodes` in jedem Kontext, `nslookup` aus einem Pod
3	S3-Speicher SeaweedFS mit vier Buckets und Konten	obs	`aws s3 ls` mit dem Konto `loki` zeigt nur `loki`
4	Prometheus, Thanos Receive, Query, Store, Compact, Ruler	obs	Thanos Query zeigt `cluster="obs"`
5	Alloy, node-exporter und kube-state-metrics, Zugang über HTTPS	app1, app2	Thanos Query zeigt alle drei Cluster
6	Grafana mit Datenquellen aus dem Repo	obs	Login unter `grafana.obs.internal`, Explore zeigt drei Cluster
7	Alarmregeln, Alertmanager, Mailpit, Dead Man’s Switch	obs	Ein absichtlicher Ausfall erzeugt eine Mail
8	Loki, Alloy sammelt Logs und Events, Log-Alarme	alle	Explore mit Loki zeigt Logs aller Cluster
9	Tempo, Alloy nimmt Traces an, Demo-App HotROD	alle	TraceQL findet Traces, Sprung vom Log zum Trace
10	Sieben fertige Dashboards	obs	Ordner **Plattform** und **Signale** in Grafana
11	Umzug der Buckets zu AWS S3 (optional)	obs	Neue Blöcke landen bei AWS
12	Backups: etcd-Snapshots und Velero	alle	Ein gelöschter Namespace kommt zurück

Kap.	Was entsteht	Wo	Woran du siehst, dass es läuft
13	Was sonst noch fehlt: Zertifikate, Profile, Rechte	alle	je Abschnitt
14	Betrieb und Fehlersuche	alle	–
15	Vom Labor zur Enterprise-Architektur: Hochverfügbarkeit	–	Ein Lesekapitel ohne Installation
16	Enterprise-Observability: Multi-Tenancy mit Thanos	–	Ein Lesekapitel mit Tests im Labor
17	Anhang: Versionen, Repo, Spickzettel	–	–

Die Reihenfolge folgt dem Muster „erst beobachten, dann scharf schalten“. Zuerst fließen Metriken, dann siehst du sie in Grafana, erst danach kommen Alarme. Logs und Traces folgen, wenn das Fundament steht.

1.8 Der Ressourcenplan

Wie viel Speicher die Komponenten brauchen, hängt stark von der Datenmenge ab. Die folgenden Werte sind **geschätzt** für ein Labor mit drei Ein-Node-Clustern und einer kleinen Demo-App. In Kapitel 14 misst du die echten Werte nach.

Komponente auf obs	RAM (geschätzt)	Disk (geschätzt)
k3s mit Traefik, CoreDNS, local-path	0,8 GB	5 GB
SeaweedFS	0,3 bis 1 GB	bis 30 GB (Limit)
Prometheus (6 h Retention)	0,8 GB	2 GB
Thanos Receive (6 h lokal)	1 GB	5 GB
Thanos Query, Store, Compact, Ruler	1 GB	5 GB
Alertmanager, Grafana, Mailpit	0,4 GB	1 GB
Loki (SingleBinary)	1 GB	5 GB
Tempo mit Metrics-Generator	1 GB	5 GB
Alloy, node-exporter, kube-state-metrics	0,4 GB	–
Summe	**rund 7 bis 8 GB**	**rund 60 GB**

Komponente auf app1 und app2	RAM (geschätzt)	Disk (geschätzt)
k3s mit Traefik, CoreDNS, local-path	0,7 GB	5 GB

Komponente auf app1 und app2	RAM (geschätzt)	Disk (geschätzt)
Alloy (Metriken, Logs, Traces)	0,3 GB	1 GB WAL
kube-state-metrics, node-exporter	0,1 GB	–
Demo-App (nur app1)	0,3 GB	–
Summe	**rund 1,5 GB**	**rund 10 GB**

Daraus ergeben sich die Größen aus dem Vorwort: 12 GB RAM für `obs` mit Luft für Spitzen, 4 GB für jeden App-Cluster. Mit 8 GB auf `obs` kommst du aus, wenn du Tempo erst in Kapitel 9 startest und die Retention knapp hältst.

WAS DU GELERNT HAST

- Metriken, Logs, Traces und Profile beantworten verschiedene Fragen. Erst zusammen erklären sie einen Fehler.
- Eine Zentrale mit schlanken App-Clustern ist einfacher zu betreiben als ein Speicher pro Cluster.
- Push per Remote Write braucht nur ausgehende Verbindungen und hält die S3-Zugangsdaten an einer Stelle.
- Jeder Cluster schreibt als eigener Mandant und trägt zusätzlich das Label `cluster`.
- Ausbleibende Daten sind bei Push unsichtbar. Dagegen hilft ein eigener Alarm.

KAPITEL 2

Drei k3s-Cluster als Fundament

Aus drei leeren Ubuntu-VMs werden drei Cluster. Dein Laptop steuert sie über die Kontexte `obs`, `app1` und `app2`, und alle kennen die Namen und die CA der Zentrale.

Dauer: 90 Minuten **Voraussetzung:** Drei Ubuntu-VMs mit SSH-Zugang, das geklonte Repo

2.1 Was du aufbaust

Jede VM wird ein eigenständiger Cluster mit genau einem Node. Die drei Cluster wissen nichts voneinander, sie teilen kein Netz und keine Konten. Verbunden werden sie erst später, über HTTPS zur Zentrale.

VM	IP	k3s-Konfiguration	Kontext
`k3s-obs`	`172.16.0.126`	`observability/k3s/config-obs.yaml`	`obs`
`k3s-app1`	`172.16.0.228`	`observability/k3s/config-app1.yaml`	`app1`
`k3s-app2`	`172.16.0.83`	`observability/k3s/config-app2.yaml`	`app2`

Du nutzt k3s so, wie es ausgeliefert wird: Flannel als Netzwerk, Traefik als Ingress, ServiceLB für die Ports 80 und 443 und `local-path` für Volumes. So bleibt der Fokus auf der Observability.

Gib jeder VM eine feste IP oder eine DHCP-Reservierung. Die Adressen stehen in den Konfigurationen und im DNS, sie dürfen sich später nicht mehr ändern.

LÄUFT AUF EINER VM SCHON K3S?

Dann entferne es zuerst mit `sudo /usr/local/bin/k3s-uninstall.sh`. Das löscht den Cluster samt Daten unwiderruflich.

2.2 Die k3s-Konfiguration

Alle drei Dateien sind gleich aufgebaut, nur Name und IP unterscheiden sich:

DATEI `observability/k3s/config-obs.yaml (Ausschnitt)`

```yaml
node-name: k3s-obs
node-ip: 172.16.0.126
tls-san:
  - 172.16.0.126
cluster-init: true
secrets-encryption: true
```

```
write-kubeconfig-mode: "0600"
cluster-cidr: "10.42.0.0/16"
service-cidr: "10.43.0.0/16"
etcd-expose-metrics: true
```

`tls-san` nimmt die IP ins Zertifikat des API-Servers auf, damit `kubectl` vom Mac aus die Verbindung prüfen kann. `cluster-init: true` startet eingebettetes etcd statt SQLite. Das brauchst du für die Snapshots in Kapitel 12. `etcd-expose-metrics` macht die etcd-Metriken auf Port 2381 der Node-IP abrufbar. Die Netze dürfen sich zwischen den Clustern wiederholen, weil keiner in den anderen hineinroutet.

MAC Schritt 1

```
scp observability/k3s/config-obs.yaml <user>@172.16.0.126:/tmp/config.yaml
scp observability/k3s/config-app1.yaml <user>@172.16.0.228:/tmp/config.yaml
scp observability/k3s/config-app2.yaml <user>@172.16.0.83:/tmp/config.yaml
```

Kopiert jeder VM ihre eigene Konfiguration. Achte darauf, dass Datei und IP zusammenpassen. Eine vertauschte Datei gibt der VM den falschen Namen und die falsche IP.

2.3 Die VM obs vorbereiten

Die folgenden Schritte zeigt das Buch für `obs`. Danach wiederholst du sie auf `app1` und `app2`.

MAC Schritt 2

```
ssh <user>@172.16.0.126
```

Öffnet die Sitzung auf der Zentrale. Alle Schritte mit dem Etikett **obs** laufen hier.

OBS Schritt 3

```
sudo hostnamectl set-hostname k3s-obs
```

Der Hostname sollte zum `node-name` aus der Konfiguration passen. Das erspart Verwirrung in Logs und Prompts.

OBS Schritt 4

```
sudo apt update && sudo apt -y upgrade
```

Bringt das System auf den aktuellen Stand.

OBS Schritt 5

```
sudo ufw status
```

Ubuntus Firewall ist meist inaktiv. Bei `inactive` machst du einfach weiter.

Ist die Firewall aktiv, müssen API-Server, Pod- und Service-Netz und auf `obs` auch HTTPS durchkommen:

OBS Schritt 6

```
sudo ufw allow 6443/tcp
sudo ufw allow 443/tcp
sudo ufw allow from 10.42.0.0/16 to any
sudo ufw allow from 10.43.0.0/16 to any
```

Port 6443 ist der API-Server für `kubectl` vom Mac. Port 443 braucht nur `obs`, denn dort nimmt Traefik die Daten der App-Cluster an. Lass Port 22 offen, sonst sperrst du dich aus.

2.4 k3s installieren

OBS Schritt 7

```
sudo install -D -m 600 -o root -g root \
  /tmp/config.yaml /etc/rancher/k3s/config.yaml
rm /tmp/config.yaml
```

`install` kopiert die Datei und setzt Rechte in einem Schritt. `-D` legt das Verzeichnis an. Die Datei muss **vor** der Installation hier liegen, sonst startet k3s ohne etcd, und das lässt sich nicht einfach nachholen.

OBS Schritt 8

```
curl -sfL https://get.k3s.io | \
  INSTALL_K3S_VERSION=v1.36.6+k3s1 sh -
```

Installiert genau die Version, mit der dieses Buch arbeitet. Ohne `INSTALL_K3S_VERSION` bekämst du den Kanal `stable`, und alle drei Cluster könnten später unterschiedliche Versionen haben.

OBS Schritt 9

```
sudo k3s kubectl get nodes -o wide
```

Fragt den Cluster direkt auf der VM ab.

DAS SOLLTEST DU SEHEN

Der Node `k3s-obs` steht nach etwa einer Minute auf `Ready`, Version `v1.36.6+k3s1`, `INTERNAL-IP` `172.16.0.126`. Startet k3s nicht, zeigt `sudo journalctl -u k3s -n 50` den Grund.

OBS Schritt 10

```
sudo k3s kubectl get pods -n kube-system
```

Zeigt die mitgelieferten Komponenten.

DAS SOLLTEST DU SEHEN

`coredns`, `local-path-provisioner`, `metrics-server` und `traefik` laufen, dazu ein `svclb-traefik`-Pod. Die beiden `helm-install`-Pods stehen auf `Completed`. Das ist richtig, sie haben Traefik installiert und sind fertig.

OBS — Schritt 11

```
sudo install -m 600 -o "$USER" \
  /etc/rancher/k3s/k3s.yaml /tmp/k3s.yaml
exit
```

Die Kubeconfig gehört root. `install` legt eine Kopie an, die dein Benutzer per `scp` holen darf, und nur er kann sie lesen. Sie enthält volle Admin-Rechte. Danach geht es zurück zum Mac.

2.5 Dasselbe auf app1 und app2

Wiederhole die Schritte dieses und des vorigen Abschnitts auf den beiden anderen VMs. Nur drei Dinge ändern sich:

	app1	app2
SSH	`ssh <user>@172.16.0.228`	`ssh <user>@172.16.0.83`
Hostname	`k3s-app1`	`k3s-app2`
Firewall	ohne Port 443	ohne Port 443

STOLPERFALLE

Wer drei Terminals offen hat, tippt leicht in das falsche. Der Hostname im Prompt hilft. Er wechselt allerdings erst bei der nächsten Anmeldung. Prüfe nach `hostnamectl` deshalb mit `hostname`, wo du bist.

2.6 Die Kubeconfigs zusammenführen

In `~/.kube/config` liegen vielleicht schon andere Kontexte. Du überschreibst die Datei deshalb nicht, sondern fügst die drei neuen Cluster hinzu.

MAC — Schritt 12

```
mkdir -p ~/.kube
[ -f ~/.kube/config ] && cp -p ~/.kube/config ~/.kube/config.bak-obs
```

Sicherung der bestehenden Datei, falls es eine gibt. Geht etwas schief, kopierst du sie zurück.

MAC Schritt 13

```
scp <user>@172.16.0.126:/tmp/k3s.yaml ~/.kube/obs.raw
scp <user>@172.16.0.228:/tmp/k3s.yaml ~/.kube/app1.raw
scp <user>@172.16.0.83:/tmp/k3s.yaml  ~/.kube/app2.raw
```

Holt die drei Kubeconfigs auf den Mac. Jede Datei bekommt den Namen ihres Clusters, damit du sie nicht verwechselst.

MAC Schritt 14

```
ssh <user>@172.16.0.126 'rm /tmp/k3s.yaml'
ssh <user>@172.16.0.228 'rm /tmp/k3s.yaml'
ssh <user>@172.16.0.83  'rm /tmp/k3s.yaml'
```

Löscht die Kopien auf den VMs wieder. Sie enthalten volle Admin-Rechte und sollen nicht in `/tmp` liegen bleiben.

MAC Schritt 15

```
sed -e 's/127\.0\.0\.1/172.16.0.126/' -e 's/: default$/: obs/' \
  ~/.kube/obs.raw > ~/.kube/obs.yaml
sed -e 's/127\.0\.0\.1/172.16.0.228/' -e 's/: default$/: app1/' \
  ~/.kube/app1.raw > ~/.kube/app1.yaml
sed -e 's/127\.0\.0\.1/172.16.0.83/' -e 's/: default$/: app2/' \
  ~/.kube/app2.raw > ~/.kube/app2.yaml
```

Jede Datei zeigt auf `127.0.0.1` und nennt Cluster, Benutzer und Kontext `default`. Der erste Ausdruck von `sed` setzt die IP der VM ein, der zweite benennt alles nach dem Cluster um. Ohne die Umbenennung hießen alle drei Einträge `default` und würden sich beim Zusammenführen gegenseitig überschreiben.

MAC Schritt 16

```
kubectl config get-contexts
```

Zeigt deine bisherigen Kontexte. Stehen `obs`, `app1` oder `app2` schon dort, entferne sie zuerst:

MAC Schritt 17

```
kubectl config delete-context obs
kubectl config delete-cluster obs
kubectl config delete-user obs
```

Nur nötig, wenn ein gleichnamiger Eintrag existiert, und dann für jeden betroffenen Namen. Alte Einträge würden beim Zusammenführen gewinnen, und `kubectl` meldete `x509`-Fehler.

MAC Schritt 18

```
KUBECONFIG=~/.kube/config:~/.kube/obs.yaml:~/.kube/app1.yaml:~/.kube/app2.yaml \
  kubectl config view --flatten > ~/.kube/config.neu
```

Mehrere Dateien in `KUBECONFIG` führt `kubectl` zusammen. `--flatten` schreibt die Zertifikate direkt in die Ausgabe, damit die neue Datei allein funktioniert.

MAC Schritt 19

```
chmod 600 ~/.kube/config.neu
mv ~/.kube/config.neu ~/.kube/config
rm ~/.kube/{obs,app1,app2}.{raw,yaml}
```

Setzt die neue Datei mit den richtigen Rechten ein und löscht die Zwischendateien mit den Admin-Zugangsdaten.

MAC Schritt 20

```
kubectl --context obs  get nodes
kubectl --context app1 get nodes
kubectl --context app2 get nodes
```

Fragt jeden Cluster über seinen eigenen Kontext ab.

DAS SOLLTEST DU SEHEN

Drei Blöcke mit je einem Node, alle `Ready`:

```
NAME       STATUS   ROLES                  AGE   VERSION
k3s-obs    Ready    control-plane,etcd   5m    v1.36.6+k3s1
NAME        STATUS   ROLES                  AGE   VERSION
k3s-app1   Ready    control-plane,etcd   5m    v1.36.6+k3s1
NAME        STATUS   ROLES                  AGE   VERSION
k3s-app2   Ready    control-plane,etcd   5m    v1.36.6+k3s1
```

Deine alten Kontexte sind weiterhin in `kubectl config get-contexts` zu sehen.

WARUM DAS WICHTIG IST

Ab hier steht in jedem Befehl `--context`. Das ist umständlicher als `kubectl config use-context`, schützt dich aber vor dem klassischen Fehler, einen Befehl für `app1` versehentlich auf der Zentrale auszuführen.

2.7 Der Namespace monitoring

Alle Observability-Komponenten liegen in jedem Cluster im Namespace `monitoring`. Lege ihn jetzt an, denn TLS-Secret und CA kommen gleich hinein.

MAC Schritt 21

```
kubectl --context obs  create namespace monitoring
kubectl --context app1 create namespace monitoring
kubectl --context app2 create namespace monitoring
```

Legt den Namespace in jedem Cluster an. Führst du den Befehl ein zweites Mal aus, meldet `kubectl` `AlreadyExists`. Das ist harmlos, der Namespace ist dann schon da.

DAS SOLLTEST DU SEHEN

Dreimal `namespace/monitoring created`.

2.8 Namen für die Zentrale

Die App-Cluster erreichen die Zentrale später unter Namen wie `receive.obs.internal`. Diese Namen gibt es in keinem öffentlichen DNS. Du trägst sie an zwei Stellen ein: im CoreDNS jedes Clusters und auf dem Mac.

k3s bringt dafür einen eingebauten Haken mit. CoreDNS lädt alle Schlüssel der ConfigMap `coredns-custom` im Namespace `kube-system`, die auf `.server` enden, als eigene Server-Blöcke.

DATEI `observability/k3s/coredns-custom.yaml`

```
apiVersion: v1
kind: ConfigMap
metadata:
  name: coredns-custom
  namespace: kube-system
data:
  obs.server: |
    obs.internal:53 {
        errors
        cache 30
        hosts {
            172.16.0.126 grafana.obs.internal
            172.16.0.126 receive.obs.internal
            172.16.0.126 loki.obs.internal
            172.16.0.126 tempo.obs.internal
            172.16.0.126 alertmanager.obs.internal
            172.16.0.126 s3.obs.internal
            fallthrough
        }
    }
```

Der Block gilt nur für die Zone `obs.internal`. Alle anderen Anfragen beantwortet CoreDNS wie bisher. Alle Namen zeigen auf die IP von `obs`, wo Traefik auf Port 443 wartet und nach Hostname weiterleitet. `s3.obs.internal` brauchst du erst in Kapitel 12 für die Backups. Er steht schon hier, damit du die Datei nur einmal anwenden musst.

MAC Schritt 22

```
kubectl --context obs  apply -f observability/k3s/coredns-custom.yaml
kubectl --context app1 apply -f observability/k3s/coredns-custom.yaml
kubectl --context app2 apply -f observability/k3s/coredns-custom.yaml
```

Wendet dieselbe Datei auf alle drei Cluster an. Auch `obs` braucht die Namen, etwa wenn Grafana später Links auf sich selbst prüft.

MAC Schritt 23

```
kubectl --context obs  -n kube-system rollout restart deploy/coredns
kubectl --context app1 -n kube-system rollout restart deploy/coredns
kubectl --context app2 -n kube-system rollout restart deploy/coredns
```

CoreDNS bemerkt die neue ConfigMap zwar von selbst, das kann aber über eine Minute dauern. Der Neustart macht es sofort wirksam.

MAC Schritt 24

```
kubectl --context app1 run dns-test --rm -it --restart=Never \
  --image=busybox:1.36 -- nslookup receive.obs.internal
```

Startet einen Wegwerf-Pod in `app1` und fragt den Namen ab. `--rm` entfernt den Pod danach.

DAS SOLLTEST DU SEHEN

Die Antwort enthält `Name: receive.obs.internal` und `Address: 172.16.0.126`. Meldet `nslookup` dagegen `NXDOMAIN`, ist die ConfigMap noch nicht angekommen oder der Schlüssel endet nicht auf `.server`.

STOLPERFALLE

Der Name `coredns-custom` ist fest. Eine ConfigMap mit anderem Namen ignoriert k3s stillschweigend. Ebenso werden Schlüssel, die auf `.override` statt `.server` enden, in den Hauptblock eingefügt. Dort wäre ein eigener Zonen-Block ein Syntaxfehler, und CoreDNS startete nicht mehr.

Auf dem Mac genügt ein Eintrag in `/etc/hosts`:

MAC Schritt 25

```
echo "172.16.0.126 grafana.obs.internal receive.obs.internal" \
  "loki.obs.internal tempo.obs.internal alertmanager.obs.internal" \
  "s3.obs.internal" | sudo tee -a /etc/hosts
```

`echo` verbindet beide Teile mit einem Leerzeichen zu einer Zeile, `tee -a` hängt sie an. Ein einfaches `sudo echo ... >> /etc/hosts` würde scheitern, weil die Umleitung nicht mit root-Rechten läuft.

MAC Schritt 26

```
dscacheutil -q host -a name grafana.obs.internal
```

Fragt den Namen so ab, wie macOS-Programme es tun.

DAS SOLLTEST DU SEHEN

Die Ausgabe enthält `ip_address: 172.16.0.126`. Steht dort nichts, hilft `sudo dscacheutil -flushcache`.

2.9 Eine eigene CA für TLS

Die App-Cluster schicken später Zugangsdaten und Messwerte über das Netz zur Zentrale. Dabei müssen zwei Dinge gelten:

- **Verschlüsselung:** Niemand im Netz soll Passwörter oder Logs mitlesen. Das leistet TLS, also HTTPS.
- **Echtheit:** Alloy in `app1` muss sicher sein, dass am anderen Ende wirklich `obs` antwortet und kein fremder Rechner, der sich die IP geschnappt hat. Dafür zeigt der Server ein Zertifikat vor, und der Client prüft, wer es unterschrieben hat.

Im Internet unterschreibt eine öffentliche Zertifizierungsstelle (CA) wie Let's Encrypt. Die stellt aber keine Zertifikate für erfundene Namen wie `obs.internal` aus. Du baust dir deshalb eine eigene kleine CA. Das sind nur zwei Dateien: ein geheimer Schlüssel, mit dem die CA unterschreibt, und ein öffentliches Zertifikat, an dem jeder ihre Unterschrift erkennt.

DATEI Die Vertrauenskette

```
ca.key  (geheim, bleibt auf dem Mac)
   | unterschreibt
   ▼
obs.internal.crt + obs.internal.key ──► Traefik auf obs zeigt es vor
   ▲
   | prüft die Unterschrift
ca.crt  (öffentlich) ──► liegt in allen Clustern und auf dem Mac
```

Ein Client vertraut dem Zertifikat von `obs`, weil es mit `ca.key` unterschrieben ist und er `ca.crt` kennt. Mehr braucht er nicht. Deshalb bekommen `app1` und `app2` nur das öffentliche `ca.crt`, nie einen Schlüssel.

Ein einziges Wildcard-Zertifikat für `*.obs.internal` deckt alle Namen der Zentrale ab, von `grafana` bis `s3`. Welche Eigenschaften die beiden Zertifikate bekommen, steht in einer kleinen Konfigurationsdatei:

DATEI observability/tls/openssl.cnf (Ausschnitt)

```
[ v3_ca ]
basicConstraints     = critical, CA:true, pathlen:0
keyUsage             = critical, keyCertSign, cRLSign

[ v3_server ]
basicConstraints       = critical, CA:false
keyUsage               = critical, digitalSignature, keyEncipherment
extendedKeyUsage       = serverAuth
subjectAltName         = @alt_names

[ alt_names ]
DNS.1 = *.obs.internal
DNS.2 = obs.internal
```

Die Datei enthält zwei Sätze von Erweiterungen, und jeder Befehl wählt mit `-extensions` einen davon aus. `v3_ca` macht ein Zertifikat zur CA: `CA:true` erlaubt das Unterschreiben, `pathlen:0` verbietet Unter-CAs, und `keyUsage` beschränkt den Schlüssel auf genau diese Aufgabe. `v3_server` beschreibt das Server-Zertifikat: `CA:false` verhindert, dass jemand damit weitere Zertifikate ausstellt, und `serverAuth` erlaubt den Einsatz für HTTPS-Server. Entscheidend ist `subjectAltName`: Browser und Go-Programme wie Alloy prüfen nur diese Namensliste, den Common Name ignorieren sie.

Jetzt erzeugst du die CA. Alle Dateien landen in `observability/tls/`.

MAC Schritt 27

```
openssl req -x509 -new -nodes -newkey rsa:4096 -days 3650 \
  -keyout observability/tls/ca.key -out observability/tls/ca.crt \
  -config observability/tls/openssl.cnf -extensions v3_ca \
  -subj "/CN=obs-lab CA"
```

Erzeugt in einem Schritt den Schlüssel der CA und ihr Zertifikat.

Die Optionen im Einzelnen:

Option	Bedeutung
`req -new`	Erzeugt eine neue Zertifikatsanfrage. `req` ist das Werkzeug von OpenSSL für Anfragen und Zertifikate.
`-x509`	Macht aus der Anfrage sofort ein fertiges Zertifikat, das sich selbst unterschreibt. Eine CA hat niemanden über sich, der sie unterschreiben könnte.
`-newkey rsa:4096`	Erzeugt dabei einen neuen RSA-Schlüssel mit 4096 Bit. Die CA lebt lange, deshalb bekommt sie einen starken Schlüssel.
`-nodes`	„no DES“: Der Schlüssel wird ohne Passwort gespeichert. Im Labor ist das bequem, in der Produktion gehört er verschlüsselt und offline.
`-days 3650`	Gültigkeit zehn Jahre. Läuft die CA ab, musst du allen Clients ein neues `ca.crt` geben.
`-keyout`, `-out`	Wohin Schlüssel und Zertifikat geschrieben werden.
`-config`, `-extensions`	Liest die Erweiterungen aus dem Abschnitt `v3_ca` der Datei oben.
`-subj`	Der Name der CA. Er erscheint später als „Aussteller“ in jedem Zertifikat, das sie unterschreibt.

Als Nächstes braucht der Server, also Traefik auf `obs`, einen eigenen Schlüssel und ein Zertifikat. Das geschieht in zwei Schritten, genau wie bei einer echten CA: Der Server stellt eine Anfrage, die CA unterschreibt sie.

MAC Schritt 28

```
openssl req -new -nodes -newkey rsa:2048 \
  -keyout observability/tls/obs.internal.key \
  -out observability/tls/obs.internal.csr \
  -config observability/tls/openssl.cnf -subj "/CN=*.obs.internal"
```

Erzeugt den Schlüssel für das Server-Zertifikat und eine Zertifikatsanfrage (CSR, Certificate Signing Request).

Der Befehl ähnelt dem vorigen, nur **ohne** `-x509`. Damit entsteht kein fertiges Zertifikat, sondern eine Anfrage: „Bitte bestätige, dass dieser öffentliche Schlüssel zu `*.obs.internal` gehört.“ Der private Schlüssel `obs.internal.key` verlässt dabei nie deinen Rechner, in der Anfrage steckt nur der öffentliche Teil. 2048 Bit genügen, denn das Server-Zertifikat tauschst du ohnehin alle zwei Jahre aus.

MAC Schritt 29

```
openssl x509 -req -in observability/tls/obs.internal.csr \
  -CA observability/tls/ca.crt -CAkey observability/tls/ca.key \
  -set_serial "0x$(openssl rand -hex 16)" -days 825 -sha256 \
  -extfile observability/tls/openssl.cnf -extensions v3_server \
  -out observability/tls/obs.internal.crt
```

Die CA unterschreibt die Anfrage. Heraus kommt das fertige Server-Zertifikat.

Option	Bedeutung
`x509 -req -in`	Liest eine Anfrage ein, statt ein Zertifikat zu bearbeiten.
`-CA`, `-CAkey`	Wer unterschreibt: das Zertifikat und der geheime Schlüssel der CA.
`-set_serial`	Jedes Zertifikat einer CA braucht eine eindeutige Seriennummer. `openssl rand -hex 16` würfelt eine. Das erspart die Datei `ca.srl`, in der OpenSSL sonst mitzählt.
`-days 825`	Gültigkeit gut zwei Jahre. Das ist das Maximum, das macOS bei Server-Zertifikaten akzeptiert.
`-sha256`	Das Hash-Verfahren der Unterschrift. Ältere Verfahren wie SHA-1 lehnen Browser ab.
`-extfile`, `-extensions`	Fügt die Erweiterungen aus `v3_server` hinzu, vor allem die Namensliste `subjectAltName`. Ohne sie wäre das Zertifikat für Alloy und Browser wertlos.

MAC Schritt 30

```
rm observability/tls/obs.internal.csr
chmod 600 observability/tls/*.key
```

Die Anfrage hat ihren Zweck erfüllt und kann weg. Die beiden Schlüssel darf danach nur noch dein Benutzer lesen.

WARUM DAS WICHTIG IST

Wer `ca.key` besitzt, kann Zertifikate für beliebige Namen unter `obs.internal` ausstellen, und alle deine Cluster würden ihnen glauben. Wer `obs.internal.key` besitzt, kann sich als Zentrale ausgeben. Beide Schlüssel sind deshalb so wertvoll wie ein Admin-Passwort. Die `.crt`-Dateien dagegen sind öffentlich, sie dürfen ins Repo.

Bevor du die Dateien verteilst, prüfst du, ob alles stimmt. Ein Fehler hier zeigt sich sonst erst viel später als rätselhafter `x509`-Fehler in Alloy.

MAC — Schritt 31

```
openssl verify -CAfile observability/tls/ca.crt observability/tls/obs.internal.crt
openssl x509 -in observability/tls/obs.internal.crt -noout -text \
  | grep -A1 "Subject Alternative Name"
```

Der erste Befehl macht genau das, was später jeder Client tut: Er prüft, ob das Server-Zertifikat von dieser CA unterschrieben und noch gültig ist. Der zweite zeigt die Namensliste, für die das Zertifikat gilt. Beide funktionieren mit dem LibreSSL von macOS und mit OpenSSL 3.

DAS SOLLTEST DU SEHEN

```
observability/tls/obs.internal.crt: OK
            X509v3 Subject Alternative Name:
                DNS:*.obs.internal, DNS:obs.internal
```

MAC — Schritt 32

```
git check-ignore -v observability/tls/ca.key observability/tls/obs.internal.key
```

Fragt Git, ob die Schlüssel vom Commit ausgeschlossen sind.

DAS SOLLTEST DU SEHEN

Für beide Dateien nennt Git die Regel `*.key` aus `.gitignore`:

```
.gitignore:2:*.key   observability/tls/ca.key
.gitignore:2:*.key   observability/tls/obs.internal.key
```

Gibt der Befehl nichts aus, würden die Schlüssel beim nächsten Commit mitgehen. Dann ergänze die Regel, bevor du weitermachst.

2.10 Zertifikat und CA in die Cluster bringen

Die Dateien liegen jetzt auf deinem Mac. Die Programme, die sie brauchen, laufen aber in den Clustern. In Kubernetes gibst du einem Pod Dateien über zwei Objekte: ein **Secret** für vertrauliche Daten und eine **ConfigMap** für alles andere. Der Pod bindet sie als Datei ein oder liest sie als Umgebungsvariable. So steht kein Schlüssel im Image oder in der Helm-Konfiguration.

Wer braucht was?

Objekt	Cluster	Wozu
Secret `obs-tls`	nur `obs`	Traefik zeigt Zertifikat und Schlüssel jedem Client vor, der `https://*.obs.internal` aufruft. Nur die Zentrale nimmt Verbindungen an, deshalb liegt der Schlüssel nur dort.
ConfigMap `obs-ca`	alle drei	Alloy, Grafana und die anderen Clients prüfen damit das Zertifikat der Zentrale. Ohne CA bricht jede Verbindung mit `x509: certificate signed by unknown authority` ab.

MAC Schritt 33

```
kubectl --context obs -n monitoring create secret tls obs-tls \
  --cert=observability/tls/obs.internal.crt \
  --key=observability/tls/obs.internal.key
```

Legt das Secret `obs-tls` vom Typ `kubernetes.io/tls` an. Dieser Typ hat genau zwei Schlüssel, `tls.crt` und `tls.key`, und Traefik weiß, wie er ihn liest. Die Ingress-Objekte der Zentrale verweisen ab Kapitel 5 darauf.

MAC Schritt 34

```
kubectl --context obs  -n monitoring create configmap obs-ca \
  --from-file=ca.crt=observability/tls/ca.crt
kubectl --context app1 -n monitoring create configmap obs-ca \
  --from-file=ca.crt=observability/tls/ca.crt
kubectl --context app2 -n monitoring create configmap obs-ca \
  --from-file=ca.crt=observability/tls/ca.crt
```

Verteilt das öffentliche CA-Zertifikat als ConfigMap `obs-ca`. `--from-file=ca.crt=...` legt fest, dass der Schlüssel in der ConfigMap `ca.crt` heißt. Bindet Alloy die ConfigMap später unter `/etc/obs-ca` ein, entsteht daraus die Datei `/etc/obs-ca/ca.crt`. Es ist kein Geheimnis, deshalb genügt eine ConfigMap.

MAC Schritt 35

```
kubectl --context obs  -n monitoring get secret obs-tls
kubectl --context obs  -n monitoring get configmap obs-ca
kubectl --context app1 -n monitoring get configmap obs-ca
kubectl --context app2 -n monitoring get configmap obs-ca
```

Kontrolliert, dass alles am richtigen Ort liegt.

DAS SOLLTEST DU SEHEN

`obs-tls` hat den Typ `kubernetes.io/tls` und `DATA 2`, also Zertifikat und Schlüssel. Die ConfigMap `obs-ca` steht in allen drei Clustern mit `DATA 1`:

```
NAME      TYPE                DATA   AGE
obs-tls   kubernetes.io/tls   2      5s
NAME     DATA   AGE
obs-ca   1      4s
```

Ob Traefik das Zertifikat wirklich ausliefert, kannst du erst testen, wenn in Kapitel 5 der erste Ingress steht. Der erste `curl` dort mit `--cacert` ist genau dieser Test.

Auf dem Mac nutzen alle Befehle im Buch die CA direkt, etwa mit `curl --cacert observability/tls/ca.crt`. Möchtest du Grafana später ohne Warnung im Browser öffnen, kannst du der CA zusätzlich im Schlüsselbund vertrauen:

MAC Schritt 36

```
sudo security add-trusted-cert -d -r trustRoot \
  -k /Library/Keychains/System.keychain observability/tls/ca.crt
```

Optional. Damit vertraut macOS allen Zertifikaten dieser CA. Hast du die CA schon einmal erzeugt, lösche den alten Eintrag „obs-lab CA“ vorher in der Schlüsselbundverwaltung unter „System“. Sonst zeigt der Browser trotzdem eine Warnung, weil er das alte Zertifikat findet.

STOLPERFALLE

Läuft das Zertifikat ab, brechen alle Verbindungen der App-Cluster gleichzeitig ab. 825 Tage sind lang, aber nicht ewig. In Kapitel 13 baust du einen Alarm, der dich 30 Tage vorher warnt.

PROBIER ES SELBST

Ein Kollege fragt, warum `app1` die CA braucht, aber kein Zertifikat. Erkläre es ihm in zwei Sätzen. Überlege dann: Was müsstest du tun, wenn ein vierter Cluster `app3` dazukommt?

WAS DU GELERNT HAST

- Die k3s-Konfiguration muss vor der Installation auf der VM liegen, und `INSTALL_K3S_VERSION` hält alle Cluster auf derselben Version.
- Umbenannte Kubeconfigs lassen sich mit `KUBECONFIG` und `--flatten` gefahrlos zusammenführen.
- `--context` in jedem Befehl schützt vor dem falschen Cluster.
- Die ConfigMap `coredns-custom` mit einem `.server`-Schlüssel ergänzt CoreDNS um eine eigene Zone.
- Eine eigene CA mit Wildcard-Zertifikat reicht für TLS im Labor. Die Clients brauchen nur das CA-Zertifikat.

KAPITEL 3

S3 im Cluster mit SeaweedFS

Auf der Zentrale läuft ein S3-Speicher mit vier Buckets. Jede Komponente hat ein eigenes Konto, das nur ihren Bucket sieht.

Dauer: 40 Minuten **Voraussetzung:** Kapitel 2

3.1 Warum S3 schon im Labor

Thanos, Loki und Tempo halten nur die jüngsten Daten lokal. Alles Ältere schreiben sie in einen Objektspeicher mit S3-Schnittstelle. Das ist der Grund, warum die Zentrale Wochen und Monate aufbewahren kann, ohne dass ihre Disk wächst wie die Datenmenge.

Im Labor übernimmt das SeaweedFS. Es läuft als ein einziger Pod, spricht S3 und braucht wenig Speicher. In Kapitel 11 ziehst du auf echtes S3 bei AWS um. Weil alle Komponenten nur die S3-Schnittstelle kennen, ändern sich dabei nur Endpunkt und Zugangsdaten.

Bucket	Konto	Secret im Namespace `monitoring`
`thanos`	`thanos`	`thanos-objstore` (Schlüssel `objstore.yml`)
`loki`	`loki`	`loki-s3` (`AWS_ACCESS_KEY_ID`, `AWS_SECRET_ACCESS_KEY`)
`tempo`	`tempo`	`tempo-s3` (`AWS_ACCESS_KEY_ID`, `AWS_SECRET_ACCESS_KEY`)
`backup`	`velero`	`velero-s3` (Schlüssel `cloud`, AWS-credentials-Format)

Dazu kommt das Secret `s3-config`. Es enthält alle vier Konten und ist die Konfiguration von SeaweedFS selbst.

WARUM DAS WICHTIG IST

Ein Konto pro Komponente klingt nach Mehraufwand. Es zahlt sich aus, sobald etwas schiefgeht. Löscht ein falsch konfigurierter Compactor Daten, trifft es nur den Bucket `thanos`. Und gelangt ein Schlüssel in ein Log, tauschst du genau diesen einen aus.

3.2 Die Zugangsdaten erzeugen

Das Skript `observability/seaweedfs/create-secrets.sh` erzeugt alle Schlüssel mit `openssl rand` und legt sie nur als Kubernetes-Secrets ab. Sie stehen in keiner Datei und erscheinen nie auf dem Bildschirm.

DATEI `observability/seaweedfs/create-secrets.sh (Ausschnitt)`

```
# Konto:Bucket
ACCOUNTS="thanos:thanos loki:loki tempo:tempo velero:backup"
...
  NEW=$(printf '%s' "$NEW" | N="$name" B="$bucket" AK="$AK" SK="$SK" jq \
    '.identities += [{
       name: env.N,
       credentials: [{accessKey: env.AK, secretKey: env.SK}],
       actions: ["Read:\(env.B)", "Write:\(env.B)", "List:\(env.B)", "Tagging:\(env.B)"]
     }]')
```

Für jedes Paar aus Konto und Bucket hängt `jq` ein Konto an die Liste an. Die Rechte in `actions` gelten nur für den genannten Bucket. Die Schlüssel übergibt das Skript über Umgebungsvariablen statt als Argumente, damit sie nicht in der Prozessliste auftauchen. Gibt es `s3-config` schon, übernimmt das Skript die vorhandenen Schlüssel. Ein zweiter Lauf ändert also nichts und ergänzt höchstens fehlende Secrets.

MAC — Schritt 1

```
./observability/seaweedfs/create-secrets.sh
```

Arbeitet standardmäßig mit dem Kontext `obs`. Für einen anderen Kontext setzt du davor `KUBE_CONTEXT=<name>`.

DAS SOLLTEST DU SEHEN

Vier Zeilen der Form `Konto thanos: neu erzeugt (Bucket thanos)`, danach `Secrets s3-config, thanos-objstore, loki-s3, tempo-s3 und velero-s3 in monitoring (Kontext obs) angelegt.`

Bei einem zweiten Lauf steht statt „neu erzeugt“ jeweils „vorhanden, Schlüssel bleiben“.

MAC — Schritt 2

```
kubectl --context obs -n monitoring get secret \
  s3-config thanos-objstore loki-s3 tempo-s3 velero-s3
```

Listet die fünf Secrets, ohne ihren Inhalt zu zeigen.

MAC — Schritt 3

```
kubectl --context obs -n monitoring get secret s3-config \
  -o jsonpath='{.data.s3\.json}' | base64 -d \
  | jq -c '.identities[] | {name, actions}'
```

Zeigt die Konten und ihre Rechte, aber nicht die Schlüssel. Der Backslash in `s3\.json` ist nötig, weil der Punkt sonst als Pfadtrenner gilt.

DAS SOLLTEST DU SEHEN

Vier Zeilen, zum Beispiel `{"name":"velero","actions":["Read:backup","Write:backup","List:backup","Tagging:backup"]}`. Kein Konto nennt einen fremden Bucket.

3.3 SeaweedFS starten

Das Manifest enthält ein PersistentVolumeClaim, ein Deployment und einen Service. Drei Stellen darin verdienen einen Blick:

DATEI `observability/seaweedfs/seaweedfs.yaml (Ausschnitt)`

```
spec:
  strategy:
    type: Recreate
  template:
    spec:
      enableServiceLinks: false
      containers:
        - name: s3
          image: chrislusf/seaweedfs:4.52
          args:
            - server
            - -dir=/data
            - -s3
            - -s3.config=/etc/seaweedfs/s3.json
            - -master.volumeSizeLimitMB=256
            - -volume.max=120
            - -volume.minFreeSpace=2GiB
```

`weed server` startet Master, Volume-Server, Filer und S3-Gateway in einem Prozess. Die Konten liest es aus `s3.json`, das aus dem Secret `s3-config` stammt. `Recreate` sorgt dafür, dass nie zwei Pods gleichzeitig auf dasselbe Datenverzeichnis zugreifen.

STOLPERFALLE

Der Service-Name `s3`. Kubernetes setzt für jeden Service im Namespace Umgebungsvariablen, hier also `S3_PORT=tcp://10.43.x.x:8333`. SeaweedFS liest Flags auch aus der Umgebung, hält das für `-s3.port` und stürzt beim Start ab. Deshalb steht im Manifest `enableServiceLinks: false`. Fehlt die Zeile, siehst du einen `CrashLoopBackOff` mit einem Fehler beim Parsen des Ports.

STOLPERFALLE

Die Größe. `local-path` setzt die Größe des PersistentVolumeClaims nicht durch. Das Volume ist einfach ein Verzeichnis auf der Node-Disk. Das echte Limit setzen die weed-Flags: 120 Volumes zu je 256 MB, also rund 30 GB. Jeder Bucket belegt beim ersten Schreiben gleich mehrere Volumes. Ein zu kleines `-volume.max` führt deshalb schnell zu Schreibfehlern, obwohl die Disk noch leer ist.

MAC Schritt 4

```
kubectl --context obs apply -f observability/seaweedfs/seaweedfs.yaml
```

Legt PVC, Deployment und Service an. Das Secret `s3-config` muss schon existieren, sonst bleibt der Pod in `ContainerCreating` stehen.

MAC Schritt 5

```
kubectl --context obs -n monitoring rollout status deploy/s3
```

Wartet, bis der Pod bereit ist. `apply` kehrt sofort zurück, dieser Befehl erst, wenn die Readiness-Probe auf Port 8333 antwortet.

MAC Schritt 6

```
kubectl --context obs -n monitoring get pod -l app=s3
kubectl --context obs -n monitoring get pvc s3-data
```

Zeigt Pod und Volume.

DAS SOLLTEST DU SEHEN

Der Pod `s3-...` steht auf `Running` und `1/1`, `RESTARTS` ist `0`. Das PVC `s3-data` ist `Bound` mit der StorageClass `local-path`.

3.4 Die Buckets anlegen

Buckets legst du mit der Verwaltungsshell `weed shell` im Pod an. Das Skript `observability/seaweedfs/create-buckets.sh` prüft zuerst, welche Buckets es schon gibt, und legt nur die fehlenden an.

MAC Schritt 7

```
./observability/seaweedfs/create-buckets.sh
```

Führt im Pod `s3.bucket.list` und für jeden fehlenden Bucket `s3.bucket.create` aus.

DAS SOLLTEST DU SEHEN

Vier Zeilen: `Bucket thanos: angelegt`, ebenso für `loki`, `tempo` und `backup`. Bei einem zweiten Lauf steht dort jeweils `vorhanden`.

MAC Schritt 8

```
kubectl --context obs -n monitoring exec deploy/s3 -- \
  sh -c "echo 's3.bucket.list' | weed shell -master=localhost:9333"
```

Fragt SeaweedFS direkt nach seinen Buckets. So siehst du, was das Skript getan hat.

3.5 Die Rechte prüfen

Ein Konto, das zu viel darf, fällt im normalen Betrieb nicht auf. Prüfe deshalb jetzt, dass das Konto `loki` seinen Bucket lesen kann und den von Thanos nicht.

Der S3-Port ist nur im Cluster erreichbar. Für den Test leitest du ihn kurz auf den Mac um. Öffne dafür ein zweites Terminal:

MAC Schritt 9

```
kubectl --context obs -n monitoring port-forward svc/s3 8333:8333
```

Macht den Service unter `localhost:8333` erreichbar, bis du ihn mit `Ctrl-C` beendest.

Zurück im ersten Terminal liest du die Schlüssel des Kontos `loki` in Shell-Variablen. Sie erscheinen dabei nicht auf dem Bildschirm.

MAC Schritt 10

```
AK=$(kubectl --context obs -n monitoring get secret loki-s3 \
  -o jsonpath='{.data.AWS_ACCESS_KEY_ID}' | base64 -d)
SK=$(kubectl --context obs -n monitoring get secret loki-s3 \
  -o jsonpath='{.data.AWS_SECRET_ACCESS_KEY}' | base64 -d)
```

Kubernetes speichert Secret-Werte Base64-kodiert, `base64 -d` macht sie wieder lesbar.

MAC Schritt 11

```
printf 'user = "%s:%s"\n' "$AK" "$SK" | curl -s -K - \
  --aws-sigv4 "aws:amz:us-east-1:s3" http://localhost:8333/loki/
```

`curl` kann S3-Anfragen selbst signieren. `-K -` liest die Zugangsdaten aus der Standardeingabe, damit sie nicht als Argument in der Prozessliste stehen. Die Anfrage listet den Inhalt des Buckets `loki`.

DAS SOLLTEST DU SEHEN

Eine kurze XML-Antwort mit `<ListBucketResult>` und `<Name>loki</Name>`. Der Bucket ist noch leer, deshalb folgen keine `<Contents>`-Einträge.

MAC Schritt 12

```
printf 'user = "%s:%s"\n' "$AK" "$SK" | curl -s -K - \
  --aws-sigv4 "aws:amz:us-east-1:s3" http://localhost:8333/thanos/
```

Dieselbe Anfrage, nur an den fremden Bucket `thanos`.

DAS SOLLTEST DU SEHEN

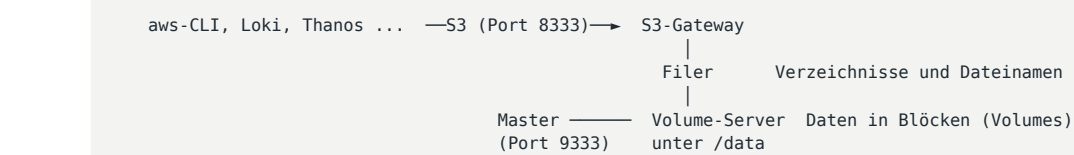
Eine XML-Fehlermeldung mit `<Code>AccessDenied</Code>`. Genau so soll es sein: Das Konto `loki` sieht nur seinen eigenen Bucket.

Lass den Port-Forward und die Variablen `AK` und `SK` noch stehen, im nächsten Abschnitt brauchst du sie wieder.

3.6 SeaweedFS im Alltag: Dateien mit der aws-CLI

Bisher hast du S3 nur mit `curl` angefragt. Für die tägliche Arbeit ist die `aws`-CLI bequemer. Sie spricht mit jedem S3-kompatiblen Speicher, nicht nur mit AWS. Du sagst ihr nur, wo der Speicher steht. Was du hier lernst, funktioniert genauso mit MinIO, Ceph, Hetzner oder echtem AWS S3, und du kannst SeaweedFS auch für eigene Projekte nutzen.

DATEI Wie SeaweedFS aufgebaut ist



`weed server` startet alle vier Teile in einem Pod. Clients sehen nur das S3-Gateway. Der Filer übersetzt Bucket und Objektname in Dateien, der Volume-Server speichert die Daten, und der Master behält den Überblick. Mit dem Master sprichst du über `weed shell`, zum Beispiel beim Anlegen von Buckets.

Die `aws`-CLI liest Zugangsdaten und Ziel aus Umgebungsvariablen. So landet nichts in einer Datei:

MAC Schritt 13

```
export AWS_ACCESS_KEY_ID="$AK" AWS_SECRET_ACCESS_KEY="$SK"
export AWS_DEFAULT_REGION=us-east-1
export AWS_ENDPOINT_URL=http://localhost:8333
```

`AWS_ENDPOINT_URL` leitet alle Befehle auf SeaweedFS statt auf AWS um. Die CLI kennt diese Variable ab Version 2.13, `aws --version` zeigt deine. Die Region verlangt die CLI, SeaweedFS ignoriert sie.

MAC Schritt 14

```
aws s3 ls
```

Listet alle Buckets, die das Konto sehen darf.

DAS SOLLTEST DU SEHEN

Eine einzige Zeile, zum Beispiel `2026-11-11 11:49:14 loki`. Die Buckets `thanos`, `tempo` und `backup` gibt es zwar, das Konto `loki` sieht sie aber nicht.

Jetzt lädst du eine Datei hoch. Ein „Ordner“ in S3 ist nur ein Teil des Objektnamens. `test/hallo.txt` ist ein einziges Objekt mit einem Schrägstrich im Namen.

MAC Schritt 15

```
echo "Hallo SeaweedFS" > hallo.txt
aws s3 cp hallo.txt s3://loki/test/hallo.txt
```

`cp` funktioniert wie das normale `cp`, nur dass eine Seite eine S3-Adresse der Form `s3://bucket/name` ist.

MAC Schritt 16

```
aws s3 ls s3://loki/
aws s3 ls s3://loki/ --recursive --human-readable
```

Die erste Zeile zeigt nur die oberste Ebene, „Ordner“ erscheinen als `PRE` (Präfix). Mit `--recursive` siehst du alle Objekte mit vollem Namen und Größe.

DAS SOLLTEST DU SEHEN

```
                           PRE test/
2026-11-11 11:49:35   16 Bytes test/hallo.txt
```

MAC Schritt 17

```
aws s3 cp s3://loki/test/hallo.txt ./zurueck.txt
cat zurueck.txt
```

Lädt das Objekt wieder herunter. Die Richtung ergibt sich aus der Reihenfolge: Quelle zuerst, Ziel danach.

Für ganze Verzeichnisse gibt es `sync`. Es überträgt nur, was sich geändert hat, und eignet sich deshalb für einfache Backups:

MAC Schritt 18

```
mkdir -p ordner && echo a > ordner/a.txt && echo b > ordner/b.txt
aws s3 sync ordner s3://loki/test/ordner/
aws s3 ls s3://loki/test/ --recursive
```

Ein zweiter Lauf von `sync` überträgt nichts mehr, solange sich lokal nichts ändert. In der Gegenrichtung, `aws s3 sync s3://loki/test/ordner/ ordner`, holst du alles zurück.

Das Konto `loki` darf nur in seinen eigenen Bucket schreiben. Probiere es mit einem fremden aus:

MAC Schritt 19

```
aws s3 cp hallo.txt s3://thanos/hallo.txt
```

Versucht, in den Bucket von Thanos zu schreiben.

DAS SOLLTEST DU SEHEN
`upload failed: ... An error occurred (AccessDenied) when calling the PutObject operation: Access Denied.`

Zum Schluss räumst du auf. Der Bucket `loki` gehört ab Kapitel 8 Loki, Testdateien haben dort nichts verloren.

MAC Schritt 20

```
aws s3 rm s3://loki/test/ --recursive
rm -r hallo.txt zurueck.txt ordner
unset AK SK AWS_ACCESS_KEY_ID AWS_SECRET_ACCESS_KEY AWS_ENDPOINT_URL
```

Löscht alle Objekte unter `test/`, die lokalen Testdateien und die Schlüssel aus deiner Shell. Beende danach den Port-Forward im zweiten Terminal mit `Ctrl-C`.

Befehl	Wozu
`aws s3 ls`	Buckets auflisten
`aws s3 ls s3://b/ --recursive`	Alle Objekte eines Buckets
`aws s3 cp datei s3://b/name`	Hochladen
`aws s3 cp s3://b/name datei`	Herunterladen
`aws s3 sync dir s3://b/pfad/`	Verzeichnis abgleichen, nur Änderungen
`aws s3 rm s3://b/name`	Löschen, mit `--recursive` alles unter einem Präfix
`aws s3 presign s3://b/name`	Zeitlich begrenzten Download-Link erzeugen

SeaweedFS für ein eigenes Projekt

Möchtest du SeaweedFS für eine eigene Anwendung nutzen, bekommt sie wie alle anderen ein eigenes Konto mit eigenem Bucket. Beide Skripte lesen ihre Listen aus einer Zeile am Anfang:

DATEI Änderung in `create-secrets.sh` und `create-buckets.sh`

```
# create-secrets.sh: Konto:Bucket
ACCOUNTS="thanos:thanos loki:loki tempo:tempo velero:backup meinprojekt:meinprojekt"
# create-buckets.sh
BUCKETS="thanos loki tempo backup meinprojekt"
```

Danach führst du `create-secrets.sh` aus, startest SeaweedFS mit `kubectl --context obs -n monitoring rollout restart deploy/s3` neu, damit es die neuen Konten liest, und führst `create-buckets.sh` aus. Vorhandene Schlüssel bleiben dabei erhalten. Für das neue Konto legt das Skript kein eigenes Secret an. Die Schlüssel stehen in `s3-config`:

MAC Schritt 21

```
kubectl --context obs -n monitoring get secret s3-config \
  -o jsonpath='{.data.s3\.json}' | base64 -d \
  | jq -r '.identities[] | select(.name == "meinprojekt") | .credentials[0]'
```

Zeigt Access Key und Secret Key des Kontos `meinprojekt`. Diesmal erscheinen die Schlüssel auf dem Bildschirm. Gib sie deiner Anwendung als Secret, nicht als Klartext in einer Datei.

Eine Anwendung im Cluster `obs` erreicht SeaweedFS unter `http://s3.monitoring.svc.cluster.local:8333`. Für Anwendungen in anderen Clustern machst du den Speicher in Kapitel 12 unter `https://s3.obs.internal` erreichbar.

STOLPERFALLE

Löschst du den SeaweedFS-Pod von Hand, startet der neue manchmal, bevor der alte die Sperre auf seine LevelDB-Dateien freigegeben hat. Der neue Pod startet dann ein- oder zweimal neu und läuft danach normal. Das ist harmlos. Für geplante Neustarts nimm `kubectl --context obs -n monitoring rollout restart deploy/s3`, dank `Recreate` wartet Kubernetes dann sauber. Direkt nach einem Neustart meldet `weed shell` ein paar Sekunden lang `raft.Server: Not current leader`. Warte kurz und wiederhole den Befehl.

3.7 Was die anderen Komponenten wissen müssen

Ab Kapitel 4 greifen Thanos, Loki, Tempo und Velero auf diesen Speicher zu. Alle nutzen dieselben Grundwerte:

Einstellung	Wert
Endpunkt	`http://s3.monitoring.svc.cluster.local:8333`
Region	`us-east-1` (SeaweedFS ignoriert sie, die Clients verlangen aber eine)
Adressierung	Pfad-Stil (`/bucket/objekt`), nicht `bucket.host`

Einstellung	Wert
TLS	aus, der Verkehr bleibt im Cluster

WARUM DAS WICHTIG IST

Der Pfad-Stil ist wichtig. Viele S3-Clients setzen den Bucket-Namen standardmäßig vor den Hostnamen, also `thanos.s3.monitoring.svc...`. Diesen Namen gibt es im Cluster-DNS nicht. Deshalb steht in `thanos-objstore` die Zeile `bucket_lookup_type: path`, und Loki und Tempo bekommen später die entsprechende Option.

PROBIER ES SELBST

Wiederhole den Test mit den Schlüsseln aus `velero-s3`. Dort liegen sie nicht als einzelne Werte, sondern in einer Datei im AWS-Format unter dem Schlüssel `cloud`. Wie kommst du mit `jsonpath`, `base64` und `awk` an die beiden Werte? Welcher Bucket muss antworten, welcher nicht?

WAS DU GELERNT HAST

- Thanos, Loki und Tempo lagern ältere Daten in S3 aus. Im Labor spielt SeaweedFS diese Rolle.
- Ein Konto pro Komponente begrenzt den Schaden bei Fehlern und bei verlorenen Schlüsseln.
- Zugangsdaten entstehen mit `openssl rand` und leben nur in Kubernetes-Secrets.
- `enableServiceLinks: false` verhindert, dass Kubernetes-Umgebungsvariablen SeaweedFS verwirren.
- Mit `curl --aws-sigv4` prüfst du S3-Rechte ohne zusätzliche Werkzeuge.
- Die `aws`-CLI arbeitet mit `AWS_ENDPOINT_URL` gegen jeden S3-Speicher: `ls`, `cp`, `sync` und `rm` genügen für den Alltag.

KAPITEL 4

Die Zentrale: Prometheus und Thanos Receive

Auf obs läuft Prometheus, schreibt seine Metriken an Thanos Receive, und Thanos legt sie dauerhaft in S3 ab. Thanos Query zeigt alles an einer Stelle.

Dauer: 75 Minuten, dazu zwei Stunden Wartezeit für den ersten Block in S3 **Voraussetzung:** Kapitel 3 (SeaweedFS mit Bucket `thanos` und Secret `thanos-objstore`)

4.1 Was du aufbaust

In diesem Kapitel entsteht der Kern der Metrik-Zentrale. Noch schreibt nur obs selbst hinein. Die App-Cluster folgen in Kapitel 5 über genau denselben Weg.

Komponente	Aufgabe
Prometheus	Sammelt die Metriken von obs ein, wertet die Standardregeln aus und schickt alles per Remote Write weiter
Alertmanager	Nimmt Alarme an, die Konfiguration folgt in Kapitel 7
Thanos Receive	Nimmt Remote Write an, trennt nach Mandant, hält 6 Stunden lokal und lädt Blöcke nach S3
Thanos Store	Macht die Blöcke in S3 abfragbar
Thanos Query	Fragt Receive, Store und Ruler gleichzeitig ab und entfernt Duplikate
Thanos Compact	Fasst Blöcke zusammen, rechnet sie herunter und löscht nach Ablauf
Thanos Ruler	Wertet Regeln über alle Cluster aus, die Regeln selbst kommen in Kapitel 7
Grafana	Startet mit Standardwerten, wird in Kapitel 6 eingerichtet

Der Weg einer Messung sieht so aus: Prometheus holt sie ab und schickt sie sofort an Receive. Receive hält sie im Speicher und auf seiner Platte. Alle zwei Stunden schreibt Receive einen fertigen Block in den Bucket `thanos`. Ab dann liefert Thanos Store die Daten aus S3, und Receive löscht seine Kopie nach sechs Stunden. Thanos Query sieht beide Quellen und setzt sie für jede Abfrage nahtlos zusammen.

WARUM DAS WICHTIG IST

Das Security-Buch nutzt den Thanos Sidecar. Er sitzt neben Prometheus und lädt dessen Blöcke hoch. Das passt, solange es nur einen Prometheus gibt. Hier schicken drei Cluster ihre Daten in die Zentrale, und die App-Cluster haben gar keinen eigenen Prometheus. Receive ist der eine Empfänger für alle, und er kennt Mandanten. Mehr Hintergrund im Buch Kubernetes Observability Hands-On.

4.2 Die Werte für kube-prometheus-stack

Die Werte liegen in `observability/central/kps/values.yaml`. Kapitel 6 und 7 ergänzen später eigene Dateien, diese hier bleibt die Grundlage. Der erste Teil beschreibt Prometheus:

DATEI `observability/central/kps/values.yaml (Ausschnitt)`

```yaml
prometheus:
  prometheusSpec:
    retention: 6h
    scrapeInterval: 30s
    evaluationInterval: 30s
    externalLabels:
      cluster: obs
    serviceMonitorSelectorNilUsesHelmValues: false
    podMonitorSelectorNilUsesHelmValues: false
    probeSelectorNilUsesHelmValues: false
    scrapeConfigSelectorNilUsesHelmValues: false
    ruleSelector:
      matchLabels:
        auswertung: lokal
    ruleNamespaceSelector: {}
    remoteWrite:
      - name: thanos-receive
        url: http://thanos-receive.monitoring.svc:19291/api/v1/receive
        headers:
          THANOS-TENANT: obs
        sendExemplars: true
    enableFeatures:
      - exemplar-storage
```

`retention: 6h` reicht, weil Prometheus hier nur noch Zwischenstation ist. `externalLabels` hängt an jede Serie `cluster="obs"`, genau wie Alloy es in Kapitel 5 für die App-Cluster tut. Der Header `THANOS-TENANT` macht obs in Receive zum Mandanten `obs`. Die vier `...NilUsesHelmValues: false` lassen Prometheus auch fremde ServiceMonitors einsammeln, etwa die von Thanos. `sendExemplars` und `exemplar-storage` bereiten den Sprung von einer Metrik zum passenden Trace in Kapitel 9 vor.

Im Rest der Datei stehen die Regel-Auswahl, der Thanos Ruler und die k3s-Besonderheiten:

DATEI `observability/central/kps/values.yaml (Ausschnitt)`

```yaml
kubeControllerManager:
  enabled: false
```

```yaml
kubeScheduler:
  enabled: false
kubeProxy:
  enabled: false
kubeEtcd:
  enabled: true
  endpoints:
    - 172.16.0.126
  service:
    port: 2381
    targetPort: 2381

defaultRules:
  create: true
  labels:
    auswertung: lokal

thanosRuler:
  enabled: true
  thanosRulerSpec:
    ruleSelector:
      matchLabels:
        auswertung: global
    ruleNamespaceSelector: {}
    queryEndpoints:
      - thanos-query.monitoring.svc.cluster.local:10902
    evaluationInterval: 30s
```

Controller-Manager, Scheduler und kube-proxy laufen bei k3s im k3s-Prozess und lauschen nur auf `127.0.0.1`. Prometheus erreicht sie nicht, sie würden dauerhaft als `down` erscheinen. etcd dagegen bietet seine Metriken dank `etcd-expose-metrics` aus Kapitel 2 auf der Node-IP an.

Die Regel-Auswahl verdient einen genaueren Blick. Ein Label-Selektor kennt kein „oder“. Deshalb bekommen die Standardregeln des Charts mit `defaultRules.labels` dasselbe Label wie deine lokalen Regeln.

Label am PrometheusRule	Wertet aus	Sieht
`auswertung: lokal`	Prometheus auf obs	Nur die Daten von obs, dafür ohne Umweg
`auswertung: global`	Thanos Ruler	Alle Cluster, über Thanos Query
kein Label	niemand	Die Regel bleibt wirkungslos

STOLPERFALLE

Die Standardregeln laufen in Prometheus auf obs. Ihre Ergebnisse, etwa die Aufnahmeregeln für CPU pro Pod, gibt es deshalb zunächst nur für `cluster="obs"`. Wie du solche Regeln für alle Cluster auswertest, zeigen die Kapitel 7 und 10.

Der Thanos Ruler fragt nicht Prometheus, sondern Thanos Query. So sieht er die Daten aller Cluster. Er gehört zum Helm-Release und läuft ab jetzt mit, Regeln und den Weg zum Alertmanager bekommt er in Kapitel 7.

4.3 kube-prometheus-stack installieren

MAC — Schritt 1

```
helm repo add prometheus-community \
  https://prometheus-community.github.io/helm-charts
helm repo update prometheus-community
```

Trägt das Chart-Repository ein und holt die aktuelle Versionsliste. Ist es schon eingetragen, meldet Helm das nur.

MAC — Schritt 2

```
helm --kube-context obs upgrade --install kps \
  prometheus-community/kube-prometheus-stack \
  -n monitoring --version 92.18.0 \
  -f observability/central/kps/values.yaml \
  --wait --timeout 10m
```

Installiert Operator, Prometheus, Alertmanager, Thanos Ruler, Grafana, kube-state-metrics und node-exporter auf obs. `--wait` kehrt erst zurück, wenn alle Pods bereit sind.

MAC — Schritt 3

```
kubectl --context obs -n monitoring get pods
```

Zeigt alle Pods im Namespace.

DAS SOLLTEST DU SEHEN

Neben `s3` laufen unter anderem `prometheus-kps-kube-prometheus-stack-prometheus-0` (2/2), `alertmanager-kps-kube-prometheus-stack-alertmanager-0`, `thanos-ruler-kps-kube-prometheus-stack-thanos-ruler-0`, `kps-grafana-...`, `kps-kube-state-metrics-...` und `kps-prometheus-node-exporter-...`. Alle stehen auf `Running`.

Prometheus versucht schon jetzt, an Receive zu schreiben. Den Empfänger gibt es noch nicht.

MAC — Schritt 4

```
kubectl --context obs -n monitoring logs \
  prometheus-kps-kube-prometheus-stack-prometheus-0 \
  -c prometheus --tail=50 | grep -i remote
```

Filtert die Meldungen zum Remote Write heraus.

DAS SOLLTEST DU SEHEN

Eine Meldung wie `level=WARN ... msg="Failed to send batch, retrying" component=remote ... no such host` für `thanos-receive.monitoring.svc`. Das ist im Moment richtig. Prometheus puffert die Daten in seinem Write-Ahead-Log und schickt sie nach, sobald Receive da ist.

4.4 Thanos Receive

Receive läuft als StatefulSet mit einer Replika. Es nimmt Daten an, verteilt sie über einen Hashring und speichert sie. In großen Installationen sind das zwei getrennte Rollen, *Router* und *Ingestor*. Hier erledigt ein Pod beides.

DATEI observability/central/thanos/receive.yaml (Ausschnitt)

```
args:
  - receive
  - --grpc-address=0.0.0.0:10901
  - --http-address=0.0.0.0:10902
  - --remote-write.address=0.0.0.0:19291
  - --tsdb.path=/data
  - --tsdb.retention=6h
  - --tsdb.max-exemplars=100000
  - --objstore.config-file=/etc/thanos/objstore.yml
  - --receive.tenant-header=THANOS-TENANT
  - --receive.tenant-label-name=tenant_id
  - --receive.default-tenant-id=unbekannt
  - --receive.hashrings-file=/etc/thanos-hashring/hashrings.json
  - --receive.local-endpoint=thanos-receive-0.thanos-receive.monitoring.svc.cluster.local:10901
  - --receive.replication-factor=1
  - --label=receive_replica="0"
```

Port `19291` nimmt Remote Write an, `10901` beantwortet Abfragen von Thanos Query per gRPC. Jeder Mandant bekommt eine eigene TSDB und das Label `tenant_id`. Wer ohne Header schreibt, landet bei `unbekannt`, so fällt ein falsch eingerichteter Sender sofort auf. `--tsdb.max-exemplars` hebt Exemplars für Kapitel 9 auf.

Der Hashring ist eine kleine JSON-Datei in einer ConfigMap derselben Datei. Er enthält genau einen Eintrag: den Pod selbst. Weil dieser Eintrag exakt mit `--receive.local-endpoint` übereinstimmt, weiß Receive, dass es die Daten selbst speichern soll, statt sie weiterzuleiten. Der Name stammt vom headless Service `thanos-receive`, der jedem Pod eines StatefulSets einen festen DNS-Namen gibt.

WARUM DAS WICHTIG IST

Ohne Hashring würde Receive ebenfalls laufen. Mit Hashring ist die Rolle aber ausdrücklich beschrieben, und der Weg zu mehr Replikas steht offen: weitere Einträge in die Liste, `--receive.replication-factor=3`, fertig. Das `receive_replica`-Label unterscheidet dann die Kopien, und Thanos Query fasst sie wieder zusammen.

MAC Schritt 5

```
kubectl --context obs -n monitoring get secret thanos-objstore
```

Prüft, ob die S3-Zugangsdaten aus Kapitel 3 da sind. Receive startet ohne sie nicht.

MAC Schritt 6

```
kubectl --context obs apply -f observability/central/thanos/receive.yaml
```

Legt Hashring-ConfigMap, StatefulSet und Service an.

MAC Schritt 7

```
kubectl --context obs -n monitoring rollout status \
  statefulset/thanos-receive --timeout=3m
```

Wartet, bis der Pod bereit ist.

MAC Schritt 8

```
kubectl --context obs -n monitoring logs \
  thanos-receive-0 --tail=30
```

Zeigt den Start von Receive.

DAS SOLLTEST DU SEHEN

Zeilen mit `level=info`, keine mit `level=error`. Gleich beim Start öffnet Receive eine leere TSDB für den Standard-Mandanten, `tenant=unbekannt msg="opening TSDB"`. Das ist normal. Nach spätestens einer Minute schreibt Prometheus, und es folgt die entscheidende Zeile:

`level=info component=receive component=multi-tsdb tenant=obs msg="TSDB is now ready"`

Fehlermeldungen zu S3 wie `Access Denied` bedeuten, dass `thanos-objstore` nicht stimmt.

So testest du Receive

Receive hat keine eigene Oberfläche. Ob er arbeitet, verraten dir zwei Endpunkte auf seinem HTTP-Port 10902. Leite ihn dafür in einem zweiten Terminal um:

MAC Schritt 9

```
kubectl --context obs -n monitoring port-forward thanos-receive-0 12902:10902
```

Port 12902 auf dem Mac, damit er nicht mit Thanos Query kollidiert, das später 10902 bekommt.

MAC Schritt 10

```
curl -s localhost:12902/-/ready; echo
curl -s localhost:12902/metrics | grep -E '^prometheus_tsdb_head_series\{'
curl -s localhost:12902/metrics | grep 'handler="receive"'
```

`/-/ready` ist dieselbe Prüfung, die Kubernetes als Readiness-Probe nutzt. `/metrics` zeigt die Messwerte von Receive selbst: wie viele Serien jeder Mandant gerade im Speicher hat und wie viele Schreibanfragen angekommen sind.

DAS SOLLTEST DU SEHEN

```
OK
prometheus_tsdb_head_series{tenant="obs"} 124765
prometheus_tsdb_head_series{tenant="unbekannt"} 0
http_requests_total{code="200",handler="receive",method="post",tenant="obs"} 406
```

Die Zahlen wachsen mit jedem Aufruf. Steht bei `obs` eine Null oder fehlt die Zeile mit `code="200"`, kommt bei Receive nichts an. Dann zeigen die Logs von Prometheus den Grund. Beende den Port-Forward danach mit `Ctrl-C`.

Damit kannst du Receive für jeden Sender nutzen, der Prometheus Remote Write spricht: Prometheus, Grafana Alloy, das OpenTelemetry Collector, Telegraf oder eigene Programme. Der Sender braucht nur die URL `/api/v1/receive` auf Port 19291 und den Header `THANOS-TENANT`.

4.5 Query, Store und Compact

Die drei übrigen Komponenten sind zustandslos. Ihre lokalen Daten lassen sich jederzeit aus S3 neu aufbauen, deshalb reicht jeweils ein `emptyDir`. Wichtig ist bei Query die Liste der Endpunkte:

DATEI `observability/central/thanos/query.yaml (Ausschnitt)`

```yaml
args:
  - query
  - --grpc-address=0.0.0.0:10901
  - --http-address=0.0.0.0:10902
  - --query.replica-label=receive_replica
  - --query.replica-label=prometheus_replica
  - --query.replica-label=thanos_ruler_replica
  - |-
    --endpoint.sd-config=endpoints:
      - address: dns+thanos-receive.monitoring.svc.cluster.local:10901
      - address: dns+thanos-store.monitoring.svc.cluster.local:10901
      - address: dnssrv+_grpc._tcp.thanos-ruler-operated.monitoring.svc.cluster.local
```

`dns+` löst einen Namen in die Pod-IPs auf, `dnssrv+` liest zusätzlich den Port aus einem SRV-Eintrag. Den Ruler bindest du ein, damit seine `ALERTS`-Serien und die Ergebnisse globaler Aufnahmeregeln in Query sichtbar sind. Die drei `replica-label` nennen die Labels, in denen sich nur Kopien derselben Daten unterscheiden.

Beim Compactor legst du fest, wie lange Daten in S3 bleiben:

DATEI `observability/central/thanos/compact.yaml (Ausschnitt)`

```yaml
args:
  - compact
  - --wait
  - --wait-interval=5m
  - --data-dir=/data
  - --objstore.config-file=/etc/thanos/objstore.yml
  - --retention.resolution-raw=30d
```

```
  - --retention.resolution-5m=90d
  - --retention.resolution-1h=365d
  - --delete-delay=1h
```

Rohdaten bleiben 30 Tage. Daraus rechnet der Compactor Mittelwerte über 5 Minuten (90 Tage) und über eine Stunde (ein Jahr). Für einen Jahresverlauf braucht niemand jeden 30-Sekunden-Wert. `--wait` hält den Prozess am Leben, statt nach einem Durchlauf zu enden.

STOLPERFALLE

Pro Bucket darf genau ein Compactor laufen. Zwei Compactors bearbeiten dieselben Blöcke gleichzeitig und beschädigen die Daten. Deshalb ist das Deployment auf eine Replika und die Strategie `Recreate` festgelegt.

MAC Schritt 11

```
kubectl --context obs apply \
  -f observability/central/thanos/store.yaml \
  -f observability/central/thanos/query.yaml \
  -f observability/central/thanos/compact.yaml
```

Legt die drei Deployments und ihre Services an.

MAC Schritt 12

```
kubectl --context obs apply \
  -f observability/central/thanos/servicemonitor.yaml
```

Ein ServiceMonitor für alle Thanos-Komponenten. Er wählt jeden Service mit dem Label `app.kubernetes.io/part-of: thanos` und liest dessen Port `http`. Das Label `job` heißt dann wie der Service, etwa `thanos-receive`.

MAC Schritt 13

```
kubectl --context obs -n monitoring get pods \
  -l app.kubernetes.io/part-of=thanos
```

Zeigt nur die Thanos-Pods.

DAS SOLLTEST DU SEHEN

Vier Pods auf `Running` und `1/1`: `thanos-receive-0`, `thanos-query-...`, `thanos-store-...` und `thanos-compact-...`. Der Thanos Ruler gehört zum Helm-Release und trägt dieses Label nicht.

4.6 Thanos Query ansehen

MAC Schritt 14

```
kubectl --context obs -n monitoring port-forward \
  svc/thanos-query 10902:10902
```

Leitet Port 10902 auf deinen Laptop um. Lass das Terminal offen und öffne http://localhost:10902 im Browser.

MAC Schritt 15

```
curl -s http://localhost:10902/api/v1/stores | jq '.data | keys'
```

Fragt in einem zweiten Terminal, welche Endpunkte Query kennt.

DAS SOLLTEST DU SEHEN

```
[
  "receive",
  "rule",
  "store"
]
```

Direkt nach dem Start fehlt `store` oft noch für etwa eine halbe Minute, bis Thanos Store den Bucket gelesen hat. Fehlt ein Typ dauerhaft, prüfe mit `kubectl --context obs -n monitoring logs deploy/thanos-query` die DNS-Auflösung der Endpunkte.

Die Seite **Endpoints** im Menü oben zeigt dasselbe wie der `curl`-Befehl, nur ausführlicher: welche Quellen Thanos Query kennt, ob sie erreichbar sind (`UP`) und welche Mandanten sie anbieten.

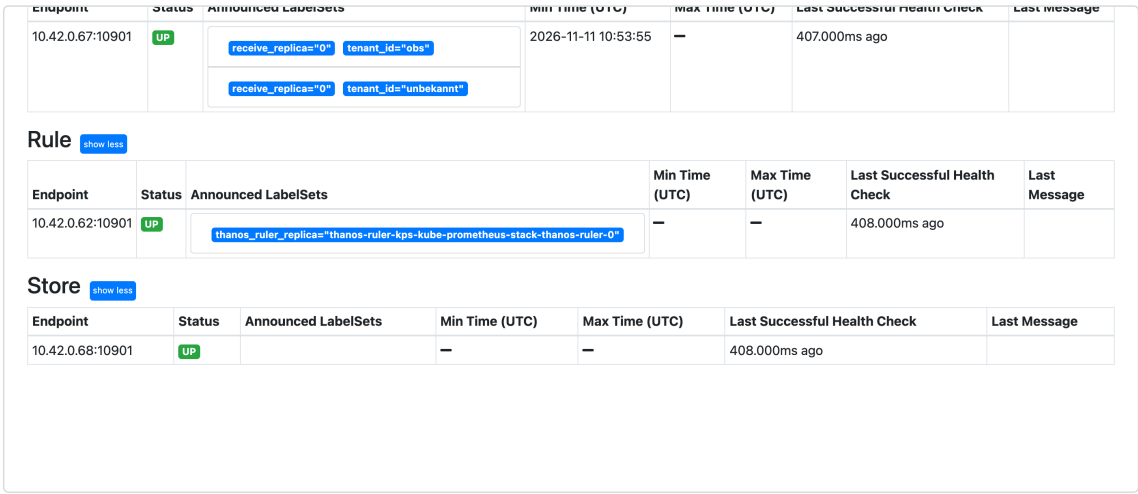


Abbildung 1: Thanos Query, Seite „Endpoints“: Receive bietet die Mandanten `obs` und `unbekannt` an, dazu kommen Ruler und Store. Store zeigt noch keine Labels, weil in S3 noch kein Block liegt.

Wechsle zurück auf **Graph**. Klicke in das Eingabefeld, gib diese Abfrage ein und klicke auf **Execute**:

```
count by (tenant_id, cluster) (up)
```

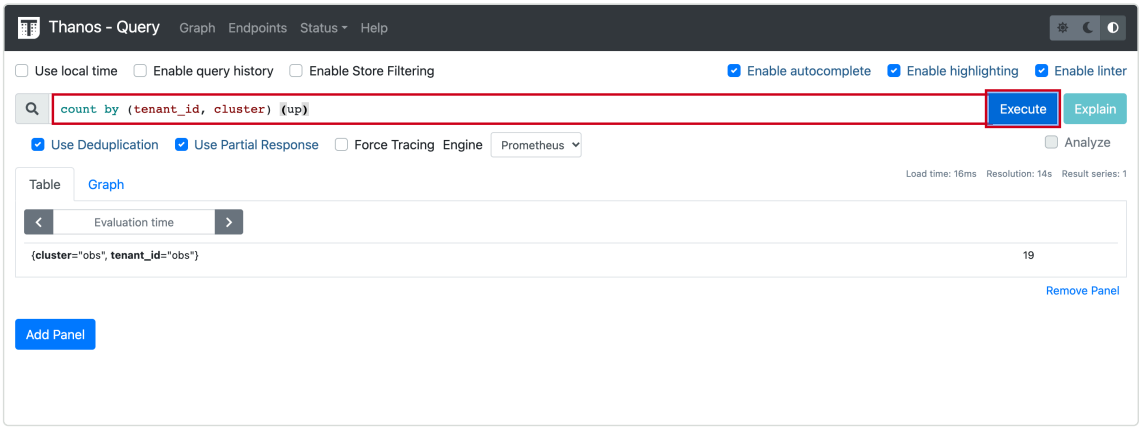


Abbildung 2: Eingabefeld und **Execute** sind rot markiert. Im Reiter **Table** erscheint das Ergebnis als Zahl.

DAS SOLLTEST DU SEHEN

Eine Zeile mit `tenant_id="obs"` und `cluster="obs"`. Die Zahl ist die Menge der Scrape-Ziele auf obs, im Labor um die 19. Siehst du stattdessen `tenant_id="unbekannt"`, fehlt der Header in `remoteWrite`.

Schalte in der Oberfläche die Deduplizierung einmal aus und wieder ein. Ohne Deduplizierung siehst du die Labels `prometheus_replica` und `receive_replica`, mit ihr verschwinden sie. Genau das bewirken die `replica-label`-Argumente.

4.7 Läuft der Remote Write sauber?

Prometheus führt über seinen Remote Write genau Buch. Zwei Abfragen in Thanos Query genügen:

```
rate(prometheus_remote_storage_samples_total{cluster="obs"}[5m])
rate(prometheus_remote_storage_samples_failed_total{cluster="obs"}[5m])
```

Wähle für Kurven vor dem Klick auf **Execute** den Reiter **Graph** unter dem Eingabefeld. Mit `-` und `+` links daneben stellst du den Zeitraum ein.

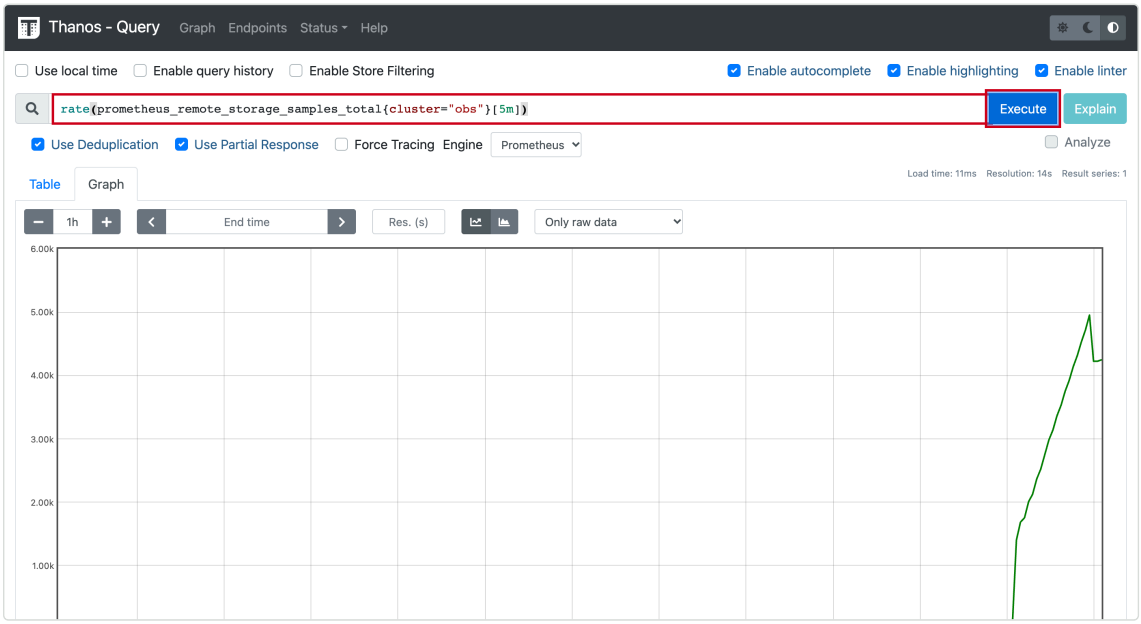


Abbildung 3: Reiter **Graph**: Prometheus schickt einige tausend Werte pro Sekunde an Receive. Die Kurve beginnt dort, wo Receive gestartet wurde.

DAS SOLLTEST DU SEHEN

Die erste Kurve liegt dauerhaft über null, im Labor bei einigen tausend Werten pro Sekunde. Die zweite ist leer oder null. Steigen die Fehler, steht der Grund in den Logs von `thanos-receive-0`.

STOLPERFALLE

Startet Receive neu, während Prometheus schreibt, meldet Receive oft kurz `out of order sample` oder `409 Conflict`. Prometheus wiederholt dann alte Daten, die Receive schon hat. Das legt sich nach wenigen Minuten von selbst. Bleibt es, zeigt Kapitel 14 die Ursachen.

4.8 Der erste Block in S3

Receive lädt Blöcke alle zwei Stunden hoch. Mach eine Pause oder arbeite mit Kapitel 5 weiter und komm danach zurück.

MAC Schritt 16

```
kubectl --context obs -n monitoring exec deploy/thanos-compact -- \
  thanos tools bucket inspect \
  --objstore.config-file=/etc/thanos/objstore.yml
```

Nutzt das Thanos-Programm im Compactor-Pod, um den Bucket zu lesen. Dort liegt die S3-Konfiguration schon, du brauchst keine Zugangsdaten auf dem Laptop.

DAS SOLLTEST DU SEHEN

Eine Tabelle mit mindestens einer Zeile. Die Spalte `RANGE` zeigt `2h0m0s`, die Spalte `LABELS` enthält `receive_replica` mit dem Wert `0` und `tenant_id` mit dem Wert `obs`. Später kommen Zeilen für `app1` und `app2` dazu und, nach einigen Tagen, längere Blöcke des Compactors.

Thanos Store findet neue Blöcke von selbst. Ab jetzt beantwortet Query Abfragen, die mehr als sechs Stunden zurückreichen, aus S3.

PROBIER ES SELBST

Schreib mit `curl` direkt an Receive, ohne Header. Leite dazu Port 19291 per `port-forward` um und sende eine leere POST-Anfrage an `/api/v1/receive`. Welche Antwort kommt zurück, und warum? Überlege danach: Welche Labels hätte eine Serie, die ein Sender ohne `THANOS-TENANT` erfolgreich schreibt?

WAS DU GELERNT HAST

- Thanos Receive nimmt Remote Write von beliebig vielen Sendern an und trennt sie über den Header `THANOS-TENANT` in Mandanten.
- Ein Hashring mit einem Eintrag macht Receive zum Router und Ingestor zugleich und lässt Raum für mehr Replikas.
- Thanos Query verbindet frische Daten aus Receive mit alten aus S3 und entfernt Kopien über die Replika-Labels.
- Ein Label am PrometheusRule entscheidet, ob Prometheus lokal oder der Thanos Ruler global auswertet.
- `thanos tools bucket inspect` zeigt, welche Blöcke wirklich in S3 liegen.

KAPITEL 5

Metriken aus den App-Clustern

app1 und app2 schicken ihre Metriken über HTTPS mit Basic Auth an die Zentrale, und eine einzige PromQL-Abfrage vergleicht alle drei Cluster.

Dauer: 75 Minuten **Voraussetzung:** Kapitel 2 (CoreDNS, TLS, `obs-ca`) und Kapitel 4 (Thanos Receive läuft)

5.1 Was du aufbaust

In den App-Clustern läuft kein Prometheus. Ein schlanker Sammler genügt: Grafana Alloy holt die Metriken ab und schickt sie per Remote Write an Receive. Zwei Exporter liefern die Rohdaten.

Wo	Was	Aufgabe
obs	Traefik-Middleware `obs-basic-auth`	Lässt nur bekannte Cluster durch
obs	Ingress `receive`, `loki`, `tempo`	HTTPS-Eingänge unter `*.obs.internal`
app1, app2	kube-state-metrics	Zustand der Kubernetes-Objekte als Metriken
app1, app2	node-exporter	CPU, Speicher, Platte und Netz des Nodes
app1, app2	Alloy mit `metrics.alloy`	Sammelt ein, hängt `cluster` an, schickt an Receive

WARUM DAS WICHTIG IST

Remote Write dreht die Richtung um. Nicht die Zentrale holt ab, sondern jeder Cluster liefert. Die Zentrale muss deshalb keinen App-Cluster erreichen können, und für einen neuen Cluster änderst du auf obs nur die Benutzerliste.

5.2 Ein Zugang pro Cluster

Jeder Cluster bekommt einen eigenen Benutzer mit eigenem Passwort. Der Benutzername ist der Clustername. Das Passwort entsteht mit `openssl rand` und liegt nur im Secret `obs-remote-auth` des jeweiligen Clusters.

MAC — Schritt 1

```
kubectl --context obs -n monitoring create secret generic obs-remote-auth \
  --from-literal=username=obs \
  --from-literal=password="$(openssl rand -hex 24)"
kubectl --context app1 -n monitoring create secret generic obs-remote-auth \
  --from-literal=username=app1 \
  --from-literal=password="$(openssl rand -hex 24)"
kubectl --context app2 -n monitoring create secret generic obs-remote-auth \
  --from-literal=username=app2 \
  --from-literal=password="$(openssl rand -hex 24)"
```

Legt in jedem Cluster das Secret an. `$(openssl rand -hex 24)` würfelt ein Passwort aus 48 Zeichen, das direkt im Secret landet. Es erscheint weder auf dem Bildschirm noch in der Shell-Historie. obs bekommt auch ein Secret, denn ab Kapitel 8 schickt Alloy auf obs Logs und Traces über denselben Weg.

STOLPERFALLE

Den Befehl nicht wiederholen, wenn die Secrets schon existieren. `create` bricht dann mit `already exists` ab, und das ist gut so: Ein neues Passwort im Cluster passt nicht mehr zur Benutzerliste auf obs. Willst du ein Passwort tauschen, löschst du das Secret und führst danach auch den nächsten Schritt erneut aus.

Auf obs braucht Traefik die Benutzerliste im htpasswd-Format. Sie enthält pro Zeile einen Namen und einen Hash des Passworts, nie das Passwort selbst.

Du baust die Liste in drei Schritten, einen pro Cluster. Jeder Schritt liest das Passwort aus dem Cluster, berechnet den Hash und hängt eine Zeile an eine Datei an:

MAC — Schritt 2

```
kubectl --context obs -n monitoring get secret obs-remote-auth \
  -o jsonpath='{.data.password}' | base64 -d \
  | openssl passwd -apr1 -stdin | sed 's/^/obs:/' > users.htpasswd
kubectl --context app1 -n monitoring get secret obs-remote-auth \
  -o jsonpath='{.data.password}' | base64 -d \
  | openssl passwd -apr1 -stdin | sed 's/^/app1:/' >> users.htpasswd
kubectl --context app2 -n monitoring get secret obs-remote-auth \
  -o jsonpath='{.data.password}' | base64 -d \
  | openssl passwd -apr1 -stdin | sed 's/^/app2:/' >> users.htpasswd
```

Lies jede Zeile von links nach rechts: Passwort aus dem Secret holen, Base64 entfernen, mit `openssl passwd -apr1` den Hash berechnen und mit `sed` den Clusternamen davorsetzen. `-stdin` hält das Passwort aus der Prozessliste heraus. Die erste Zeile legt die Datei mit `>` neu an, die beiden anderen hängen mit `>>` an.

MAC Schritt 3

```
kubectl --context obs -n monitoring create secret generic \
  obs-basic-auth-users --from-file=users=users.htpasswd
rm users.htpasswd
```

Legt das Secret auf obs an und löscht die Datei wieder. Sie enthält nur Hashes, keine Passwörter, gehört aber trotzdem nicht auf die Platte.

MAC Schritt 4

```
kubectl --context obs -n monitoring get secret obs-basic-auth-users \
  -o jsonpath='{.data.users}' | base64 -d | cut -d: -f1
```

Zeigt nur die Benutzernamen, nicht die Hashes.

DAS SOLLTEST DU SEHEN

Drei Zeilen: `obs`, `app1` und `app2`.

5.3 Middleware und Ingress

Die Middleware verweist auf die Benutzerliste. Jeder Ingress, der sie einbindet, verlangt Basic Auth.

DATEI `observability/central/ingress/middleware.yaml`

```yaml
apiVersion: traefik.io/v1alpha1
kind: Middleware
metadata:
  name: obs-basic-auth
  namespace: monitoring
spec:
  basicAuth:
    secret: obs-basic-auth-users
    removeHeader: true
```

`removeHeader: true` entfernt den `Authorization`-Header, bevor die Anfrage an Receive geht. Das Passwort reist also nicht weiter als nötig.

DATEI `observability/central/ingress/receive.yaml (Ausschnitt)`

```yaml
metadata:
  name: receive
  namespace: monitoring
  annotations:
    traefik.ingress.kubernetes.io/router.entrypoints: websecure
    traefik.ingress.kubernetes.io/router.tls: "true"
    traefik.ingress.kubernetes.io/router.middlewares: monitoring-obs-basic-auth@kubernetescrd
spec:
  tls:
    - hosts: [receive.obs.internal]
      secretName: obs-tls
  rules:
```

```
    - host: receive.obs.internal
      http:
        paths:
          - path: /api/v1/receive
            pathType: Prefix
            backend:
              service: {name: thanos-receive, port: {number: 19291}}
```

Die Middleware nennst du in der Form `<namespace>-<name>@kubernetescrd`. Nach außen ist nur der Schreibpfad `/api/v1/receive` offen, und nur über HTTPS mit dem Zertifikat aus `obs-tls`. Abfragen laufen weiter nur im Cluster über Thanos Query.

`loki.yaml` und `tempo.yaml` im selben Verzeichnis sind genauso gebaut. Sie führen `loki.obs.internal` mit dem Pfad `/loki/api/v1/push` auf den Service `loki` (Port 3100) und `tempo.obs.internal` mit dem Pfad `/v1/traces` auf den Service `tempo` (Port 4318). Diese Services entstehen erst in den Kapiteln 8 und 9. Du legst alle Eingänge trotzdem jetzt an, dann ist die Tür zur Zentrale an einer Stelle beschrieben.

MAC Schritt 5

```
kubectl --context obs apply -f observability/central/ingress/
```

Wendet Middleware und alle drei Ingress-Objekte an.

MAC Schritt 6

```
kubectl --context obs -n monitoring get ingress
```

Zeigt die drei Eingänge.

DAS SOLLTEST DU SEHEN

`receive`, `loki` und `tempo` mit Klasse `traefik`, den passenden Hostnamen und Port `80, 443`. Unter `ADDRESS` steht `172.16.0.126`.

5.4 Von außen prüfen

Bevor ein Cluster schreibt, prüfst du die Tür vom Mac aus. Der Name `receive.obs.internal` steht seit Kapitel 2 in deiner `/etc/hosts`.

MAC Schritt 7

```
curl -s -o /dev/null -w '%{http_code}\n' \
  --cacert observability/tls/ca.crt \
  -X POST https://receive.obs.internal/api/v1/receive
```

Eine Anfrage ohne Zugangsdaten. `-w '%{http_code}'` gibt nur den HTTP-Status aus. `--cacert` prüft das Zertifikat gegen deine eigene CA.

DAS SOLLTEST DU SEHEN

`401`. Traefik weist die Anfrage ab, bevor sie Receive erreicht. Meldet `curl` stattdessen einen Zertifikatsfehler, stimmt `obs-tls` nicht. Kommt `404`, fehlt der Ingress.

MAC — Schritt 8

```
PW=$(kubectl --context app1 -n monitoring get secret obs-remote-auth \
  -o jsonpath='{.data.password}' | base64 -d)
```

Holt das Passwort von app1 in eine Shell-Variable, ohne es anzuzeigen.

MAC — Schritt 9

```
curl -s -o /dev/null -w '%{http_code}\n' \
  --cacert observability/tls/ca.crt -u "app1:$PW" \
  -X POST https://receive.obs.internal/api/v1/receive
unset PW
```

Dieselbe Anfrage mit Benutzer und Passwort. Danach verschwindet das Passwort wieder aus der Shell.

DAS SOLLTEST DU SEHEN

Nicht mehr `401`, sondern `400`. Traefik hat dich durchgelassen, und Receive lehnt den leeren Inhalt ab, weil er keine gültige Remote-Write-Nachricht ist. Genau so soll es sein.

STOLPERFALLE

Basic Auth beweist nur, *welcher* Cluster schreibt. Den Mandanten wählt der Sender selbst über den Header `THANOS-TENANT`. Ein Cluster mit gültigem Zugang könnte also unter fremdem Namen schreiben. Im Labor ist das hinnehmbar, die Aufgabe am Ende des Kapitels zeigt, wie Traefik den Mandanten erzwingt.

5.5 Exporter in app1 und app2

Auf obs hat kube-prometheus-stack beide Exporter mitgebracht. In den App-Clustern installierst du sie einzeln aus denselben Charts. Welche Versionen aktuell sind, fragst du Helm:

MAC — Schritt 10

```
helm repo update prometheus-community
helm search repo prometheus-community/kube-state-metrics
helm search repo prometheus-community/prometheus-node-exporter
```

Zeigt die neueste Chart- und App-Version. Stand dieses Buches: `kube-state-metrics` Chart `9.0.0` (App `2.20.0`) und `prometheus-node-exporter` Chart `5.0.0` (App `1.12.1`). Die Befehle unten nutzen genau diese Versionen.

DATEI `observability/cluster/exporters/node-exporter.yaml (Ausschnitt)`

```
prometheusScrape: false
extraArgs:
  - --collector.filesystem.mount-points-exclude=^/(dev|proc|sys|run/containerd/.+|var/lib/docker/.+|var/
lib/kubelet/.+)($|/)
  - --collector.filesystem.fs-types-exclude=^(autofs|binfmt_misc|bpf|...|erofs)$
```

Die Filter stammen aus kube-prometheus-stack. Ohne sie meldet node-exporter jeden Container-Mount als eigenes Dateisystem. `prometheusScrape: false` lässt die Annotation `prometheus.io/scrape` weg, denn Alloy sucht die Pods gezielt über ihr Label. Die Datei für kube-state-metrics enthält nur diesen Schalter und Ressourcen-Grenzen.

MAC Schritt 11

```
helm --kube-context app1 upgrade --install kube-state-metrics \
  prometheus-community/kube-state-metrics \
  -n monitoring --version 9.0.0 \
  -f observability/cluster/exporters/kube-state-metrics.yaml --wait
helm --kube-context app2 upgrade --install kube-state-metrics \
  prometheus-community/kube-state-metrics \
  -n monitoring --version 9.0.0 \
  -f observability/cluster/exporters/kube-state-metrics.yaml --wait
```

Installiert kube-state-metrics in beiden App-Clustern. Die beiden Befehle unterscheiden sich nur im Kontext.

MAC Schritt 12

```
helm --kube-context app1 upgrade --install node-exporter \
  prometheus-community/prometheus-node-exporter \
  -n monitoring --version 5.0.0 \
  -f observability/cluster/exporters/node-exporter.yaml --wait
helm --kube-context app2 upgrade --install node-exporter \
  prometheus-community/prometheus-node-exporter \
  -n monitoring --version 5.0.0 \
  -f observability/cluster/exporters/node-exporter.yaml --wait
```

Installiert node-exporter als DaemonSet. Er läuft im Netz des Hosts auf Port 9100.

MAC Schritt 13

```
kubectl --context app1 -n monitoring get pods
```

Zeigt die Pods in app1.

DAS SOLLTEST DU SEHEN

`kube-state-metrics-...` und `node-exporter-prometheus-node-exporter-...` stehen auf `Running` und `1/1`. In app2 sieht es genauso aus.

5.6 Wie Alloy mehrere Dateien lädt

Die Konfiguration von Alloy wächst über drei Kapitel: Metriken jetzt, Logs in Kapitel 8, Traces in Kapitel 9. Jedes Thema bekommt eine eigene Datei in `observability/cluster/alloy/`. Damit das funktioniert, greifen drei Dinge ineinander.

1. **Eine ConfigMap, ein Schlüssel pro Datei.** Du legst die ConfigMap `alloy-config` selbst mit `kubectl` an. Jede `.alloy`-Datei wird ein eigener Schlüssel.
2. **Das Chart hängt die ganze ConfigMap ein.** Alle Schlüssel erscheinen als Dateien unter `/etc/alloy`, und das Chart startet `alloy run /etc/alloy/<key>`.
3. **Alloy liest ein Verzeichnis.** Laut Doku von `alloy run` lädt Alloy aus einem Verzeichnis alle `*.alloy`-Dateien als eine gemeinsame Konfiguration und ignoriert Unterverzeichnisse.

Setzt du `key` auf einen Punkt, entsteht `alloy run /etc/alloy/.`, also das Verzeichnis selbst. Die versteckten Unterverzeichnisse, mit denen Kubernetes ConfigMaps einhängt, überspringt Alloy.

DATEI `observability/cluster/alloy/values.yaml (Ausschnitt)`

```yaml
alloy:
  configMap:
    create: false
    name: alloy-config
    key: "."
  extraEnv:
    - name: CLUSTER_NAME
      valueFrom:
        secretKeyRef:
          name: obs-remote-auth
          key: username
    # OBS_USER und OBS_PASSWORD ebenso aus obs-remote-auth
  mounts:
    varlog: true
    extra:
      - name: obs-ca
        mountPath: /etc/obs-ca
        readOnly: true
controller:
  type: daemonset
  volumes:
    extra:
      - name: obs-ca
        configMap:
          name: obs-ca
```

Weil der Benutzername gleich dem Clusternamen ist, liest `CLUSTER_NAME` aus demselben Secret. So gilt eine einzige Werte-Datei für alle drei Cluster, Dateien pro Cluster brauchst du nicht. `varlog` und die

OTLP-Ports im Rest der Datei sind schon für die Kapitel 8 und 9 vorbereitet, damit sich der Helm-Befehl nie mehr ändert.

WARUM DAS WICHTIG IST

Alloy verlangt, dass jeder Komponentenname über alle Dateien hinweg eindeutig ist. Deshalb beginnen alle Namen in `metrics.alloy` mit `metrics_`, in `logs.alloy` mit `logs_` und in `traces.alloy` mit `traces_`. Zwei Dateien können sich so nie in die Quere kommen.

5.7 Die Datei metrics.alloy

Die Datei hat drei Teile: Kubelet und cAdvisor, die beiden Exporter und den Versand. Lies sie wie jede Alloy-Datei von unten nach oben, `forward_to` zeigt immer zum nächsten Glied der Kette.

DATEI `observability/cluster/alloy/metrics.alloy (Ausschnitt)`

```
discovery.kubernetes "metrics_nodes" {
  role = "node"
  selectors {
    role  = "node"
    field = "metadata.name=" + sys.env("HOSTNAME")
  }
}

prometheus.scrape "metrics_cadvisor" {
  targets           = discovery.relabel.metrics_cadvisor.output
  job_name          = "kubelet"
  metrics_path      = "/metrics/cadvisor"
  scheme            = "https"
  scrape_interval   = "30s"
  honor_labels      = true
  bearer_token_file = "/var/run/secrets/kubernetes.io/serviceaccount/token"
  tls_config {
    ca_file              = "/var/run/secrets/kubernetes.io/serviceaccount/ca.crt"
    insecure_skip_verify = true
  }
  forward_to = [prometheus.remote_write.metrics_obs.receiver]
}
```

`HOSTNAME` setzt das Chart auf den Node-Namen. Der Feld-Selektor sorgt dafür, dass jeder Alloy-Pod nur den Kubelet seines eigenen Nodes abfragt. Ein `discovery.relabel` davor setzt die Labels `node` und `metrics_path`. Der Kubelet selbst läuft genauso, nur mit `/metrics`. Das Label `metrics_path` brauchen die Dashboards von kube-prometheus-stack, um beide Quellen zu unterscheiden. `insecure_skip_verify` entspricht der Voreinstellung von kube-prometheus-stack für den Kubelet.

DATEI `observability/cluster/alloy/metrics.alloy (Ausschnitt)`

```
discovery.kubernetes "metrics_pods" {
  role = "pod"
  namespaces {
    names = ["monitoring"]
  }
```

```
  selectors {
    role  = "pod"
    field = "spec.nodeName=" + sys.env("HOSTNAME")
  }
}

discovery.relabel "metrics_kube_state_metrics" {
  targets = discovery.kubernetes.metrics_pods.targets
  rule {
    source_labels = ["__meta_kubernetes_pod_label_app_kubernetes_io_name"]
    regex         = "kube-state-metrics"
    action        = "keep"
  }
  rule {
    source_labels = ["__meta_kubernetes_pod_container_port_name"]
    regex         = "http"
    action        = "keep"
  }
}

prometheus.scrape "metrics_kube_state_metrics" {
  targets         = discovery.relabel.metrics_kube_state_metrics.output
  job_name        = "kube-state-metrics"
  scrape_interval = "30s"
  honor_labels    = true
  forward_to      = [prometheus.remote_write.metrics_obs.receiver]
}
```

`keep` behält nur Ziele, deren Label auf `regex` passt. Übrig bleibt der Port `http` von kube-state-metrics. node-exporter folgt demselben Muster mit dem Label `prometheus-node-exporter`, dem Port `metrics` und `job_name = "node-exporter"`. `honor_labels` schützt die Labels `namespace` und `pod`, die kube-state-metrics für die beschriebenen Objekte setzt.

DATEI `observability/cluster/alloy/metrics.alloy (Ausschnitt)`

```
prometheus.remote_write "metrics_obs" {
  external_labels = {
    cluster = sys.env("CLUSTER_NAME"),
  }
  endpoint {
    url = "https://receive.obs.internal/api/v1/receive"
    headers = {
      "THANOS-TENANT" = sys.env("CLUSTER_NAME"),
    }
    basic_auth {
      username = sys.env("OBS_USER")
      password = sys.env("OBS_PASSWORD")
    }
    tls_config {
      ca_file = "/etc/obs-ca/ca.crt"
    }
  }
}
```

`external_labels` hängt `cluster` an jede Serie, der Header bestimmt den Mandanten in Receive. Zugangsdaten und Clustername stammen aus Umgebungsvariablen, die Datei selbst enthält kein Geheimnis. Die CA aus der ConfigMap `obs-ca` prüft das Zertifikat von `receive.obs.internal`.

GETESTET MIT ALLOY 1.21

Die Alloy-Dateien dieses Buches liefen mit dem Chart `1.15.0`, das Alloy `v1.21.3` mitbringt. Ein Tippfehler zeigt sich beim Start: Der Pod wird nicht bereit, und `kubectl logs` nennt Datei und Zeile.

5.8 Alloy installieren

Zuerst legst du die ConfigMap an, danach Alloy. In app1 und app2 enthält sie vorerst nur `metrics.alloy`.

MAC — Schritt 14

```
kubectl --context app1 -n monitoring create configmap alloy-config \
  --from-file=observability/cluster/alloy/metrics.alloy \
  --dry-run=client -o yaml | kubectl --context app1 apply -f -
kubectl --context app2 -n monitoring create configmap alloy-config \
  --from-file=observability/cluster/alloy/metrics.alloy \
  --dry-run=client -o yaml | kubectl --context app2 apply -f -
```

`--from-file` mit einer Datei nimmt den Dateinamen als Schlüssel, hier `metrics.alloy`. `create ... --dry-run=client -o yaml` baut die ConfigMap nur als Text, `apply` legt sie an oder aktualisiert sie. So funktioniert derselbe Befehl beim ersten Mal und bei jeder späteren Änderung.

MAC — Schritt 15

```
helm repo add grafana https://grafana.github.io/helm-charts
helm repo update grafana
```

Trägt das Grafana-Repository ein. Alloy liegt dort, nicht bei `grafana-community`.

MAC — Schritt 16

```
helm --kube-context app1 upgrade --install alloy grafana/alloy \
  -n monitoring --version 1.15.0 \
  -f observability/cluster/alloy/values.yaml --wait
helm --kube-context app2 upgrade --install alloy grafana/alloy \
  -n monitoring --version 1.15.0 \
  -f observability/cluster/alloy/values.yaml --wait
```

Installiert Alloy als DaemonSet in beiden App-Clustern. Das Chart bringt die Rechte für die Kubernetes-Discovery und für die Kubelet-Metriken mit.

MAC — Schritt 17

```
kubectl --context app1 -n monitoring logs ds/alloy -c alloy --tail=30
```

Zeigt den Start von Alloy in app1.

DAS SOLLTEST DU SEHEN

Keine Zeilen mit `level=error`. Steht dort `401`, passt das Passwort nicht zur Benutzerliste. `x509: certificate signed by unknown authority` heißt, dass `/etc/obs-ca/ca.crt` fehlt oder falsch ist. `no such host` deutet auf CoreDNS (Kapitel 2).

So testest du Alloy

Alloy hat eine eigene Oberfläche. Sie zeigt, welche Komponenten geladen sind und ob sie arbeiten. Leite sie in einem zweiten Terminal auf den Mac um:

MAC Schritt 18

```
kubectl --context app1 -n monitoring port-forward svc/alloy 12345:12345
```

Öffne danach http://localhost:12345 im Browser.

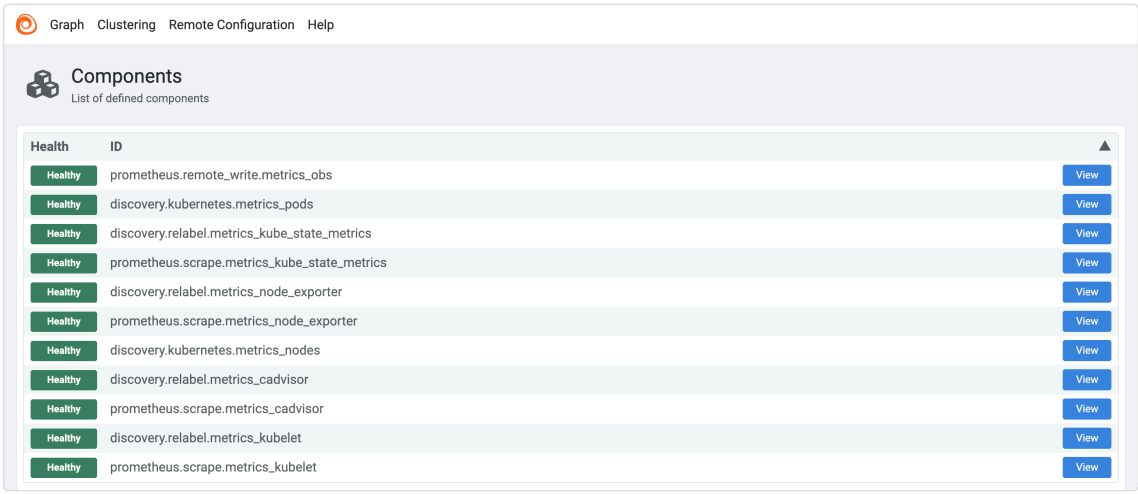


Abbildung 4: Startseite der Alloy-Oberfläche: jede Komponente aus `metrics.alloy` mit ihrem Zustand. Alle müssen `Healthy` sein.

Mit **View** öffnest du eine Komponente und siehst ihre aktuellen Argumente und Ausgaben, bei `discovery.relabel` etwa die gefundenen Ziele. Der Menüpunkt **Graph** zeigt die ganze Kette so, wie du sie geschrieben hast:

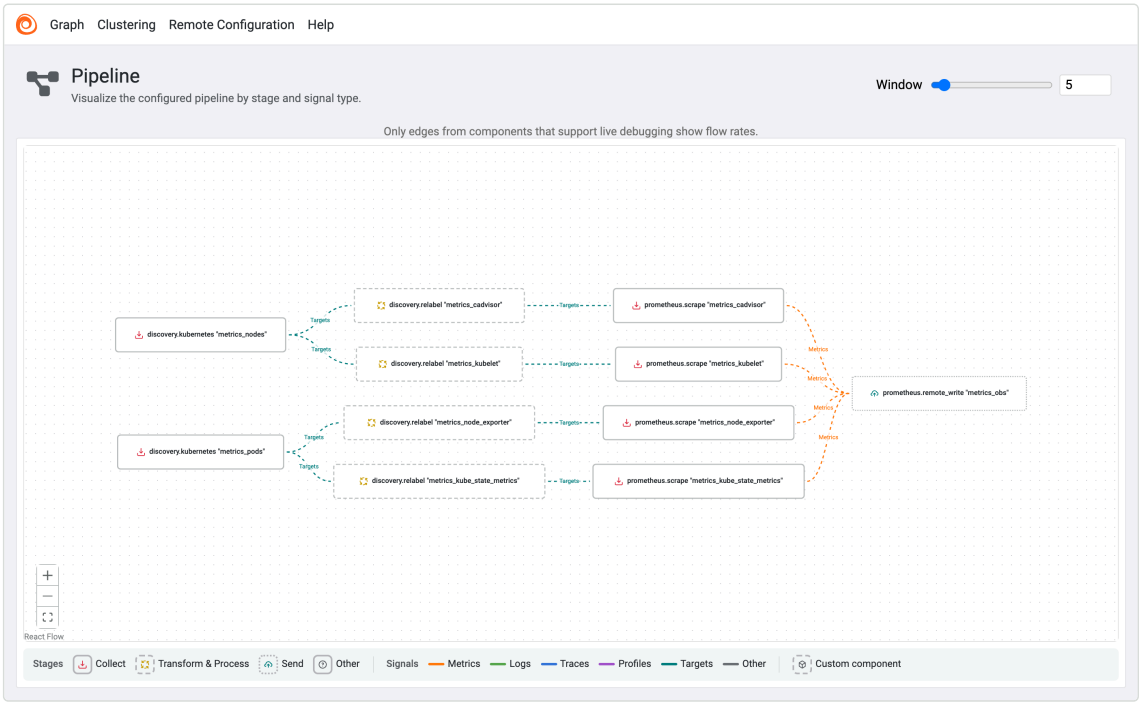


Abbildung 5: Menüpunkt **Graph**: links die Discovery, in der Mitte Relabel und Scrape, rechts der Versand an Receive. Orange Linien tragen Metriken, türkisfarbene die Liste der Ziele.

Ist eine Komponente rot, steht der Grund in ihrer Detailansicht. Beende den Port-Forward danach mit `Ctrl-C`.

5.9 Die Konfiguration aktualisieren

Diesen Ablauf nutzt du jedes Mal, wenn sich eine `.alloy`-Datei ändert oder eine neue dazukommt. Die Kapitel 8 und 9 wiederholen ihn mit ihren Dateien.

MAC Schritt 19

```
kubectl --context app1 -n monitoring create configmap alloy-config \
  --from-file=observability/cluster/alloy/metrics.alloy \
  --dry-run=client -o yaml | kubectl --context app1 apply -f -
kubectl --context app2 -n monitoring create configmap alloy-config \
  --from-file=observability/cluster/alloy/metrics.alloy \
  --dry-run=client -o yaml | kubectl --context app2 apply -f -
```

Schreibt die ConfigMap neu. Für jede weitere Datei hängst du eine eigene Zeile `--from-file=...` an. Nicht genannte Dateien verschwinden aus der ConfigMap.

MAC Schritt 20

```
kubectl --context app1 -n monitoring rollout restart ds/alloy
kubectl --context app1 -n monitoring rollout status ds/alloy
kubectl --context app2 -n monitoring rollout restart ds/alloy
kubectl --context app2 -n monitoring rollout status ds/alloy
```

Startet Alloy neu und wartet, bis alle Pods wieder bereit sind.

WARUM DAS WICHTIG IST

Das Chart bringt einen `config-reloader` mit, der Alloy bei Änderungen von selbst neu laden lässt. Das dauert aber bis zu einer Minute, und bei einem Fehler arbeitet Alloy still mit der alten Konfiguration weiter. Nach einem Neustart siehst du sofort, ob die neue Datei gilt: Der Pod wird bereit oder eben nicht.

So sieht die ConfigMap am Ende des Buches aus:

Cluster	`--from-file` für
app1, app2	`metrics.alloy`, `logs.alloy` (Kapitel 8), `traces.alloy` (Kapitel 9)
obs	`logs.alloy` (Kapitel 8), `traces.alloy` (Kapitel 9), kein `metrics.alloy`

5.10 Alle Cluster in einer Abfrage

Wechsle zurück zu Thanos Query auf obs.

Öffne Thanos Query wie in Kapitel 4 per `port-forward` unter http://localhost:10902.

```
count by (tenant_id, cluster) (up)
```

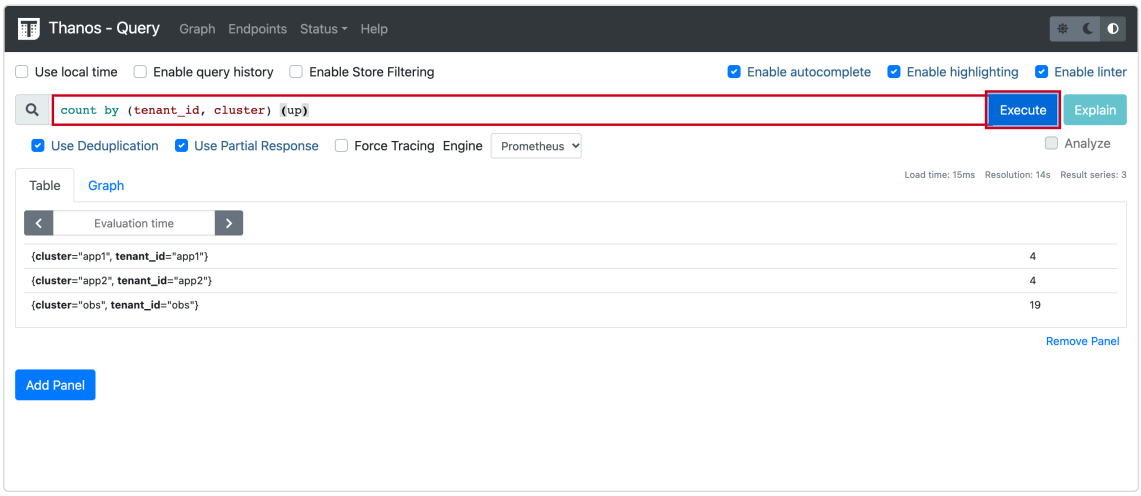


Abbildung 6: Dieselbe Abfrage wie in Kapitel 4, jetzt mit drei Zeilen.

DAS SOLLTEST DU SEHEN

Drei Zeilen: je eine für `obs`, `app1` und `app2`, und in jeder Zeile sind `tenant_id` und `cluster` gleich. In app1 und app2 zählt jede Zeile vier Ziele: Kubelet, cAdvisor, node-exporter und kube-state-metrics.

Jetzt vergleichst du die Cluster. Das Muster ist immer gleich: `sum by (cluster)` fasst alles innerhalb eines Clusters zusammen und behält die Cluster getrennt.

```
sum by (cluster) (
  rate(container_cpu_usage_seconds_total{container!=""}[5m])
)
sum by (cluster) (node_memory_MemAvailable_bytes)
  / sum by (cluster) (node_memory_MemTotal_bytes)
sum by (cluster) (kube_pod_status_phase{phase="Running"})
```

DAS SOLLTEST DU SEHEN

Jede Abfrage liefert drei Kurven, eine pro Cluster. Im Labor sahen die Werte so aus:

Abfrage	obs	app1	app2
CPU-Kerne	0,38	0,04	0,04
freier Speicher	0,66	0,66	0,64
laufende Pods	17	9	8

Bei CPU und Pods liegt obs deutlich höher, dort läuft die ganze Zentrale. Beim freien Anteil des Speichers liegen alle gleichauf, weil obs dreimal so viel RAM hat.

PROBIER ES SELBST

Erzwinge den Mandanten auf obs. Traefik kann den angemeldeten Benutzernamen in einen Header schreiben: Die Option `headerField` der Basic-Auth-Middleware setzt ihn und überschreibt dabei einen mitgeschickten Wert. Lege eine zweite Middleware `receive-tenant` an, die dieselbe Benutzerliste nutzt und zusätzlich `headerField: THANOS-TENANT` setzt. Ersetze am Ingress `receive` die bisherige Middleware durch die neue. Zwei Basic-Auth-Middlewares hintereinander funktionieren nicht, denn die erste entfernt den `Authorization`-Header. Ändere dann in app1 testweise den Header in `metrics.alloy` auf `"THANOS-TENANT" = "app2"` und aktualisiere die Konfiguration. Welches `tenant_id` zeigt Thanos Query für `cluster="app1"`? Stell danach alles zurück.

WAS DU GELERNT HAST

- Ein Benutzer pro Cluster in einer htpasswd-Liste trennt die Zugänge, die Passwörter leben nur in Secrets.
- Nach außen ist nur der Schreibpfad offen, und ein `curl` ohne Zugangsdaten muss `401` liefern.
- Alloy lädt mit `key: "."` alle `.alloy`-Dateien einer ConfigMap. Eindeutige Präfixe halten die Dateien auseinander.
- Eine Änderung an Alloy heißt: ConfigMap neu schreiben, DaemonSet neu starten.
- `sum by (cluster)` vergleicht alle Cluster in einer einzigen Abfrage.

KAPITEL 6

Grafana als Code

Grafana startet mit Datenquellen, Ordnern und Login aus dem Repo. Nichts, was zählt, entsteht durch Klicks in der Oberfläche.

Dauer: 45 Minuten **Voraussetzung:** Kapitel 4 und 5: kube-prometheus-stack läuft auf obs, Thanos Query kennt alle drei Cluster

6.1 Warum keine Klicks

Grafana lädt zum Klicken ein. Eine Datenquelle ist schnell angelegt, ein Panel schnell gebaut. Nach drei Monaten weiß niemand mehr, wer was geändert hat, und nach einem Neuaufbau ist alles weg. Deshalb gilt in diesem Buch eine Regel: **Die Quelle der Wahrheit ist das Repo.** Die Oberfläche ist zum Lesen und Ausprobieren da.

Was	Wo im Repo	Wie es in Grafana landet
Login, Ingress, Sidecars	`observability/central/kps/grafana.yaml`	Helm-Werte für `kps`
Admin-Passwort	nirgends	Secret `grafana-admin`, erzeugt mit `openssl rand`
Datenquellen	`observability/grafana/datasources/`	ConfigMaps mit Label `grafana_datasource: "1"`
Dashboards	`observability/dashboards/`	ConfigMaps mit Label `grafana_dashboard: "1"` (Kapitel 10)

Im Grafana-Pod laufen neben Grafana zwei kleine Sidecar-Container. Sie beobachten die Kubernetes-API und schreiben jede passende ConfigMap als Datei in Grafanas Provisioning-Verzeichnis. Grafana liest diese Dateien und lädt sie neu. Du legst also nur ConfigMaps an, den Rest erledigen die Sidecars.

WARUM DAS WICHTIG IST

Grafana braucht so keinen eigenen Speicher. Fällt der Pod aus oder ziehst du auf einen neuen Cluster um, startet Grafana leer und baut sich aus den ConfigMaps in Sekunden wieder auf. Das spart ein PVC und eine Backup-Aufgabe in Kapitel 12.

6.2 Das Admin-Passwort

Grafana hat einen lokalen Administrator. Sein Passwort gehört in ein Secret, nicht in die Werte-Datei.

MAC — Schritt 1

```
kubectl --context obs -n monitoring create secret generic grafana-admin \
  --from-literal=admin-user=admin \
  --from-literal=admin-password="$(openssl rand -hex 20)"
```

Erzeugt ein zufälliges Passwort mit 40 Hex-Zeichen und legt es direkt im Cluster ab. Es erscheint weder auf dem Bildschirm noch in einer Datei.

STOLPERFALLE

Grafana übernimmt das Passwort nur beim allerersten Start in seine Datenbank. Ohne PVC ist das bei jedem Neustart so, deshalb wirkt eine Änderung am Secret nach `kubectl rollout restart`. Mit PVC müsstest du das Passwort zusätzlich mit `grafana cli admin reset-admin-password` ändern.

6.3 Die Helm-Werte

Die Grafana-Werte liegen in einer eigenen Datei. So bleibt `values.yaml` aus Kapitel 4 übersichtlich, und du siehst auf einen Blick, was Grafana betrifft.

DATEI `observability/central/kps/grafana.yaml (Ausschnitt)`

```
grafana:
  admin:
    existingSecret: grafana-admin
    userKey: admin-user
    passwordKey: admin-password
  persistence:
    enabled: false
  ingress:
    enabled: true
    ingressClassName: traefik
    annotations:
      traefik.ingress.kubernetes.io/router.entrypoints: websecure
      traefik.ingress.kubernetes.io/router.tls: "true"
    hosts:
      - grafana.obs.internal
    tls:
      - secretName: obs-tls
        hosts:
          - grafana.obs.internal
  grafana.ini:
    server:
      root_url: https://grafana.obs.internal
    users:
      allow_sign_up: false
  sidecar:
    dashboards:
      label: grafana_dashboard
```

```
      labelValue: "1"
      searchNamespace: ALL
      folderAnnotation: grafana_folder
      annotations:
        grafana_folder: Kubernetes
      multicluster:
        global:
          enabled: true
      provider:
        foldersFromFilesStructure: true
        allowUiUpdates: false
    datasources:
      label: grafana_datasource
      labelValue: "1"
      defaultDatasourceEnabled: true
      isDefaultDatasource: false
      alertmanager:
        enabled: false
```

Die Datei enthält zusätzlich Ressourcen-Grenzen und Analytics-Schalter. Die wichtigen Stellen:

Schlüssel	Wirkung
`admin.existingSecret`	Login aus dem Secret `grafana-admin`
`persistence.enabled: false`	Keine Datenbank auf Disk, alles kommt aus ConfigMaps
`ingress`	Traefik liefert Grafana unter `https://grafana.obs.internal` mit dem Zertifikat aus `obs-tls` aus
`folderAnnotation`	Die Annotation `grafana_folder` an einer ConfigMap bestimmt den Ordner in Grafana
`annotations`	Die rund 30 mitgelieferten Dashboards des Charts landen im Ordner `Kubernetes`
`multicluster.global.enabled`	Die mitgelieferten Dashboards zeigen die Auswahl `cluster` an, sonst ist sie versteckt
`allowUiUpdates: false`	Dashboards aus ConfigMaps lassen sich in der UI nicht überschreiben
`isDefaultDatasource: false`	Die lokale Prometheus-Datenquelle bleibt, Standard wird Thanos
`alertmanager.enabled: false`	Die Alertmanager-Datenquelle kommt aus dem Repo statt vom Chart

STOLPERFALLE

`foldersFromFilesStructure` und `folderAnnotation` gehören zusammen. Setzt du nur die Annotation, landen trotzdem alle Dashboards im Ordner **Dashboards**, denn Grafana übernimmt die Ordnerstruktur der Sidecar-Dateien nur mit diesem Schalter.

6.4 Datenquellen als ConfigMap

Eine Datenquelle ist eine kleine YAML-Datei im Provisioning-Format von Grafana, verpackt in eine ConfigMap.

DATEI `observability/grafana/datasources/thanos.yaml`

```yaml
apiVersion: v1
kind: ConfigMap
metadata:
  name: grafana-ds-thanos
  namespace: monitoring
  labels:
    grafana_datasource: "1"
data:
  thanos.yaml: |
    apiVersion: 1
    datasources:
      - name: Thanos
        uid: thanos
        type: prometheus
        access: proxy
        url: http://thanos-query.monitoring.svc:10902
        isDefault: true
        editable: false
        jsonData:
          prometheusType: Thanos
          httpMethod: POST
          timeInterval: 30s
          exemplarTraceIdDestinations:
            - name: traceID
              datasourceUid: tempo
```

Thanos Query spricht die Prometheus-API, deshalb ist der Typ `prometheus`. `prometheusType: Thanos` schaltet in Grafana die passenden Abfrage-Optionen frei. `timeInterval` sagt Grafana, wie dicht die Daten liegen, damit `$__rate_interval` nie kürzer als zwei Proben wird. Die feste `uid: thanos` brauchst du für Links zwischen Datenquellen: `exemplarTraceIdDestinations` führt von einem Exemplar mit dem Label `traceID` zum Trace in Tempo, das in Kapitel 9 dazukommt. Bis dahin ist der Eintrag einfach ohne Wirkung.

Daneben liegt `alertmanager.yaml`. Sie zeigt auf `http://alertmanager-operated.monitoring.svc:9093`, den Service, den der Prometheus Operator für jeden Alertmanager anlegt. Damit siehst du in Kapitel 7 Alarme und Silences auch in Grafana. Die Datenquellen für Loki und Tempo kommen in Kapitel 8 und 9 ins selbe Verzeichnis.

MAC — Schritt 2

```
kubectl --context obs apply \
  -f observability/grafana/datasources/thanos.yaml \
  -f observability/grafana/datasources/alertmanager.yaml
```

Legt beide ConfigMaps an. Die Dateien für Loki und Tempo liegen schon im Verzeichnis, sie kommen erst in Kapitel 8 und 9 dran. Der Sidecar bemerkt sie sofort und lässt Grafana die Datenquellen neu laden. Grafana selbst läuft vielleicht noch mit alten Werten, das ändert der nächste Schritt.

6.5 Grafana neu ausrollen

MAC Schritt 3

```
helm --kube-context obs upgrade kps \
  prometheus-community/kube-prometheus-stack \
  --version 92.18.0 -n monitoring \
  -f observability/central/kps/values.yaml \
  -f observability/central/kps/grafana.yaml \
  --wait --timeout 10m
```

Spätere Dateien überschreiben frühere. Ab Kapitel 7 kommt `alerting.yaml` als dritte Datei dazu. Lass nie eine Datei weg, sonst setzt Helm deren Werte auf die Standardwerte zurück.

MAC Schritt 4

```
kubectl --context obs -n monitoring rollout status deploy/kps-grafana
```

Wartet, bis der neue Grafana-Pod bereit ist.

MAC Schritt 5

```
kubectl --context obs -n monitoring logs deploy/kps-grafana \
  -c grafana-sc-datasources --tail=20
```

Zeigt das Protokoll des Datenquellen-Sidecars. Der Containername `grafana-sc-datasources` stammt aus dem Grafana-Chart.

DAS SOLLTEST DU SEHEN

Zeilen, in denen der Sidecar `thanos.yaml` und `alertmanager.yaml` nach `/etc/grafana/provisioning/datasources/` schreibt, außerdem `datasource.yaml`, die Datei des Charts für die lokale Prometheus-Datenquelle. Danach ruft er Grafana auf:

```
"msg": "None sent to http://localhost:3000/api/admin/provisioning/datasources/reload.
        Response: 200 OK {\"message\":\"Datasources config reloaded\"}"
```

Davor stehen oft einige `WARNING ... Retrying`-Zeilen. Der Sidecar startet schneller als Grafana und versucht es einfach erneut, das ist harmlos.

6.6 Der erste Login

Die Namen unter `obs.internal` hast du in Kapitel 2 in `/etc/hosts` eingetragen, die CA kennt dein Browser, wenn du sie im Schlüsselbund als vertrauenswürdig markiert hast.

MAC Schritt 6

```
kubectl --context obs -n monitoring get secret grafana-admin \
  -o jsonpath='{.data.admin-password}' | base64 -d | pbcopy
```

Kopiert das Passwort in die Zwischenablage, ohne es anzuzeigen. Öffne dann https://grafana.obs.internal und melde dich als `admin` an.

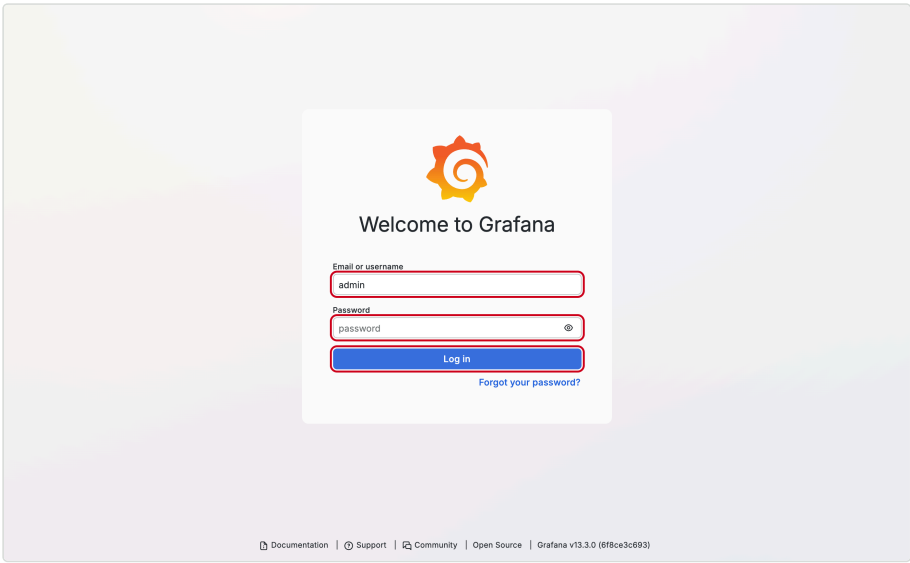


Abbildung 7: Die Anmeldeseite von Grafana: oben `admin` eintragen, darunter das Passwort mit `Cmd-V` einfügen, dann **Log in**.

Zeigt der Browser stattdessen eine Zertifikatswarnung, vertraut er deiner CA noch nicht. Hol den optionalen Schritt mit dem Schlüsselbund aus Kapitel 2 nach und lade die Seite neu.

Dieselbe Prüfung geht ohne Browser über die HTTP-API von Grafana:

MAC Schritt 7

```
GF_PW=$(kubectl --context obs -n monitoring get secret grafana-admin \
  -o jsonpath='{.data.admin-password}' | base64 -d)
```

Legt das Passwort nur in einer Shell-Variable ab.

MAC Schritt 8

```
curl -s --cacert observability/tls/ca.crt -u "admin:$GF_PW" \
  https://grafana.obs.internal/api/datasources \
  | jq -r '.[] | "\(.name)\t\(.type)\tStandard=\(.isDefault)"'
```

Listet alle Datenquellen. `--cacert` prüft das Zertifikat gegen deine eigene CA, statt die Prüfung mit `-k` abzuschalten.

DAS SOLLTEST DU SEHEN

Drei Zeilen, alphabetisch sortiert:

```
Alertmanager  alertmanager  Standard=false
Prometheus    prometheus    Standard=false
Thanos        prometheus    Standard=true
```

Unter **Dashboards** gibt es den Ordner **Kubernetes**.

In der Oberfläche findest du dieselbe Liste im Menü links unter **Connections → Data sources**:

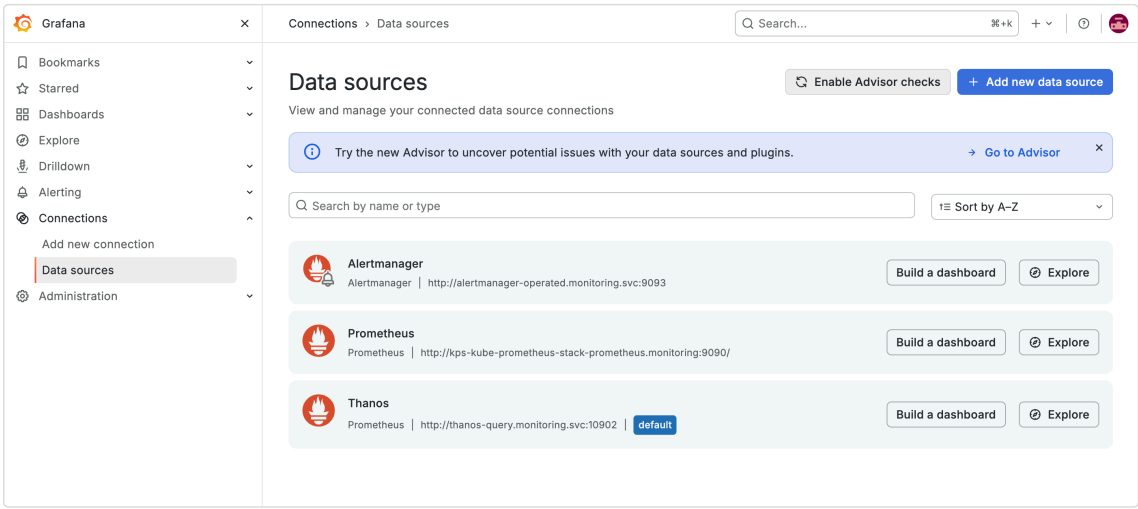


Abbildung 8: **Connections → Data sources**: Thanos trägt das Etikett `default`. Klickst du eine Datenquelle an, steht oben der Hinweis, dass sie per Provisioning angelegt wurde und sich nicht bearbeiten lässt.

MAC Schritt 9

```
unset GF_PW
```

Entfernt das Passwort aus der Shell.

6.7 Eine Abfrage über alle Cluster

Klicke im Menü links auf **Explore**. Oben links wählst du die Datenquelle, darunter liegt das Eingabefeld für die Abfrage. Steht dort ein Schalter **Builder** und **Code**, wähle **Code**, dann kannst du PromQL direkt tippen. Wähle **Thanos**, gib diese Abfrage ein und drücke `Shift-Enter` oder den blauen Knopf oben rechts:

```
count by (cluster) (up)
```

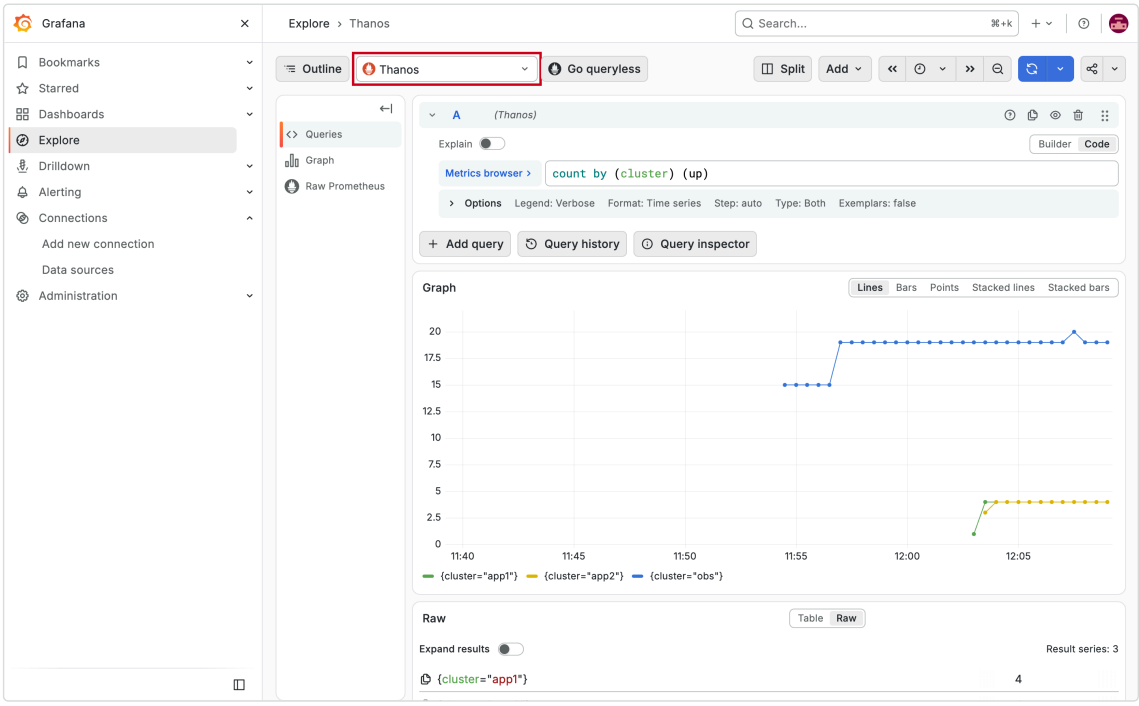


Abbildung 9: **Explore**: Datenquelle und Eingabefeld sind rot markiert. Der Graph zeigt eine Linie pro Cluster, die Tabelle darunter den aktuellen Wert.

DAS SOLLTEST DU SEHEN

Drei Zeilen: `obs`, `app1` und `app2`, jeweils mit der Zahl der Scrape-Targets. Fehlt ein Cluster, liegt das Problem vor Grafana. Prüfe dann zuerst Kapitel 5.

Wechselst du auf die Datenquelle **Prometheus**, siehst du nur `obs`. Das ist der lokale Prometheus mit seinen sechs Stunden Daten. Er ist praktisch, wenn du wissen willst, ob ein Fehler im Remote Write oder in Thanos liegt.

Öffne nun über **Dashboards** den Ordner **Kubernetes** und darin das Dashboard **Kubernetes / Compute Resources / Cluster**. Oben findest du die Auswahl **cluster**. Stell sie auf `app1`.

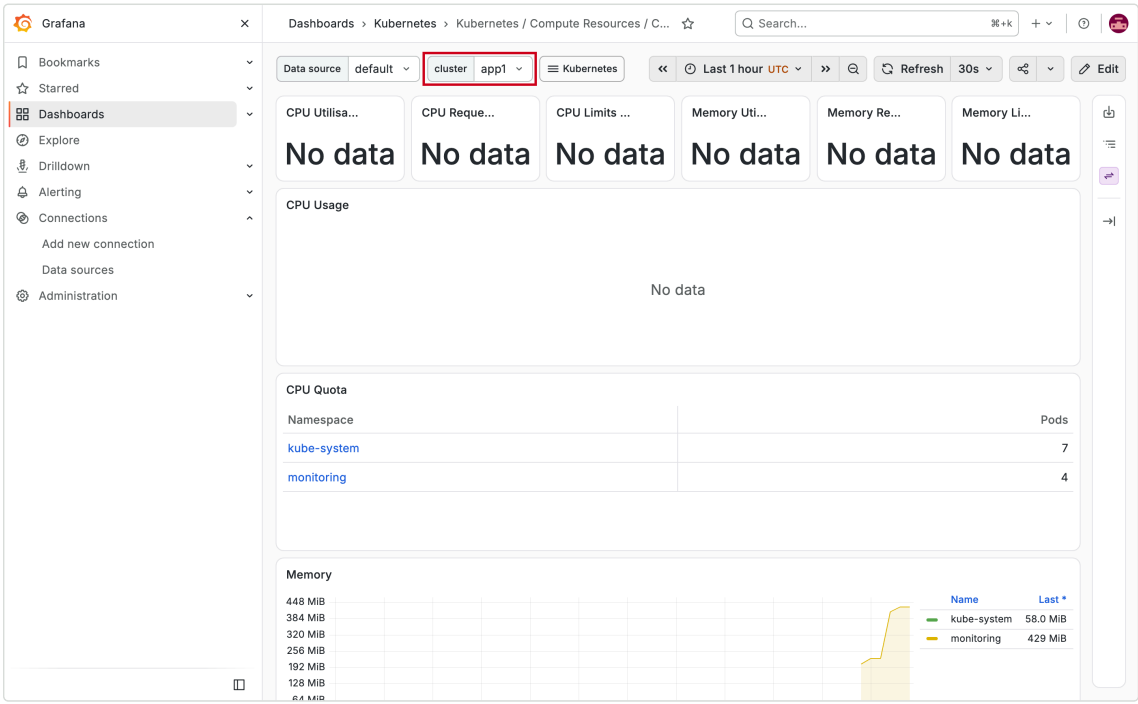


Abbildung 10: Die Auswahl **cluster** ist rot markiert. Für `app1` bleiben die oberen Kacheln und **CPU Usage** leer, **CPU Quota** und **Memory** zeigen Daten.

Die leeren Panels sind kein Fehler deines Aufbaus. Sie nutzen Aufnahmeregeln, die nur der Prometheus auf obs auswertet, also nur für `cluster="obs"`. Wie die eigenen Dashboards das vermeiden, steht in Kapitel 10.

6.8 Ändern, ohne zu klicken

Du willst ein Dashboard anpassen? Der Weg ist immer derselbe:

1. In Grafana das Dashboard öffnen, **Edit** wählen und so lange ändern, bis es passt.
2. Speichern geht nicht. Wähle stattdessen oben rechts **Export → Export as code**.
3. Klappe **Advanced options** auf und stelle **Model** auf **Classic**. Grafana 13 bietet sonst das neue Format `V2 Resource` an, das nicht zu den Dateien im Repo passt. Lass **Export for sharing externally** aus und klicke **Download file** oder **Copy to clipboard**.
4. Ersetze die Datei unter `observability/dashboards/` und prüfe sie mit `jq empty`.
5. Mit `git diff` die Änderung ansehen, mit `kubectl apply -k` ausrollen, committen.

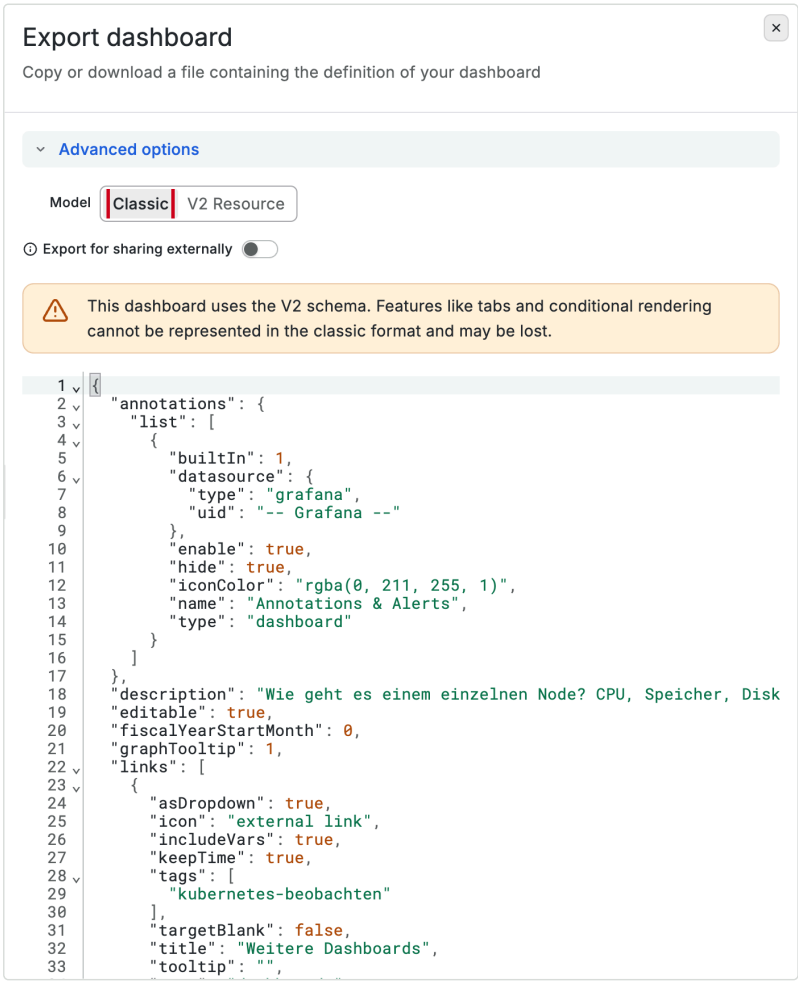


Abbildung 11: **Export as code** mit aufgeklappten **Advanced options**: **Classic** ist gewählt, der Schalter für das Teilen bleibt aus. Den Hinweis zum V2-Schema kannst du für die Dashboards dieses Buches ignorieren, sie nutzen weder Tabs noch bedingtes Anzeigen.

STOLPERFALLE

Schalte beim Export **Export for sharing externally** nicht ein. Diese Variante ersetzt die Datenquellen durch Platzhalter wie `${DS_THANOS}` und legt einen Block `__inputs` an. Den füllt nur der Import-Dialog aus, der Sidecar nicht. Das Dashboard zeigt dann überall „Datasource not found“. Die eigenen Dashboards nutzen stattdessen eine Variable `datasource`, die beim Laden die Standard-Datenquelle wählt.

Bei Datenquellen ist es noch einfacher: Datei ändern, `kubectl apply -f`, fertig. Der Sidecar lädt neu, ein Neustart ist nicht nötig.

PROBIER ES SELBST

Lege in `observability/grafana/datasources/` eine dritte Datenquelle `Thanos (roh)` an, die auf denselben Dienst zeigt, aber in `jsonData` den Eintrag `customQueryParameters: "dedup=false"` setzt. Mit ihr siehst du die Daten vor der Deduplizierung. Wende sie an und prüfe mit der API-Abfrage von oben, ob sie erscheint. Was passiert, wenn du in der neuen Datei aus Versehen auch `isDefault: true` setzt? Schau ins Log des Containers `grafana`.

WAS DU GELERNT HAST

- Grafana braucht keinen eigenen Speicher, wenn alles aus ConfigMaps kommt.
- Sidecars machen aus ConfigMaps mit einem bestimmten Label Datenquellen und Dashboards.
- Die Annotation `grafana_folder` sortiert Dashboards in Ordner, aber nur zusammen mit `foldersFromFilesStructure`.
- Das Admin-Passwort gehört in ein Secret, erzeugt mit `openssl rand`.
- Änderungen entstehen in der UI, werden aber als JSON exportiert und über das Repo ausgerollt.

KAPITEL 7

Alarme, die ankommen

Regeln über alle Cluster lösen Alarme aus, der Alertmanager sortiert sie nach Cluster und Schwere, und eine Mail kommt an. Fällt die ganze Kette aus, meldet sich ein Dienst von außen.

Dauer: 90 Minuten **Voraussetzung:** Kapitel 6: Grafana mit den Datenquellen Thanos und Alertmanager

7.1 Die Kette

Ein Dashboard schaut niemand um drei Uhr nachts an. Dafür gibt es Alarme. Damit einer ankommt, muss eine ganze Kette funktionieren:

Glied	Aufgabe in diesem Labor
Prometheus auf obs	Wertet die Standardregeln des Charts und Regeln mit `auswertung: lokal` aus. Sieht nur `obs`.
Thanos Ruler	Wertet Regeln mit `auswertung: global` aus. Fragt Thanos Query, sieht also alle Cluster.
Alertmanager	Bündelt, sortiert, unterdrückt und verschickt Alarme.
Mailpit	Fängt im Labor alle Mails ab. Später ersetzt du es durch einen echten Mailserver.
healthchecks.io	Dead Man's Switch: schlägt Alarm, wenn der Alertmanager sich **nicht** mehr meldet.

WARUM DAS WICHTIG IST

Prometheus auf obs hält nur sechs Stunden Daten von `obs` selbst. Eine Regel wie „Node-Speicher in app1 fast voll" kann er nicht auswerten, denn die Daten von app1 liegen in Thanos Receive. Der Thanos Ruler stellt dieselbe Frage an Thanos Query und bekommt die Antwort aus allen Clustern. Deshalb leben die globalen Regeln dort.

7.2 Mailpit als Test-Empfänger

Mailpit ist ein kleiner SMTP-Server mit Weboberfläche. Er nimmt jede Mail an und stellt keine einzige zu. Ideal, um Routing zu testen, ohne Postfächer zu fluten.

Die Datei `observability/central/mailpit/mailpit.yaml` enthält ein Deployment mit dem Image `axllent/mailpit:v1.33.1` und einen Service `mailpit` mit den Ports 1025 (SMTP) und 8025 (Oberfläche und API). Der Container läuft ohne Root und mit schreibgeschütztem Dateisystem, `MP_MAX_MESSAGES` löscht alte Mails.

MAC Schritt 1

```
kubectl --context obs apply -f observability/central/mailpit/
kubectl --context obs -n monitoring rollout status deploy/mailpit
```

Startet Mailpit und wartet, bis es bereit ist.

MAC Schritt 2

```
kubectl --context obs -n monitoring port-forward svc/mailpit 8025:8025
```

Läuft im Vordergrund weiter. Öffne ein zweites Terminal für die nächsten Schritte. Die Oberfläche erreichst du jetzt unter http://localhost:8025.

DAS SOLLTEST DU SEHEN

Mailpit zeigt einen leeren Posteingang. Mailpit kannst du auch für andere Projekte nutzen: Jede Anwendung, die Mails über SMTP verschickt, schickst du im Test an `mailpit.monitoring.svc:1025` und siehst das Ergebnis hier, ohne dass eine Mail das Labor verlässt.

7.3 Der Dead Man's Switch

Was passiert, wenn der Alertmanager selbst ausfällt, oder der ganze Cluster obs? Dann kommt keine Mail, und Stille sieht aus wie „alles gut". Dagegen hilft ein Alarm, der **immer** feuert: `Watchdog` aus den Standardregeln des Charts. Der Alertmanager schickt ihn jede Minute an einen Dienst außerhalb des Labors. Bleibt das Signal aus, schlägt dieser Dienst Alarm.

Lege bei healthchecks.io ein kostenloses Konto und einen Check an. Stell **Period** auf 5 Minuten und **Grace** auf 5 Minuten, und hinterlege dort deine echte Mail-Adresse oder dein Handy. Kopiere die Ping-URL des Checks, sie sieht aus wie `https://hc-ping.com/<uuid>`.

MAC Schritt 3

```
read -rs HC_URL
```

Liest die URL ein, ohne sie anzuzeigen. Füge sie ein und drücke Enter. Die URL ist ein Geheimnis: Wer sie kennt, kann deinen Check grün halten.

MAC Schritt 4

```
printf '%s' "$HC_URL" | kubectl --context obs -n monitoring \
  create secret generic alertmanager-deadmansswitch --from-file=url=/dev/stdin
unset HC_URL
```

Legt das Secret mit dem Schlüssel `url` an. Der Wert läuft über die Standardeingabe und taucht nicht in der Prozessliste auf.

STOLPERFALLE

Eine falsche URL fällt sofort auf. Antwortet die Ping-URL mit einem Fehler, etwa weil du beim Kopieren ein Zeichen verloren hast, feuern nach wenigen Minuten `AlertmanagerFailedToSendAlerts` und `AlertmanagerClusterFailedToSendAlerts` mit dem Integrationstyp `webhook`. Prüfe dann die URL mit `curl -fsS <url>`. Hast du kein Konto bei healthchecks.io, trage zum Üben `http://thanos-query.monitoring.svc:10902/-/healthy` ein. Dieser Endpunkt beantwortet jeden POST mit `200`, schützt dich aber natürlich nicht vor einem Ausfall.

STOLPERFALLE

Das Secret muss existieren, bevor du gleich `helm upgrade` ausführst. Der Alertmanager-Pod hängt es als Datei ein. Fehlt es, bleibt der Pod in `ContainerCreating` hängen, und `kubectl describe pod` meldet `secret "alertmanager-deadmansswitch" not found`.

7.4 Die Alertmanager-Konfiguration

Die Konfiguration steht in `observability/central/kps/alerting.yaml` unter `alertmanager.config`. Das Herz ist der Routing-Baum.

DATEI `observability/central/kps/alerting.yaml (Ausschnitt: route)`

```yaml
route:
  receiver: plattform
  group_by: [cluster, alertname]
  group_wait: 30s
  group_interval: 5m
  repeat_interval: 4h
  routes:
    - receiver: deadmansswitch
      matchers:
        - alertname = "Watchdog"
      group_wait: 0s
      group_interval: 1m
      repeat_interval: 1m
    - receiver: "null"
      matchers:
        - alertname = "InfoInhibitor"
    - receiver: "null"
      matchers:
        - severity = "info"
    - receiver: plattform
      matchers:
        - cluster = "obs"
      routes:
        - receiver: plattform-kritisch
          matchers:
            - severity = "critical"
          group_wait: 10s
          repeat_interval: 1h
    - receiver: apps
      matchers:
```

```
      - cluster =~ "app1|app2"
    routes:
      - receiver: apps-kritisch
        matchers:
          - severity = "critical"
        group_wait: 10s
        repeat_interval: 1h
```

Der Alertmanager geht die Routen von oben nach unten durch und nimmt die erste, die passt. Unterrouten erben alle Werte, die sie nicht selbst setzen.

Einstellung	Bedeutung
`group_by: [cluster, alertname]`	Eine Mail pro Cluster und Alarmname. Ohne `cluster` stünden `app1` und `app2` in derselben Mail, und eine Silence für einen Cluster träfe beide.
`group_wait`, `group_interval`	Wartezeit vor der ersten Mail einer Gruppe, und Abstand, wenn neue Alarme in die Gruppe kommen.
`repeat_interval`	Nach dieser Zeit kommt eine Erinnerung, solange der Alarm feuert. Kritisch jede Stunde, sonst alle vier.
Watchdog-Route	So oft wie möglich, damit healthchecks.io jede Minute ein Lebenszeichen bekommt.
`severity = "info"`	Info-Alarme siehst du in der Oberfläche, aber sie wecken niemanden.

Die Empfänger dazu sind kurz. Die Mails gehen über `mailpit.monitoring.svc:1025`, eingestellt unter `global` mit `smtp_smarthost` und `smtp_require_tls: false`.

DATEI `observability/central/kps/alerting.yaml (Ausschnitt: receivers)`

```
  receivers:
    - name: "null"
    - name: deadmansswitch
      webhook_configs:
        - url_file: /etc/alertmanager/secrets/alertmanager-deadmansswitch/url
          send_resolved: false
    - name: apps-kritisch
      email_configs:
        - to: apps@obs.internal, bereitschaft@obs.internal
          send_resolved: true
```

`url_file` liest die URL aus der Datei, die `alertmanagerSpec.secrets` aus dem Secret einhängt. So steht sie weder im Repo noch in der Helm-Release. Die übrigen Empfänger `plattform`, `plattform-kritisch` und `apps` sind genauso gebaut. Die Adressen gibt es nicht, Mailpit nimmt sie trotzdem an.

Folgealarme unterdrücken

Fällt ein ganzer Cluster aus, fallen dessen Dienste mit aus. Du willst dann eine Mail „Cluster weg“ und nicht zwanzig über einzelne Pods. Das erledigen Inhibit-Regeln: Solange ein Quell-Alarm feuert, werden passende Ziel-Alarme stumm geschaltet.

DATEI observability/central/kps/alerting.yaml (Ausschnitt: inhibit_rules)

```yaml
    inhibit_rules:
      - source_matchers:
          - alertname = "ClusterSchweigt"
        target_matchers:
          - alertname != "ClusterSchweigt"
        equal: [cluster]
      - source_matchers:
          - severity = "critical"
        target_matchers:
          - severity =~ "warning|info"
        equal: [cluster, namespace, alertname]
```

`equal: [cluster]` ist entscheidend: Nur Alarme mit **demselben** Wert für `cluster` werden unterdrückt. Ohne diese Zeile würde ein Ausfall von app2 auch alle Alarme aus app1 und obs verschlucken. Die zweite Regel stammt aus den Standardwerten des Charts, hier ergänzt um `cluster`.

STOLPERFALLE

Helm führt Werte-Dateien zusammen, aber nur Maps, keine Listen. Setzt du `inhibit_rules` in deiner Datei, ersetzt sie die Liste des Charts komplett. Deshalb stehen in `alerting.yaml` auch die drei übrigen Standardregeln, jeweils mit `cluster` in `equal`. Dasselbe gilt für `routes` und `receivers`.

Der Thanos Ruler

Den Ruler hat Kapitel 4 schon eingeschaltet: `values.yaml` wählt Regeln mit `auswertung: global`, setzt `queryEndpoints` auf Thanos Query und wertet alle 30 Sekunden aus. Es fehlt noch, wohin er Alarme schickt.

DATEI observability/central/kps/alerting.yaml (Ausschnitt: thanosRuler)

```yaml
thanosRuler:
  thanosRulerSpec:
    ruleSelectorNilUsesHelmValues: false
    ruleSelector:
      matchLabels:
        auswertung: global
    objectStorageConfig:
      existingSecret:
        name: thanos-objstore
        key: objstore.yml
    alertmanagersConfig:
      secret:
        alertmanagers:
          - api_version: v2
            scheme: http
            static_configs:
              - alertmanager-operated.monitoring.svc:9093
```

`alertmanagersConfig` schickt die Alarme an den Alertmanager. Ohne diesen Block wertet der Ruler zwar aus, aber niemand erfährt davon. `objectStorageConfig` lädt die Blöcke des Rulers alle zwei Stunden in den Bucket `thanos`, so bleibt der Alarmverlauf länger als die 24 Stunden lokaler Retention erhalten.

`ruleSelectorNilUsesHelmValues: false` verhindert, dass das Chart zusätzlich nach dem Label `release: kps` filtert.

Die Alarme, die der Ruler erzeugt, speichert er als Serie `ALERTS` in seiner eigenen kleinen TSDB. Damit Grafana sie über Thanos Query findet, hat Kapitel 4 den Ruler bereits als Endpunkt von `thanos-query` eingetragen, mit `thanos_ruler_replica` als Replika-Label.

7.5 Ausrollen

MAC Schritt 5

```
helm --kube-context obs upgrade kps \
  prometheus-community/kube-prometheus-stack \
  --version 92.18.0 -n monitoring \
  -f observability/central/kps/values.yaml \
  -f observability/central/kps/grafana.yaml \
  -f observability/central/kps/alerting.yaml \
  --wait --timeout 10m
```

Ab jetzt immer mit allen drei Dateien. Der Operator schreibt die Konfiguration in ein Secret, der Alertmanager lädt sie ohne Neustart.

MAC Schritt 6

```
kubectl --context obs -n monitoring get pods \
  -l 'app.kubernetes.io/name in (alertmanager,thanos-ruler)'
```

Zeigt die Pods von Alertmanager und Thanos Ruler.

DAS SOLLTEST DU SEHEN

Zwei Pods, beide `Running`: `alertmanager-kps-kube-prometheus-stack-alertmanager-0` mit `2/2` Containern und `thanos-ruler-kps-kube-prometheus-stack-thanos-ruler-0`, ebenfalls mit `2/2`. Der zweite Container ist jeweils der Config-Reloader.

Nach ein bis zwei Minuten wird der Check bei healthchecks.io grün. Der Watchdog kommt an.

STOLPERFALLE

Mailpit füllt sich möglicherweise sofort mit Alarmen von `obs`. Das sind Standardregeln des Charts, die auf k3s nicht passen, etwa für Komponenten, die k3s im Hauptprozess betreibt. Kapitel 4 hat die bekannten Fälle abgeschaltet. Taucht trotzdem einer auf, schalte die Regel in `values.yaml` unter `defaultRules.disabled` mit ihrem Namen ab, etwa `KubeProxyDown: true`, statt sie dauerhaft stummzuschalten.

7.6 Globale Regeln

Die Regeln liegen in `observability/rules/`, eine Datei pro Thema. Die wichtigste prüft, ob ein Cluster überhaupt noch Daten schickt.

DATEI `observability/rules/cluster.yaml (Ausschnitt)`

```yaml
apiVersion: monitoring.coreos.com/v1
kind: PrometheusRule
metadata:
  name: global-cluster
  namespace: monitoring
  labels:
    auswertung: global
spec:
  groups:
    - name: cluster
      rules:
        - alert: ClusterSchweigt
          expr: |
            group by (cluster) (max_over_time(up{cluster!=""}[6h]))
            unless
            group by (cluster) (up{cluster!=""})
          for: 3m
          labels:
            severity: critical
          annotations:
            summary: "Cluster {{ $labels.cluster }} liefert keine Metriken mehr"
            runbook_url: https://github.com/thomaszachmann/k3s/blob/main/
              observability/rules/runbooks.md#clusterschweigt
```

Die erste Hälfte liefert jeden Cluster, der in den letzten sechs Stunden irgendeine `up`-Serie hatte. `unless` entfernt alle, die **jetzt** noch eine haben. Übrig bleiben die Cluster, die verstummt sind. Das funktioniert für jeden neuen Cluster ohne Änderung an der Regel. Die Grenze: Schweigt ein Cluster länger als sechs Stunden, fällt er aus dem Fenster, und der Alarm endet von selbst. Die `runbook_url` steht in der Datei in einer Zeile, hier ist sie nur umbrochen.

Datei	Alarm	Schwere	Bedingung
`cluster.yaml`	`ClusterSchweigt`	critical	keine `up`-Serie mehr seit 3 Minuten
`knoten.yaml`	`NodeSpeicherKnapp`	warning	RAM über 90 % für 10 Minuten
	`NodeSpeicherKritisch`	critical	RAM über 97 % für 5 Minuten
	`NodeDiskKnapp`	warning	weniger als 15 % frei auf `/`
`workloads.yaml`	`PodCrashLoop`	warning	Container 10 Minuten in `CrashLoopBackOff`
	`DeploymentUnvollstaendig`	warning	weniger bereite als gewünschte Replikas für 15 Minuten
`thanos.yaml`	`ReceiveFehler`	critical	über 5 % der Schreibanfragen mit 5xx
	`ReceiveLehntAb`	warning	über 5 % mit 4xx, etwa 409 oder 429

Datei	Alarm	Schwere	Bedingung
	`ReceiveUploadFehler`	warning	Block-Upload ins S3 gescheitert
	`CompactorAngehalten`	critical	`thanos_compact_halted` ist 1

Jeder Alarm hat eine Annotation `runbook_url`. Sie zeigt auf einen Abschnitt in `observability/rules/runbooks.md`: Was bedeutet der Alarm, was prüfst du zuerst, wie behebst du es. Der Link steht in jeder Mail und in der Oberfläche des Alertmanagers. Ein Alarm ohne Runbook ist um drei Uhr nachts nur halb so viel wert.

MAC Schritt 7

```
kubectl --context obs apply -f observability/rules/
```

`kubectl` überspringt `runbooks.md`, weil es nur `.yaml`, `.yml` und `.json` liest. Prometheus auf obs ignoriert die Regeln, weil er nur `auswertung: lokal` auswählt.

MAC Schritt 8

```
kubectl --context obs get --raw \
  "/api/v1/namespaces/monitoring/services/thanos-ruler-operated:web/proxy/api/v1/rules" \
  | jq -r '.data.groups[].rules[] | "\(.name)\t\(.health)\t\(.state)"'
```

Fragt die Regel-API des Rulers über den Service-Proxy des API-Servers ab. Das spart ein `port-forward`.

DAS SOLLTEST DU SEHEN

Zehn Zeilen, jede mit `ok` als Gesundheit und meist `inactive` als Zustand. Direkt nach dem `apply` stehen sie bis zu einer Minute auf `unknown unknown`: Der Ruler hat die Dateien geladen, aber noch nicht ausgewertet. Warte kurz und frag erneut. Steht dort `err`, zeigt `.lastError` in derselben Antwort den Grund, oft ein Tippfehler im Ausdruck oder Thanos Query nicht erreichbar. Fehlt eine Regel ganz, prüfe das Label `auswertung: global`.

7.7 Ein Alarm von Anfang bis Ende

Jetzt löst du absichtlich einen Alarm aus: Du nimmst Alloy in app2 vom Netz. Ein DaemonSet lässt sich nicht auf null skalieren, also gibst du ihm einen `nodeSelector`, den kein Node erfüllt.

MAC Schritt 9

```
kubectl --context app2 -n monitoring patch daemonset alloy --type merge \
  -p '{"spec":{"template":{"spec":{"nodeSelector":{"obs-pause":"true"}}}}}'
```

Der Alloy-Pod in app2 wird beendet und nicht neu gestartet. app2 schickt keine Metriken mehr.

Mit `kubectl --context app2 -n monitoring get daemonset alloy` siehst du, dass `DESIRED` und `CURRENT` auf `0` stehen.

Jetzt heißt es warten. Die Zeiten sind im Labor gemessen, ab dem `patch`:

Nach etwa	Was passiert
0 Minuten	Alloy ist weg. Die letzten Proben von app2 liegen in Thanos Receive.
5 Minuten	Die letzten Proben sind älter als das Lookback-Fenster von Abfragen. `up{cluster="app2"}` liefert nichts mehr.
5 Minuten 19 Sekunden	Bei der nächsten Auswertung (alle 30 Sekunden) sieht der Ruler die Bedingung erfüllt. `ClusterSchweigt` steht auf `pending`.
8 Minuten 24 Sekunden	Nach `for: 3m` wechselt der Alarm auf `firing` und geht an den Alertmanager.
8 Minuten 34 Sekunden	Nach `group_wait` von 10 Sekunden verschickt der Alertmanager die Mail.

In Grafana kannst du zusehen: **Explore** mit Thanos und `count by (cluster) (up)` zeigt app2 ab etwa Minute fünf nicht mehr. Unter **Alerting → Alert rules** gibst du in das Suchfeld `datasource:Thanos` ein. Dann siehst du die Regeln des Rulers mit ihrem Zustand:

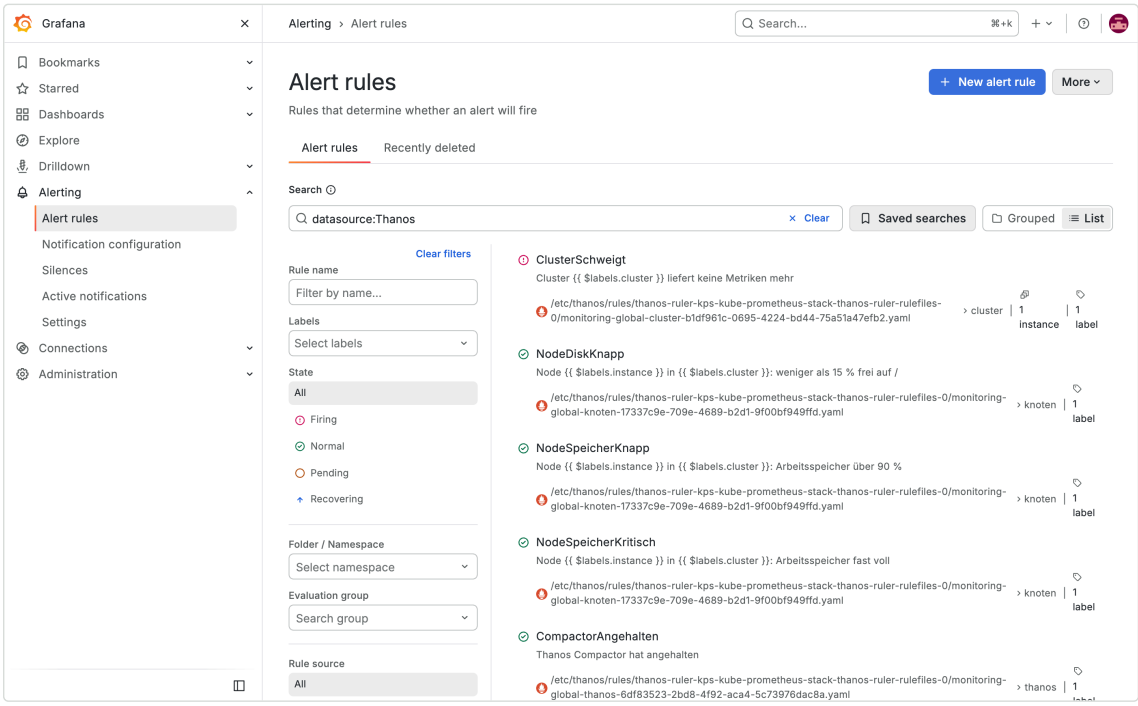


Abbildung 12: **Alerting → Alert rules** mit dem Filter `datasource:Thanos`: `ClusterSchweigt` feuert (rotes Symbol), die anderen Regeln sind grün. Grafana zeigt hier die Regeln des Rulers nur an, bearbeiten kannst du sie nur im Repo.

Den Alertmanager fragst du über seinen Ingress ab. Er ist per Basic Auth geschützt, du nutzt den Benutzer `obs` aus Kapitel 5.

MAC Schritt 10

```
AM_PW=$(kubectl --context obs -n monitoring get secret obs-remote-auth \
  -o jsonpath='{.data.password}' | base64 -d)
```

Liest das Passwort für den Benutzer `obs` in eine Variable.

MAC Schritt 11

```
curl -s --cacert observability/tls/ca.crt -u "obs:$AM_PW" \
  https://alertmanager.obs.internal/api/v2/alerts \
  | jq -r '.[] | "\(.labels.alertname)\t\(.labels.cluster)\t\(.labels.severity)\t\(.status.state)"'
```

Listet alle Alarme, die der Alertmanager gerade kennt, mit Cluster, Schwere und Zustand.

DAS SOLLTEST DU SEHEN

Eine Zeile `ClusterSchweigt app2 critical active`, dazu `Watchdog obs none active`. Der Zustand `active` heißt: weder stumm geschaltet noch unterdrückt. Folgealarme aus app2 hätten hier `suppressed`.

MAC Schritt 12

```
kubectl --context obs get --raw \
  "/api/v1/namespaces/monitoring/services/mailpit:http/proxy/api/v1/messages" \
  | jq -r '.messages[] | "\(.Subject)  ->  \([.To[].Address] | join(", "))"'
```

Fragt die API von Mailpit ab, wieder über den Service-Proxy.

DAS SOLLTEST DU SEHEN

```
[FIRING:1] ClusterSchweigt app2 (critical)  ->  apps@obs.internal,
bereitschaft@obs.internal
```

Die Route `cluster =~ "app1|app2"` und darunter `severity = "critical"` haben gegriffen.

In der Oberfläche von Mailpit unter http://localhost:8025 klickst du die Mail an und siehst sie so, wie sie im Postfach der Bereitschaft ankäme:

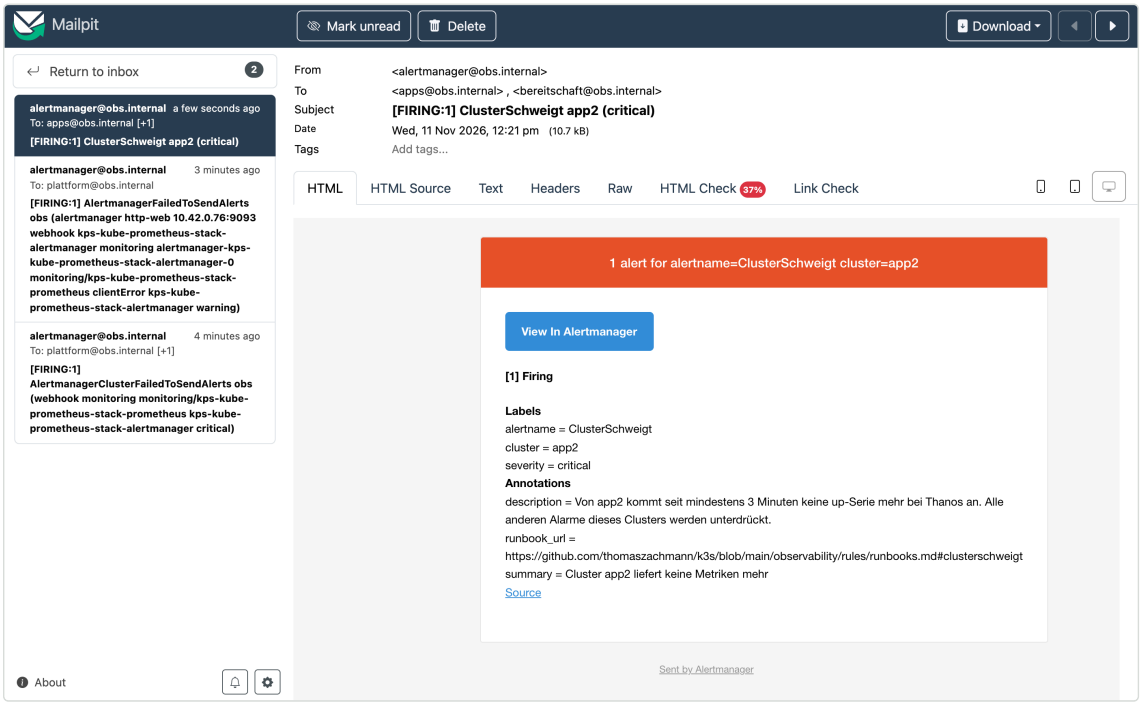


Abbildung 13: Die Alarm-Mail in Mailpit: Labels, Beschreibung und der Runbook-Link. **View In Alertmanager** führt direkt zur Oberfläche des Alertmanagers.

Der Alertmanager hat auch eine eigene Oberfläche. Öffne https://alertmanager.obs.internal und melde dich mit dem Benutzer `obs` und dem Passwort aus `obs-remote-auth` an, also demselben, das gerade in `AM_PW` steht:

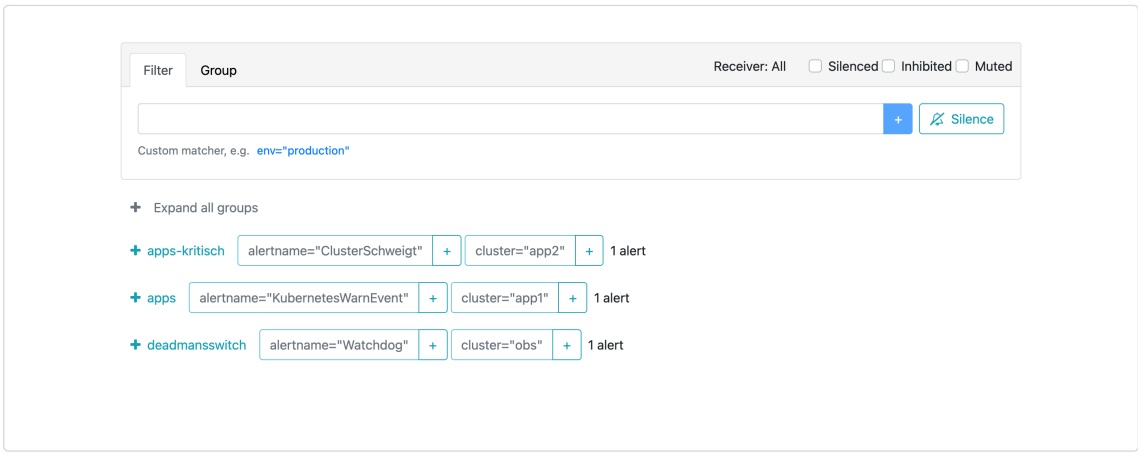


Abbildung 14: Der Alertmanager gruppiert nach Empfänger: `apps-kritisch` hat den Alarm aus app2, `deadmansswitch` den Watchdog. Der Eintrag `apps` ist ein Log-Alarm, den du in Kapitel 8 baust. Mit den Kästchen **Silenced** und **Inhibited** oben rechts blendest du auch stumme Alarme ein.

Jetzt holst du Alloy zurück. Der Patch entfernt genau den Schlüssel, den du eben gesetzt hast.

MAC Schritt 13

```
kubectl --context app2 -n monitoring patch daemonset alloy --type json \
  -p '[{"op":"remove","path":"/spec/template/spec/nodeSelector/obs-pause"}]'
```

Alloy startet wieder und schickt Metriken, auch die gepufferten aus seinem WAL, soweit es welche hat.

DAS SOLLTEST DU SEHEN

Nach ein bis drei Minuten liegt in Mailpit eine zweite Mail mit `[RESOLVED] ClusterSchweigt app2 (critical)` im Betreff. Das bewirkt `send_resolved: true`. Im Labor kam sie nach 52 Sekunden.

7.8 Stummschalten ohne amtool

Planst du Wartung, willst du keine Alarme. Dafür gibt es Silences: Sie schalten alle Alarme stumm, deren Labels zu den Matchern passen. Den Alertmanager-Befehl `amtool` brauchst du dafür nicht. Es gibt drei Wege: die Oberfläche des Alertmanagers, Grafana unter **Alerting → Silences** mit der Datenquelle Alertmanager, und die HTTP-API.

In der Oberfläche klickst du oben rechts auf **New Silence**. Trage unter **Duration** die Dauer ein, unter **Matchers** einen oder mehrere Matcher wie `cluster="app1"` (jeweils mit `+` bestätigen) und unter **Comment** den Grund. **Preview Alerts** zeigt, welche Alarme betroffen wären, **Create** legt die Silence an.

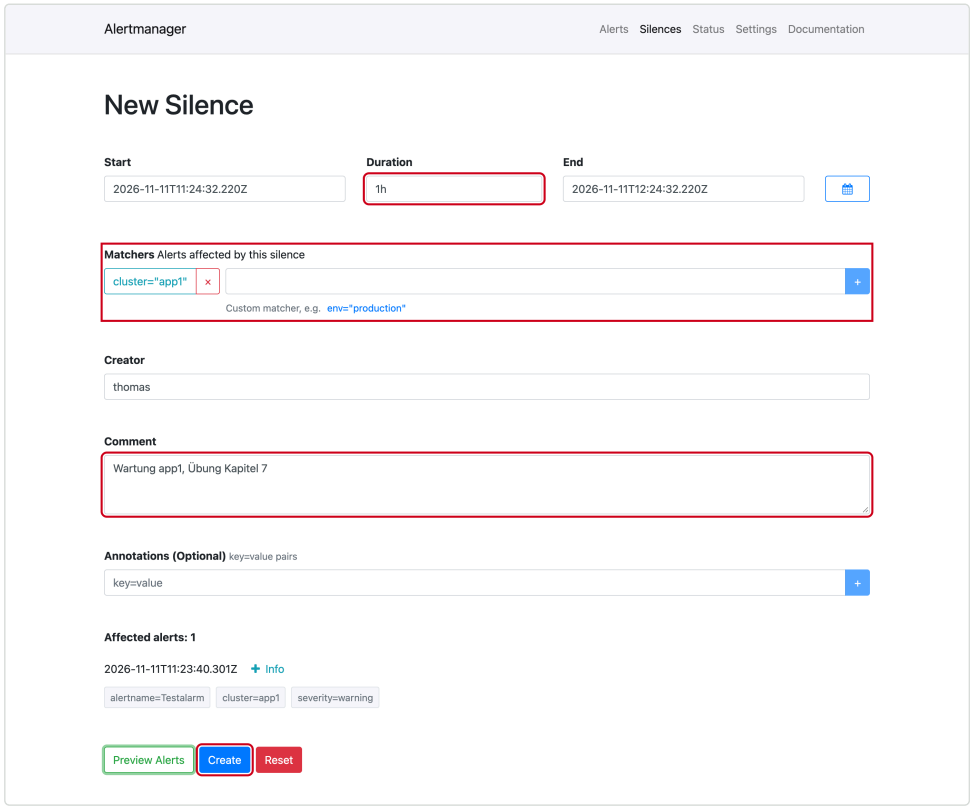


Abbildung 15: Das Formular **New Silence**: Dauer, Matcher und Kommentar sind die Pflichtangaben. **Preview Alerts** zeigt hier schon den Testalarm, den du im nächsten Schritt über die API anlegst.

Die API ist der Weg, den du in Skripte einbauen kannst. Den nutzt du jetzt.

Zum Üben brauchst du einen Alarm. Den schickst du selbst an die API, so wie es der Ruler tut:

MAC

Schritt 14

```
jq -n '[{labels: {alertname: "Testalarm", cluster: "app1",
  severity: "warning"}, annotations: {summary: "Nur ein Test"}}]' \
  | curl -s --cacert observability/tls/ca.crt -u "obs:$AM_PW" \
    -H 'Content-Type: application/json' -d @- \
    https://alertmanager.obs.internal/api/v2/alerts
```

Legt einen Alarm an. Ohne `endsAt` gilt er nach `resolve_timeout` (fünf Minuten) als erledigt, wenn ihn niemand erneut schickt.

MAC Schritt 15

```
jq -n --arg start "$(date -u +%Y-%m-%dT%H:%M:%SZ)" \
  --arg ende "$(date -u -v+1H +%Y-%m-%dT%H:%M:%SZ)" \
  '{matchers: [{name: "cluster", value: "app1", isRegex: false, isEqual: true}],
    startsAt: $start, endsAt: $ende, createdBy: "thomas",
    comment: "Wartung app1, Übung Kapitel 7"}' \
  | curl -s --cacert observability/tls/ca.crt -u "obs:$AM_PW" \
    -H 'Content-Type: application/json' -d @- \
    https://alertmanager.obs.internal/api/v2/silences
```

Schaltet für eine Stunde alles mit `cluster="app1"` stumm. `date -v+1H` ist die macOS-Schreibweise, unter Linux hieße es `date -u -d '+1 hour' ...`. Die Antwort enthält die `silenceID`.

MAC Schritt 16

```
curl -s --cacert observability/tls/ca.crt -u "obs:$AM_PW" \
  https://alertmanager.obs.internal/api/v2/alerts \
  | jq -r '.[] | select(.labels.alertname=="Testalarm")
      | "\(.status.state) \(.status.silencedBy)"'
```

Zeigt, ob der Testalarm stumm ist und durch welche Silence.

DAS SOLLTEST DU SEHEN

`suppressed` und eine Liste mit der ID deiner Silence. In Mailpit kommt keine Mail für `Testalarm` an.

Alle Silences liefert `GET /api/v2/silences`. Mit `curl -X DELETE .../api/v2/silence/<id>` beendest du eine vorzeitig, beachte das Singular `silence` im Pfad.

STOLPERFALLE

Eine Silence auf `cluster="app1"` verschluckt **alles** aus app1, auch einen echten Ausfall während der Wartung. Gib deshalb immer ein Ende an und halte Silences so eng wie möglich, etwa mit einem zusätzlichen Matcher auf `alertname` oder `namespace`.

MAC Schritt 17

```
unset AM_PW
```

Entfernt das Passwort aus der Shell.

PROBIER ES SELBST

Prüfe die Inhibit-Regel selbst. Schicke wie oben zwei Alarme gleichzeitig an die API: `ClusterSchweigt` mit `cluster: "app2"` und `severity: "critical"`, dazu `Testalarm` mit `cluster: "app2"`. Welcher Zustand steht beim Testalarm, und was steht in `.status.inhibitedBy`? Wiederhole es mit `cluster: "app1"` beim Testalarm. Warum wird er jetzt zugestellt?

WAS DU GELERNT HAST

- Globale Regeln gehören in den Thanos Ruler, weil nur er über Thanos Query alle Cluster sieht.
- `group_by` und `equal` brauchen das Label `cluster`, sonst vermischen sich Alarme verschiedener Cluster.
- Inhibit-Regeln machen aus einem Clusterausfall eine Mail statt zwanzig.
- Ein Watchdog mit Dead Man's Switch meldet, wenn die Alarmkette selbst ausfällt. Die URL gehört in ein Secret.
- Silences legst du über Oberfläche oder API an, immer mit Ende und so eng wie möglich.

KAPITEL 8

Logs mit Loki und Alloy

Die Logs aller Pods und die Kubernetes-Events aus allen drei Clustern landen in Loki auf `obs`*. Du fragst sie mit LogQL ab, und ein Log-Alarm erreicht den zentralen Alertmanager.*

Dauer: 90 Minuten **Voraussetzung:** Kapitel 3, 5, 6 und 7

8.1 Was du aufbaust

Metriken sagen dir, dass ein Pod neu startet. Warum er neu startet, steht im Log. Bisher liegen diese Logs nur auf dem jeweiligen Node, und nach einem Neustart des Pods sind sie weg. Jetzt sammelst du sie zentral.

Komponente	Wo	Aufgabe
Loki	`obs`	Speichert Logs im Bucket `loki`, ein Mandant pro Cluster
Alloy (`logs.alloy`)	alle drei	Liest Pod-Logs und Events über die Kubernetes-API, schickt sie an Loki
Loki-Ruler	`obs`	Wertet LogQL-Regeln aus, schickt Alarme an den Alertmanager
Grafana	`obs`	Datenquelle Loki, fragt alle Mandanten zugleich ab

Loki unterscheidet sich in einem Punkt grundlegend von einer Volltextsuche: Es indiziert nur die **Labels** eines Log-Streams, nicht den Text. Ein Stream ist die Menge aller Zeilen mit genau derselben Label-Kombination. Bei einer Abfrage wählt Loki zuerst die Streams über ihre Labels aus und durchsucht dann nur deren Zeilen. Das macht Loki billig im Betrieb, verlangt aber Disziplin bei den Labels.

8.2 Wenige, stabile Labels

Jede neue Label-Kombination erzeugt einen neuen Stream. Für jeden Stream hält Loki eigene Puffer im Speicher und schreibt eigene kleine Blöcke ins S3. Ein Label wie `user_id` oder `trace_id` würde aus einem Stream Tausende machen. Dieses Buch setzt deshalb genau fünf Labels:

Label	Herkunft	Werte im Labor
`cluster`	Umgebungsvariable `CLUSTER_NAME` in Alloy	3
`namespace`	Namespace des Pods	rund 5 pro Cluster
`pod`	Name des Pods	rund 20 pro Cluster

Label	Herkunft	Werte im Labor
`container`	Name des Containers	meist 1 bis 2 pro Pod
`app`	Pod-Label `app.kubernetes.io/name`, sonst `app`	rund 15

`pod` ist das einzige Label mit spürbarer Veränderung, weil jeder neue Pod einen neuen Namen bekommt. Das ist gewollt, denn nach dem Pod suchst du am häufigsten. Alles andere bleibt im Text der Zeile.

In Grafana wirst du drei weitere Labels sehen, die du nicht selbst gesetzt hast. `job` und `instance` hängt Alloy an: `job` ist für alle Pod-Logs gleich, `instance` ist `namespace/pod:container`. `service_name` leitet Loki selbst aus `app` ab. Keines davon erzeugt neue Streams, denn ihr Wert ergibt sich aus den fünf Labels oben.

Die Logstufe, also `info` oder `error`, legt Alloy als **strukturierte Metadaten** mit dem Namen `level` ab. Sie hängen an der einzelnen Zeile, erzeugen keinen neuen Stream und lassen sich trotzdem filtern. Loki rät die Stufe zusätzlich selbst und legt sie als `detected_level` daneben. Meist sind beide gleich. `level` stammt aus deiner Regel in Alloy und ist deshalb der verlässlichere Wert.

WARUM DAS WICHTIG IST

Zu viele Streams sind der häufigste Grund, warum Loki langsam wird oder Daten ablehnt. Die Werte-Datei unten begrenzt deshalb jeden Mandanten auf 5000 Streams. Bei Verstoß antwortet Loki mit HTTP 429, und Alloy meldet das im Log. Ein solcher Fehler zeigt fast immer ein falsch gewähltes Label an, kein zu kleines Limit.

8.3 Loki auf obs installieren

Bucket `loki` und Secret `loki-s3` hast du in Kapitel 3 angelegt. Die Werte-Datei folgt dem Muster der anderen Komponenten: ein Prozess, S3 als Speicher, keine Caches.

DATEI `observability/central/loki/values.yaml (Ausschnitt)`

```yaml
deploymentMode: Monolithic

loki:
  auth_enabled: true
  commonConfig:
    replication_factor: 1
  storage:
    type: s3
    s3:
      endpoint: http://s3.monitoring.svc.cluster.local:8333
      accessKeyId: ${AWS_ACCESS_KEY_ID}
      secretAccessKey: ${AWS_SECRET_ACCESS_KEY}
      s3ForcePathStyle: true
```

```
    querier:
      multi_tenant_queries_enabled: true
    limits_config:
      retention_period: 168h
      max_global_streams_per_user: 5000
    rulerConfig:
      alertmanager_url: http://alertmanager-operated.monitoring.svc:9093
      # ... Regeln lokal unter /rules, siehe Abschnitt Log-Alarme

  singleBinary:
    replicas: 1
    extraEnvFrom:
      - secretRef:
          name: loki-s3
```

`auth_enabled: true` schaltet die Mandanten ein. Jede Anfrage braucht dann den Header `X-Scope-OrgID`, sonst lehnt Loki sie ab. `multi_tenant_queries_enabled` erlaubt Abfragen mit mehreren Mandanten, getrennt durch `|`. Die S3-Schlüssel setzt Loki beim Start aus den Umgebungsvariablen ein, das Chart startet Loki dafür mit `-config.expand-env=true`. Die vollständige Datei schaltet außerdem Gateway, Caches und Canary ab, legt das Schema `v13` fest und konfiguriert den Compactor.

Die Helm-Charts für Loki und Tempo pflegt inzwischen die Community in einem eigenen Repository, nicht mehr Grafana Labs im Repository `grafana`, aus dem Alloy stammt. Trag es einmal ein:

MAC Schritt 1

```
helm repo add grafana-community https://grafana-community.github.io/helm-charts
helm repo update grafana-community
```

Ohne diesen Schritt meldet der nächste Befehl `repo grafana-community not found`.

MAC Schritt 2

```
helm --kube-context obs upgrade --install loki grafana-community/loki \
  -n monitoring --version 21.0.0 \
  -f observability/central/loki/values.yaml --wait --timeout 10m
```

Installiert Loki als StatefulSet `loki` mit einem Pod. Im Pod läuft neben Loki ein zweiter Container, der Regeln aus ConfigMaps einsammelt. Den brauchst du für die Alarme.

MAC Schritt 3

```
kubectl --context obs -n monitoring logs loki-0 -c loki --tail=30
```

Zeigt die letzten Zeilen des Loki-Containers.

DAS SOLLTEST DU SEHEN

In den ersten Sekunden nach dem Start meldet Loki `failed to fast-join the memberlist cluster` und `empty ring`. Loki sucht dabei sich selbst über einen DNS-Namen, den Kubernetes erst vergibt, wenn der Pod bereit ist. Das legt sich von allein. Danach kommen keine Zeilen mit `level=error` mehr. Meldungen wie `AccessDenied` oder `NoSuchBucket` weisen auf Konto oder Bucket aus Kapitel 3 hin. Hinweise des Rulers auf fehlende Regeln sind bis zum Abschnitt über Alarme normal, der Ordner `/rules` ist noch leer.

Jetzt prüfst du die Mandanten. Öffne dafür ein zweites Terminal:

MAC — Schritt 4

```
kubectl --context obs -n monitoring port-forward svc/loki 3100:3100
```

Macht Loki unter `localhost:3100` erreichbar, bis du mit `Ctrl-C` beendest.

MAC — Schritt 5

```
curl -s localhost:3100/loki/api/v1/labels
```

Eine Anfrage ohne Mandant. Erwartet ist die Antwort `no org id`: Loki verweigert Anfragen ohne Header.

MAC — Schritt 6

```
curl -s -H "X-Scope-OrgID: obs" localhost:3100/loki/api/v1/labels
```

Dieselbe Anfrage für den Mandanten `obs`. Jetzt kommt `{"status":"success"}` zurück, noch ohne Labels, denn Alloy schreibt erst ab dem nächsten Abschnitt.

8.4 Alloy schickt die Logs

Alloy liest die Logs nicht aus Dateien auf dem Node, sondern über die Kubernetes-API, genau wie `kubectl logs`. Dafür braucht Alloy keine Root-Rechte und keinen Zugriff auf Dateien des Nodes. Den Mount von `/var/log` aus der Werte-Datei von Kapitel 5 nutzt `logs.alloy` nicht. Für Ein-Node-Cluster wie im Labor ist das die einfachste Lösung. Bei großen Clustern mit viel Log-Volumen belastet dieser Weg den API-Server, dort liest man besser die Dateien unter `/var/log/pods`.

DATEI `observability/cluster/alloy/logs.alloy (Ausschnitt)`

```
discovery.kubernetes "logs_pods" {
  role = "pod"
  // ... nur Pods auf dem eigenen Node
}

discovery.relabel "logs_pods" {
```

```
  targets = discovery.kubernetes.logs_pods.targets
  rule {
    source_labels = ["__meta_kubernetes_namespace"]
    target_label  = "namespace"
  }
  // ... ebenso pod, container und app
}

loki.source.kubernetes "logs_pods" {
  targets    = discovery.relabel.logs_pods.output
  forward_to = [loki.process.logs_pods.receiver]
}

loki.process "logs_pods" {
  // ... zieht level aus der Zeile, legt es als Metadaten ab
  forward_to = [loki.write.logs_loki.receiver]
}

loki.source.kubernetes_events "logs_events" {
  job_name   = "kubernetes-events"
  log_format = "logfmt"
  forward_to = [loki.write.logs_loki.receiver]
}

loki.write "logs_loki" {
  endpoint {
    url       = "https://loki.obs.internal/loki/api/v1/push"
    tenant_id = sys.env("CLUSTER_NAME")
    basic_auth {
      username = sys.env("OBS_USER")
      password = sys.env("OBS_PASSWORD")
    }
    tls_config {
      ca_file = "/etc/obs-ca/ca.crt"
    }
  }
  external_labels = { cluster = sys.env("CLUSTER_NAME") }
}
```

Lies die Kette von oben nach unten: Pods finden, Labels setzen, Logs lesen, Logstufe herausziehen, schreiben. `tenant_id` setzt den Header `X-Scope-OrgID` auf den Clusternamen, `external_labels` hängt an jeden Stream das Label `cluster`. Die Events laufen ohne Umweg direkt zu `loki.write`. Alle Namen beginnen mit `logs_`, damit sie sich nicht mit `metrics.alloy` aus Kapitel 5 überschneiden. Die vollständige Datei liest außerdem nur Pods auf dem eigenen Node und verwirft beendete Pods.

Auch `obs` schickt über `https://loki.obs.internal` und nicht direkt an den Service. So ist `logs.alloy` in allen drei Clustern identisch, und der Weg über Traefik wird bei jedem Cluster gleich geprüft.

Wie in Kapitel 5 liegen alle `.alloy`-Dateien eines Clusters in der ConfigMap `alloy-config`, und Alloy lädt sie gemeinsam. In `app1` und `app2` kommt `logs.alloy` zu `metrics.alloy` dazu:

MAC Schritt 7

```
kubectl --context app1 -n monitoring create configmap alloy-config \
  --from-file=observability/cluster/alloy/metrics.alloy \
  --from-file=observability/cluster/alloy/logs.alloy \
  --dry-run=client -o yaml | kubectl --context app1 apply -f -
kubectl --context app1 -n monitoring rollout restart ds/alloy
kubectl --context app1 -n monitoring rollout status ds/alloy
```

Baut die ConfigMap für app1 lokal neu, ersetzt die alte, startet Alloy neu und wartet, bis der Pod wieder läuft. Von selbst bemerkt Alloy die Änderung auch, aber erst nach bis zu einer Minute.

MAC Schritt 8

```
kubectl --context app2 -n monitoring create configmap alloy-config \
  --from-file=observability/cluster/alloy/metrics.alloy \
  --from-file=observability/cluster/alloy/logs.alloy \
  --dry-run=client -o yaml | kubectl --context app2 apply -f -
kubectl --context app2 -n monitoring rollout restart ds/alloy
kubectl --context app2 -n monitoring rollout status ds/alloy
```

Dasselbe für app2.

Auf `obs` läuft Alloy noch gar nicht, denn Metriken sammelt dort Prometheus. Die Voraussetzungen sind aber da: das Secret `obs-remote-auth` mit dem Benutzer `obs` aus Kapitel 5 und die ConfigMap `obs-ca` aus Kapitel 2.

MAC Schritt 9

```
kubectl --context obs -n monitoring create configmap alloy-config \
  --from-file=observability/cluster/alloy/logs.alloy \
  --dry-run=client -o yaml | kubectl --context obs apply -f -
```

Auf `obs` enthält die ConfigMap nur `logs.alloy`, ohne `metrics.alloy`.

MAC Schritt 10

```
helm --kube-context obs upgrade --install alloy grafana/alloy \
  -n monitoring --version 1.15.0 \
  -f observability/cluster/alloy/values.yaml --wait
```

Dieselbe Werte-Datei wie in den App-Clustern. Was Alloy tut, bestimmen allein die Dateien in der ConfigMap.

MAC Schritt 11

```
kubectl --context app1 -n monitoring logs ds/alloy -c alloy --tail=50 \
  | grep -E 'logs_|level=error'
```

Filtert die Meldungen der neuen Komponenten und alle Fehler.

DAS SOLLTEST DU SEHEN

Meldungen mit `component_id=loki.source.kubernetes.logs_pods` oder `loki.write.logs_loki`, aber keine Zeilen mit `level=error`. `server returned HTTP status 401` heißt, die Basic-Auth-Daten passen nicht. `x509: certificate signed by unknown authority` heißt, die ConfigMap `obs-ca` fehlt oder ist nicht eingebunden.

MAC Schritt 12

```
curl -s -H "X-Scope-OrgID: obs"  localhost:3100/loki/api/v1/label/cluster/values
curl -s -H "X-Scope-OrgID: app1" localhost:3100/loki/api/v1/label/cluster/values
curl -s -H "X-Scope-OrgID: app2" localhost:3100/loki/api/v1/label/cluster/values
```

Fragt für jeden Mandanten die Werte des Labels `cluster` ab. Der Port-Forward aus dem zweiten Terminal muss noch laufen.

DAS SOLLTEST DU SEHEN

Drei Zeilen, jede mit genau ihrem eigenen Clusternamen:

```
{"status":"success","data":["obs"]}
{"status":"success","data":["app1"]}
{"status":"success","data":["app2"]}
```

Jeder Mandant sieht nur seine eigenen Daten. Zum Vergleich: Mit `-H "X-Scope-OrgID: obs|app1|app2"` bekommst du alle drei Werte in einer Antwort.

STOLPERFALLE

Den Header `X-Scope-OrgID` setzt der Absender selbst. Mit den Zugangsdaten von `app1` könnte jemand also auch als `app2` schreiben. Basic Auth schützt vor Fremden, nicht vor einem falsch konfigurierten eigenen Cluster. Wie man den Mandanten fest an den Benutzer bindet, zeigt Kapitel 13.

8.5 Logs in Grafana

Die Datenquelle legst du wie in Kapitel 6 als ConfigMap ab:

DATEI `observability/grafana/datasources/loki.yaml (Ausschnitt)`

```
    - name: Loki
      uid: loki
      type: loki
      url: http://loki.monitoring.svc:3100
      jsonData:
        httpHeaderName1: X-Scope-OrgID
        derivedFields:
          - name: TraceID
            matcherType: regex
            matcherRegex: 'trace_?id"?[=:]\s*"?(\w+)'
```

```
            datasourceUid: tempo
            url: '$${__value.raw}'
        secureJsonData:
          httpHeaderValue1: obs|app1|app2
```

Grafana schickt bei jeder Abfrage den Header `X-Scope-OrgID: obs|app1|app2` und sieht damit alle drei Mandanten. Das Feld `derivedFields` macht aus einer Trace-ID im Log-Text einen Link zu Tempo. Er funktioniert ab Kapitel 9. In Provisioning-Dateien ersetzt Grafana `$NAME` durch Umgebungsvariablen, deshalb steht dort `$$` für ein einzelnes Dollarzeichen.

MAC Schritt 13

```
kubectl --context obs apply -f observability/grafana/datasources/loki.yaml
```

Der Sidecar von Grafana bemerkt die ConfigMap und lädt die Datenquelle nach, ohne Neustart.

Öffne grafana.obs.internal, dann **Explore**, wähle **Loki** und gib ein:

```
{cluster="app1", namespace="kube-system"}
```

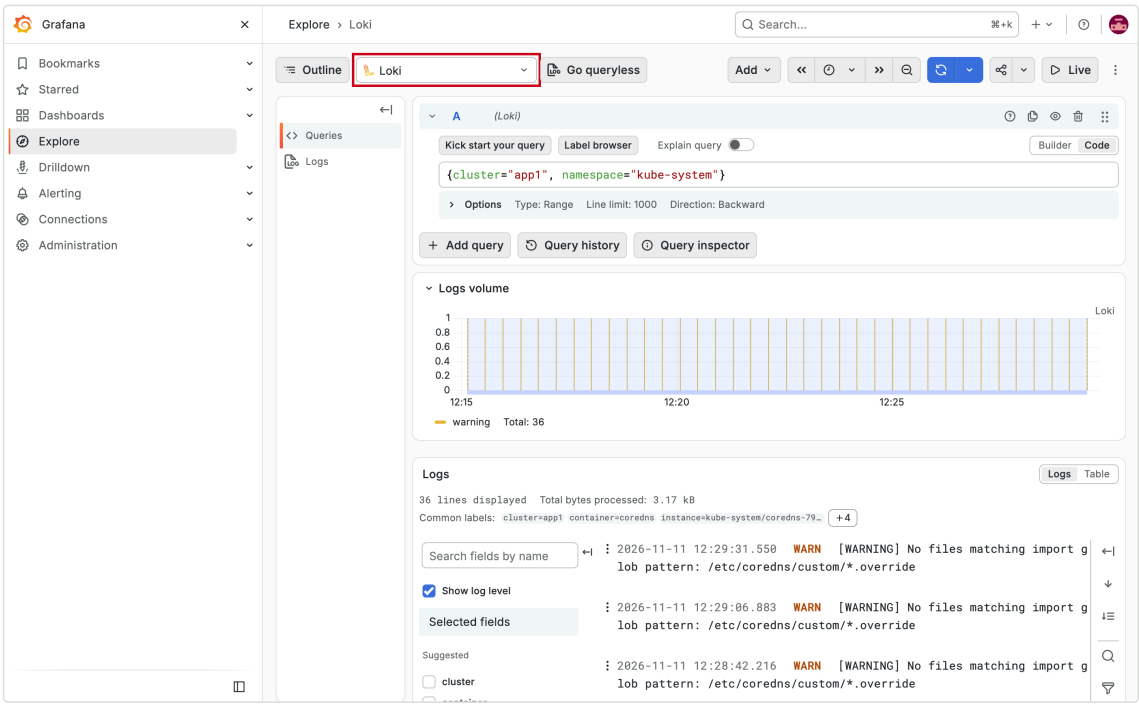


Abbildung 16: **Explore** mit der Datenquelle **Loki**: oben die Menge der Zeilen über die Zeit, darunter die Zeilen selbst. Die Warnung von CoreDNS über fehlende `*.override`-Dateien ist harmlos, k3s sucht diese Dateien immer.

DAS SOLLTEST DU SEHEN

Log-Zeilen von CoreDNS, Traefik und dem Metrics-Server aus `app1`. Klappst du eine Zeile auf, stehen darunter die Labels und die strukturierten Metadaten `level` und `detected_level`. Zusätzlich zeigt Grafana das Label `__tenant_id__`. Loki fügt es bei Abfragen über mehrere Mandanten hinzu.

Über **Label browser** über dem Eingabefeld klickst du dir Abfragen zusammen, ohne die Labels auswendig zu kennen. Schalte für LogQL wie bei PromQL auf **Code**.

8.6 LogQL in vier Schritten

LogQL baut auf denselben Ideen auf wie PromQL. Eine Abfrage beginnt immer mit einem Stream-Selektor in geschweiften Klammern. Danach folgen, durch `|` getrennt, Filter und Verarbeitungsschritte.

1. Streams wählen. Mindestens ein Label muss genau passen oder ein nicht leeres Muster haben:

```
{namespace="monitoring", app=~"alloy|loki"}
```

2. Zeilen filtern. `|=` sucht Text, `!=` schließt aus, `|~` nimmt einen regulären Ausdruck. Filter sind schnell, weil Loki dafür nichts zerlegen muss:

```
{cluster="app1"} |~ "(?i)error|timeout" != "level=debug"
```

3. Zeilen zerlegen. `| logfmt` und `| json` machen aus Feldern im Text vorübergehende Labels, nach denen du filtern kannst:

```
{namespace="monitoring", app="alloy"} | logfmt | level="error"
```

Auf strukturierte Metadaten filterst du ohne Parser, weil sie schon da sind:

```
{cluster=~"app1|app2"} | level="error"
```

4. Zählen statt lesen. Mit `count_over_time` und `rate` wird aus Log-Zeilen eine Zeitreihe. Das ist die Grundlage für Diagramme und Alarme:

```
sum by (cluster, namespace) (
  count_over_time({namespace=~".+"} |~ "(?i)error" [5m])
)
```

Mit `| line_format "{{.msg}}"` zeigst du am Ende nur noch die Felder, die dich interessieren.

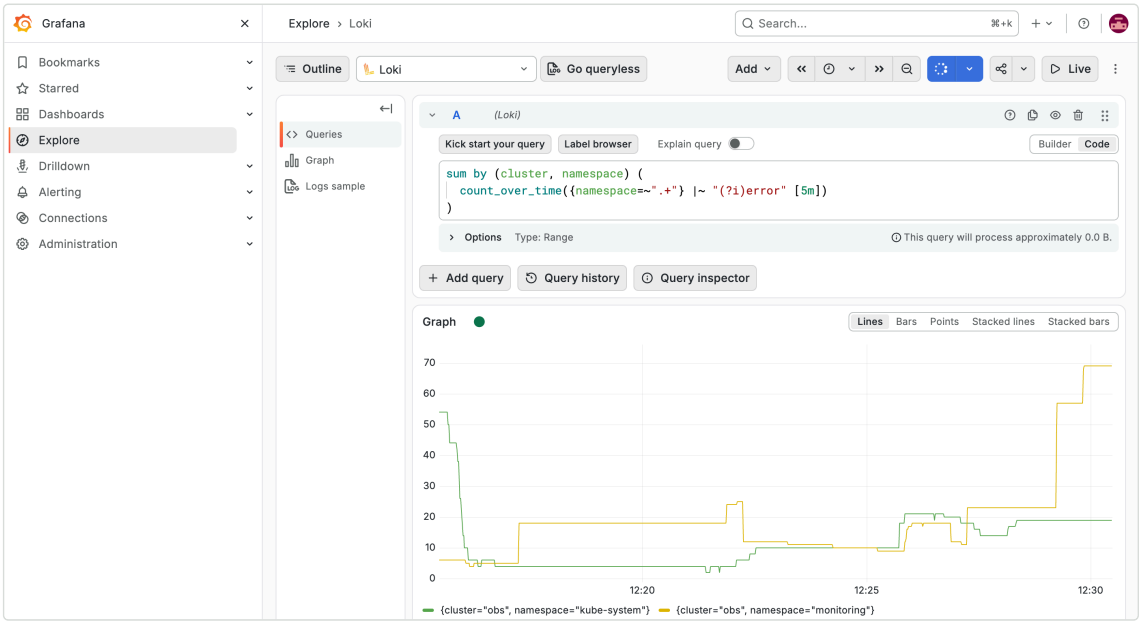


Abbildung 17: Die Zähl-Abfrage aus Schritt 4 in **Explore**: Aus Log-Zeilen wird eine Kurve pro Cluster und Namespace. Genau so eine Kurve wertet gleich der Loki-Ruler aus.

STOLPERFALLE

`{cluster="app1"} |= "error"` liest alle Logs von `app1` im gewählten Zeitraum. Bei einer Stunde ist das im Labor kein Problem, bei 30 Tagen schon. Schränke zuerst über Labels ein, dann über Text. Grafana zeigt unter **Query inspector** an, wie viele Bytes Loki gelesen hat.

8.7 Kubernetes-Events

Events sind die Notizen, die Kubernetes zu Objekten schreibt: Pod geplant, Image gezogen, Container neu gestartet, Speicher voll. Der API-Server löscht sie nach einer Stunde. In Loki bleiben sie so lange wie alle anderen Logs.

```
{job="kubernetes-events"} | logfmt | type="Warning"
```

Welche Gründe gab es in der letzten Stunde, je Cluster?

```
sum by (cluster, reason) (
  count_over_time({job="kubernetes-events"} | logfmt | type="Warning" [1h])
)
```

Die Felder im Text heißen unter anderem `kind`, `name`, `reason`, `type` und `msg`. In einem frischen Cluster gibt es kaum Warnungen. Fehlen auch die Events mit `type="Normal"`, prüfe in den Alloy-Logs die Komponente `logs_events`.

8.8 Log-Alarme über den Loki-Ruler

Manches siehst du nur im Log, etwa eine Häufung von Fehlermeldungen oder bestimmte Events. Dafür hat Loki einen eigenen Ruler. Er wertet LogQL-Regeln aus und schickt Alarme an denselben Alertmanager wie Prometheus und der Thanos Ruler. Routing, Mailpit und Stummschalten aus Kapitel 7 gelten also auch hier.

Der Ruler arbeitet pro Mandant. Eine Regel im Mandanten `app1` sieht nur die Logs von `app1`. Abfragen über mehrere Mandanten wie in Grafana sind für Regeln nicht vorgesehen. Deshalb lädst du dieselbe Regeldatei dreimal, einmal pro Mandant.

DATEI `observability/rules/loki/log-alarme.yaml (Ausschnitt)`

```yaml
groups:
  - name: logs
    rules:
      - alert: LogFehlerHaeufig
        expr: |
          sum by (cluster, namespace, pod) (
            rate({namespace=~".+"} |~ "(?i)(error|exception|panic)" [5m])
          ) > 1
        for: 10m
        labels:
          severity: warning
      - alert: KubernetesWarnEvent
        expr: |
          sum by (cluster, namespace, reason) (
            count_over_time(
              {job="kubernetes-events"} | logfmt
                | type="Warning"
                | reason=~"BackOff|FailedScheduling|FailedMount|OOMKilling|Evicted" [10m]
            )
          ) > 0
        for: 5m
        labels:
          severity: warning
```

Das Format ist dasselbe wie bei Prometheus-Regeln, nur steht in `expr` LogQL. Weil jede Regel nach `cluster` gruppiert, trägt der Alarm dieses Label, und das Routing aus Kapitel 7 greift. Die erste Regel ist bewusst grob. Passe das Muster an deine Anwendungen an, bevor du dich auf sie verlässt.

DATEI `observability/rules/loki/kustomization.yaml (Ausschnitt)`

```yaml
generatorOptions:
  disableNameSuffixHash: true
  labels:
    loki_rule: "1"
configMapGenerator:
  - name: loki-regeln-app1
    files: [log-alarme.yaml]
    options:
      annotations: {loki_rule_folder: app1}
  # ... ebenso für obs und app2
```

Kustomize erzeugt drei ConfigMaps aus derselben Datei. Der Regel-Sidecar im Loki-Pod sucht ConfigMaps mit dem Label `loki_rule` und legt sie in den Ordner aus der Annotation `loki_rule_folder`. Der Ruler erwartet genau diese Struktur: `/rules/<mandant>/<datei>`.

MAC Schritt 14

```
kubectl --context obs apply -k observability/rules/loki/
```

Legt die drei ConfigMaps im Namespace `monitoring` an.

MAC Schritt 15

```
kubectl --context obs -n monitoring exec loki-0 -c loki-sc-rules -- ls -R /rules
```

Zeigt, was der Sidecar abgelegt hat. Der Befehl läuft im Sidecar, nicht im Container `loki`: Das Loki-Image enthält kein `ls`. Beide Container teilen sich das Verzeichnis `/rules`.

DAS SOLLTEST DU SEHEN

Drei Ordner `obs`, `app1` und `app2`, in jedem die Datei `log-alarme.yaml`. Der Ruler liest den Ordner regelmäßig neu ein, ein Neustart ist nicht nötig.

Dateien im Ordner heißen noch nicht, dass der Ruler sie geladen hat. Das verrät dir seine API:

MAC Schritt 16

```
curl -s -H "X-Scope-OrgID: app1" \
  localhost:3100/prometheus/api/v1/rules | jq -c '.data.groups[].rules[].name'
```

Fragt den Ruler, welche Regeln er für den Mandanten `app1` geladen hat. Der Port-Forward muss noch laufen. Die Ruler-API nimmt nur einen Mandanten pro Anfrage, `obs|app1|app2` liefert hier nichts.

DAS SOLLTEST DU SEHEN

Die Namen `"LogFehlerHaeufig"` und `"KubernetesWarnEvent"`. Fehlen sie, obwohl die Datei im Ordner liegt, hat Loki sie nicht lesen können. Die Ursache steht dann im Log von `loki-0`, meist ein YAML- oder LogQL-Fehler.

Jetzt löst du einen Alarm aus. Ein Pod, der sofort endet, landet in `CrashLoopBackOff`, und Kubernetes schreibt `BackOff`-Events:

MAC Schritt 17

```
kubectl --context app1 run kaputt --image=busybox:1.36 \
  --restart=Always -- sh -c "exit 1"
```

Startet einen Pod in `default`, der immer wieder abstürzt.

MAC Schritt 18

```
curl -s -H "X-Scope-OrgID: app1" \
  localhost:3100/prometheus/api/v1/alerts | jq '.data.alerts[].labels'
```

Fragt den Ruler nach aktiven Alarmen des Mandanten `app1`. Der Port-Forward muss noch laufen.

DAS SOLLTEST DU SEHEN

Nach etwa einer Minute ein Alarm `KubernetesWarnEvent` mit `cluster: "app1"`, `namespace: "default"` und `reason: "BackOff"`, zuerst im Zustand `pending`. Im Labor wechselte er nach 6 Minuten auf `firing`, 30 Sekunden später lag in Mailpit `[FIRING:1] KubernetesWarnEvent app1 (default loki BackOff warning)`. Kommt keine Mail, obwohl der Alarm feuert, schau im Alertmanager nach: Eine noch laufende Silence aus Kapitel 7 auf `cluster="app1"` schaltet ihn stumm. Daneben feuert meist auch der Metrik-Alarm aus Kapitel 7 für den Crashloop. Das ist kein Fehler: Zwei Signale bestätigen dasselbe Problem.

MAC Schritt 19

```
kubectl --context app1 delete pod kaputt
```

Räumt auf. Der Alarm löst sich auf, sobald die Events aus dem Zehn-Minuten-Fenster fallen.

8.9 Aufbewahrung und Compactor

Loki schreibt Logs in Blöcken ins S3 und legt dazu einen Index an. Gelöscht wird nichts von selbst. Das übernimmt der **Compactor**, der im selben Prozess läuft. Er fasst die kleinen Index-Dateien zusammen und löscht mit `retention_enabled: true` alles, was älter ist als `retention_period`, hier sieben Tage.

`retention_delete_delay: 2h` lässt zwischen Markieren und Löschen eines Blocks zwei Stunden Zeit, in denen du einen Fehler noch bemerken kannst. Löschaufträge legt der Compactor laut `delete_request_store` im S3 ab. Zeilen, die älter als sieben Tage sind, nimmt Loki wegen `reject_old_samples_max_age` gar nicht erst an.

STOLPERFALLE

Ohne `retention_enabled: true` wächst der Bucket unbegrenzt, auch wenn `retention_period` gesetzt ist. Lege für den Loki-Bucket auch keine eigene S3-Lifecycle-Regel an, die Dateien nach Alter löscht. Der Index wüsste davon nichts und zeigte auf fehlende Blöcke.

Die Aufbewahrung lässt sich pro Mandant ändern, ohne Neustart über die Laufzeit-Konfiguration. Das brauchst du, wenn ein Cluster aus rechtlichen Gründen kürzer oder länger aufbewahren muss. Kapitel 13 greift das beim Thema Datenschutz wieder auf.

PROBIER ES SELBST

`app2` soll nur drei Tage aufbewahren. Ergänze in `observability/central/loki/values.yaml` unter `loki` einen Block `runtimeConfig` mit `overrides`, darunter den Mandanten `app2` und dort `retention_period: 72h`. Prüfe die Datei mit `helm template`, bevor du `helm upgrade` ausführst. Wie kontrollierst du danach, ob Loki den Wert übernommen hat? Tipp: Loki zeigt die aktive Laufzeit-Konfiguration unter dem Pfad `/runtime_config`.

Beende zum Schluss den Port-Forward im zweiten Terminal mit `Ctrl-C`.

WAS DU GELERNT HAST

- Loki indiziert nur Labels. Wenige, stabile Labels halten die Zahl der Streams klein, alles andere bleibt im Text oder in strukturierten Metadaten.
- Jeder Cluster schreibt als eigener Mandant. Grafana fragt mit `obs|app1|app2` alle zugleich ab.
- Eine LogQL-Abfrage wählt zuerst Streams, filtert dann Text und zählt am Ende Zeilen.
- Der Loki-Ruler arbeitet pro Mandant und schickt Alarme an denselben Alertmanager wie Prometheus.
- Gelöscht wird nur, wenn der Compactor die Retention ausführt.

KAPITEL 9

Traces mit Tempo und OpenTelemetry

Eine Demo-App in `app1` schickt Traces über Alloy an Tempo auf `obs`. Du findest sie mit TraceQL, siehst den Service-Graphen und springst zwischen Logs, Traces und Metriken hin und her.

Dauer: 90 Minuten **Voraussetzung:** Kapitel 8

9.1 Was du aufbaust

Ein **Trace** ist der Weg einer einzelnen Anfrage durch deine Dienste. Er besteht aus **Spans**. Jeder Span ist ein Arbeitsschritt mit Start, Dauer, Status und Attributen, etwa ein HTTP-Aufruf oder eine Datenbankabfrage. Alle Spans eines Traces teilen sich dieselbe Trace-ID. Steht diese ID auch im Log, kommst du vom Log zum Trace.

Komponente	Wo	Aufgabe
HotROD	`app1`, Namespace `demo`	Demo-App aus dem Jaeger-Projekt, erzeugt Traces per OpenTelemetry
Alloy (`traces.alloy`)	alle drei	Nimmt OTLP an, ergänzt Kubernetes-Daten und `cluster`, schickt an Tempo
Tempo	`obs`	Speichert Traces im Bucket `tempo`, ein Mandant pro Cluster
Metrics-Generator	`obs`, in Tempo	Erzeugt aus Spans Metriken und schreibt sie an Thanos Receive
Grafana	`obs`	Datenquelle Tempo mit Sprüngen zu Loki und Thanos

Anwendungen schicken ihre Spans nie direkt an die Zentrale, sondern an das Alloy im eigenen Cluster. Die App kennt damit nur einen Namen im Cluster und keine Zugangsdaten. Alloy ergänzt, was die App nicht weiß, etwa Namespace und Cluster, und kümmert sich um TLS, Basic Auth und Wiederholungen.

9.2 Tempo auf obs installieren

Bucket `tempo` und Secret `tempo-s3` stammen aus Kapitel 3.

DATEI observability/central/tempo/values.yaml (Ausschnitt)

```yaml
tempo:
  multitenancyEnabled: true
  retention: 168h
  storage:
    trace:
      backend: s3
      s3:
        bucket: tempo
        endpoint: s3.monitoring.svc.cluster.local:8333
        access_key: ${AWS_ACCESS_KEY_ID}
        secret_key: ${AWS_SECRET_ACCESS_KEY}
        insecure: true
        forcepathstyle: true
  queryFrontend:
    multi_tenant_queries_enabled: true
  metricsGenerator:
    enabled: true
    processor:
      service_graphs:
        dimensions: [cluster]
      span_metrics:
        dimensions: [cluster]
    storage:
      path: /var/tempo/generator/wal
      remote_write:
        - url: http://thanos-receive.monitoring.svc:19291/api/v1/receive
          send_exemplars: true
          headers:
            THANOS-TENANT: obs
  overrides:
    defaults:
      metrics_generator:
        processors: [service-graphs, span-metrics]
  extraArgs:
    config.expand-env: "true"
  extraEnvFrom:
    - secretRef:
        name: tempo-s3
```

`multitenancyEnabled` verlangt wie bei Loki den Header `X-Scope-OrgID`. Anders als bei Loki musst du `config.expand-env` selbst setzen, sonst stünde `${AWS_ACCESS_KEY_ID}` wörtlich in der Konfiguration. Der Endpunkt steht ohne `http://`, `insecure: true` regelt das Protokoll. Den Metrics-Generator schaltest du an zwei Stellen ein: unter `metricsGenerator` die Konfiguration, unter `overrides` die Prozessoren für alle Mandanten.

WARUM DAS WICHTIG IST

Der Metrics-Generator schreibt als Mandant `obs` an Thanos Receive, nicht als `app1` oder `app2`. Er läuft auf `obs`, und Tempo könnte den Mandanten nur im Header `X-Scope-OrgID` weitergeben. Thanos Receive liest aber `THANOS-TENANT`. Die Herkunft geht trotzdem nicht verloren: Über `dimensions: [cluster]` wird das Span-Attribut `cluster` zum Label jeder erzeugten Serie.

MAC Schritt 1

```
helm --kube-context obs upgrade --install tempo grafana-community/tempo \
  -n monitoring --version 3.2.1 \
  -f observability/central/tempo/values.yaml --wait --timeout 10m
```

Installiert Tempo als StatefulSet `tempo` mit einem Pod. Verteiler, Speicher, Abfrage und Metrics-Generator laufen in diesem einen Prozess.

MAC Schritt 2

```
kubectl --context obs -n monitoring logs tempo-0 --tail=30 \
  | grep -iE 'level=(error|warn)'
```

Zeigt nur Warnungen und Fehler der letzten 30 Zeilen.

DAS SOLLTEST DU SEHEN

Im Leerlauf meldet Tempo alle 15 Sekunden ein Paar `level=warn ... no jobs found` und `level=error msg="error calling scheduler" ... no jobs found`. Das ist der Hintergrunddienst für die Kompaktierung, der nachfragt, ob es Arbeit gibt. Er hat einfach noch keine, das ist harmlos. Andere Fehler sind es nicht: `AccessDenied` weist auf das S3-Konto hin. Fehler mit `remote_write` und `connection refused` bedeuten, dass Thanos Receive nicht erreichbar ist. Das prüfst du in Kapitel 4 nach.

9.3 Alloy nimmt Traces an

`traces.alloy` baut eine Kette aus fünf Gliedern: annehmen, Kubernetes-Daten ergänzen, `cluster` setzen, bündeln, senden.

DATEI `observability/cluster/alloy/traces.alloy (Ausschnitt)`

```
otelcol.receiver.otlp "traces_in" {
  grpc {
    endpoint = "0.0.0.0:4317"
  }
  http {
    endpoint = "0.0.0.0:4318"
  }
  output {
    traces = [otelcol.processor.k8sattributes.traces_k8s.input]
  }
}

otelcol.processor.k8sattributes "traces_k8s" {
  extract {
    metadata = ["k8s.namespace.name", "k8s.pod.name",
                "k8s.deployment.name"]
  }
  pod_association {
    source {
      from = "connection"
```

```
    }
  }
  output {
    traces = [otelcol.processor.transform.traces_cluster.input]
  }
}

otelcol.processor.transform "traces_cluster" {
  trace_statements {
    context    = "resource"
    statements = [
      string.format(`set(attributes["cluster"], "%s")`,
                    sys.env("CLUSTER_NAME")),
    ]
  }
  output {
    traces = [otelcol.processor.batch.traces_batch.input]
  }
}

// otelcol.processor.batch "traces_batch" -> traces_tempo

otelcol.exporter.otlphttp "traces_tempo" {
  client {
    endpoint = "https://tempo.obs.internal"
    auth     = otelcol.auth.basic.traces_obs.handler
    headers  = { "X-Scope-OrgID" = sys.env("CLUSTER_NAME") }
    tls {
      ca_file = "/etc/obs-ca/ca.crt"
    }
  }
}
```

`k8sattributes` erkennt den sendenden Pod an seiner IP-Adresse und hängt Namespace, Pod und Deployment an. Genau diese Attribute braucht Grafana später, um zu einem Span die passenden Logs zu finden. `transform` setzt das Ressourcen-Attribut `cluster`. Der Exporter schickt OTLP über HTTPS an Traefik, `otelcol.auth.basic` liefert Benutzer und Passwort aus `obs-remote-auth`. Dort stehen `username` und `password` direkt in der Komponente, obwohl die Doku sie als veraltet führt und den Block `client_auth` empfiehlt. Mit `client_auth` bricht Alloy bis mindestens Version 1.21.3 beim Start mit `no credential source provided` ab, weil es immer auch die Server-Seite der Anmeldung aufbaut und diese leer bleibt (Issue `grafana/alloy#5793`). Die nötigen Leserechte für Pods und Namespaces bringt das Alloy-Chart schon mit.

STOLPERFALLE

`k8sattributes` muss direkt hinter dem Empfänger stehen. Hinter `batch` ist die IP-Adresse des Absenders verloren, weil Spans verschiedener Pods dann gemischt in einem Paket liegen. Die Spans kommen trotzdem an, nur ohne Namespace und Pod. Der Sprung zu den Logs findet dann nichts.

Die Ports 4317 und 4318 hat der Alloy-Service seit Kapitel 5. Jetzt kommt `traces.alloy` in die ConfigMap, auf dieselbe Weise wie in Kapitel 8:

MAC Schritt 3

```
kubectl --context app1 -n monitoring create configmap alloy-config \
  --from-file=observability/cluster/alloy/metrics.alloy \
  --from-file=observability/cluster/alloy/logs.alloy \
  --from-file=observability/cluster/alloy/traces.alloy \
  --dry-run=client -o yaml | kubectl --context app1 apply -f -
kubectl --context app1 -n monitoring rollout restart ds/alloy
kubectl --context app1 -n monitoring rollout status ds/alloy
```

In app1 liegen jetzt alle drei Dateien in der ConfigMap.

MAC Schritt 4

```
kubectl --context app2 -n monitoring create configmap alloy-config \
  --from-file=observability/cluster/alloy/metrics.alloy \
  --from-file=observability/cluster/alloy/logs.alloy \
  --from-file=observability/cluster/alloy/traces.alloy \
  --dry-run=client -o yaml | kubectl --context app2 apply -f -
kubectl --context app2 -n monitoring rollout restart ds/alloy
kubectl --context app2 -n monitoring rollout status ds/alloy
```

Dasselbe für app2.

MAC Schritt 5

```
kubectl --context obs -n monitoring create configmap alloy-config \
  --from-file=observability/cluster/alloy/logs.alloy \
  --from-file=observability/cluster/alloy/traces.alloy \
  --dry-run=client -o yaml | kubectl --context obs apply -f -
kubectl --context obs -n monitoring rollout restart ds/alloy
kubectl --context obs -n monitoring rollout status ds/alloy
```

Auf `obs` ohne `metrics.alloy`, wie gehabt.

MAC Schritt 6

```
kubectl --context app1 -n monitoring logs ds/alloy -c alloy --tail=50 \
  | grep -E 'traces_|level=error'
```

Prüft, ob die neuen Komponenten ohne Fehler laufen.

DAS SOLLTEST DU SEHEN

Meldungen mit `traces_` im Namen, etwa zum Start des Empfängers auf `0.0.0.0:4317` und `0.0.0.0:4318`, aber keine Zeilen mit `level=error`. Ein Tippfehler in der Datei erscheint hier mit Zeilennummer, und Alloy behält dann die alte Konfiguration.

9.4 Die Demo-App

HotROD ist eine kleine Fahrdienst-App aus dem Jaeger-Projekt. Ein Aufruf von `/dispatch` geht durch die Dienste `frontend`, `customer`, `driver` und `route`, dazu kommen simulierte Aufrufe an

MySQL und Redis. Einige Redis-Aufrufe enden absichtlich mit einem Fehler, die MySQL-Abfrage dauert absichtlich rund 300 ms. Alle vier Dienste laufen in einem Pod.

DATEI `observability/demo/hotrod.yaml (Ausschnitt)`

```
    containers:
      - name: hotrod
        image: jaegertracing/example-hotrod:2.22.0
        args: ["all"]
        env:
          - name: OTEL_EXPORTER_OTLP_ENDPOINT
            value: http://alloy.monitoring.svc.cluster.local:4318
```

`all` startet alle Dienste in einem Prozess. `OTEL_EXPORTER_OTLP_ENDPOINT` ist die Standard-Variable des OpenTelemetry-SDK. HotROD hängt `/v1/traces` selbst an und schickt unverschlüsselt, weil die Adresse mit `http://` beginnt. Das ist in Ordnung, denn die Verbindung bleibt im Cluster. Die Datei legt außerdem den Namespace `demo` und einen Service an.

HotROD erzeugt nur Traces, wenn jemand eine Fahrt bestellt. Das übernimmt ein kleiner Lastgenerator in `observability/demo/last.yaml`. Er ruft alle drei Sekunden `/dispatch` für einen der vier Kunden auf.

MAC Schritt 7

```
kubectl --context app1 apply -f observability/demo/
```

Legt Namespace, HotROD und den Lastgenerator an.

MAC Schritt 8

```
kubectl --context app1 -n demo rollout status deploy/hotrod
```

Wartet, bis HotROD bereit ist.

MAC Schritt 9

```
kubectl --context app1 -n demo logs deploy/hotrod --tail=5
```

Zeigt die letzten Log-Zeilen von HotROD.

DAS SOLLTEST DU SEHEN

Zeilen wie diese:

```
INFO  frontend/best_eta.go:103  Dispatch successful  {"service": "frontend",
      "trace_id": "ff85bb590c413868cc304494136a3399", "span_id": "13df3b88240783d8",
      "driver": "T734673C", "eta": "2m0s"}
```

Am Ende jeder Zeile steht ein JSON-Block mit `"trace_id"` und `"span_id"`. Diese IDs nutzt du später für den Sprung vom Log zum Trace.

So testest du Tempo ohne Grafana

Tempo hat eine HTTP-API auf Port 3200. Wie bei Loki brauchst du den Header `X-Scope-OrgID`. Leite den Port in einem zweiten Terminal um:

MAC — Schritt 10

```
kubectl --context obs -n monitoring port-forward svc/tempo 3200:3200
```

Lass das Terminal offen.

MAC — Schritt 11

```
curl -s -G -H "X-Scope-OrgID: app1" localhost:3200/api/search \
  --data-urlencode 'q={ resource.service.name = "frontend" }' \
  --data-urlencode limit=3 | jq -c '.traces[] | [.rootServiceName, .rootTraceName, .durationMs]'
```

Sucht mit TraceQL nach drei Traces des Dienstes `frontend`. `-G` hängt die Parameter an die URL, `--data-urlencode` maskiert die Sonderzeichen.

DAS SOLLTEST DU SEHEN

Etwa eine Minute nach dem Start von HotROD drei Zeilen wie `["frontend","GET /dispatch",759]`: Dienst, Name des Wurzel-Spans und Dauer in Millisekunden. Bleibt die Liste leer, ist die Kette vor Tempo unterbrochen. Prüfe die Alloy-Logs auf `401` oder `x509`.

PROBIER ES SELBST

Leite HotROD auf den Mac um, mit `kubectl --context app1 -n demo port-forward svc/hotrod 8080:8080`, und öffne http://localhost:8080. Klicke ein paarmal schnell hintereinander auf einen Kunden. Was fällt dir an den angezeigten Zeiten auf? Die Erklärung findest du gleich im Trace.

9.5 Traces in Grafana

DATEI `observability/grafana/datasources/tempo.yaml (Ausschnitt)`

```
    - name: Tempo
      uid: tempo
      type: tempo
      url: http://tempo.monitoring.svc:3200
      jsonData:
        httpHeaderName1: X-Scope-OrgID
        tracesToLogsV2:
          datasourceUid: loki
          filterByTraceID: true
          tags:
            - {key: cluster, value: cluster}
            - {key: k8s.namespace.name, value: namespace}
            - {key: k8s.pod.name, value: pod}
        serviceMap:
          datasourceUid: thanos
```

```
      nodeGraph:
        enabled: true
    secureJsonData:
      httpHeaderValue1: obs|app1|app2
```

Auch hier fragt Grafana alle Mandanten zugleich ab. `tracesToLogsV2` übersetzt Attribute eines Spans in Loki-Labels: Aus `k8s.namespace.name` wird `namespace`, aus `k8s.pod.name` wird `pod`. `serviceMap` sagt Grafana, wo die Metriken für den Service-Graphen liegen. Die vollständige Datei enthält zusätzlich einen Sprung zu den Span-Metriken.

MAC Schritt 12

```
kubectl --context obs apply -f observability/grafana/datasources/tempo.yaml
```

Der Sidecar lädt die Datenquelle nach.

Öffne **Explore**, wähle **Tempo**, stelle **Query type** auf **TraceQL** und gib ein:

```
{ resource.cluster = "app1" && resource.service.name = "frontend" }
```

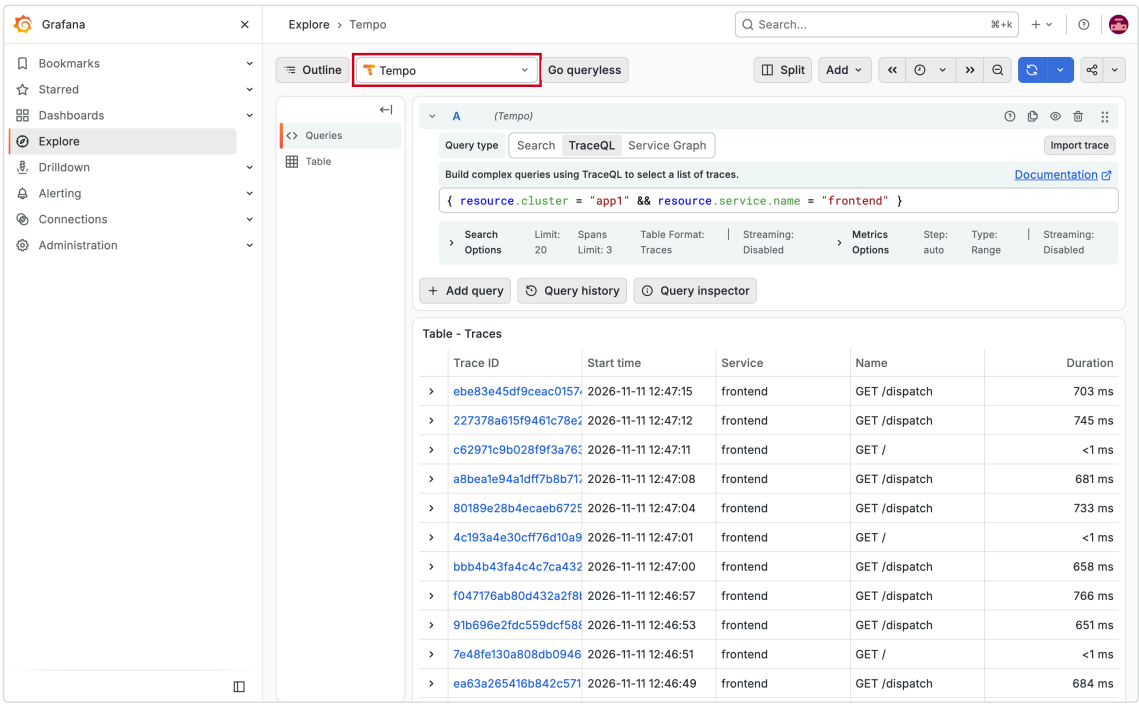


Abbildung 18: **Explore** mit Tempo: **Query type** steht auf **TraceQL**. Jede Zeile ist ein Trace, ein Klick auf die Trace-ID öffnet ihn.

DAS SOLLTEST DU SEHEN

Eine Liste von Traces, deren Wurzel-Span `GET /dispatch` heißt, etwa alle drei Sekunden einer. Dazwischen stehen kurze Traces `GET /`, die Abfragen der Startseite. Ein Klick öffnet den Wasserfall: Ganz oben steht `frontend`, darunter `customer` mit dem MySQL-Span von rund 300 ms, dann `driver` mit vielen kurzen Redis-Spans und mehrere parallele `route`-Aufrufe. Unter **Resource attributes** eines Spans stehen `cluster`, `k8s.namespace.name` und `k8s.pod.name`.

STOLPERFALLE

Erscheint nichts, geh die Kette von vorn nach hinten durch: HotROD-Log auf Verbindungsfehler zu `alloy.monitoring.svc`, dann das Alloy-Log auf `401` oder `x509`, dann das Tempo-Log. Erst nach etwa einer Minute sind neue Traces in der Suche sichtbar.

9.6 TraceQL-Grundlagen

TraceQL sucht nach Spans und gibt die Traces zurück, zu denen sie gehören. Eine Abfrage steht in geschweiften Klammern. Attribute haben einen Bereich als Präfix: `resource.` für Attribute des Dienstes, `span.` für Attribute des einzelnen Spans. Dazu kommen eingebaute Felder wie `name`, `duration` und `status`.

Langsame Abfragen beim Dienst `customer`:

```
{ resource.service.name = "customer" && duration > 250ms }
```

Alle Spans mit Fehler, in allen Clustern:

```
{ status = error }
```

Traces, in denen unter `frontend` irgendwo ein Redis-Fehler auftritt. Der Operator `>>` heißt „ist Nachfahre von“:

```
{ resource.service.name = "frontend" }
  >> { resource.service.name = "redis-manual" && status = error }
```

Traces aus dem Namespace `demo` mit besonders vielen Spans:

```
{ resource.k8s.namespace.name = "demo" } | count() > 20
```

WARUM DAS WICHTIG IST

Mit TraceQL fragst du nach Mustern, nicht nach einzelnen IDs. „Zeig mir alle Anfragen, bei denen Redis ausfiel und die trotzdem schnell waren“ wäre mit Logs allein eine mühsame Suche. Hier ist es eine Zeile.

9.7 Service-Graph und Span-Metriken

Der Metrics-Generator wertet jede ankommende Span aus und schreibt zwei Arten von Metriken an Thanos Receive:

Metrik	Inhalt
`traces_spanmetrics_calls_total`	Anzahl der Spans je Dienst, Span-Name und Status
`traces_spanmetrics_latency_bucket`	Histogramm der Span-Dauer, mit Exemplars
`traces_service_graph_request_total`	Aufrufe von Dienst A nach Dienst B
`traces_service_graph_request_failed_total`	davon fehlgeschlagen

Wähle in **Explore** die Datenquelle **Thanos** und frage ab:

```
sum by (cluster, service) (rate(traces_spanmetrics_calls_total[5m]))
```

DAS SOLLTEST DU SEHEN

Je eine Serie für `frontend`, `customer`, `driver`, `route`, `mysql` und `redis-manual`, alle mit `cluster="app1"` und zusätzlich `tenant_id="obs"`. Kommen keine Serien an, prüfe das Tempo-Log auf Fehler beim Schreiben an Thanos Receive.

Wechsle zurück zur Datenquelle **Tempo** und wähle den Abfragetyp **Service Graph**.



Abbildung 19: Der Service-Graph: `user` ruft `frontend`, das ruft `route`, `driver` und `customer`. Jeder Knoten zeigt mittlere Dauer und Anfragen pro Sekunde. Der rote Anteil am Ring unten ist die Fehlerquote von Redis.

DAS SOLLTEST DU SEHEN

Ein Graph mit den Diensten als Knoten und Pfeilen von `frontend` zu `customer`, `driver` und `route`. Über dem Graphen steht eine Tabelle mit Rate und Dauer je Operation. `mysql` und `redis-manual` erscheinen als eigene Knoten, aber nicht immer mit Pfeil, weil HotROD diese Aufrufe nur simuliert.

9.8 Von der Metrik zum Trace: Exemplars

Ein **Exemplar** ist eine einzelne Messung, an der eine Trace-ID hängt. Der Metrics-Generator speichert zu jedem Bucket des Latenz-Histogramms einige Beispiele unter dem Label `traceID`. Thanos Receive bewahrt sie auf, und die Datenquelle Thanos verweist für dieses Label auf Tempo. Beides ist seit Kapitel 4 und 6 vorbereitet.

```
histogram_quantile(0.95, sum by (le, service) (
  rate(traces_spanmetrics_latency_bucket{cluster="app1"}[5m])
))
```

Schalte unter der Abfrage den Schalter **Exemplars** ein.

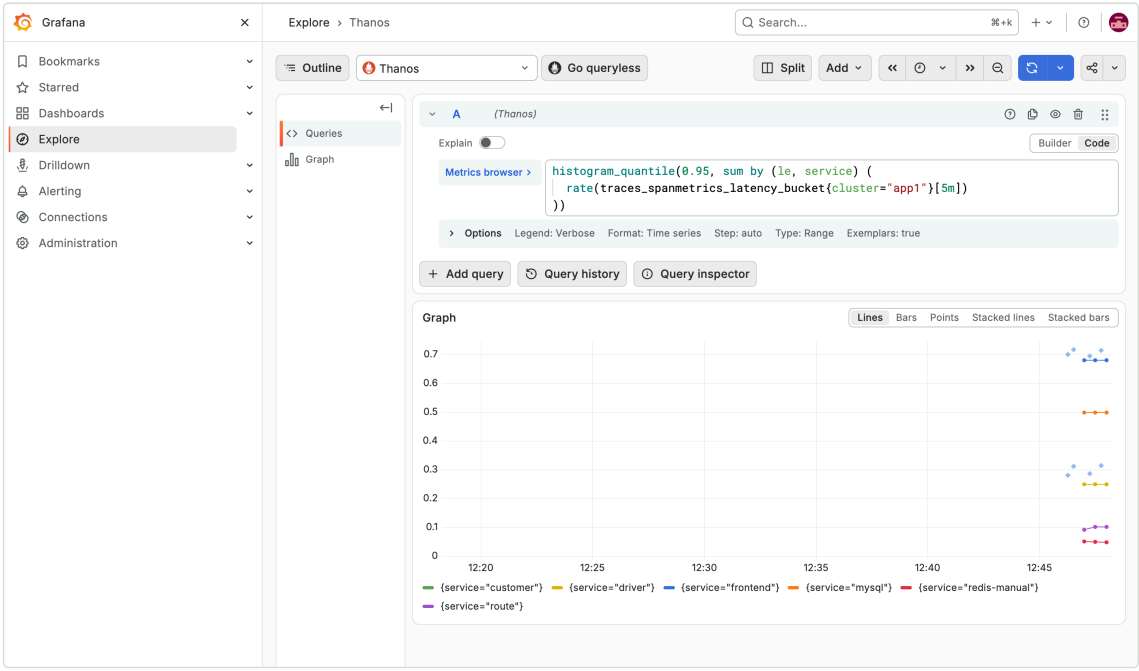


Abbildung 20: Das 95. Perzentil je Dienst. Die blassen Rauten sind Exemplars, jede steht für eine echte Anfrage mit Trace-ID.

DAS SOLLTEST DU SEHEN

Im Diagramm erscheinen kleine Rauten über den Linien. Fährst du mit der Maus darüber, zeigt Grafana die `traceID` und einen Knopf, der den Trace in Tempo öffnet. So kommst du von „das 95. Perzentil steigt“ direkt zu einer Anfrage, die langsam war.

9.9 Von Logs zu Traces und zurück

Die Datenquelle Loki aus Kapitel 8 erkennt `trace_id` im Log-Text. HotROD schreibt bei jedem Redis-Fehler eine Zeile `redis timeout` mit der Trace-ID:

```
{cluster="app1", namespace="demo"} |= "redis timeout"
```

Klicke auf eine Zeile. Darunter öffnen sich die Details. Unter **Links** steht **Trace in Tempo öffnen**:

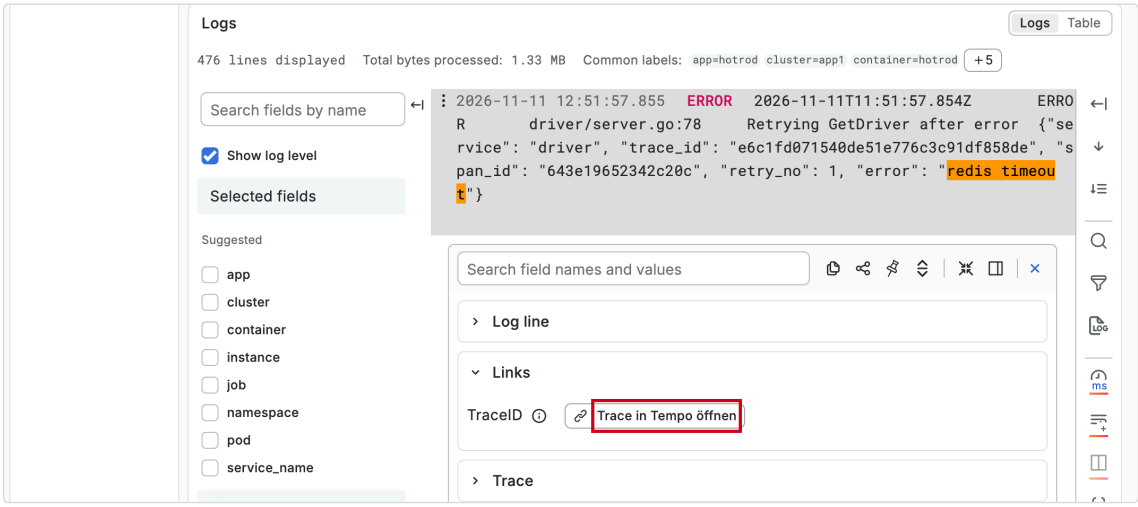


Abbildung 21: Die aufgeklappte Log-Zeile. Den Link **Trace in Tempo öffnen** erzeugt das Feld `derivedFields` der Loki-Datenquelle aus der `trace_id` im Text.

Ein Klick öffnet den Trace in einer geteilten Ansicht neben den Logs. Dort findest du den fehlerhaften Redis-Span rot markiert:

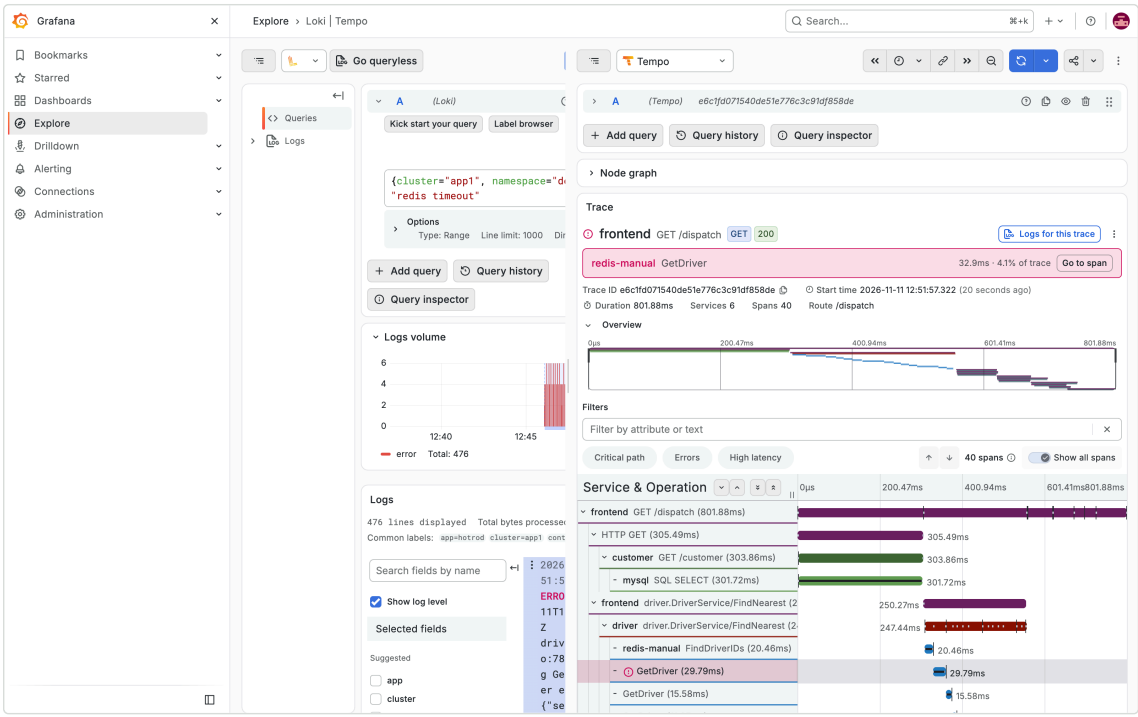


Abbildung 22: Links die Logs, rechts der Trace derselben Anfrage. Oben rot der Fehler-Span `GetDriver` von `redis-manual`, darunter der Wasserfall mit dem MySQL-Span von 300 ms. **Logs for this trace** oben rechts führt den umgekehrten Weg.

Der Weg zurück: Klicke im Trace oben rechts auf **Logs for this trace** oder bei einem Span von `driver` auf das Log-Symbol. Grafana baut daraus eine Loki-Abfrage mit `cluster`, `namespace` und `pod` und filtert auf die Trace-ID.

DAS SOLLTEST DU SEHEN

Im rechten Bereich erscheinen genau die Log-Zeilen dieser einen Anfrage, nicht alle Logs des Pods. Erscheint die Meldung, dass Labels fehlen, fehlen dem Span die Attribute aus `k8sattributes`. Prüfe dann die Reihenfolge in `traces.alloy`.

PROBIER ES SELBST

Erzeuge gleichzeitige Anfragen mit `kubectl --context app1 -n demo scale deploy/hotrod-last --replicas=5`. Beobachte im Exemplar-Diagramm, wie sich die Latenz von `customer` verändert, und öffne über eine Raute einen langsamen Trace. Wo wartet die Anfrage? Tipp: Vergleiche die Startzeit des MySQL-Spans mit der seines Eltern-Spans. Skaliere danach zurück auf `1`.

WAS DU GELERNT HAST

- Ein Trace zeigt den Weg einer Anfrage, ein Span einen Schritt darin. Die Trace-ID verbindet beides mit den Logs.
- Anwendungen schicken OTLP an das Alloy im eigenen Cluster. Alloy ergänzt Kubernetes-Daten und `cluster` und schickt als Mandant an Tempo.
- TraceQL sucht nach Mustern über Attribute, Dauer, Status und die Struktur eines Traces.
- Der Metrics-Generator macht aus Spans Metriken für Service-Graph und Exemplars und schreibt sie als Mandant `obs` an Thanos.
- Grafana springt von Metrik zu Trace, von Log zu Trace und von Trace zu Log, wenn die Attribute zusammenpassen.

KAPITEL 10

Fertige Dashboards

Sieben eigene Dashboards aus dem Repo beantworten die Fragen, die du im Alltag wirklich stellst. Dazu weißt du, welche mitgelieferten und welche Community-Dashboards sich lohnen.

Dauer: 45 Minuten **Voraussetzung:** Kapitel 6 und 7, für die Dashboards **Logs** und **Services** auch Kapitel 8 und 9

10.1 Warum eigene Dashboards

kube-prometheus-stack bringt rund 30 Dashboards mit, und auf grafana.com gibt es Tausende. Trotzdem lohnen sich eigene. Die mitgelieferten sind für einen Cluster mit eigenem Prometheus gebaut. Viele ihrer Panels nutzen Recording Rules, also vorberechnete Serien wie `node_namespace_pod_container:container_cpu_usage_seconds_total:sum_rate5m`. Diese Serien berechnet in deinem Labor nur Prometheus auf obs, und zwar nur für obs. Wählst du in **Kubernetes / Compute Resources / Cluster** den Cluster app1, bleiben deshalb einige Panels leer.

Man könnte diese Regeln zusätzlich im Thanos Ruler auswerten. Das sind aber rund zwei Dutzend Regeln mit Joins über mehrere Metriken, und für obs entstünden dieselben Serien doppelt, einmal von Prometheus und einmal vom Ruler. Dieses Buch geht deshalb den anderen Weg: Die eigenen Dashboards kommen ganz ohne Recording Rules aus.

Die Dashboards in `observability/dashboards/` folgen vier Regeln:

- **Eine Frage pro Dashboard.** Der Titel sagt, was du erfährst, die Beschreibung jedes Panels, wie du es liest.
- **Nur Rohdaten.** Alle Abfragen nutzen Serien, die Alloy, kube-state-metrics und node-exporter direkt liefern. Sie funktionieren deshalb für jeden Cluster.
- **Zwei Variablen.** `datasource` wählt die Datenquelle, `cluster` den Cluster. Keine feste UID einer Datenquelle steht im JSON.
- **Verlinkt.** Oben rechts führt **Weitere Dashboards** zu den anderen, mit derselben Auswahl und demselben Zeitraum.

Datei	Ordner	Frage
`multi-cluster.json`	Plattform	Sind alle Cluster da, gesund und liefern sie Daten?

Datei	Ordner	Frage
`node.json`	Plattform	Wie geht es einem einzelnen Node?
`workloads.json`	Plattform	Was läuft in einem Namespace, was verbraucht es, was ist kaputt?
`thanos-speicher.json`	Plattform	Kommen die Metriken an, und landen sie im S3?
`alarme.json`	Plattform	Was feuert gerade, und kommen die Benachrichtigungen an?
`logs.json`	Signale	Was steht in den Logs, und wo häufen sich Fehler?
`services.json`	Signale	Wie viele Anfragen, Fehler und welche Latenz hat ein Service?

10.2 So ist ein Dashboard gebaut

Jedes Dashboard beginnt mit denselben zwei Variablen:

DATEI `observability/dashboards/plattform/multi-cluster.json (Ausschnitt)`

```
"templating": { "list": [
  { "name": "datasource", "label": "Datenquelle",
    "type": "datasource", "query": "prometheus",
    "current": { "text": "default", "value": "default" } },
  { "name": "cluster", "label": "Cluster", "type": "query",
    "datasource": { "type": "prometheus", "uid": "${datasource}" },
    "query": "label_values(up, cluster)",
    "refresh": 2, "multi": true, "includeAll": true, "allValue": ".*" }
]}
```

Eine Variable vom Typ `datasource` listet alle Datenquellen des Typs `prometheus`. Der Wert `default` steht für die Standard-Datenquelle, bei dir Thanos. Das ist keine feste UID: In einer anderen Grafana-Instanz wählt dasselbe Dashboard deren Standard. `refresh: 2` liest die Cluster bei jeder Änderung des Zeitraums neu. Kommt ein vierter Cluster dazu, erscheint er von selbst.

Jedes Panel verweist dann auf die Variable statt auf eine feste Datenquelle:

```
"datasource": { "type": "prometheus", "uid": "${datasource}" },
"targets": [{ "expr": "count(group by (cluster) (up{cluster=~\"$cluster\"}))",
              "instant": true }]
```

Das `=~` ist wichtig. Bei Mehrfachauswahl setzt Grafana für `$cluster` einen regulären Ausdruck wie `app1|app2` ein, bei **All** den Wert aus `allValue`, also `.*`. Mit `=` statt `=~` fände die Abfrage dann nichts.

STOLPERFALLE

Lässt du `current` leer, also `"current": {}`, wählt Grafana 13 nicht zuverlässig die Standard-Datenquelle. Im Labor landete das Dashboard bei `Prometheus`, dem lokalen Prometheus auf obs. Seine Serien tragen kein Label `cluster`, denn das hängt erst der Remote Write an. Das Dashboard öffnet dann mit leerer Cluster-Auswahl und lauter leeren Panels.

STOLPERFALLE

Ein Stat-Panel mit `count(...)` zeigt **No data** statt `0`, wenn nichts passt, denn `count` über eine leere Menge ist leer und nicht null. Deshalb enden solche Abfragen mit `or vector(0)`. Ohne das sieht „keine Alarme“ genauso aus wie „Abfrage kaputt“.

10.3 Einspielen

Die Dashboards landen als zwei ConfigMaps im Cluster. Die Datei `kustomization.yaml` baut sie mit einem `configMapGenerator`:

DATEI `observability/dashboards/kustomization.yaml (Ausschnitt)`

```yaml
namespace: monitoring
generatorOptions:
  disableNameSuffixHash: true
  labels:
    grafana_dashboard: "1"
configMapGenerator:
  - name: dashboards-plattform
    options:
      annotations:
        grafana_folder: Plattform
    files:
      - plattform/multi-cluster.json
      - plattform/node.json
      # ...
  - name: dashboards-signale
    options:
      annotations:
        grafana_folder: Signale
    files:
      - signale/logs.json
      - signale/services.json
```

Das Label holt der Sidecar aus Kapitel 6, die Annotation bestimmt den Ordner. `disableNameSuffixHash` hält die Namen stabil, sonst hinge Kustomize bei jeder Änderung einen neuen Hash an und ließe die alte ConfigMap liegen.

MAC Schritt 1

```
kubectl --context obs apply -k observability/dashboards/ --server-side
```

`--server-side` ist nötig: Ein normales `apply` speichert die ganze ConfigMap zusätzlich in einer Annotation, und die darf höchstens 256 KB groß sein. Die ConfigMap `dashboards-plattform` ist schon rund 150 KB groß (gemessen an der Ausgabe von `kubectl kustomize`). Mit einem weiteren großen Dashboard wäre die Grenze überschritten, also gewöhnst du dir `--server-side` gleich an.

MAC Schritt 2

```
kubectl --context obs -n monitoring logs deploy/kps-grafana \
  -c grafana-sc-dashboard --tail=20
```

Der Dashboard-Sidecar meldet jede Datei, die er schreibt.

DAS SOLLTEST DU SEHEN

Sieben Zeilen, in denen der Sidecar die Dateien unter `/tmp/dashboards/Plattform/` und `/tmp/dashboards/Signale/` ablegt. In Grafana stehen unter **Dashboards** die Ordner **Kubernetes**, **Plattform** und **Signale**. Jedes Dashboard öffnet mit `default` als Datenquelle und einer gefüllten Auswahl **Cluster**.

10.4 Die Dashboards im Einzelnen

Multi-Cluster-Überblick

Dein Einstieg jeden Morgen. Oben sechs Kennzahlen: wie viele Cluster Daten liefern, welcher Anteil der Targets erreichbar ist, wie viele Alarme feuern, das älteste Datenalter, Nodes und laufende Pods. Darunter eine Tabelle mit einer Zeile pro Cluster, deren Zellen rot werden, wenn etwas nicht stimmt. Im unteren Teil CPU und Speicher je Cluster sowie Proben pro Sekunde und aktive Serien je Mandant in Thanos Receive.

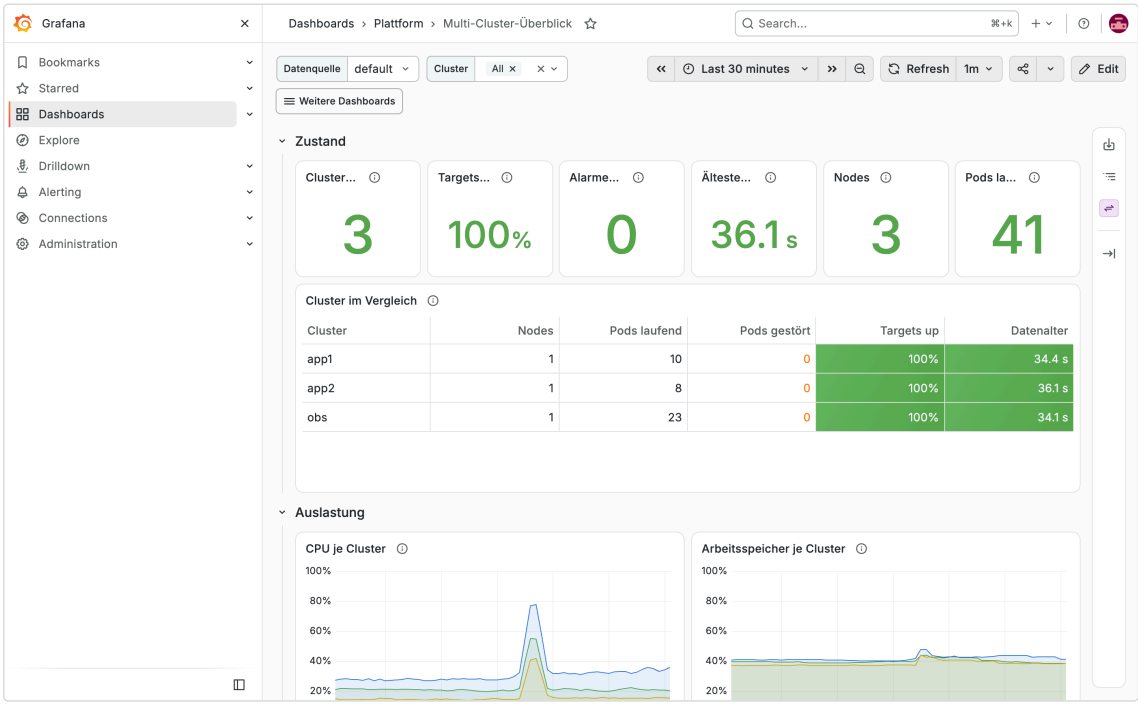


Abbildung 23: Der Multi-Cluster-Überblick im Normalbetrieb: drei Cluster, alle Targets erreichbar, das Datenalter überall grün. Wie dasselbe Dashboard während eines Ausfalls aussieht, zeigt Kapitel 15.

WARUM DAS WICHTIG IST

Das Panel **Ältester Datenpunkt** rechnet für jeden Cluster, wie lange sein letzter `up`-Wert her ist. Es zeigt dasselbe wie der Alarm `ClusterSchweigt` aus Kapitel 7, aber sofort und ohne `for`-Zeit. Ein Wert über zwei Minuten heißt: Dieser Cluster schickt gerade nichts.

Die Abfrage braucht dafür einen Trick. Ein einfaches `time() - timestamp(up)` findet nur Serien, die in den letzten fünf Minuten einen Wert hatten. Ein schweigender Cluster verschwindet damit nach fünf Minuten aus der Tabelle, statt rot zu werden. Deshalb schaut die Abfrage mit einer Unterabfrage sechs Stunden zurück:

```
time() - max by (cluster) (
  max_over_time(timestamp(up{cluster=~"$cluster"})[6h:1m]))
```

`[6h:1m]` wertet `timestamp(up)` sechs Stunden lang einmal pro Minute aus, `max_over_time` nimmt davon den jüngsten Zeitpunkt. Weil die Unterabfrage nur minütlich rechnet, zeigt auch ein gesunder Cluster bis zu gut einer Minute an. Das ist der Grund für die Schwelle von zwei Minuten.

Node

Wähle Cluster und Node. Die obere Reihe zeigt Laufzeit, Kerne, Arbeitsspeicher und die aktuelle Belegung von CPU, RAM und `/`. Darunter CPU nach Modus (viel `iowait` heißt: Die Disk bremst),

Load im Vergleich zur Zahl der Kerne, Speicher aufgeteilt in belegt, Cache und verfügbar, die Belegung aller Dateisysteme sowie Disk- und Netzwerk-Durchsatz. Senden und Schreiben zeigen nach unten, so liest du beide Richtungen in einem Panel.

Workloads

Wähle einen Cluster und einen oder mehrere Namespaces. Oben laufende und gestörte Pods, nicht bereite Container und Neustarts der letzten Stunde. Die Tabelle **Anforderung und Verbrauch je Pod** stellt CPU und RAM den Requests und Limits gegenüber. Liegt der Verbrauch weit unter dem Request, verschenkst du Platz. Liegt er nah am Limit, droht `OOMKilled`. Unten drei Tabellen für die Fehlersuche: Neustarts, wartende Container mit Grund (etwa `CrashLoopBackOff` oder `ImagePullBackOff`) und Deployments mit gewünschten und verfügbaren Replikas.

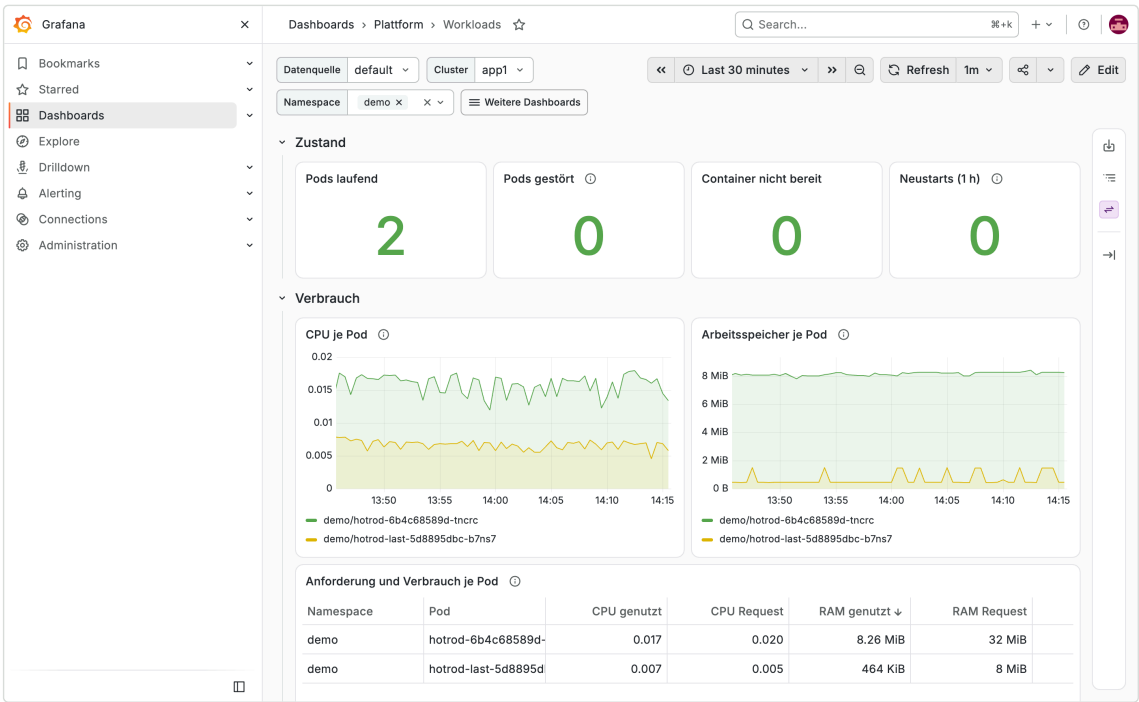


Abbildung 24: **Workloads** für `app1` und den Namespace `demo`: oben der Zustand, darunter CPU und Speicher je Pod, unten der Vergleich mit den Requests.

Thanos-Speicher

Die Gesundheit der Speicherkette auf obs. Oben Schreibanfragen pro Sekunde, Fehlerquote 5xx, abgelehnte Anfragen 4xx, Serien gesamt, Upload-Fehler und ob der Compactor angehalten hat. Darunter Antworten nach HTTP-Code, Schreiblatenz p50 und p99, Proben und Serien je Mandant. Im zweiten Teil Block-Uploads, S3-Fehler je Komponente, die Zahl der Blöcke im Store

Gateway und fehlgeschlagene Kompaktierungen. Zum Schluss Speicher und CPU der Thanos-Pods und von SeaweedFS. Die Variable `cluster` filtert hier die Mandanten.

Alarme

Oben die Zahl kritischer, warnender und wartender Alarme, aktive Silences, der Watchdog (muss `1` sein) und Zustellfehler der letzten Stunde. Darunter die Tabelle aller feuernden Alarme mit Cluster, Schwere und Namespace, dann der Verlauf je Cluster und je Alarmname, die Benachrichtigungen je Kanal und die Fehler bei der Regelauswertung in Prometheus und Thanos Ruler.

Logs

Nutzt die Datenquelle Loki aus Kapitel 8. Die Variablen `cluster` und `namespace` liest das Dashboard aus den Labels in Loki, dazu kommt ein Suchfeld für einen regulären Ausdruck. Du siehst die Log-Zeilen je Namespace als Balken, Zeilen mit `error`, `fatal` oder `panic` je Cluster, die zehn gesprächigsten Pods und darunter die Log-Zeilen selbst.

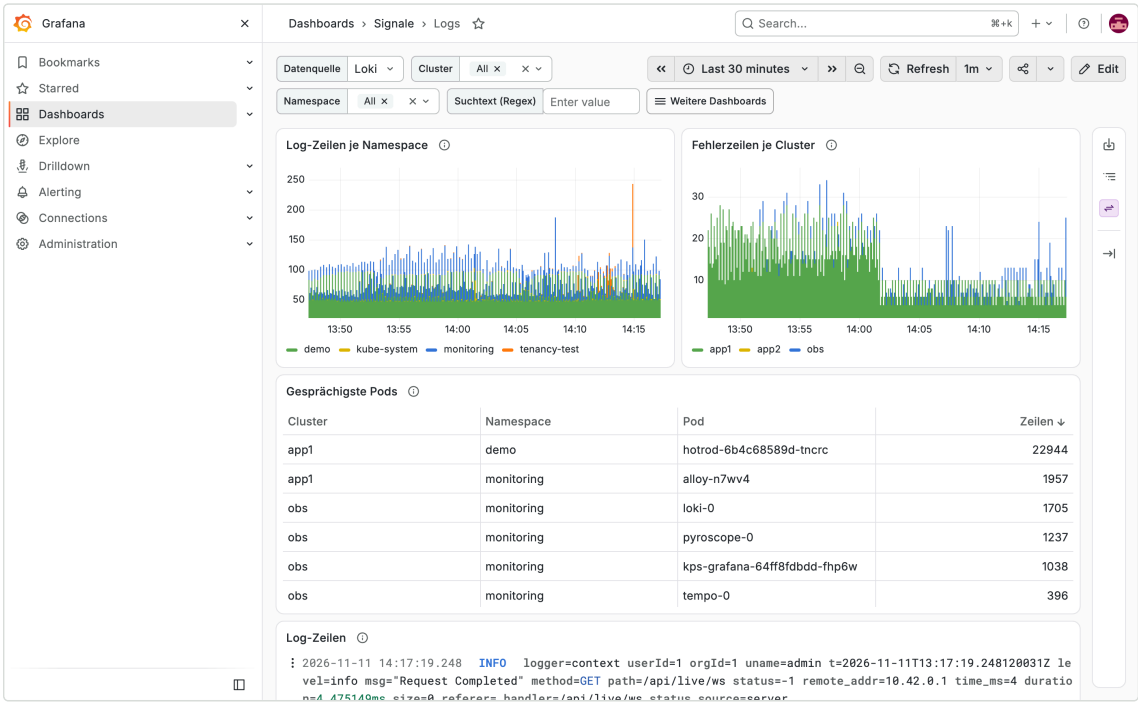


Abbildung 25: Das Dashboard **Logs** über alle Cluster: Zeilen je Namespace, Fehlerzeilen je Cluster, die gesprächigsten Pods und darunter die Zeilen selbst.

STOLPERFALLE

Loki mag kein `.*`. Bei der Auswahl **All** setzt Grafana den Wert aus `allValue` ein. Für Prometheus ist `.*` richtig, für Loki nicht: Loki verlangt in jedem Selektor mindestens einen Matcher, der keine leeren Werte zulässt, und antwortet sonst mit `queries require at least one regexp or equality matcher that does not have an empty-compatible value`. Alle Panels zeigen dann ein rotes Warndreieck. Im Logs-Dashboard steht deshalb `"allValue": ".+"`.

STOLPERFALLE

Loki trennt die Cluster als Mandanten. Damit das Dashboard alle drei auf einmal zeigt, muss die Loki-Datenquelle aus Kapitel 8 den Header `X-Scope-OrgID: obs|app1|app2` senden, und in Loki müssen Abfragen über mehrere Mandanten erlaubt sein. Sonst zeigt die Auswahl `cluster` nur einen Wert.

Services

Die RED-Methode für jeden Service: **Rate**, **Errors**, **Duration**. Die Daten kommen nicht aus der Anwendung, sondern aus Tempos Metrics-Generator (Kapitel 9). Er macht aus jeder Span die Serien `traces_spanmetrics_calls_total` und `traces_spanmetrics_latency_bucket`, und aus Aufrufen zwischen Services die Serien `traces_service_graph_request_total`. Du siehst Anfragen pro Sekunde, Fehlerquote und Latenz p95 je Service, eine Tabelle der Endpunkte und eine Tabelle „Wer ruft wen?“. Die Punkte im Latenz-Panel sind Exemplars: Ein Klick öffnet den passenden Trace in Tempo.

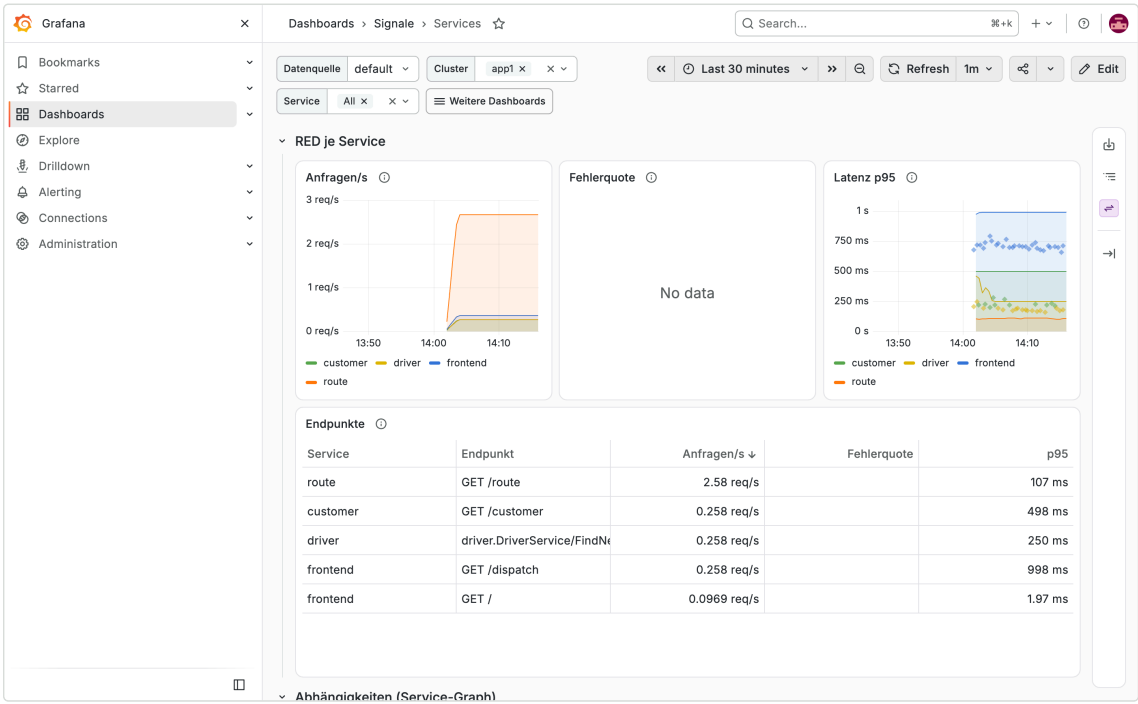


Abbildung 26: **Services** für `app1`. Die Kurven beginnen erst um 14 Uhr, davor lag der Ausfalltest aus Kapitel 15, in dem keine Traces ankamen. **Fehlerquote** zeigt **No data**: Die Redis-Fehler von HotROD hängen an Client-Spans, das Panel zählt nur Spans vom Typ Server, und die waren fehlerfrei. Die Rauten im Latenz-Panel sind Exemplars.

STOLPERFALLE

Der Metrics-Generator schreibt alle Span-Metriken als Mandant `obs`, egal aus welchem Cluster die Spans stammen. `tenant_id` hilft hier also nicht weiter. Die Abfragen filtern deshalb auf das Label `cluster`, das Tempo aus dem Ressourcen-Attribut `cluster` als Dimension übernimmt (Kapitel 9). Fehlt diese Dimension, bleibt das Dashboard leer, sobald du einen Cluster auswählst.

PROBIER ES SELBST

Öffne das Dashboard **Workloads**, wähle `app1` und den Namespace `demo`. Welcher Pod hat den größten Abstand zwischen RAM-Request und tatsächlichem Verbrauch? Prüfe deine Antwort mit `kubectl --context app1 -n demo top pods` und vergleiche mit `kubectl get pod -o yaml`. Warum weichen die Werte von `kubectl top` und vom Dashboard leicht ab?

10.5 Mitgelieferte und Community-Dashboards

Die eigenen Dashboards ersetzen nicht alles. Die folgenden sind einen Blick wert. Die ersten liegen schon im Ordner **Kubernetes**.

Dashboard	Herkunft	Wofür, und was in diesem Labor gilt
Node Exporter / Nodes	kps	Ausführliche Node-Ansicht aus Rohdaten mit Auswahl `cluster`, funktioniert für alle Cluster
Kubernetes / Kubelet	kps	Pods, Volumes und Latenz des Kubelets, ohne Recording Rules
Kubernetes / Persistent Volumes	kps	Belegung der PVCs. Bei local-path oft leer, weil der Kubelet für hostPath-Volumes keine Volume-Statistik liefert
Kubernetes / Networking / Pod und Cluster	kps	Netzwerkverkehr aus cAdvisor-Daten, ohne Recording Rules
Kubernetes / Compute Resources / …	kps	Sehr gut für obs. Für app1 und app2 bleiben die Panels mit Recording Rules leer
Node Exporter / USE Method	kps	USE-Methode je Node und Cluster, nur für obs, weil vollständig aus Recording Rules
Alertmanager / Overview	kps	Eingehende Alarme und Benachrichtigungen des Alertmanagers
Prometheus / Remote Write	kps	Warteschlange und Fehler des Remote Write von Prometheus auf obs
CoreDNS	kps	Anfragen und Fehler von CoreDNS
Node Exporter Full (ID 1860)	grafana.com	Der Klassiker mit jeder Metrik des node-exporter
Kubernetes / Views / Global, Namespaces, Nodes, Pods (ID 15757 bis 15760)	grafana.com	Moderne, übersichtliche Kubernetes-Ansichten. Prüfe nach dem Einspielen, ob sie eine Auswahl `cluster` anbieten, sonst mischen sie alle Cluster

Community-Dashboards holst du genauso ins Repo wie deine eigenen. Lade das JSON einer bestimmten Revision herunter, zum Beispiel mit `curl -sL https://grafana.com/api/dashboards/1860/revisions/<revision>/download`, prüfe es mit `jq empty` und lege es in einen eigenen Ordner mit eigenem `configMapGenerator`. Enthält es einen Block `__inputs`, ersetze die Platzhalter wie `${DS_PROMETHEUS}` durch eine Variable `datasource`, aus demselben Grund wie in Kapitel 6.

STOLPERFALLE

Ein Community-Dashboard kennt deine Job-Namen nicht. Viele erwarten `job="node-exporter"` oder `job="kubelet"`. Genau deshalb hat Kapitel 5 diese Namen für app1 und app2 übernommen. Bleibt ein Panel trotzdem leer, öffne es mit **Edit** und vergleiche die Labels in der Abfrage mit denen, die **Explore** für dieselbe Metrik zeigt.

PROBIER ES SELBST

Ergänze das Dashboard **Node** um ein Panel „Offene Dateien“ mit der Abfrage `node_filefd_allocated{cluster="$cluster", instance="$instance"}`. Baue es in Grafana, exportiere das JSON wie in Kapitel 6 beschrieben, ersetze `observability/dashboards/plattform/node.json`, prüfe mit `jq empty` und spiele es mit `kubectl apply -k` ein. Was zeigt `git diff` außer deinem neuen Panel noch an, und warum?

WAS DU GELERNT HAST

- Mitgelieferte Dashboards nutzen oft Recording Rules, die es nur für den Cluster mit eigenem Prometheus gibt.
- Eigene Dashboards mit Rohdaten und einer Variable `cluster` funktionieren für jeden Cluster.
- Die Variable `datasource` ersetzt feste UIDs, dann laufen Dashboards in jeder Grafana-Instanz.
- Mehrfachauswahl braucht `=~` in der Abfrage, und `or vector(0)` macht aus „keine Daten“ eine Null.
- Große Dashboards spielst du mit `kubectl apply --server-side` ein.

KAPITEL 11

S3 auf AWS

Die Langzeitdaten ziehen von SeaweedFS in vier verschlüsselte AWS-Buckets um, und jede Komponente darf nur an ihren eigenen.

Dauer: 90 Minuten **Voraussetzung:** Kapitel 3 bis 9, ein AWS-Konto und die `aws` CLI auf dem Mac

SeaweedFS hat dich bis hierher gut getragen. Es hat aber einen Haken: Es liegt auf derselben VM wie alles andere. Stirbt die Disk von `k3s-obs`, sind Metriken, Logs und Traces weg, und die Sicherungen aus Kapitel 12 gleich mit. AWS S3 löst genau das. Die Daten liegen außerhalb deines Labors, mehrfach gespiegelt, und du zahlst nur, was du belegst.

NICHT IM LABOR DURCHGESPIELT

Anders als die Kapitel 2 bis 10 lief dieses Kapitel nicht gegen ein echtes AWS-Konto. Die Befehle folgen der AWS-Doku und sind syntaktisch geprüft, aber Ausgaben und Fehlermeldungen können abweichen. Gehe Schritt für Schritt vor und vergleiche nach jedem Schritt mit dem Kasten „Das solltest du sehen“.

Der Umbau ist kleiner, als er klingt. Thanos, Loki und Tempo sprechen schon S3. Du tauschst nur Endpunkt, Bucket und Schlüssel aus.

WARUM DAS WICHTIG IST

Wer Observability-Daten in die Cloud legt, öffnet einen neuen Angriffsweg. Ein falsch gesetzter Bucket ist öffentlich lesbar, ein zu breiter Schlüssel löscht alles. Deshalb baust du in diesem Kapitel erst die Riegel ein und schreibst danach die ersten Daten.

11.1 Die aws CLI vorbereiten

Für die Einrichtung brauchst du die AWS CLI v2 und einen IAM-Benutzer mit Administrationsrechten, nicht das Root-Konto. Seine Schlüssel liegen nur in `~/.aws/credentials` auf deinem Mac, nie im Repo.

MAC Schritt 1

```
aws configure --profile obs-admin
```

Fragt Access Key, Secret Key, Region (`eu-central-1`) und Ausgabeformat (`json`) ab. Das Profil heißt `obs-admin`, damit es nicht mit anderen Projekten kollidiert.

MAC — Schritt 2

```
export AWS_PROFILE=obs-admin AWS_REGION=eu-central-1 PRAEFIX=tz7
aws sts get-caller-identity
```

Bucket-Namen sind weltweit eindeutig. Wähle deshalb ein eigenes, kurzes Präfix aus Kleinbuchstaben und Ziffern statt `tz7`. Alle Befehle dieses Kapitels laufen in derselben Terminal-Sitzung. Die zweite Zeile fragt AWS, wer du gerade bist.

DAS SOLLTEST DU SEHEN

Eine JSON-Ausgabe mit `Account` und einer `Arn`, die auf `:user/...` endet. Steht dort `:root`, arbeitest du mit dem Root-Konto. Lege dann zuerst einen eigenen Admin-Benutzer an.

11.2 Vier Buckets anlegen

Wie in SeaweedFS bekommt jede Komponente ihren eigenen Bucket. So kannst du Rechte, Lebensdauer und Versionierung pro Bucket festlegen.

Bucket	Nutzer	Besonderheit
`<praefix>-obs-thanos`	Thanos	Ablauf nach der Compactor-Retention
`<praefix>-obs-loki`	Loki	Ablauf nach der Loki-Retention
`<praefix>-obs-tempo`	Tempo	Ablauf nach der Tempo-Retention
`<praefix>-obs-backup`	k3s, Velero	Versionierung an

MAC — Schritt 3

```
aws s3api create-bucket --bucket "$PRAEFIX-obs-thanos" \
  --create-bucket-configuration LocationConstraint=eu-central-1
aws s3api create-bucket --bucket "$PRAEFIX-obs-loki" \
  --create-bucket-configuration LocationConstraint=eu-central-1
aws s3api create-bucket --bucket "$PRAEFIX-obs-tempo" \
  --create-bucket-configuration LocationConstraint=eu-central-1
aws s3api create-bucket --bucket "$PRAEFIX-obs-backup" \
  --create-bucket-configuration LocationConstraint=eu-central-1
```

Legt die vier Buckets in Frankfurt an. `$PRAEFIX` hast du oben gesetzt, die Shell ersetzt es in jedem Befehl. Außerhalb von `us-east-1` verlangt AWS die Region zusätzlich als `LocationConstraint`.

Meldet AWS `BucketAlreadyExists`, hat jemand anderes auf der Welt diesen Namen schon. Wähle dann ein anderes Präfix.

11.3 Die Riegel vor dem ersten Byte

Neue Buckets sind heute schon privat und verschlüsselt. Du setzt alles trotzdem ausdrücklich. Dann steht es im Befehlsverlauf, und eine geänderte Voreinstellung bei AWS ändert nichts an deinen Buckets.

MAC — Schritt 4

```
BLOCK=BlockPublicAcls=true,IgnorePublicAcls=true,BlockPublicPolicy=true,RestrictPublicBuckets=true
aws s3api put-public-access-block --bucket "$PRAEFIX-obs-thanos" \
  --public-access-block-configuration "$BLOCK"
aws s3api put-public-access-block --bucket "$PRAEFIX-obs-loki" \
  --public-access-block-configuration "$BLOCK"
aws s3api put-public-access-block --bucket "$PRAEFIX-obs-tempo" \
  --public-access-block-configuration "$BLOCK"
aws s3api put-public-access-block --bucket "$PRAEFIX-obs-backup" \
  --public-access-block-configuration "$BLOCK"
```

Block Public Access mit allen vier Schaltern. Selbst eine versehentlich öffentliche Policy oder ACL greift dann nicht mehr.

MAC — Schritt 5

```
aws s3api put-bucket-encryption --bucket "$PRAEFIX-obs-thanos" \
  --server-side-encryption-configuration file://observability/aws/encryption.json
aws s3api put-bucket-encryption --bucket "$PRAEFIX-obs-loki" \
  --server-side-encryption-configuration file://observability/aws/encryption.json
aws s3api put-bucket-encryption --bucket "$PRAEFIX-obs-tempo" \
  --server-side-encryption-configuration file://observability/aws/encryption.json
aws s3api put-bucket-encryption --bucket "$PRAEFIX-obs-backup" \
  --server-side-encryption-configuration file://observability/aws/encryption.json
```

`observability/aws/encryption.json` setzt `"SSEAlgorithm": "AES256"`. Das ist SSE-S3: AWS verschlüsselt jedes Objekt beim Schreiben mit eigenen Schlüsseln.

WARUM DAS WICHTIG IST

SSE-KMS mit eigenem Schlüssel klingt sicherer. Es bringt dir zwei Dinge: ein Protokoll jeder Entschlüsselung in CloudTrail und die Möglichkeit, den Schlüssel zu sperren. Dafür kostet jeder Lese- und Schreibzugriff eine KMS-Anfrage, und jede Komponente braucht zusätzlich `kms:Decrypt` und `kms:GenerateDataKey`. Thanos Store und Loki lesen sehr viele kleine Stücke. Für ein Labor ist SSE-S3 deshalb die bessere Wahl. In einer Firma mit Prüfpflichten nimmst du SSE-KMS mit Bucket Key, der die Zahl der KMS-Anfragen stark senkt.

Verschlüsselung im Bucket hilft nicht, wenn die Daten unverschlüsselt hinfließen. Eine Bucket-Policy verbietet deshalb jeden Zugriff ohne TLS.

DATEI observability/aws/bucket-policy-tls.json (Ausschnitt)

```
"Effect": "Deny",
"Principal": "*",
"Action": "s3:*",
"Resource": ["arn:aws:s3:::BUCKET", "arn:aws:s3:::BUCKET/*"],
"Condition": {"Bool": {"aws:SecureTransport": "false"}}
```

, erklaerung: [Ein `Deny` schlägt jedes `Allow`. `BUCKET` ist ein Platzhalter, der Befehl setzt den echten Namen ein.]

MAC Schritt 6

```
sed "s|BUCKET|$PRAEFIX-obs-thanos|g" observability/aws/bucket-policy-tls.json > /tmp/policy.json
aws s3api put-bucket-policy --bucket "$PRAEFIX-obs-thanos" --policy file:///tmp/policy.json
```

Setzt die TLS-Pflicht für den Bucket `thanos`. `sed` setzt den echten Namen in die Vorlage ein, die Datei in `/tmp` enthält keine Geheimnisse.

MAC Schritt 7

```
sed "s|BUCKET|$PRAEFIX-obs-loki|g" observability/aws/bucket-policy-tls.json > /tmp/policy.json
aws s3api put-bucket-policy --bucket "$PRAEFIX-obs-loki" --policy file:///tmp/policy.json
sed "s|BUCKET|$PRAEFIX-obs-tempo|g" observability/aws/bucket-policy-tls.json > /tmp/policy.json
aws s3api put-bucket-policy --bucket "$PRAEFIX-obs-tempo" --policy file:///tmp/policy.json
sed "s|BUCKET|$PRAEFIX-obs-backup|g" observability/aws/bucket-policy-tls.json > /tmp/policy.json
aws s3api put-bucket-policy --bucket "$PRAEFIX-obs-backup" --policy file:///tmp/policy.json
```

Dasselbe für die übrigen drei Buckets, je zwei Zeilen pro Bucket.

MAC Schritt 8

```
aws s3api put-bucket-versioning --bucket "$PRAEFIX-obs-backup" \
  --versioning-configuration Status=Enabled
```

Versionierung nur für den Backup-Bucket. Überschreibt oder löscht jemand eine Sicherung, bleibt die alte Version erhalten.

DAS SOLLTEST DU SEHEN

`aws s3api get-public-access-block --bucket "$PRAEFIX-obs-backup"` zeigt vier Mal `true`. `get-bucket-versioning` zeigt für den Backup-Bucket `"Status": "Enabled"`, für die anderen drei nichts. Das ist richtig so.

STOLPERFALLE

Schalte die Versionierung nicht für Thanos, Loki und Tempo ein. Der Thanos Compactor fasst Blöcke ständig zusammen und löscht die alten. Mit Versionierung bleiben alle gelöschten Blöcke als alte Versionen liegen, und du zahlst sie weiter, ohne sie je zu lesen.

11.4 Lebensdauer: Lifecycle-Regeln

Eine Lifecycle-Regel löscht Objekte nach einer festen Zahl von Tagen oder verschiebt sie in eine günstigere Speicherklasse. Das klingt nach der idealen Retention. Für Thanos, Loki und Tempo ist sie es aber nicht.

WARUM DAS WICHTIG IST

Jedes dieser Tools führt eigene Buchhaltung über seine Objekte. Thanos kennt jeden Block über seine `meta.json`, Loki führt einen Index über seine Chunks, Tempo eine Blockliste pro Mandant. Löscht S3 ein Objekt, bevor das Tool es selbst aufgibt, zeigt der Index ins Leere. Abfragen brechen mit „object not found“ ab, und der Thanos Compactor hält mit einem Fehler an. Deshalb gilt: Die Retention macht das Tool. Die Lifecycle-Regel ist nur ein Sicherheitsnetz und muss **später** greifen als die Tool-Retention.

Bucket	Retention im Tool	Lifecycle-Ablauf
`thanos`	Compactor: Rohdaten 30 Tage, 5-Minuten-Daten 90 Tage, 1-Stunden-Daten 365 Tage (Kapitel 4)	400 Tage
`loki`	`retention_period: 168h`, also 7 Tage (Kapitel 8)	60 Tage
`tempo`	`retention: 168h`, also 7 Tage (Kapitel 9)	30 Tage
`backup`	Velero-TTL und etcd-Retention (Kapitel 12)	alte Versionen nach 30 Tagen

DATEI `observability/aws/lifecycle-thanos.json (Ausschnitt)`

```
{ "ID": "abgebrochene-uploads", "Status": "Enabled",
  "Filter": {"Prefix": ""},
  "AbortIncompleteMultipartUpload": {"DaysAfterInitiation": 7} },
{ "ID": "sicherheitsnetz-nach-thanos-retention", "Status": "Enabled",
  "Filter": {"Prefix": ""},
  "Expiration": {"Days": 400} }
```

, erklaerung: [Die erste Regel räumt halbe Uploads weg. Sie entstehen, wenn ein Pod mitten im Hochladen eines großen Blocks stirbt, sind unsichtbar und kosten trotzdem. Die zweite Regel liegt 35 Tage hinter der längsten Thanos-Retention. `lifecycle-loki.json` und `lifecycle-tempo.json` sind genauso gebaut, nur mit anderen Tagen.]

Erhöhst du später die Retention in Loki oder Tempo, passt du zuerst die Lifecycle-Regel an. Sonst löscht S3 Daten, die das Tool noch für gültig hält.

MAC Schritt 9

```
aws s3api put-bucket-lifecycle-configuration --bucket "$PRAEFIX-obs-thanos" \
  --lifecycle-configuration file://observability/aws/lifecycle-thanos.json
aws s3api put-bucket-lifecycle-configuration --bucket "$PRAEFIX-obs-loki" \
  --lifecycle-configuration file://observability/aws/lifecycle-loki.json
aws s3api put-bucket-lifecycle-configuration --bucket "$PRAEFIX-obs-tempo" \
  --lifecycle-configuration file://observability/aws/lifecycle-tempo.json
aws s3api put-bucket-lifecycle-configuration --bucket "$PRAEFIX-obs-backup" \
  --lifecycle-configuration file://observability/aws/lifecycle-backup.json
```

Setzt pro Bucket die passende Regel. Jeder Bucket bekommt seine eigene Datei.

DAS SOLLTEST DU SEHEN

`aws s3api get-bucket-lifecycle-configuration --bucket "$PRAEFIX-obs-backup"` zeigt die Regel `alte-versionen-und-uploads` mit `NoncurrentVersionExpiration` von 30 Tagen. Alte Versionen einer Sicherung verschwinden also erst nach einem Monat.

STOLPERFALLE

Ein Übergang nach `STANDARD_IA` oder Glacier spart Speicherkosten, schadet hier aber. Thanos Store und der Compactor lesen auch alte Blöcke regelmäßig, und jeder Lesezugriff auf IA kostet extra. Objekte in Glacier Flexible Retrieval kann Thanos gar nicht mehr lesen. Übergänge lohnen sich nur für Daten, die du fast nie liest, etwa alte Sicherungen.

11.5 Ein Benutzer, eine Policy pro Bucket

Thanos, Loki und Tempo laufen auf `obs` und teilen sich den IAM-Benutzer `obs-lab`. Er bekommt für jeden Bucket eine eigene, kleine Policy. Die Sicherungen schreiben dagegen alle drei Cluster, deshalb bekommen sie einen zweiten Benutzer `obs-lab-backup`. Ein gestohlener Schlüssel aus `app1` reicht dann nicht an deine Metriken heran.

DATEI `observability/aws/iam-policy-thanos.json (Ausschnitt)`

```
{ "Effect": "Allow",
  "Action": ["s3:ListBucket"],
  "Resource": "arn:aws:s3:::PRAEFIX-obs-thanos" },
{ "Effect": "Allow",
  "Action": ["s3:GetObject", "s3:PutObject", "s3:DeleteObject"],
  "Resource": "arn:aws:s3:::PRAEFIX-obs-thanos/*" }
```

, erklaerung: [`ListBucket` gilt für den Bucket, die drei Objektrechte nur für seinen Inhalt. Mehr braucht Thanos nicht. Tempo bekommt zusätzlich `s3:GetObjectTagging` und `s3:PutObjectTagging` .]

Die Backup-Policy erlaubt Schreiben und Löschen, verbietet aber ausdrücklich `s3:DeleteObjectVersion`, `s3:PutBucketVersioning` und `s3:PutLifecycleConfiguration`. Wer den Backup-

Schlüssel stiehlt, kann Sicherungen also nur scheinbar löschen. Die alten Versionen bleiben 30 Tage liegen, und weder Versionierung noch Lifecycle lassen sich mit diesem Schlüssel abschalten.

MAC Schritt 10

```
aws iam create-user --user-name obs-lab
aws iam create-user --user-name obs-lab-backup
```

Legt die beiden technischen Benutzer an. Sie haben kein Passwort und können sich nicht in der Konsole anmelden.

MAC Schritt 11

```
sed "s|PRAEFIX|$PRAEFIX|g" observability/aws/iam-policy-thanos.json > /tmp/policy.json
aws iam put-user-policy --user-name obs-lab \
  --policy-name obs-thanos --policy-document file:///tmp/policy.json
sed "s|PRAEFIX|$PRAEFIX|g" observability/aws/iam-policy-loki.json > /tmp/policy.json
aws iam put-user-policy --user-name obs-lab \
  --policy-name obs-loki --policy-document file:///tmp/policy.json
sed "s|PRAEFIX|$PRAEFIX|g" observability/aws/iam-policy-tempo.json > /tmp/policy.json
aws iam put-user-policy --user-name obs-lab \
  --policy-name obs-tempo --policy-document file:///tmp/policy.json
```

Hängt drei Inline-Policies an `obs-lab`, eine pro Bucket. Inline-Policies verschwinden mit dem Benutzer, du räumst später also nichts doppelt auf.

MAC Schritt 12

```
sed "s|PRAEFIX|$PRAEFIX|g" observability/aws/iam-policy-backup.json > /tmp/policy.json
aws iam put-user-policy --user-name obs-lab-backup \
  --policy-name obs-backup --policy-document file:///tmp/policy.json
```

Die Backup-Policy für den zweiten Benutzer. Seinen Schlüssel erzeugst du erst in Kapitel 12.

DAS SOLLTEST DU SEHEN

`aws iam list-user-policies --user-name obs-lab` nennt `obs-loki`, `obs-tempo` und `obs-thanos`.

Jetzt der heikle Moment: der Zugangsschlüssel. AWS zeigt den geheimen Teil genau einmal. Er wandert deshalb direkt in zwei Shell-Variablen und von dort in Kubernetes-Secrets, ohne je auf dem Bildschirm oder in einer Datei zu landen.

MAC Schritt 13

```
read -r AK SK <<< "$(aws iam create-access-key --user-name obs-lab \
  --query 'AccessKey.[AccessKeyId,SecretAccessKey]' --output text)"
```

Erzeugt den Schlüssel und liest beide Teile in `AK` und `SK`. Schließe das Terminal erst, wenn alle Secrets dieses Kapitels angelegt sind.

DAS SOLLTEST DU SEHEN

`echo "${#AK} ${#SK}"` gibt nur die Längen aus: `20 40`. Steht dort `0 0`, ist der Befehl fehlgeschlagen. Ein Benutzer darf höchstens zwei Schlüssel haben. Prüfe mit `aws iam list-access-keys --user-name obs-lab` und lösche überzählige mit `aws iam delete-access-key`.

11.6 Thanos umziehen

Die Reihenfolge ist wichtig. Erst hält der Compactor an, dann kopierst du die Blöcke, dann schaltest du um, und zum Schluss kopierst du noch einmal, was in der Zwischenzeit dazugekommen ist. So läuft nie ein Compactor auf zwei Buckets gleichzeitig.

MAC — Schritt 14

```
kubectl --context obs -n monitoring scale deployment thanos-compact --replicas=0
```

Hält den Compactor an. Er ist der Einzige, der Blöcke löscht und umschreibt. Receive und Store laufen weiter.

MAC — Schritt 15

```
kubectl --context obs -n monitoring port-forward svc/s3 8333:8333
```

Macht SeaweedFS auf `localhost:8333` erreichbar. Lass dieses Terminal offen und arbeite in einem zweiten weiter. Dort musst du `AWS_PROFILE`, `AWS_REGION` und `PRAEFIX` erneut setzen.

MAC — Schritt 16

```
SW=$(kubectl --context obs -n monitoring get secret s3-config \
  -o jsonpath='{.data.s3\.json}' | base64 -d)
SWAK=$(echo "$SW" | jq -r '.identities[] | select(.name=="thanos") | .credentials[0].accessKey')
SWSK=$(echo "$SW" | jq -r '.identities[] | select(.name=="thanos") | .credentials[0].secretKey')
```

Liest die Schlüssel des SeaweedFS-Kontos `thanos` aus `s3-config` in zwei Variablen, ohne sie anzuzeigen.

MAC — Schritt 17

```
AWS_ACCESS_KEY_ID=$SWAK AWS_SECRET_ACCESS_KEY=$SWSK \
  aws s3 sync s3://thanos ./umzug/thanos --endpoint-url http://localhost:8333
```

Lädt alle Blöcke aus SeaweedFS in den Ordner `umzug/thanos`. Die beiden Variablen gelten nur für diesen einen Aufruf. Laut Kapitel 3 sind das höchstens einige Gigabyte.

STOLPERFALLE

`umzug/` liegt im Repo-Verzeichnis. Trag es in `.gitignore` ein, bevor du `git add` tippst, sonst landen Gigabytes an Metriken in Git. Alternativ legst du den Ordner unter `/tmp` an.

MAC Schritt 18

```
aws s3 sync ./umzug/thanos "s3://$PRAEFIX-obs-thanos"
```

Lädt die Blöcke nach AWS. Jetzt gilt wieder dein Profil `obs-admin`.

DATEI `observability/aws/thanos-objstore-aws.yml`

```
type: S3
config:
  bucket: PRAEFIX-obs-thanos
  endpoint: s3.eu-central-1.amazonaws.com
  region: eu-central-1
  insecure: false
  bucket_lookup_type: virtual-hosted
```

, erklaerung: [Gegenüber SeaweedFS ändern sich vier Dinge: der Bucket, der AWS-Endpunkt, `insecure: false` für HTTPS und die virtuelle Adressierung `<bucket>.s3...` statt Pfad-Stil. Die Schlüsselzeilen fehlen absichtlich.]

MAC Schritt 19

```
{ sed "s|PRAEFIX|$PRAEFIX|" observability/aws/thanos-objstore-aws.yml
  printf '  access_key: %s\n  secret_key: %s\n' "$AK" "$SK"; } |
kubectl --context obs -n monitoring create secret generic thanos-objstore \
  --from-file=objstore.yml=/dev/stdin --dry-run=client -o yaml |
kubectl --context obs -n monitoring apply -f -
```

Baut das neue `objstore.yml` aus Vorlage und Schlüsseln und ersetzt das Secret. `printf` ist in die Shell eingebaut. Die Schlüssel tauchen deshalb weder in der Prozessliste noch in einer Datei auf.

MAC Schritt 20

```
kubectl --context obs -n monitoring rollout restart \
  statefulset/thanos-receive deployment/thanos-store
```

Secrets werden nur beim Start gelesen. Beim Herunterfahren lädt Receive seine letzten Blöcke noch nach SeaweedFS hoch, danach schreibt es nach AWS.

MAC Schritt 21

```
AWS_ACCESS_KEY_ID=$SWAK AWS_SECRET_ACCESS_KEY=$SWSK \
  aws s3 sync s3://thanos ./umzug/thanos --endpoint-url http://localhost:8333
aws s3 sync ./umzug/thanos "s3://$PRAEFIX-obs-thanos"
```

Der zweite Durchgang holt, was Receive beim Herunterfahren noch nach SeaweedFS geschrieben hat. `sync` überträgt nur, was neu ist.

MAC Schritt 22

```
kubectl --context obs -n monitoring scale deployment thanos-compact --replicas=1
```

Startet den Compactor wieder, jetzt gegen AWS.

DAS SOLLTEST DU SEHEN

`kubectl --context obs -n monitoring logs deployment/thanos-store --tail=20` zeigt keine Zeilen mit `level=error`. In Grafana zeigt eine Abfrage über die letzten sieben Tage, etwa `sum by (cluster) (up)`, eine durchgehende Linie ohne Lücke vor dem Umzug. Lücken von wenigen Minuten während des Neustarts sind normal.

11.7 Loki und Tempo umschalten

Loki und Tempo lesen ihre Schlüssel aus `loki-s3` und `tempo-s3`. Endpunkt und Bucket stehen in den Helm-Werten. Für AWS kommen zwei kleine Zusatzdateien dazu.

DATEI `observability/aws/loki-aws.yaml`

```
loki:
  storage:
    bucketNames:
      chunks: PRAEFIX-obs-loki
      ruler: PRAEFIX-obs-loki
      admin: PRAEFIX-obs-loki
    s3:
      endpoint: s3.eu-central-1.amazonaws.com
      region: eu-central-1
      s3ForcePathStyle: false
      insecure: false
```

, erklaerung: [Wird **nach** `observability/central/loki/values.yaml` angegeben und überschreibt nur diese Felder. `tempo-aws.yaml` setzt dasselbe unter `tempo.storage.trace.s3` .]

MAC Schritt 23

```
printf 'AWS_ACCESS_KEY_ID=%s\nAWS_SECRET_ACCESS_KEY=%s\n' "$AK" "$SK" |
  kubectl --context obs -n monitoring create secret generic loki-s3 \
    --from-env-file=/dev/stdin --dry-run=client -o yaml |
  kubectl --context obs -n monitoring apply -f -
printf 'AWS_ACCESS_KEY_ID=%s\nAWS_SECRET_ACCESS_KEY=%s\n' "$AK" "$SK" |
  kubectl --context obs -n monitoring create secret generic tempo-s3 \
    --from-env-file=/dev/stdin --dry-run=client -o yaml |
  kubectl --context obs -n monitoring apply -f -
```

Ersetzt die SeaweedFS-Schlüssel in beiden Secrets durch den AWS-Schlüssel von `obs-lab`.

MAC Schritt 24

```
sed "s|PRAEFIX|$PRAEFIX|g" observability/aws/loki-aws.yaml |
helm --kube-context obs upgrade loki grafana-community/loki --version 21.0.0 \
  -n monitoring -f observability/central/loki/values.yaml -f -
```

`-f -` liest die zweite Wertedatei von der Standardeingabe, nachdem `sed` das Präfix eingesetzt hat. Das Upgrade startet Loki neu, und Loki liest dabei auch das neue Secret.

MAC Schritt 25

```
sed "s|PRAEFIX|$PRAEFIX|g" observability/aws/tempo-aws.yaml |
helm --kube-context obs upgrade tempo grafana-community/tempo --version 3.2.1 \
  -n monitoring -f observability/central/tempo/values.yaml -f -
```

Dasselbe für Tempo.

Zum Schluss löscht `unset AK SK SWAK SWSK` alle Schlüssel aus der Shell. Ab jetzt stehen sie nur noch in den Secrets.

DAS SOLLTEST DU SEHEN

In Grafana unter **Explore** liefert die Datenquelle Loki mit `{cluster="app1"}` neue Zeilen, und Tempo findet Traces der letzten Minuten. `aws s3 ls "s3://$PRAEFIX-obs-loki/" --recursive | head` zeigt neue Objekte.

PROBIER ES SELBST

Ziehe auch die alten Logs und Traces um. Das Muster kennst du von Thanos. Hinweise: Die SeaweedFS-Schlüssel für `loki` und `tempo` stehen in `s3-config`, wie bei Thanos. Halte Loki vorher mit `kubectl scale statefulset loki --replicas=0` an, damit sich der Bucket während der Kopie nicht ändert. Überlege: Warum reicht es bei Loki nicht, nur die Chunks zu kopieren, und welcher Ordner muss unbedingt mit?

11.8 Was kostet das?

Eine feste Zahl kann dir niemand nennen. Die Preise ändern sich, und sie hängen von Region und Speicherklasse ab. Den Rechenweg kannst du aber selbst gehen. Schlag die aktuellen Preise für `eu-central-1` auf der offiziellen Seite aws.amazon.com/s3/pricing nach.

Posten	So misst du die Menge	Rechnung
Speicher	`aws s3 ls s3://<bucket> --recursive --summarize` (Zeile `Total Size`)	GB × Preis pro GB und Monat
Schreibanfragen	Thanos: `sum(increase(thanos_objstore_bucket_operations_total{operation="upload"}[30d]))`	Anzahl ÷ 1000 × Preis pro 1000 PUT
Leseanfragen	Dieselbe Metrik mit `operation=~"get|get_range"`	Anzahl ÷ 1000 × Preis pro 1000 GET
Datenabfluss	Was Store Gateway, Loki und Tempo aus S3 lesen	GB × Preis pro GB Datentransfer ins Internet

WARUM DAS WICHTIG IST

Der letzte Posten wird gern vergessen. Deine VMs stehen nicht in AWS. Jede Grafana-Abfrage über alte Daten zieht Blöcke aus Frankfurt zu dir nach Hause, und das ist Datentransfer ins Internet. Speicher ist billig, Abfluss nicht. Ein Dashboard mit 30-Tage-Zeitraum und automatischer Aktualisierung kann teurer werden als der ganze Speicher. Lege dir deshalb in der AWS-Konsole unter **Billing** ein Budget mit Warnung an, bevor die erste Rechnung kommt.

11.9 Spiegelung, wenn eine Kopie nicht reicht

Auch eine Region kann ausfallen. Für Sicherungen bietet AWS deshalb die Replikation in einen zweiten Bucket an, auch in einer anderen Region. Beide Buckets brauchen Versionierung, und du zahlst Speicher und Übertragung doppelt. Für Metriken, Logs und Traces lohnt sich das im Labor nicht. Für den Backup-Bucket reicht oft eine einfachere Lösung: SeaweedFS bleibt als zweite, lokale Kopie, die du gelegentlich mit `aws s3 sync` abgleichst.

WAS DU GELERNT HAST

- Ein Bucket pro Komponente erlaubt eigene Rechte, Lebensdauer und Versionierung.
- Block Public Access, SSE-S3 und eine TLS-Pflicht setzt du, bevor das erste Byte fließt.
- Die Retention steuert das Tool. Eine Lifecycle-Regel ist nur das Sicherheitsnetz und muss später greifen.
- Schlüssel gehen per `printf` und Standardeingabe direkt in Secrets, nie in Dateien oder auf den Bildschirm.
- Bei Speicher außerhalb des Labors kostet vor allem das Lesen, nicht das Liegen.

KAPITEL 12

Backup und Restore

Du sicherst genau das, was sich nicht neu bauen lässt, und holst es in einer Übung zurück.

Dauer: 90 Minuten **Voraussetzung:** Kapitel 3 und 9, für die AWS-Variante Kapitel 11

Eine Sicherung, die nie zurückgespielt wurde, ist eine Hoffnung. Dieses Kapitel endet deshalb nicht beim ersten grünen Haken, sondern bei zwei Wiederherstellungen: einmal den ganzen Datastore eines Clusters, einmal einen gelöschten Namespace.

12.1 Was gesichert werden muss und was nicht

Der größte Teil deines Stacks braucht keine Sicherung. Er steht im Git oder liegt schon in S3. Geh die Liste einmal bewusst durch.

Was	Warum	Sicherung
Helm-Werte, Manifeste, Dashboards, Regeln	Liegen in diesem Repo	Git
Metriken, Logs, Traces	Liegen in S3. Ein Verlust der letzten Stunden ist verschmerzbar	Keine, oder S3 selbst (siehe unten)
etcd jedes Clusters	Enthält alle Secrets, die du per `openssl rand` erzeugt hast, und alles, was nicht im Git steht	k3s-Snapshots nach S3
k3s-Token	Ohne ihn lässt sich ein Snapshot auf neuer Hardware nicht entschlüsseln	Passwort-Manager
Namespace `demo`	Steht als Beispiel für Anwendungen, die nicht aus Git kommen	Velero

Was	Warum	Sicherung
Grafana-Datenbank	Kein PVC (Kapitel 6), alles kommt aus ConfigMaps	Keine
PVC von SeaweedFS	Nur wenn du nicht auf AWS umgezogen bist: Hier liegen dann alle Langzeitdaten auf einer einzigen Disk	`aws s3 sync` nach außen

WARUM DAS WICHTIG IST

Die Secrets sind der Grund, warum etcd so wichtig ist. Die Passwörter für Basic Auth, S3 und Grafana gibt es nur im Cluster. Ist etcd weg, musst du sie neu erzeugen und in allen drei Clustern neu verteilen. Mit einem Snapshot dauert das fünf Minuten.

12.2 SeaweedFS von außen erreichbar machen

Bisher ist SeaweedFS nur im Cluster `obs` erreichbar, unter `s3.monitoring.svc`. Die Snapshots von `app1` und `app2` und Velero in `app1` kommen dort nicht hin. Du gibst S3 deshalb einen eigenen Namen hinter Traefik: `s3.obs.internal`.

Die Datei `observability/backup/s3-ingress.yaml` ist gebaut wie die Ingresse aus Kapitel 5: nur HTTPS über `websecure`, Zertifikat aus `obs-tls`, Ziel ist der Service `s3` auf Port 8333. Es fehlt nur bewusst die Basic-Auth-Middleware.

WARUM DAS WICHTIG IST

S3 authentifiziert jede einzelne Anfrage selbst. Der Client signiert sie mit seinem Schlüssel (Signature Version 4) und schickt die Signatur im Header `Authorization`. Basic Auth bräuchte genau diesen Header für sich. Beides zusammen geht also nicht, und es ist auch nicht nötig: Ohne gültige Signatur antwortet SeaweedFS mit `AccessDenied`.

MAC — Schritt 1

```
kubectl --context obs apply -f observability/backup/s3-ingress.yaml
```

Legt den Ingress auf `obs` an.

In den Pods der Cluster löst CoreDNS den Namen `s3.obs.internal` schon auf, und dein Mac kennt ihn ebenfalls. Beides hast du in Kapitel 2 eingetragen.

k3s selbst läuft aber nicht im Cluster, sondern als Dienst auf der VM. Für die etcd-Snapshots fragt es den Resolver des Betriebssystems und nicht CoreDNS. Jede VM braucht deshalb einen eigenen Eintrag in `/etc/hosts`:

MAC Schritt 2

```
ssh -t <user>@172.16.0.126 'echo "172.16.0.126 s3.obs.internal" | sudo tee -a /etc/hosts'
ssh -t <user>@172.16.0.228 'echo "172.16.0.126 s3.obs.internal" | sudo tee -a /etc/hosts'
ssh -t <user>@172.16.0.83  'echo "172.16.0.126 s3.obs.internal" | sudo tee -a /etc/hosts'
```

Ergänzt den Eintrag auf allen drei VMs. Alle zeigen auf die IP von obs. `-t` braucht `sudo` für die Passwortabfrage.

MAC Schritt 3

```
curl -s --cacert observability/tls/ca.crt https://s3.obs.internal/ | head -c 200
```

Eine Anfrage ohne Signatur vom Mac, über TLS mit deiner CA.

DAS SOLLTEST DU SEHEN

Eine kurze XML-Antwort von SeaweedFS, typischerweise mit `AccessDenied`. Das ist der Erfolg: TLS, Traefik und der Service funktionieren. Ein `404 page not found` in Klartext kommt von Traefik und heißt, dass der Ingress nicht greift. Ein Zertifikatsfehler heißt, dass `curl` die CA nicht bekommen hat.

ENTSCHEIDUNG: EIN KONTO FÜR ALLE SICHERUNGEN

Kapitel 3 hat für die Sicherungen ein einziges SeaweedFS-Konto `velero` angelegt, das nur auf den Bucket `backup` darf. Alle drei Cluster nutzen dieses Konto. Jeder schreibt aber in seinen eigenen Ordner: `etcd/obs`, `etcd/app1`, `etcd/app2` und `velero/app1`. Der Grund: SeaweedFS vergibt Rechte pro Bucket, nicht pro Ordner. Drei Konten würden also nichts trennen, solange es nur einen Bucket gibt. Der Preis: Wer den Schlüssel aus `app1` stiehlt, kann auch die Sicherungen von `app2` löschen. Auf AWS fängt die Versionierung aus Kapitel 11 genau diesen Fall ab.

12.3 etcd-Snapshots nach S3

NICHT IM LABOR DURCHGESPIELT

Die Schritte dieses und des nächsten Abschnitts laufen per SSH direkt auf den VMs. Sie wurden beim Testlauf dieses Buches nicht ausgeführt, nur gegen die k3s-Doku geprüft. Der Velero-Teil weiter unten lief dagegen vollständig im Labor.

k3s macht schon von sich aus alle zwölf Stunden einen Snapshot, aber nur lokal auf der VM. Stirbt die Disk, ist er mit weg. Du stellst auf alle sechs Stunden um und lässt k3s jede Kopie zusätzlich nach S3 schieben.

Die Zugangsdaten sollen nicht in die k3s-Config auf der VM. k3s kann sie deshalb aus einem Secret im Namespace `kube-system` lesen. Zuerst holst du die Schlüssel des Kontos `velero` aus `obs`.

MAC — Schritt 4

```
CLOUD=$(kubectl --context obs -n monitoring get secret velero-s3 \
  -o jsonpath='{.data.cloud}' | base64 -d)
AK=$(echo "$CLOUD" | awk -F= '/aws_access_key_id/ {print $2}')
SK=$(echo "$CLOUD" | awk -F= '/aws_secret_access_key/ {print $2}')
echo "${#AK} ${#SK}"
```

Liest die Schlüssel in Shell-Variablen und gibt nur ihre Längen aus. Laut Kapitel 3 sind das `32 64`.

MAC — Schritt 5

```
kubectl --context obs -n kube-system create secret generic k3s-etcd-s3 \
  --type=etcd.k3s.cattle.io/s3-config-secret \
  --from-literal=etcd-s3-endpoint=s3.obs.internal \
  --from-literal=etcd-s3-bucket=backup \
  --from-literal=etcd-s3-folder=etcd/obs \
  --from-file=etcd-s3-endpoint-ca=observability/tls/ca.crt \
  --from-file=etcd-s3-access-key=<(printf %s "$AK") \
  --from-file=etcd-s3-secret-key=<(printf %s "$SK")
```

Das Secret für obs, mit dem Ordner `etcd/obs`.

MAC — Schritt 6

```
kubectl --context app1 -n kube-system create secret generic k3s-etcd-s3 \
  --type=etcd.k3s.cattle.io/s3-config-secret \
  --from-literal=etcd-s3-endpoint=s3.obs.internal \
  --from-literal=etcd-s3-bucket=backup \
  --from-literal=etcd-s3-folder=etcd/app1 \
  --from-file=etcd-s3-endpoint-ca=observability/tls/ca.crt \
  --from-file=etcd-s3-access-key=<(printf %s "$AK") \
  --from-file=etcd-s3-secret-key=<(printf %s "$SK")
kubectl --context app2 -n kube-system create secret generic k3s-etcd-s3 \
  --type=etcd.k3s.cattle.io/s3-config-secret \
  --from-literal=etcd-s3-endpoint=s3.obs.internal \
  --from-literal=etcd-s3-bucket=backup \
  --from-literal=etcd-s3-folder=etcd/app2 \
  --from-file=etcd-s3-endpoint-ca=observability/tls/ca.crt \
  --from-file=etcd-s3-access-key=<(printf %s "$AK") \
  --from-file=etcd-s3-secret-key=<(printf %s "$SK")
```

Dasselbe für app1 und app2. Die drei Befehle unterscheiden sich nur in Kontext und Ordner. Der Typ `etcd.k3s.cattle.io/s3-config-secret` ist Pflicht. Die CA steht direkt als PEM im Secret. Die beiden Schlüssel kommen über `<(printf ...)` als Datei, damit sie nicht in der Prozessliste auftauchen. Region und Adressierung bleiben auf den Standardwerten `us-east-1` und `auto`, und `auto` wählt für fremde Endpunkte den Pfad-Stil.

DATEI `observability/backup/k3s-etcd-s3.yaml`

```
etcd-snapshot-schedule-cron: "0 */6 * * *"
etcd-snapshot-retention: 8
etcd-snapshot-compress: true
etcd-s3: true
etcd-s3-config-secret: k3s-etcd-s3
```

, erklaerung: [Ein Drop-in für k3s. Alle sechs Stunden ein Snapshot, lokal bleiben die letzten acht, komprimiert als ZIP. `etcd-s3: true` schaltet die Kopie ein, alles Weitere steht im Secret.]

STOLPERFALLE

Setze hier keine weitere `etcd-s3-*`-Option, auch nicht `etcd-s3-retention`. Laut k3s-Doku ignoriert k3s das Secret, sobald irgendeine andere S3-Option gesetzt ist. Die Snapshots landen dann ohne Fehlermeldung nur lokal.

k3s liest neben `/etc/rancher/k3s/config.yaml` auch alle Dateien in `/etc/rancher/k3s/config.yaml.d/`. Deine Haupt-Config aus Kapitel 2 bleibt also unverändert.

MAC — Schritt 7

```
scp observability/backup/k3s-etcd-s3.yaml <user>@172.16.0.126:/tmp/etcd-s3.yaml
scp observability/backup/k3s-etcd-s3.yaml <user>@172.16.0.228:/tmp/etcd-s3.yaml
scp observability/backup/k3s-etcd-s3.yaml <user>@172.16.0.83:/tmp/etcd-s3.yaml
```

Kopiert das Drop-in auf alle drei VMs.

MAC — Schritt 8

```
ssh <user>@172.16.0.228
```

Meldet dich auf `k3s-app1` an. Die nächsten Schritte mit dem Etikett **app1** laufen dort.

APP1 — Schritt 9

```
sudo install -D -m 600 /tmp/etcd-s3.yaml \
  /etc/rancher/k3s/config.yaml.d/etcd-s3.yaml
sudo systemctl restart k3s
```

Legt die Datei mit Rechten nur für root ab und startet k3s neu. Die Pods laufen dabei weiter. Wiederhole diesen Schritt auf `k3s-obs` und `k3s-app2`.

APP1 — Schritt 10

```
sudo k3s etcd-snapshot save --name manuell
sudo k3s etcd-snapshot ls
```

Ein Snapshot von Hand, damit du nicht sechs Stunden warten musst. Danach die Liste aller Snapshots, lokal und in S3.

DAS SOLLTEST DU SEHEN

Zwei Zeilen mit `manuell-k3s-app1-...`: eine mit einem Pfad unter `/var/lib/rancher/k3s/server/db/snapshots/`, eine mit `s3://backup/etcd/app1/...`. Fehlt die S3-Zeile, zeigt `sudo journalctl -u k3s | grep -i s3 | tail` den Grund. Häufig ist es der fehlende `/etc/hosts`-Eintrag.

Sichere jetzt auch das Server-Token aus `/var/lib/rancher/k3s/server/token`, für jede VM einzeln, in deinem Passwort-Manager und nirgendwo sonst.

STOLPERFALLE

Der Snapshot von `obs` liegt im SeaweedFS von `obs`, also auf derselben Disk. Für Bedienfehler hilft er trotzdem, gegen einen Plattentod nicht. Erst mit AWS (am Ende dieses Kapitels) liegt auch er außer Haus.

12.4 Die Wiederherstellung üben

Du übst an `app2`, dort ist am wenigsten zu verlieren. Der Plan: eine Markierung anlegen, Snapshot ziehen, Markierung löschen, Snapshot zurückspielen. Ist die Markierung danach wieder da, hat es funktioniert.

MAC — Schritt 11

```
kubectl --context app2 create configmap restore-probe --from-literal=stand=vorher
```

Die Markierung, im Namespace `default`.

MAC — Schritt 12

```
ssh <user>@172.16.0.83
```

Öffne eine zweite Sitzung auf `k3s-app2`. Die Befehle mit dem Etikett **Mac** laufen weiter im ersten Terminal.

APP2 — Schritt 13

```
sudo k3s etcd-snapshot save --name vor-uebung
sudo ls /var/lib/rancher/k3s/server/db/snapshots/
```

Zieht den Snapshot und zeigt den vollständigen Dateinamen. Er beginnt mit `vor-uebung-k3s-app2-` und endet mit `.zip`.

MAC — Schritt 14

```
kubectl --context app2 delete configmap restore-probe
```

Der „Unfall“.

APP2 Schritt 15

```
sudo systemctl stop k3s
sudo k3s server --cluster-reset --etcd-s3=false \
  --cluster-reset-restore-path=/var/lib/rancher/k3s/server/db/snapshots/<datei>
```

k3s muss stehen, bevor du den Datastore zurücksetzt. Dann spielt die zweite Zeile den lokalen Snapshot zurück. Setze für `<datei>` den Namen aus dem vorletzten Schritt ein. `--etcd-s3=false` ist nötig, weil im Drop-in `etcd-s3: true` steht. Sonst würde k3s die Datei in S3 suchen. Der Befehl beendet sich selbst und meldet, dass du k3s jetzt ohne diese Optionen starten sollst.

APP2 Schritt 16

```
sudo systemctl start k3s
```

Startet k3s normal.

DAS SOLLTEST DU SEHEN

`kubectl --context app2 get configmap restore-probe -o jsonpath='{.data.stand}'` auf dem Mac gibt `vorher` aus. Der Cluster steht wieder auf dem Stand des Snapshots. Pods, die nach dem Snapshot entstanden sind, räumt Kubernetes in den nächsten Minuten auf.

PROBIER ES SELBST

Spiele denselben Snapshot aus S3 zurück. Dabei hilft das Secret nicht, denn beim Restore läuft noch kein Cluster, aus dem k3s es lesen könnte. Die S3-Angaben müssen deshalb als Optionen auf die Kommandozeile: `--etcd-s3`, `--etcd-s3-endpoint`, `--etcd-s3-endpoint-ca` (hier als Dateipfad auf der VM), `--etcd-s3-bucket`, `--etcd-s3-folder` und die beiden Schlüssel. Als Restore-Pfad gibst du dann nur den Dateinamen an, ohne Ordner. Überlege vorher: Wo stehen die Schlüssel während des Befehls sichtbar, und wie räumst du danach auf?

12.5 Velero für Anwendungen

Ein etcd-Snapshot sichert alles oder nichts. Willst du nur einen Namespace zurückholen, ohne den Rest des Clusters zurückzudrehen, brauchst du Velero. Velero liest die Objekte über die Kubernetes-API, packt sie in ein Archiv und legt es in S3 ab.

Velero läuft in `app1` im eigenen Namespace `velero`. Das ist die Voreinstellung der Velero-CLI und trennt die Sicherung sauber von der Observability in `monitoring`.

MAC Schritt 17

```
brew install velero
kubectl --context app1 create namespace velero
```

Installiert die CLI, ihre Version sollte zur Server-Version 1.18 passen. Dann der Namespace für Velero.

MAC Schritt 18

```
kubectl --context obs -n monitoring get secret velero-s3 \
  -o jsonpath='{.data.cloud}' | base64 -d |
kubectl --context app1 -n velero create secret generic velero-s3 \
  --from-file=cloud=/dev/stdin --dry-run=client -o yaml |
kubectl --context app1 apply -f -
```

Kopiert das Secret `velero-s3` von `obs` nach `app1`, ohne dass der Inhalt auf dem Bildschirm erscheint. Der Schlüssel `cloud` enthält eine AWS-credentials-Datei.

MAC Schritt 19

```
kubectl --context app1 -n velero create secret generic obs-ca \
  --from-file=ca.crt=observability/tls/ca.crt
```

Velero liest die CA für `s3.obs.internal` aus einem Secret, nicht aus einer ConfigMap.

DATEI `observability/backup/velero-values.yaml (Ausschnitt)`

```
initContainers:
  - name: velero-plugin-for-aws
    image: velero/velero-plugin-for-aws:v1.14.4
credentials:
  useSecret: true
  existingSecret: velero-s3
configuration:
  backupStorageLocation:
    - name: default
      provider: aws
      bucket: backup
      prefix: velero/app1
      default: true
      caCertRef: {name: obs-ca, key: ca.crt}
      config:
        region: us-east-1
        s3ForcePathStyle: "true"
        s3Url: https://s3.obs.internal
  defaultBackupTTL: 168h0m0s
snapshotsEnabled: false
deployNodeAgent: false
schedules:
  demo-taeglich:
    schedule: "0 3 * * *"
    template:
      ttl: 168h0m0s
      includedNamespaces: [demo]
```

, erklaerung: [Das AWS-Plugin spricht jedes S3, auch SeaweedFS. `s3ForcePathStyle` ist für SeaweedFS nötig. Volume-Snapshots sind aus, weil `local-path` keine kennt. Der Zeitplan sichert `demo` jede Nacht um drei, jede Sicherung lebt sieben Tage.]

STOLPERFALLE

Das Plugin muss zur Velero-Version passen. Laut Kompatibilitätstabelle im Repository `velero-plugin-for-aws` gehört zu Velero 1.18 die Plugin-Reihe 1.14. Das Chart nennt im Kommentar noch `v1.13.1` als Beispiel. Mit der falschen Reihe startet Velero, aber der Speicherort bleibt `Unavailable`.

MAC Schritt 20

```
helm --kube-context app1 upgrade --install velero vmware-tanzu/velero \
  --version 12.2.4 -n velero -f observability/backup/velero-values.yaml --wait
```

Installiert Velero 1.18.2. Das Chart legt auch die CRDs an.

MAC Schritt 21

```
velero --kubecontext app1 backup-location get
```

Zeigt den Speicherort und seinen Zustand.

DAS SOLLTEST DU SEHEN

```
NAME      PROVIDER   BUCKET/PREFIX        PHASE       LAST VALIDATED   ACCESS MODE
DEFAULT
default   aws        backup/velero/app1   Available   ...              ReadWrite
true
```

Eine Zeile `default` mit Provider `aws`, Bucket `backup/velero/app1` und Phase `Available`. Steht dort `Unavailable`, zeigt `kubectl --context app1 -n velero logs deploy/velero | grep -i error` den Grund. Typisch sind ein Zertifikatsfehler (CA-Secret fehlt) oder `AccessDenied` (falsches Secret).

MAC Schritt 22

```
velero --kubecontext app1 backup create demo-manuell \
  --include-namespaces demo --wait
```

Eine Sicherung von Hand. `--wait` wartet, bis sie fertig ist.

DAS SOLLTEST DU SEHEN

`velero --kubecontext app1 backup describe demo-manuell --details --cacert observability/tls/ca.crt` zeigt `Phase: Completed`, `Errors: 0` und eine Liste mit Deployments, Services und dem Namespace `demo`. Für `--details` lädt die CLI direkt aus S3, deshalb braucht sie die CA und den `/etc/hosts`-Eintrag auf dem Mac.

Jetzt der Ernstfall. Du löschst den ganzen Namespace.

MAC Schritt 23

```
kubectl --context app1 delete namespace demo
```

HotROD und der Lastgenerator sind weg. In Grafana reißt die Linie der Traces ab.

MAC Schritt 24

```
velero --kubecontext app1 restore create demo-zurueck \
  --from-backup demo-manuell --wait
```

Holt alle Objekte aus der Sicherung zurück.

DAS SOLLTEST DU SEHEN

`Restore completed with status: Completed.` Danach zeigt `kubectl --context app1 -n demo get pods` die Pods `hotrod-...` und `hotrod-last-...`, im Labor nach gut 30 Sekunden auf `Running`. Die Pods tragen sogar wieder denselben Namen wie vorher, denn Velero stellt die Objekte so wieder her, wie sie gesichert wurden. Nach ein paar weiteren Minuten kommen in Grafana wieder Traces aus `app1` an.

Velero sichert hier nur Objekte, keine Daten in Volumes. Für die Demo-App reicht das. Für PVCs auf `local-path` bräuchtest du das File System Backup mit `deployNodeAgent: true`.

12.6 Alarm, wenn die Sicherung ausbleibt

Eine Sicherung, die still scheitert, merkst du erst beim Restore. Velero meldet deshalb Metriken auf Port 8085. Alloy in `app1` holt sie mit einer kleinen Zusatzdatei ab und schickt sie wie alle anderen Metriken an Thanos Receive.

DATEI `observability/backup/backup.alloy (Ausschnitt)`

```
prometheus.scrape "backup_velero" {
  targets         = discovery.relabel.backup_velero.output
  job_name        = "velero"
  scrape_interval = "60s"
  forward_to      = [prometheus.remote_write.metrics_obs.receiver]
}
```

, erklaerung: [Davor stehen eine Pod-Suche im Namespace `velero` und eine Regel, die nur den Port `http-monitoring` behält. Der Versand nutzt `metrics_obs` aus `metrics.alloy`. Alle Namen tragen das Präfix `backup_`.]

MAC Schritt 25

```
kubectl --context app1 -n monitoring create configmap alloy-config \
  --from-file=observability/cluster/alloy/metrics.alloy \
  --from-file=observability/cluster/alloy/logs.alloy \
  --from-file=observability/cluster/alloy/traces.alloy \
  --from-file=observability/backup/backup.alloy \
  --dry-run=client -o yaml | kubectl --context app1 apply -f -
kubectl --context app1 -n monitoring rollout restart ds/alloy
```

Die ConfigMap aus Kapitel 5, 8 und 9 bekommt `backup.alloy` dazu. Nur in `app1`, denn nur dort läuft Velero.

MAC Schritt 26

```
kubectl --context obs apply -f observability/backup/velero-rules.yaml
```

Drei Alarme mit `auswertung: global`, ausgewertet vom Thanos Ruler.

Sie melden eine gescheiterte letzte Sicherung, eine erfolgreiche Sicherung, die älter als 25 Stunden ist, und gescheiterte Versuche in der letzten Stunde.

DAS SOLLTEST DU SEHEN

In Grafana liefert `velero_backup_last_successful_timestamp{cluster="app1"}` eine Serie mit `schedule="demo-taeglich"`, sobald der Zeitplan einmal gelaufen ist. Davor gibt es die Serie nur für manuelle Sicherungen mit leerem `schedule`.

Für die etcd-Snapshots gibt es keine fertige Metrik. Prüfe sie deshalb regelmäßig von Hand mit `k3s etcd-snapshot ls`, oder schau in S3 nach dem Datum der neuesten Datei.

12.7 Variante: Sicherungen auf AWS

Mit Kapitel 11 hast du den Bucket `<praefix>-obs-backup` mit Versionierung und den Benutzer `obs-lab-backup`. Der Umstieg tauscht nur Schlüssel und Ziel.

Erzeuge zuerst wie in Kapitel 11 mit `aws iam create-access-key` einen Schlüssel für `obs-lab-backup` und lies ihn mit `read -r AK SK` in die Shell.

MAC Schritt 27

```
printf '[default]\naws_access_key_id=%s\naws_secret_access_key=%s\n' "$AK" "$SK" |
kubectl --context app1 -n velero create secret generic velero-s3 \
  --from-file=cloud=/dev/stdin --dry-run=client -o yaml |
kubectl --context app1 apply -f -
```

Ersetzt das Velero-Secret in `app1` durch den AWS-Schlüssel.

MAC Schritt 28

```
sed "s|PRAEFIX|$PRAEFIX|" observability/backup/velero-aws.yaml |
helm --kube-context app1 upgrade velero vmware-tanzu/velero --version 12.2.4 \
  -n velero -f observability/backup/velero-values.yaml -f - --wait
```

`velero-aws.yaml` ersetzt den Speicherort komplett: Bucket in `eu-central-1`, kein `s3Url`, keine eigene CA.

Für die etcd-Snapshots baust du das Secret `k3s-etcd-s3` neu, mit `etcd-s3-endpoint=s3.eu-central-1.amazonaws.com`, `etcd-s3-region=eu-central-1`, dem Bucket

`$PRAEFIX-obs-backup` und ohne `etcd-s3-endpoint-ca`. Das Drop-in bleibt unverändert. Zum Schluss `unset AK SK`.

WAS DU GELERNT HAST

- Gesichert wird nur, was nicht im Git oder in S3 steht: vor allem etcd mit seinen Secrets und das k3s-Token.
- S3 braucht keine Basic Auth, denn jede Anfrage ist signiert. Der Name muss aber für Pods und für den Dienst auf der VM auflösbar sein.
- k3s liest die S3-Zugangsdaten für Snapshots aus einem Secret, beim Restore aber nur von der Kommandozeile.
- Velero sichert einzelne Namespaces. Das Plugin muss zur Velero-Version passen.
- Eine Sicherung zählt erst, wenn die Wiederherstellung geübt ist und ein Alarm ihr Ausbleiben meldet.

KAPITEL 13

Was sonst noch fehlt

Du prüfst deine Endpunkte von außen, überwachst die Überwachung, hältst die Serienzahl im Griff und formulierst ein erstes SLO.

Dauer: 2 Stunden, die Abschnitte sind unabhängig voneinander **Voraussetzung:** Kapitel 4 bis 9

Der Stack läuft. Metriken, Logs und Traces kommen an, Alarme gehen raus. Dieses Kapitel schließt die Lücken, die man erst bemerkt, wenn es zu spät ist: ein abgelaufenes Zertifikat, ein Cluster, der still verstummt, eine Anwendung, die plötzlich Millionen Serien erzeugt. Jeder Abschnitt steht für sich. Nimm dir zuerst den vor, der dir am meisten Sorgen macht.

13.1 Von außen prüfen: der Blackbox-Exporter

Alle bisherigen Metriken kommen von innen. Prometheus weiß, dass der Grafana-Pod läuft. Ob Grafana über `https://grafana.obs.internal` auch erreichbar ist, weiß er nicht. Dazwischen liegen DNS, Traefik, das Zertifikat und der Ingress. Der Blackbox-Exporter prüft genau diesen Weg, so wie ihn ein Nutzer geht.

DATEI `observability/extras/blackbox-values.yaml (Ausschnitt)`

```yaml
config:
  modules:
    http_2xx:
      prober: http
      timeout: 5s
      http:
        preferred_ip_protocol: ip4
        fail_if_not_ssl: true
        tls_config:
          ca_file: /etc/obs-ca/ca.crt
    http_basic_auth:
      prober: http
      http:
        valid_status_codes: [200, 401]
        fail_if_not_ssl: true
        tls_config:
          ca_file: /etc/obs-ca/ca.crt
extraConfigmapMounts:
  - name: obs-ca
    mountPath: /etc/obs-ca
    configMap: obs-ca
    readOnly: true
```

```
, erklaerung: [Zwei Prüfarten. http_2xx erwartet eine erfolgreiche Antwort. http_basic_auth
ist für Endpunkte hinter Basic Auth: Eine 401 beweist dort, dass DNS, TLS und Traefik
funktionieren. Beide prüfen das Zertifikat gegen deine CA aus Kapitel 2.]
```

MAC — Schritt 1

```
helm --kube-context obs upgrade --install blackbox \
  prometheus-community/prometheus-blackbox-exporter --version 11.21.1 \
  -n monitoring -f observability/extras/blackbox-values.yaml --wait
```

Installiert den Exporter auf `obs`. Er heißt danach `blackbox-prometheus-blackbox-exporter` und lauscht auf Port 9115.

Was geprüft wird, steht nicht im Exporter, sondern in `Probe`-Objekten. Der Prometheus-Operator macht daraus Scrape-Jobs.

DATEI `observability/extras/probes.yaml (Ausschnitt)`

```yaml
kind: Probe
metadata:
  name: obs-endpunkte
  namespace: monitoring
spec:
  jobName: blackbox
  interval: 60s
  module: http_basic_auth
  prober:
    url: blackbox-prometheus-blackbox-exporter.monitoring.svc:9115
  targets:
    staticConfig:
      static:
        - https://receive.obs.internal/api/v1/receive
        - https://loki.obs.internal/loki/api/v1/push
        - https://tempo.obs.internal/v1/traces
        - https://alertmanager.obs.internal/
```

```
, erklaerung: [Eine zweite Probe obs-grafana prüft https://grafana.obs.internal/api/health mit
http_2xx .]
```

MAC — Schritt 2

```
kubectl --context obs apply -f observability/extras/probes.yaml
```

Legt beide Probes an. Prometheus übernimmt sie nach etwa einer Minute.

MAC — Schritt 3

```
kubectl --context obs -n monitoring port-forward svc/blackbox-prometheus-blackbox-exporter 9115
```

Öffnet die Oberfläche des Exporters. Lass das Terminal offen.

MAC — Schritt 4

```
curl -s 'http://localhost:9115/probe?module=http_2xx&target=https://grafana.obs.internal/api/health' \
  | grep -E '^probe_(success|ssl_earliest|http_status)'
```

Löst eine Prüfung von Hand aus und filtert die wichtigsten Werte.

DAS SOLLTEST DU SEHEN

`probe_success 1`, `probe_http_status_code 200` und `probe_ssl_earliest_cert_expiry` mit einem Unix-Zeitstempel rund 825 Tage nach dem Tag, an dem du das Zertifikat in Kapitel 2 ausgestellt hast. Steht `probe_success 0`, zeigt dieselbe Anfrage mit `&debug=true` am Ende der URL das vollständige Protokoll der Prüfung.

Jetzt der Alarm, den Kapitel 2 angekündigt hat. Das Wildcard-Zertifikat gilt 825 Tage. Das ist lang genug, um es zu vergessen.

DATEI `observability/extras/blackbox-rules.yaml (Ausschnitt)`

```
metadata:
  labels:
    auswertung: lokal
spec:
  groups:
    - name: blackbox
      rules:
        - alert: EndpunktNichtErreichbar
          expr: probe_success{job="blackbox"} == 0
          for: 5m
        - alert: ZertifikatLaeuftBaldAb
          expr: (probe_ssl_earliest_cert_expiry{job="blackbox"} - time()) / 86400 < 30
          for: 1h
```

```
, erklaerung: [Warnung 30 Tage vor Ablauf, ein zweiter, kritischer Alarm bei 7 Tagen.
Die Rechnung zieht die aktuelle Zeit vom Ablaufdatum ab und teilt durch die Sekunden
eines Tages.]
```

WARUM DAS WICHTIG IST

Diese Regeln tragen `auswertung: lokal`. Prometheus auf `obs` wertet sie also selbst aus, ohne Umweg über Thanos. Ein Alarm „Endpunkt nicht erreichbar“ soll auch dann feuern, wenn gerade Thanos der Grund ist.

MAC Schritt 5

```
kubectl --context obs apply -f observability/extras/blackbox-rules.yaml
```

Legt die Regeln an.

PROBIER ES SELBST

Teste den Zertifikatsalarm, ohne zwei Jahre zu warten. Ändere in einer Kopie der Regel die Schwelle von `30` auf `900` Tage und wende sie unter anderem Namen an. Nach einer Stunde erscheint der Alarm in Alertmanager und in Mailpit. Lösche die Kopie danach wieder.

13.2 Wer überwacht die Überwachung?

Ein Monitoring, das ausfällt, meldet nichts. Das sieht genauso aus wie ein gesunder Cluster. Gegen diesen Fall hast du in Kapitel 7 schon die wichtigste Sicherung gebaut: den Watchdog. Er feuert immer, und der externe Dead Man's Switch schlägt Alarm, wenn er ausbleibt. Damit erfährst du, wenn die ganze Kette steht. Was der Watchdog nicht sieht, sind Teilausfälle.

Teilausfall	Woran du ihn erkennst
Ein App-Cluster schickt nichts mehr	Alarm `ClusterSchweigt` aus Kapitel 7, Panel **Ältester Datenpunkt** im Multi-Cluster-Dashboard
Receive lehnt Daten ab	Alarme `ReceiveFehler` und `ReceiveLehntAb` aus Kapitel 7
Receive kann nicht nach S3 hochladen	Alarm `ReceiveUploadFehler` aus Kapitel 7
Compactor hat aufgegeben	Alarm `CompactorAngehalten` aus Kapitel 7, Grundlage ist `thanos_compact_halted == 1`
etcd auf `obs` ist krank	Standardregeln des Charts, etwa `etcdNoLeader`. Die Daten kommen über Port 2381, den die k3s-Config mit `etcd-expose-metrics` öffnet
Ein Eingang ist von außen nicht erreichbar	Alarm `EndpunktNichtErreichbar` aus dem vorigen Abschnitt
Alloy verliert Daten	Logs von Alloy, siehe Kapitel 14, und die Zähler für verworfene Daten aus Kapitel 15

Die Regeln für die meisten Teilausfälle hast du also schon. Sie liegen in `observability/rules/` und laufen im Thanos Ruler, denn nur er sieht alle Cluster. Lege sie nicht ein zweites Mal an: Zwei Regeln mit demselben Namen und derselben Bedingung lösen jeden Alarm doppelt aus.

Wie gut die Kette im Ernstfall greift, zeigt der Ausfalltest in Kapitel 15: Dort ist obs zwei Stunden lang für die App-Cluster unerreichbar, und du siehst, welcher Alarm wann feuert und was dabei verloren geht.

Alloy kann auch über sich selbst berichten, mit der Komponente `prometheus.exporter.self`. Dann siehst du etwa, wie viele Samples Alloy nicht loswird. In diesem Buch fehlt das bewusst, um `metrics.alloy` kurz zu halten. Es ist eine gute nächste Übung.

13.3 Kardinalität: wenn Serien explodieren

Jede einzigartige Kombination aus Metrikname und Labels ist eine Serie. Ein Label mit der Benutzer-ID oder der vollen URL macht aus einer Metrik schnell hunderttausend Serien. Receive braucht dann mehr Speicher, als die VM hat, und alle Cluster leiden mit.

MAC Schritt 6

```
kubectl --context obs -n monitoring port-forward svc/thanos-query 10902
```

Öffnet die Oberfläche von Thanos Query auf `localhost:10902`. Du kannst die folgenden Abfragen genauso in Grafana unter **Explore** stellen.

Die wichtigste Abfrage zeigt die zehn Metriken mit den meisten Serien:

```
topk(10, count by (__name__)({__name__=~".+"}))
```

Diese Abfrage ist selbst teuer, denn sie fasst jede Serie an. Stelle sie nur von Hand und nie in ein Dashboard mit automatischer Aktualisierung. Hast du eine verdächtige Metrik gefunden, suchst du das schuldige Label:

```
count by (cluster, job) (kube_pod_container_status_restarts_total)
```

Die Serien pro Mandant zeigt dir Receive selbst:

```
sum by (tenant) (prometheus_tsdb_head_series{job="thanos-receive"})
```

DAS SOLLTEST DU SEHEN

Im Labor sahen die Serien je Mandant so aus:

```
["app1","55441"]   ["app2","54581"]   ["obs","139825"]   ["unbekannt","0"]
```

`obs` hat mehr als doppelt so viele Serien wie ein App-Cluster, denn dort sammelt Prometheus auch die Metriken von Thanos, Loki, Tempo und Grafana ein. Ganz oben in der `topk`-Liste standen Histogramme des API-Servers, allen voran `apiserver_request_duration_seconds_bucket` mit über 17000 Serien. Ein Histogramm erzeugt eine Serie pro Grenze `le`, pro Ressource und pro Verb, das summiert sich schnell. Die genauen Zahlen hängen von deinem Cluster ab.

Gegen einen Ausreißer hilft ein Limit pro Mandant. Thanos Receive liest es aus einer Datei.

DATEI `observability/extras/receive-limits.yaml (Ausschnitt)`

```yaml
write:
  global:
    max_concurrency: 30
  default:
    request:
      series_limit: 5000
      samples_limit: 20000
  tenants:
    obs:
      request:
        series_limit: 0
        samples_limit: 0
```

, erklaerung: [Pro Remote-Write-Anfrage höchstens 5000 Serien und 20000 Samples. Was darüber liegt, lehnt Receive mit HTTP 413 ab. `0` heißt unbegrenzt, für `obs` selbst. Ein Limit für die Gesamtzahl aktiver Serien (`head_series_limit`) gibt es auch, es braucht aber eine Meta-Monitoring-Abfrage gegen einen zweiten Prometheus.]

Die Datei liegt als ConfigMap im Repo. Damit Receive sie liest, ergänzt du in `observability/central/thanos/receive.yaml` drei Dinge: das Argument `--receive.limits-config-file=/etc/thanos-limits/limits.yaml`, ein Volume aus der ConfigMap `thanos-receive-limits` und den passenden `volumeMount`.

PROBIER ES SELBST

Baue die Ergänzung selbst ein und wende beide Dateien an. Prüfe danach in den Logs von `thanos-receive`, dass die Limits geladen wurden. Was passiert in Alloy, wenn eine Anfrage mit 413 abgelehnt wird? Tipp: Alloy versucht es bei 413 nicht erneut, die Daten sind verloren. Deshalb gehören Limits großzügig gesetzt.

13.4 SLOs mit Burn-Rate-Alarmen

Bisher alarmierst du auf Ursachen: Pod startet neu, Disk wird voll. Ein SLO dreht das um. Es beschreibt, was der Nutzer erlebt, und alarmiert nur, wenn das Versprechen in Gefahr ist.

Die Begriffe in einem Satz: Das **SLI** ist die Messgröße, hier der Anteil fehlerfreier Anfragen an den `frontend`-Dienst von HotROD. Das **SLO** ist das Ziel, hier 99 Prozent über 30 Tage. Das **Fehlerbudget** ist der Rest: 1 Prozent der Anfragen dürfen scheitern.

Die **Burn-Rate** sagt, wie schnell du das Budget verbrauchst. Bei Burn-Rate 1 ist es nach genau 30 Tagen leer. Bei 14,4 ist nach einer Stunde schon 2 Prozent des Monatsbudgets weg. Gute Alarme schauen auf zwei Zeitfenster gleichzeitig: ein langes, damit ein kurzer Ausreißer nicht weckt, und ein kurzes, damit der Alarm endet, sobald das Problem behoben ist.

Burn-Rate	Langes Fenster	Kurzes Fenster	Bedeutung
14,4	1 Stunde	5 Minuten	2 % des Budgets in einer Stunde, sofort handeln
6	6 Stunden	30 Minuten	5 % in sechs Stunden, sofort handeln
3	1 Tag	2 Stunden	10 % an einem Tag, heute noch anschauen
1	3 Tage	6 Stunden	10 % in drei Tagen, ein Ticket

Die Werte stammen aus dem SRE Workbook von Google. Die Daten liefern die Span-Metriken von Tempo aus Kapitel 9.

DATEI `observability/extras/slo-rules.yaml (Ausschnitt)`

```
- record: slo:demo_fehlerquote:rate1h
  expr: |
    sum by (cluster) (rate(traces_spanmetrics_calls_total{service="frontend",
        span_kind="SPAN_KIND_SERVER", status_code="STATUS_CODE_ERROR"}[1h]))
```

```
      /
    sum by (cluster) (rate(traces_spanmetrics_calls_total{service="frontend",
       span_kind="SPAN_KIND_SERVER"}[1h]))
# ... dieselbe Regel fuer 5m, 30m, 2h, 6h, 1d und 3d
- alert: DemoFehlerbudgetBrenntSchnell
  expr: |
    slo:demo_fehlerquote:rate1h > (14.4 * 0.01)
      and
    slo:demo_fehlerquote:rate5m > (14.4 * 0.01)
  for: 2m
```

, erklaerung: [Recording Rules rechnen die Fehlerquote für sieben Fenster einmal pro Minute vor. Die Alarme vergleichen dann nur noch fertige Werte. `0.01` ist das Fehlerbudget. Die Datei enthält alle vier Zeilen der Tabelle.]

MAC

Schritt 7

```
kubectl --context obs apply -f observability/extras/slo-rules.yaml
```

Die Regeln tragen `auswertung: global`. Die Span-Metriken kommen über Receive, deshalb sieht sie nur der Thanos Ruler.

DAS SOLLTEST DU SEHEN

Nach zwei, drei Minuten liefert `slo:demo_fehlerquote:rate5m` in Grafana einen Wert pro Cluster mit HotROD. Ist die Abfrage leer, gab es im Fenster keinen einzigen Fehler: Dann ist der Zähler leer, und die Division liefert nichts. Das ist kein Fehler der Regel.

STOLPERFALLE

Ein SLO auf 99,99 Prozent klingt gut. Bei einer Demo-App mit einer Anfrage alle drei Sekunden sind das rund 860000 Anfragen im Monat. Schon 87 Fehler leeren das Budget. Fang mit einem Ziel an, das du heute schon erreichst, und verschärfe es später.

PROBIER ES SELBST

Ergänze ein zweites SLO für die Latenz: 95 Prozent der Anfragen an `frontend` sollen schneller als 500 Millisekunden sein. Die Daten stehen in `traces_spanmetrics_latency_bucket`. Überlege zuerst, wie du „Anteil schneller als 500 ms“ aus einem Histogramm mit `le`-Grenzen ausrechnest, bevor du die Regel schreibst.

13.5 Profiling mit Pyroscope (optional)

Metriken sagen, **dass** ein Dienst viel CPU braucht, Traces, **welche** Anfrage langsam ist. Profile zeigen, **welche Funktion** im Code die Zeit verbraucht. Grafana Pyroscope speichert solche Profile und zeigt sie als Flammendiagramm. Es ist das vierte Signal, und das jüngste.

Für einen ersten Blick reicht ein einzelner Pod auf `obs`.

MAC Schritt 8

```
helm --kube-context obs upgrade --install pyroscope grafana/pyroscope \
  --version 2.4.3 -n monitoring -f observability/extras/pyroscope-values.yaml --wait
kubectl --context obs apply -f observability/extras/pyroscope-datasource.yaml
```

Installiert Pyroscope mit 5 GiB lokalem Speicher und ohne das mitgelieferte zweite Alloy. Die Datenquelle erscheint nach einer Minute in Grafana.

Daten sammelt Alloy, entweder per eBPF für alle Prozesse eines Nodes (`pyroscope.ebpf`, braucht privilegierte Rechte) oder indem es Go-Programme über ihre pprof-Schnittstelle abfragt (`pyroscope.scrape`). Beides ist eine eigene `.alloy`-Datei wert. Wie man Profile liest, zeigt das Buch „Kubernetes Observability Hands-On“ ausführlicher. Willst du Pyroscope nicht behalten, entfernst du es mit `helm uninstall pyroscope` und löschst das PVC.

13.6 Absicherung und Datenschutz

Observability-Daten sind Betriebsdaten, aber nicht harmlos. Logs enthalten IP-Adressen, Benutzernamen, manchmal E-Mail-Adressen in URLs oder ganze Anfragen. Das sind personenbezogene Daten im Sinne der DSGVO. Fünf Fragen solltest du für dein Setup beantworten können.

Was steht in den Logs? Such einmal gezielt danach, etwa mit `{cluster="app1"} |~ "[a-zA-Z0-9._%+-]+@[a-zA-Z0-9.-]+"` in Grafana. Am besten entfernst du solche Daten schon im Cluster, bevor sie das Haus verlassen. In Alloy geht das mit einer Stufe `stage.replace` in `loki.process`, die ein Muster durch einen Platzhalter ersetzt.

Wie lange liegt was? Die Aufbewahrung steht in deinen Werten: 7 Tage für Logs und Traces, bis zu 365 Tage für verdichtete Metriken. Schreib diese Zahlen und ihren Zweck auf. Das ist der Kern dessen, was ein Verarbeitungsverzeichnis von dir wissen will. Einzelne Log-Zeilen löschst du vor Ablauf über die Lösch-API von Loki. Einen Teil der Voraussetzungen, `delete_request_store`, hast du in Kapitel 8 schon gesetzt. Was sonst noch nötig ist, steht in der Loki-Doku unter „Log entry deletion“.

Brauchen die Cluster unterschiedliche Fristen, kann Loki das pro Mandant. Im Chart setzt du dafür unter `loki.runtimeConfig` einen Block `overrides`, darin pro Mandant ein eigenes `retention_period`, etwa `app2: {retention_period: 72h}`. Loki liest diese Datei im laufenden Betrieb neu ein. Bei Thanos gilt die Retention des Compactors dagegen für den ganzen Bucket. Unterschiedliche Fristen pro Mandant gehen dort nur mit mehreren Compactors, die sich über `--selector.relabel-config` nach dem Label `tenant_id` aufteilen. Das ist für ein Labor zu viel Aufwand.

Wer darf was sehen? In Grafana hat jeder angemeldete Benutzer mindestens die Rolle **Viewer** und sieht damit alle Datenquellen. Lege Teams an und vergib Rechte pro Ordner. Wer nur die Logs von `app1` sehen darf, bekommt eine eigene Loki-Datenquelle mit `X-Scope-OrgID: app1` statt der Mehrmandanten-Abfrage `obs|app1|app2`.

Kann sich ein Cluster als anderer ausgeben? Ja, im jetzigen Aufbau schon. Basic Auth prüft, **wer** schreibt. Den Mandanten bestimmt aber der Header, den Alloy mitschickt, also `THANOS-TENANT` oder `X-Scope-OrgID`. Wer das Passwort von `app1` kennt, kann mit dem Header `app2` in dessen Mandanten schreiben. Für ein Labor mit drei eigenen Clustern ist das vertretbar. Für mehr brauchst du eine Instanz, die den Header aus der Anmeldung ableitet und nicht dem Client glaubt. Traefik bietet dafür in der Basic-Auth-Middleware die Option `headerField`: Sie schreibt den angemeldeten Benutzernamen in einen Header deiner Wahl. Weil deine Benutzer genauso heißen wie die Mandanten, wäre das mit je einer Middleware pro Header-Namen eine schlanke Lösung. Teste aber selbst, was mit einem Header passiert, den der Client schon mitschickt, bevor du dich darauf verlässt. Die robuste Alternative ist ein eigener Proxy vor Receive, Loki und Tempo, etwa der Enterprise Gateway von Grafana oder oauth2-proxy mit einer Zuordnung von Benutzer zu Mandant.

Wie oft wechseln die Zugänge? Die Basic-Auth-Passwörter der Cluster laufen nie ab. Wechsle sie regelmäßig, mindestens aber, wenn jemand das Team verlässt. Weil der Benutzername zugleich der Clustername ist, kann es pro Cluster nur einen Eintrag in `obs-basic-auth-users` geben. Tausche deshalb das Passwort dort und in `obs-remote-auth` direkt nacheinander und starte Alloy sofort neu. In der Lücke dazwischen lehnt Traefik mit `401` ab, und diese Daten gehen verloren. Bei den S3-Schlüsseln geht es ohne Lücke: Ein AWS-Benutzer darf zwei Schlüssel gleichzeitig haben, ein SeaweedFS-Konto mehrere Einträge unter `credentials`.

WARUM DAS WICHTIG IST

Rotation scheitert selten an der Technik, sondern an der Reihenfolge. Wo zwei Geheimnisse gleichzeitig gelten dürfen, gilt: erst das neue ergänzen, dann alle Nutzer umstellen, dann das alte entfernen. So entsteht kein Zeitfenster, in dem niemand schreiben darf.

13.7 Ausblick: GitOps

Du hast in diesem Buch jede Änderung mit `kubectl apply` und `helm upgrade` vom Mac aus ausgerollt. Das ist zum Lernen richtig, denn du siehst jeden Schritt. Im Alltag wird es mühsam: Hat jemand etwas von Hand geändert? Ist `app2` auf demselben Stand wie `app1`? Werkzeuge wie Flux oder Argo CD drehen den Weg um. Sie laufen in jedem Cluster, lesen dieses Repo und gleichen

den Cluster laufend mit dem Stand im Git ab. Weicht etwas ab, setzen sie es zurück. Deine Ordnerstruktur unter `observability/` ist dafür schon gut vorbereitet, denn alles liegt als Datei vor. Offen bleiben die Secrets, die du bewusst nie ins Git geschrieben hast. Dafür brauchst du dann ein Werkzeug wie External Secrets oder SOPS. Wie GitOps mit Argo CD für eine Plattform mit vielen Kunden und Clustern aussieht, inklusive Onboarding neuer Cluster, zeigt Kapitel 16.

WAS DU GELERNT HAST

- Der Blackbox-Exporter prüft den Weg eines Nutzers von außen, inklusive Zertifikatsablauf.
- Der Watchdog meldet den Totalausfall, Teilausfälle brauchen eigene Regeln wie `ClusterSchweigt`.
- Kardinalität findest du mit `topk` über `count by (__name__)`, und Limits pro Mandant schützen Receive.
- Burn-Rate-Alarme mit zwei Fenstern melden, wenn das Fehlerbudget in Gefahr ist, nicht bei jedem Ausreißer.
- Logs enthalten personenbezogene Daten. Aufbewahrung, Zugriff und Rotation gehören dokumentiert.

KAPITEL 14

Betrieb und Fehlersuche

Du aktualisierst den Stack in einer sicheren Reihenfolge, misst seinen Verbrauch und findest typische Fehler nach einem festen Muster.

Dauer: 30 Minuten Lesen, danach als Nachschlagewerk **Voraussetzung:** Ein laufender Stack aus Kapitel 2 bis 9

14.1 Updates in der richtigen Reihenfolge

Ein Update betrifft selten nur eine Komponente. Alloy spricht mit Receive, Grafana mit Thanos Query, der Compactor mit den Blöcken, die Receive hochlädt. Die Grundregel: Erst wird aktualisiert, wer empfängt, dann wer sendet. Ein neuer Empfänger versteht fast immer noch die alten Sender, umgekehrt ist das nicht garantiert.

Schritt	Was und warum
0	etcd-Snapshot auf allen drei VMs (Kapitel 12). Das ist dein Rückweg.
1	Release Notes lesen, ab deiner Version bis zur Zielversion. Achte auf „breaking“ und auf geänderte Werte-Namen.
2	SeaweedFS. Alles andere hängt am Speicher.
3	Thanos: erst Store, Compactor und Query, dann Receive.
4	kube-prometheus-stack. Vorher die CRDs aktualisieren, das macht Helm bei einem Upgrade nicht. Die Befehle stehen im README des Charts unter „Upgrading“.
5	Loki und Tempo.
6	Alloy, kube-state-metrics und node-exporter, erst in `app2`, dann `app1`, zuletzt `obs`.
7	k3s selbst, eine VM nach der anderen, `app2` zuerst, `obs` zuletzt.

MAC Schritt 1

```
helm --kube-context obs  list -A
helm --kube-context app1 list -A
helm --kube-context app2 list -A
```

Zeigt alle Releases mit Chart- und App-Version, einen Block pro Cluster. Vergleiche sie mit der Versions-Tabelle im Anhang.

MAC Schritt 2

```
helm repo update
helm search repo grafana-community/loki --versions | head -5
```

Zeigt die neuesten Chart-Versionen. Spring nicht über mehrere Hauptversionen auf einmal.

MAC Schritt 3

```
helm template loki grafana-community/loki --version <neu> \
  -f observability/central/loki/values.yaml > /dev/null
```

Rendert die neue Version mit deinen Werten, ohne etwas zu installieren. Ein Fehler hier kostet dich nichts, im Cluster schon.

STOLPERFALLE

Aktualisiere nie alles in einem Rutsch. Nach jedem Schritt wartest du, bis alle Pods `Running` sind, und schaust in Grafana, ob Daten aus allen drei Clustern ankommen. Erst dann kommt der nächste Schritt. Geht etwas schief, weißt du genau, welcher es war.

14.2 Ressourcen messen statt schätzen

Die Speicherangaben in Kapitel 1 waren Schätzungen. Jetzt, wo alles läuft, misst du. k3s bringt den metrics-server mit, daher funktioniert `kubectl top` sofort.

MAC Schritt 4

```
kubectl --context obs top nodes
kubectl --context obs -n monitoring top pods --sort-by=memory
```

Momentaufnahme von Node und Pods, die größten Verbraucher oben.

DAS SOLLTEST DU SEHEN

Ganz oben stehen typischerweise Prometheus, Thanos Receive, Loki und Tempo. Liegt der Node über 85 Prozent Speicher, wird es eng. Dann hilft eine größere VM mehr als jede Feineinstellung.

Eine Momentaufnahme lügt, wenn gerade nichts los ist. Der Verlauf in Grafana sagt mehr. Diese Abfragen zeigen Speicher und CPU pro Pod und die Lücke zur Anfrage im Manifest:

```
sum by (pod) (container_memory_working_set_bytes{cluster="obs",
namespace="monitoring", container!=""})

sum by (pod) (rate(container_cpu_usage_seconds_total{cluster="obs",
namespace="monitoring", container!=""}[5m]))

sum by (pod) (kube_pod_container_resource_requests{cluster="obs",
namespace="monitoring", resource="memory"})
```

Setze die Anfragen (`requests`) auf den typischen Verbrauch über eine Woche und das Speicherlimit auf etwa das Doppelte des höchsten Werts. Der Platz auf der Disk ist die dritte Größe:

```
node_filesystem_avail_bytes{cluster="obs", mountpoint="/"} /
node_filesystem_size_bytes{cluster="obs", mountpoint="/"}
```

OBS Schritt 5

```
sudo du -sh /var/lib/rancher/k3s/storage/*
```

`local-path` legt jedes PVC als Ordner auf der VM ab. So siehst du, welches am meisten belegt.

14.3 Typische Fehler

Geh bei jedem Problem von außen nach innen vor: Was sagt Grafana, was sagen die Logs des Senders, was die Logs des Empfängers? Die Tabelle sammelt die Fehler, die in diesem Aufbau am häufigsten auftreten.

Symptom	Ursache	Prüfen mit	Lösung
Receive-Log: `out of order sample`	Zwei Sender schreiben dieselbe Serie, oder die Uhr einer VM geht falsch	`kubectl logs sts/thanos-receive -n monitoring | grep -i "out of order"`, auf der VM `timedatectl`	Eindeutige Labels (`cluster`, Replika-Label), nur ein Scraper pro Ziel, NTP an
Alloy-Log: HTTP `429` oder `413`	Limit in Receive überschritten, zu viele Serien	`kubectl logs ds/alloy -c alloy -n monitoring | grep -E "429|413"`	Kardinalität senken (Kapitel 13), Limit anheben
Receive nimmt nichts an	Falsches Passwort (`401`), fehlender Header (Mandant `unbekannt`) oder unbekannte CA (`x509`)	Alloy-Log nach `receive` filtern, in Grafana `count by (tenant_id) (up)`	`obs-remote-auth` gegen Basic Auth prüfen, Header in `metrics.alloy`, ConfigMap `obs-ca`
`thanos_compact_halted 1`, Log: `overlaps found`	Zwei Blöcke mit gleichen Labels und überlappender Zeit, etwa nach doppeltem Upload	`kubectl logs deploy/thanos-compact -n monitoring | grep -i overlap`	Doppelten Block löschen oder mit `thanos tools bucket mark` ausschließen, Compactor neu starten
Loki: `entry too far behind`	Zeilen sind älter als das erlaubte Fenster hinter der neuesten Zeile ihres Streams, oft	`kubectl logs ds/alloy -c alloy -n monitoring | grep "too far behind"`	Meist harmlos und einmalig. Tritt es dauernd auf, Labels und Zeitstempel prüfen

Symptom	Ursache	Prüfen mit	Lösung
	nach einem Alloy-Neustart		
Tempo zeigt keine Traces	App sendet nicht, `traces.alloy` fehlt in der ConfigMap oder falscher Mandant in der Datenquelle	Alloy-Log nach `traces_` filtern, `sum by (tenant) (rate(tempo_distributor_spans_received_total[5m]))`	OTLP-Endpunkt der App, ConfigMap neu bauen, Mandant in Grafana
Keine Span-Metriken oder kein Exemplar-Link für `app1`	Der Metrics-Generator schreibt als Mandant `obs`. Ein Filter auf `tenant_id="app1"` findet nichts. Exemplars hängen unter dem Label `traceID`	`count by (tenant_id, cluster) (traces_spanmetrics_calls_total)`	Nach `cluster` filtern, nicht nach `tenant_id`. In der Datenquelle Thanos muss `traceID` auf Tempo verweisen
Alloy-Pod nicht `Ready`	Syntaxfehler, doppelter Komponentenname, fehlendes Secret	`kubectl logs ds/alloy -c alloy -n monitoring | head -30`	Datei korrigieren, Präfix pro Datei einhalten, Secret anlegen
`no such host` für `*.obs.internal`	`coredns-custom` fehlt oder Schlüssel endet nicht auf `.server`	Siehe Befehl unten	ConfigMap anwenden, `kubectl rollout restart deploy/coredns -n kube-system`
`x509: certificate has expired`	Wildcard-Zertifikat abgelaufen	Siehe Befehl unten	Neu ausstellen wie in Kapitel 2, Secret `obs-tls` ersetzen

MAC — Schritt 6

```
kubectl --context app1 run dnstest --rm -it --restart=Never \
  --image=busybox:1.37 -- nslookup receive.obs.internal
```

Startet einen Wegwerf-Pod und fragt CoreDNS. Erwartet ist die Adresse `172.16.0.126`.

MAC — Schritt 7

```
openssl s_client -connect 172.16.0.126:443 -servername grafana.obs.internal \
  </dev/null 2>/dev/null | openssl x509 -noout -enddate
```

Zeigt das Ablaufdatum des Zertifikats, das Traefik gerade ausliefert. Den Alarm dafür hast du in Kapitel 13 gebaut.

STOLPERFALLE

Ein angehaltener Compactor fällt zuerst nicht auf. Receive lädt weiter hoch, Abfragen funktionieren. Aber nichts wird mehr verdichtet und nichts gelöscht, und der Bucket wächst ungebremst. Der Alarm `CompactorAngehalten` aus Kapitel 7 ist deshalb einer der wichtigsten.

PROBIER ES SELBST

Erzeuge absichtlich einen der Fehler aus der Tabelle, etwa ein falsches Passwort in `obs-remote-auth` in `app2`. Finde ihn nur mit den Befehlen aus der Spalte „Prüfen mit“, ohne nachzusehen, was du geändert hast. Wie lange hat es gedauert, bis ein Alarm kam?

14.4 Aufräumen

Wenn du das Labor abbaust, geh in umgekehrter Reihenfolge vor. So bleibt nichts zurück, was weiter Geld kostet.

MAC — Schritt 8

```
aws s3 rb "s3://$PRAEFIX-obs-thanos" --force
aws s3 rb "s3://$PRAEFIX-obs-loki" --force
aws s3 rb "s3://$PRAEFIX-obs-tempo" --force
```

Leert und löscht die drei Buckets ohne Versionierung. Nur wenn du Kapitel 11 gemacht hast.

STOLPERFALLE

Beim Backup-Bucket funktioniert `--force` nicht vollständig. Er hat Versionierung, und `rb --force` löscht nur die aktuellen Versionen. Leere ihn in der AWS-Konsole mit **Empty** und lösche ihn danach. Genau diesen Schutz hast du in Kapitel 11 gewollt.

Ein IAM-Benutzer lässt sich erst löschen, wenn seine Schlüssel und Policies weg sind. Du gehst deshalb in drei Schritten vor, zuerst für `obs-lab`:

MAC — Schritt 9

```
aws iam list-access-keys --user-name obs-lab \
  --query 'AccessKeyMetadata[].AccessKeyId' --output text
aws iam list-user-policies --user-name obs-lab \
  --query 'PolicyNames' --output text
```

Zeigt die IDs der Zugangsschlüssel und die Namen der Policies. Du brauchst sie im nächsten Schritt.

MAC Schritt 10

```
aws iam delete-access-key --user-name obs-lab --access-key-id <ID aus der Ausgabe>
aws iam delete-user-policy --user-name obs-lab --policy-name <Name aus der Ausgabe>
aws iam delete-user --user-name obs-lab
```

Löscht Schlüssel, Policy und zuletzt den Benutzer. Hat der Benutzer mehrere Schlüssel oder Policies, wiederholst du die ersten beiden Zeilen für jeden.

Danach wiederholst du beide Schritte für den Benutzer `obs-lab-backup`.

APP1 Schritt 11

```
sudo /usr/local/bin/k3s-uninstall.sh
```

Entfernt k3s samt allen Daten von der VM. Auf allen drei VMs ausführen. Danach sind auch SeaweedFS und alle PVCs weg.

MAC Schritt 12

```
kubectl config delete-context obs
kubectl config delete-context app1
kubectl config delete-context app2
```

Entfernt die drei Kontexte aus `~/.kube/config`. Die Einträge für Cluster und Benutzer listet `kubectl config get-clusters` und `kubectl config get-users`, du löschst sie mit `delete-cluster` und `delete-user`.

Zum Schluss entfernst du die Zeilen mit `obs.internal` aus `/etc/hosts` auf dem Mac und löschst den privaten CA-Schlüssel `observability/tls/ca.key`. Ohne ihn kann niemand mehr Zertifikate ausstellen, denen deine Rechner vertrauen.

WAS DU GELERNT HAST

- Updates laufen in fester Reihenfolge: Snapshot, Release Notes, Empfänger vor Sendern, ein Schritt nach dem anderen.
- Ressourcen misst du mit `kubectl top` für den Moment und mit PromQL für den Verlauf.
- Die meisten Fehler zeigen sich in den Logs von Alloy oder Receive. Die Tabelle sagt dir, wonach du suchst.
- Ein angehaltener Compactor ist leise, aber teuer. Er braucht einen eigenen Alarm.
- Aufräumen heißt auch: versionierte Buckets, IAM-Schlüssel und den CA-Schlüssel nicht vergessen.

KAPITEL 15

Vom Labor zur Enterprise-Architektur: Hochverfügbarkeit und Datenverlust vermeiden

Du weißt, was mit Metriken, Logs und Traces passiert, wenn die Zentrale ausfällt, welche Bausteine dein Labor zu einer hochverfügbaren Plattform machen und wie du einen Standortausfall übersteht.

Dauer: 60 Minuten Lesen **Voraussetzung:** Kapitel 1 bis 12. Dieses Kapitel installiert nichts.

Bis hierher hast du eine Plattform gebaut, die funktioniert und die du verstehst. Sie hat aber eine Schwäche, die du von Anfang an kanntest: Alles Wichtige läuft auf einer einzigen VM. Fällt `obs` aus, sieht niemand mehr etwas, und ein Teil der Daten ist verloren.

Dieses Kapitel zeigt, wie aus dem Labor eine Plattform wird, die auch im Ernstfall trägt. Es beginnt mit einer Messung im Labor: Was passiert wirklich, wenn `obs` eine Stunde lang nicht erreichbar ist? Danach gehst du die Bausteine einzeln durch und siehst, wie jeder redundant wird. Am Ende steht eine Referenzarchitektur mit Aufwand und Kosten.

SKIZZEN, KEINE FERTIGEN REZEPTE

Die Laborkapitel enthalten getestete Befehle. Dieses Kapitel nicht. Die YAML-Ausschnitte hier zeigen, **welche** Einstellung zählt, nicht eine fertige Konfiguration für deinen Betrieb. Sie wurden nicht auf einem hochverfügbaren Cluster getestet. Prüfe jede Einstellung gegen die Doku deiner Versionen und teste sie mit den Ausfalltests am Ende dieses Kapitels, bevor du dich darauf verlässt.

15.1 Wo das Labor an seine Grenzen stößt

Ein **Single Point of Failure** ist ein Teil, dessen Ausfall das Ganze lahmlegt. Im Labor ist das fast jede Komponente:

Komponente	Im Labor	Was ein Ausfall bewirkt
VM `obs`	ein Node	Alles auf der Zentrale steht: Speichern, Abfragen, Alarme
Traefik	ein Pod	Kein Cluster kann mehr schreiben, Grafana ist nicht erreichbar
Thanos Receive	eine Replika	Keine neuen Metriken, Alloy puffert
Loki	ein Prozess, `replication_factor: 1`	Keine neuen Logs, Alloy puffert kurz
Tempo	ein Prozess	Keine neuen Traces, Alloy puffert sehr kurz
SeaweedFS	ein Pod, eine Kopie jeder Datei	Kein Upload, keine alten Daten; Plattendefekt heißt Datenverlust
Alertmanager	eine Replika	Keine Benachrichtigungen. Nur der Dead Man's Switch merkt es
Grafana	eine Replika, ohne Datenbank	Keine Ansicht, aber kein Datenverlust
Alloy in den App-Clustern	ein Pod pro Node, Puffer im Container	Neustart löscht den Puffer

Die letzte Zeile ist leicht zu übersehen, und sie betrifft dich auch ohne Ausfall der Zentrale. Darauf kommt der Abschnitt über Pufferung zurück.

15.2 Gemessen: obs fällt eine Stunde aus

Bevor du Hochverfügbarkeit planst, solltest du wissen, was ein Ausfall heute kostet. Deshalb wurde das Labor für dieses Kapitel absichtlich gestört.

Der Versuch. Traefik auf `obs` wurde auf null Replikas skaliert:

```
kubectl --context obs -n kube-system scale deploy traefik --replicas=0
```

Für `app1` und `app2` ist die Zentrale damit unerreichbar, Verbindungen laufen ins Leere. Auf `obs` selbst läuft alles weiter, so lässt sich danach genau nachzählen, was angekommen ist. Nach 66 Minuten wurde Traefik wieder gestartet. Während des Ausfalls hat ein kleines Skript alle fünf Minuten die Zähler von Alloy in `app1` mitgeschrieben. Danach wurde für jedes Signal gezählt, was aus der Ausfallzeit gespeichert ist. Die Demo-App HotROD lief die ganze Zeit mit gleichbleibender Last.

Was während des Ausfalls passiert

Nach	Beobachtung
2 Minuten	Alloy hält 12.423 Metrik-Proben im WAL zurück. Erste Traces stehen in der Warteschlange.
5 bis 7 Minuten	Die ersten 242 Spans sind endgültig verworfen. Ihre fünf Minuten für Wiederholungen sind abgelaufen.
8 Minuten	`ClusterSchweigt` feuert für `app1` und `app2`. Der Alarm kommt aus der Zentrale, die ja noch läuft.
12 Minuten	Alloy verwirft die ersten 47 Log-Zeilen. Nach 37 Minuten sind es 16.348.
42 Minuten	Die Warteschlange für Traces ist voll, 1000 Anfragen. Neue Spans lehnt Alloy jetzt sofort ab.
62 Minuten	Verworfen: 24.621 Log-Zeilen, 2.074 Spans nach Ablauf der Wiederholungen, 23.592 Spans ohne Platz in der Warteschlange. Metriken: keine.

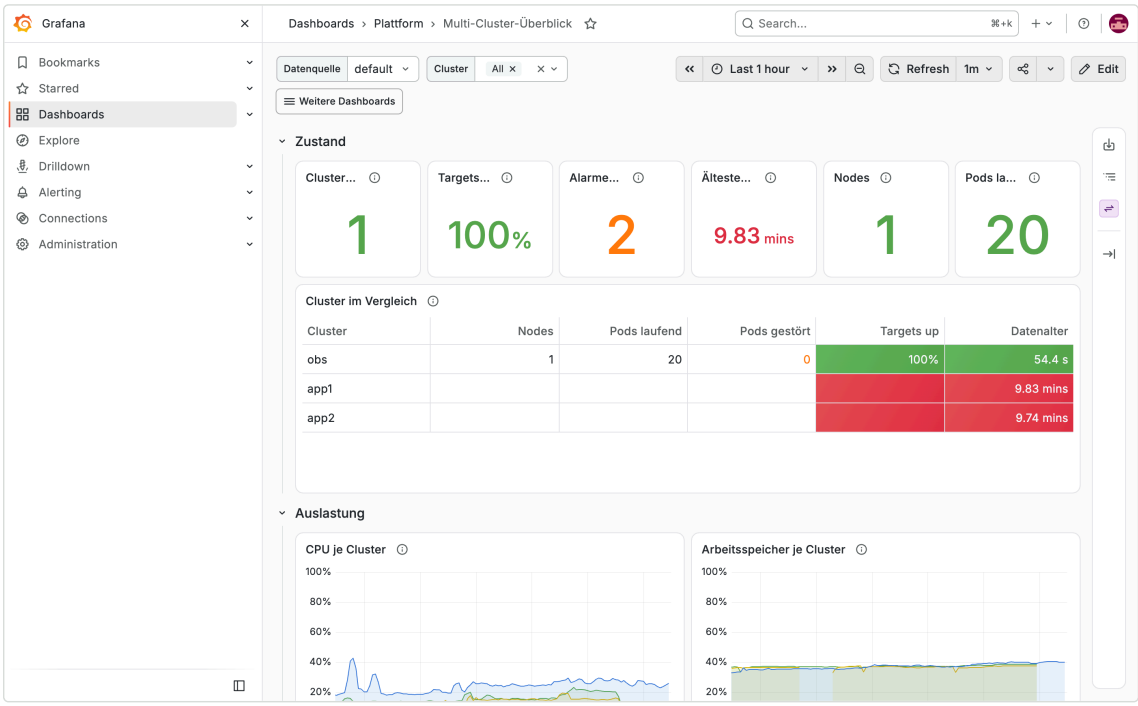


Abbildung 27: Das Multi-Cluster-Dashboard aus Kapitel 10 nach knapp zehn Minuten Ausfall. Für `app1` und `app2` gibt es keine aktuellen Werte mehr, das Datenalter steht rot bei fast zehn Minuten.

Was nach dem Ausfall ankommt

Zwei Minuten nach dem Neustart von Traefik hatte Alloy die Metriken aus dem WAL vollständig nachgeliefert, ohne eine einzige abgelehnte Probe. Dann wurde gezählt:

Signal	Aus 66 Minuten Ausfall gespeichert	Grund
Metriken	**alles**: in jedem 5-Minuten-Fenster alle 10 Proben von `up`	WAL auf der Platte. Jeder Cluster ist ein eigener Mandant, deshalb lagen die nachgelieferten Proben nicht „zu weit zurück“.
Logs	**etwa die Hälfte**: die erste Minute und die letzten 33 Minuten. Rund 32 Minuten fehlen.	Puffer im Speicher, nach zehn Versuchen verworfen. Siehe die Erklärung unten.
Traces	**nichts**	Warteschlange im Speicher, nach fünf Minuten verworfen, danach voll.

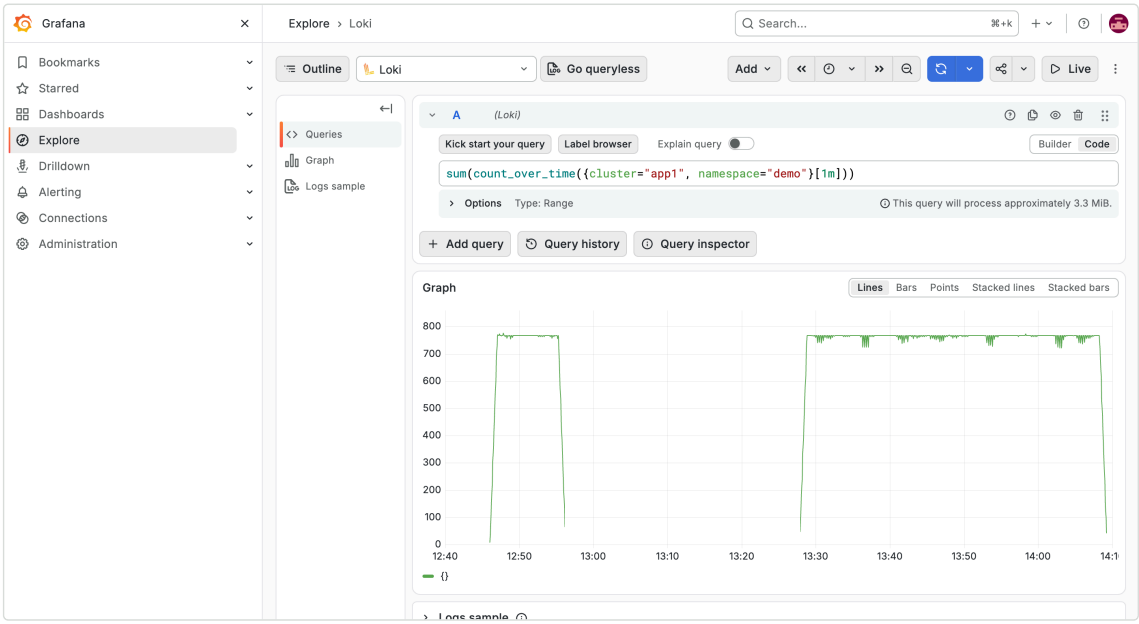


Abbildung 28: Log-Zeilen der Demo-App pro Minute in Loki, nach dem Ausfall abgefragt. Links die Zeit vor dem Ausfall, in der Mitte die Lücke von gut einer halben Stunde, rechts ab dem letzten Drittel des Ausfalls wieder vollständig. Die Achse zeigt Ortszeit.

Warum die Logs zur Hälfte zurückkamen. Die Zeilen der ersten halben Stunde hatte die Quelle `loki.source.kubernetes` schon gelesen und an `loki.write` übergeben. Dort scheiterten sie nach zehn Versuchen und wurden verworfen. Danach war der Puffer von `loki.write` voll, und die Quelle konnte keine neuen Zeilen mehr loswerden. Sie hörte auf zu lesen, und ihre gemerkte Position blieb stehen. Nach dem Ausfall las sie ab dieser Position weiter, über die Kubernetes-API, die die Logs der laufenden Pods noch hatte. Das ist die Deutung des Verhaltens anhand der Zähler und des Quellcodes von Alloy, nicht eine dokumentierte Zusage. Wäre ein Pod in der Zwischenzeit neu gestartet oder sein Log rotiert worden, wären auch diese Zeilen verloren gewesen.

WARUM DAS WICHTIG IST

Die Messung zeigt das Muster, das du dir merken solltest: **Metriken überstehen einen Ausfall von einer Stunde, Logs nur zum Teil und Traces gar nicht.** Bei längeren Ausfällen kommen für Metriken zwei Grenzen dazu: `max_keepalive_time` (8 Stunden) und der Neustart von Alloy, der den WAL im Labor löscht. Welche Daten du verlieren darfst, ist eine Entscheidung, die du vor dem Ausfall triffst, nicht danach.

15.3 Wie Alloy puffert, und wo die Grenzen liegen

Alloy behandelt die drei Signale völlig unterschiedlich. Das ist kein Versehen, sondern folgt aus den Komponenten, die du in den Kapiteln 5, 8 und 9 eingesetzt hast.

Metriken: der Write-Ahead-Log

`prometheus.remote_write` schreibt jede Probe zuerst in einen **Write-Ahead-Log** (WAL) auf der Platte und schickt sie erst danach los. Antwortet Receive nicht oder mit einem 5xx-Fehler, versucht Alloy es wieder, mit wachsenden Pausen bis `max_backoff` (Standard 5 Sekunden). Die Proben bleiben im WAL, bis sie angekommen sind.

Einstellung	Standard	Bedeutung
`wal.max_keepalive_time`	`8h`	Proben, die älter sind, löscht Alloy aus dem WAL, auch wenn sie nie angekommen sind
`wal.min_keepalive_time`	`5m`	So lange bleiben Proben mindestens im WAL
`wal.truncate_frequency`	`2h`	Wie oft Alloy den WAL aufräumt
`queue_config.max_backoff`	`5s`	Längste Pause zwischen zwei Versuchen
`queue_config.retry_on_http_429`	`true`	Bei „zu viele Anfragen“ nach `Retry-After` erneut senden

Zwei Grenzen musst du kennen. Erstens wirft Alloy Proben weg, die Receive mit einem 4xx-Fehler außer 429 ablehnt. Die Doku sagt dazu: „Other HTTP 4xx status code responses are never considered recoverable errors.“ Genau so einen Fehler erzeugt Receive, wenn Proben zu alt sind (dazu gleich mehr). Zweitens liegt der WAL dort, wo Alloy seinen Speicher hat.

STOLPERFALLE

Der WAL überlebt im Labor keinen Neustart. Die Werte aus Kapitel 5 starten Alloy mit `--storage.path=/tmp/alloy`, und `/tmp/alloy` liegt im Dateisystem des Containers, nicht auf einem Volume. Jeder Neustart des Pods, auch das `rollout restart` nach einer Änderung der Konfiguration, löscht damit alles, was noch nicht angekommen war. Für den Betrieb gehört der Speicherpfad auf ein `hostPath`-Volume des Nodes oder auf ein PersistentVolume. Dann übersteht der Puffer auch einen Neustart.

Logs: Puffer im Speicher, begrenzte Geduld

`loki.write` sammelt Zeilen im Arbeitsspeicher und schickt sie in Paketen. Scheitert ein Paket, versucht Alloy es mit wachsender Pause erneut, standardmäßig höchstens zehnmal (`max_backoff_retries = 10`, Pausen von 500 ms bis höchstens 5 Minuten). Danach verwirft es das Paket und zählt die Zeilen in `loki_write_dropped_entries_total`. Rechnerisch gibt Alloy ein Paket nach rund achteinhalb Minuten auf.

Dazu kommt `loki.source.kubernetes`, die Quelle der Pod-Logs. Sie merkt sich, bis wohin sie gelesen hat, und liest nach einem Neustart ab dieser Stelle weiter. Die Stelle rückt aber vor, sobald eine Zeile an `loki.write` übergeben ist, nicht erst, wenn Loki sie bestätigt hat. Was im Speicher von `loki.write` verloren geht, liest die Quelle also nicht erneut.

Es gibt zwei Stellschrauben, beide mit Vorsicht zu genießen:

- `max_backoff_retries = 0` wiederholt unbegrenzt. Dann wächst der Speicher von Alloy während eines Ausfalls, und irgendwann blockiert die Quelle.
- Ein `wal`-Block für `loki.write` existiert, ist aber **experimentell**. Alloy muss dafür mit `--stability.level=experimental` laufen, und die Funktion kann sich in einer späteren Version ändern oder wegfallen.

Traces: kurze Warteschlange im Speicher

`otelcol.exporter.otlphttp` hält eine Warteschlange von 1000 Anfragen im Speicher (`sending_queue`). Scheitert eine Anfrage, wiederholt der Exporter sie mit Pausen von 5 bis 30 Sekunden, aber höchstens fünf Minuten lang (`retry_on_failure.max_elapsed_time`). Danach ist die Anfrage verloren, sie erscheint in `otelcol_exporter_send_failed_spans_total`.

Eine persistente Warteschlange ist möglich: Die Komponente `otelcol.storage.file` speichert die Warteschlange auf der Platte, und `sending_queue` verweist mit `storage` darauf. `otelcol.storage.file` ist allerdings als **Public preview** eingestuft, also noch nicht allgemein freigegeben.

DATEI Skizze: persistente Warteschlange für Traces (nicht getestet)

```
otelcol.storage.file "traces_queue" {
  // Verzeichnis unter dem Speicherpfad von Alloy, der persistent sein muss
}

otelcol.exporter.otlphttp "traces_tempo" {
  sending_queue {
    storage    = otelcol.storage.file.traces_queue.handler
    queue_size = 5000
  }
  retry_on_failure {
    max_elapsed_time = "0s"   // unbegrenzt wiederholen
  }
  // client { ... } wie in Kapitel 9
}
```

Die Skizze zeigt nur das Prinzip. Ob die Warteschlange einen Ausfall von zwei Stunden übersteht, hängt von Datenmenge, `queue_size` und Plattenplatz ab. Eine garantierte Pufferdauer gibt es nicht.

Zusammengefasst

	Metriken	Logs	Traces
Puffer	WAL auf der Platte	Speicher	Speicher
Gibt auf nach	`max_keepalive_time` (8 h) oder bei 4xx	10 Versuchen (rund 8,5 min)	`max_elapsed_time` (5 min)
Persistent machbar	ja, Speicherpfad auf Volume	nur experimentell	mit `otelcol.storage.file` (Preview)
Zähler für Verluste	`prometheus_remote_storage_sampl`	`loki_write_dropped_entries_tota`	`otelcol_exporter_send_failed_spans_total`

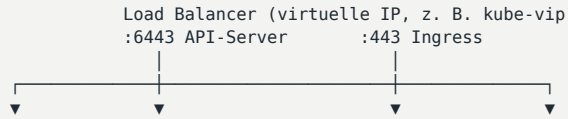
WARUM DAS WICHTIG IST

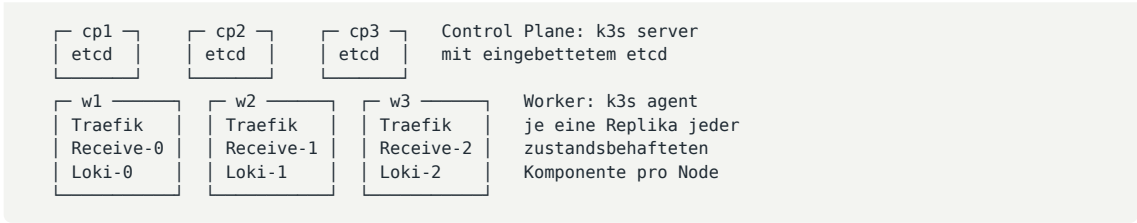
Keine Pufferung ist ein Ersatz für eine erreichbare Zentrale. Puffer überbrücken Minuten, vielleicht Stunden, aber sie haben eine Grenze, und diese Grenze hängt von Datenmenge und Plattenplatz ab. Wer Datenverlust vermeiden will, macht deshalb zuerst die Empfänger hochverfügbar und nutzt Puffer nur für die verbleibenden kurzen Lücken.

15.4 Ein hochverfügbarer Kubernetes-Cluster für obs

Die Grundlage ist ein Cluster, der den Ausfall eines Nodes übersteht.

DATEI Skizze: obs als hochverfügbarer Cluster





Drei Server-Nodes bilden das Quorum für etcd: Zwei von dreien müssen erreichbar sein. Die Worker tragen die Last. Jede wichtige Komponente läuft mindestens dreimal, je einmal pro Worker.

Baustein	Wozu und worauf du achtest
Drei Control-Plane-Nodes	k3s mit eingebettetem etcd braucht eine ungerade Zahl von Servern. Der erste startet mit `--cluster-init`, die anderen treten mit `--server` und demselben Token bei. Bei drei Servern darf einer ausfallen.
Feste Adresse für den API-Server	Ein Load Balancer oder eine virtuelle IP vor den drei Servern. Diese Adresse gehört mit `--tls-san` ins Zertifikat, und die Kubeconfig zeigt auf sie, nicht auf einen einzelnen Node.
Drei Worker	Getrennt von der Control Plane, damit eine speicherhungrige Abfrage nicht etcd ausbremst.
Redundanter Ingress	Traefik mit mindestens zwei Replikas auf verschiedenen Nodes, davor ein Load Balancer, der nur gesunde Nodes anspricht. Ohne Load Balancer bleibt die eine IP aus `/etc/hosts` ein Single Point of Failure.
Pod-Anti-Affinity	Verhindert, dass zwei Replikas derselben Komponente auf einem Node landen. Sonst nimmt ein Node-Ausfall alle Kopien mit.
PodDisruptionBudgets	Erlauben bei Wartung, etwa `kubectl drain`, immer nur so viele Pods zu stoppen, dass das Quorum erhalten bleibt.
Speicher für Volumes	`local-path` bindet ein Volume an einen Node. Das ist für Receive, Loki und Tempo mit Replikation in Ordnung, denn die Kopien liegen auf anderen Nodes. Eine verteilte Speicherlösung wie Longhorn oder Ceph RBD ist nicht zwingend.

DATEI Skizze: Anti-Affinity und PodDisruptionBudget für Receive (nicht getestet)

```
# im StatefulSet thanos-receive, spec.template.spec:
affinity:
  podAntiAffinity:
    requiredDuringSchedulingIgnoredDuringExecution:
      - labelSelector:
          matchLabels: {app.kubernetes.io/name: thanos-receive}
        topologyKey: kubernetes.io/hostname
---
```

```
apiVersion: policy/v1
kind: PodDisruptionBudget
metadata: {name: thanos-receive, namespace: monitoring}
spec:
  maxUnavailable: 1
  selector:
    matchLabels: {app.kubernetes.io/name: thanos-receive}
```

15.5 Thanos Receive mit Replikation

Mehrere Receive-Pods allein machen noch keine Hochverfügbarkeit. Entscheidend ist, wie sie zusammenarbeiten.

Der Hashring aus Kapitel 4 bekommt so viele Einträge wie Pods. Für neue Installationen empfiehlt die Thanos-Doku den Algorithmus `ketama` (`--receive.hashrings-algorithm=ketama`). Der Standard `hashmod` verteilt beim Hinzufügen eines Pods fast alle Serien neu.

Der Replikationsfaktor `--receive.replication-factor=3` schreibt jede Serie auf drei Pods. Receive bestätigt eine Schreibanfrage, sobald ein Quorum geantwortet hat: bei drei Kopien zwei. Fällt ein Pod aus, geht das Schreiben weiter. Fallen zwei aus, antwortet Receive mit `503`, und Alloy puffert und wiederholt.

Router und Ingestor trennen. In großen Installationen nehmen eigene Pods die Anfragen an und verteilen sie (Router), andere speichern (Ingestor). Router sind zustandslos und lassen sich beliebig skalieren und neu starten, ohne dass Daten in Bewegung geraten. Ein Receive ohne `--receive.local-endpoint`, aber mit Hashring, arbeitet als reiner Router.

Deduplizierung in Thanos Query. Jeder Ingestor hängt sein eigenes Label an, etwa `receive_replica="0"`, `"1"` und `"2"`. Thanos Query fasst mit `--query.replica-label=receive_replica` die drei Kopien wieder zu einer Serie zusammen. Das hast du in Kapitel 4 schon vorbereitet.

Warum reicht es nicht, einfach `replicas: 3` ins StatefulSet zu schreiben?

- Ohne passenden Hashring wissen die Pods nichts voneinander. Jeder speichert, was bei ihm ankommt, und ein Ausfall verliert genau diesen Anteil.
- Ohne `replication-factor` gibt es keine Kopien.
- Ohne PersistentVolume pro Pod verliert ein neu gestarteter Ingestor seine TSDB und seinen WAL, also alles seit dem letzten Upload nach S3, bis zu zwei Stunden.
- Ohne Anti-Affinity liegen womöglich alle Kopien auf einem Node.
- Ohne Load Balancer vor Traefik bleibt der Eingang ein einzelner Punkt.

STOLPERFALLE

Nachgelieferte Daten können abgewiesen werden. Eine TSDB nimmt nur Proben an, die höchstens rund eine Stunde hinter der jüngsten Probe dieses Mandanten liegen. Ältere gelten als „out of bounds“, Receive antwortet mit `409`, und Alloy verwirft sie, weil 4xx nicht wiederholt wird. Dass jeder Cluster bei dir ein eigener Mandant ist, hilft: Der Kopf der TSDB von `app1` wandert während eines Ausfalls nicht weiter, weil niemand anderes in diesen Mandanten schreibt. Teilen sich mehrere Sender einen Mandanten, wird die Nachlieferung nach langen Ausfällen abgewiesen. Thanos kennt dafür ein Zeitfenster für Proben außer der Reihe (`--tsdb.out-of-order.time-window`). Es ist experimentell und verlangt, dass der Compactor mit `--compact.enable-vertical-compaction` läuft, sonst hält er an.

15.6 Loki hochverfügbar

Loki kennt drei Betriebsarten:

Betriebsart	Wann
Monolithisch	Ein Prozess mit allen Rollen. Laut Doku geeignet bis etwa 20 GB Logs pro Tag. Mehrere Instanzen mit gemeinsamem S3 und `replication_factor: 3` ergeben eine hochverfügbare Variante.
Simple Scalable	Getrennte Pfade für Schreiben, Lesen und Hintergrundarbeit. Wird mit Loki 4.0 entfernt, deshalb nicht mehr für neue Installationen.
Microservices	Jede Rolle einzeln skaliert. Für sehr große Datenmengen, mit entsprechend viel Betriebsaufwand.

Für die meisten Plattformen ist der **monolithische Betrieb mit drei Instanzen** der richtige nächste Schritt. Dafür ändern sich im Wesentlichen drei Dinge gegenüber Kapitel 8:

- `replication_factor: 3`. Der Distributor schreibt jede Zeile an drei Ingester und wartet auf zwei Bestätigungen.
- Ein PersistentVolume pro Instanz für den WAL des Ingesters. Der WAL ist bei Loki standardmäßig an. Nach einem Neustart spielt der Ingester ihn wieder ein, statt die Zeilen im Speicher zu verlieren, die noch nicht im S3 liegen.
- Optional Zonen: Mit `zone_awareness_enabled` verteilt Loki die drei Kopien auf verschiedene Zonen, etwa Brandabschnitte oder Rechenzentren.

STOLPERFALLE

Späte Logs haben ein Zeitfenster. Loki lehnt eine Zeile ab, wenn sie mehr als die halbe `max_chunk_age` hinter der jüngsten Zeile desselben Streams liegt, standardmäßig also mehr als etwa eine Stunde. Die Antwort ist `400` , und Alloy wiederholt sie nicht. Grafana rät davon ab, `max_chunk_age` zu erhöhen. Für Logs heißt das: Ein Puffer, der länger als eine Stunde zurückhält, hilft nur, wenn in der Zwischenzeit nichts Neueres für denselben Stream angekommen ist.

Der **Compactor** von Loki, der die Aufbewahrung durchsetzt, darf nur einmal laufen. Das gilt auch bei drei Instanzen.

15.7 Tempo hochverfügbar

Mit Tempo 3 hat sich die Architektur grundlegend geändert. Die früheren **Ingester** gibt es nicht mehr.

- Im **verteilten Betrieb** schreibt der Distributor jede Span in Kafka. Kafka ist der WAL: Der Distributor bestätigt erst, wenn Kafka die Daten hat. Daraus bauen **Block-Builder** die Blöcke für S3, und **Live-Stores** beantworten Abfragen auf frischen Daten. Weil Kafka die Daten sichert, hält Tempo selbst nur eine Kopie.
- Im **monolithischen Betrieb**, wie im Labor, gibt es kein Kafka. Der Distributor übergibt die Spans im selben Prozess an den Live-Store, der sie zuerst im Speicher sammelt und dann in einen lokalen WAL schreibt.

Hochverfügbarkeit heißt bei Tempo also: Kafka hochverfügbar betreiben (drei Broker, Replikation) und Distributor, Block-Builder und Live-Store mehrfach starten. Das ist deutlich mehr Betriebsaufwand als bei Loki. Der **Backend-Scheduler**, der die Kompaktierung steuert, läuft wie der Compactor bei Thanos und Loki nur einmal. Seine Meldungen `no jobs found` kennst du aus Kapitel 9.

Eine gute Nachricht: Spans dürfen jederzeit nachkommen. Tempo fügt sie bei einer Abfrage nach Trace-ID zusammen. Anders als bei Metriken und Logs gibt es kein Zeitfenster, das späte Daten abweist.

15.8 S3-Speicher im Vergleich

	SeaweedFS, ein Pod	SeaweedFS verteilt	Ceph RGW	AWS S3
Kopien	eine	einstellbar, z. B. `010` = zweite Kopie in anderem Rack	einstellbar, Replikation oder Erasure Coding	mindestens drei Availability Zones
Ausfall eines Rechners	Speicher weg	übersteht ihn mit Kopien	übersteht ihn	für dich unsichtbar
Zweiter Standort	nein	mit Kopie `100` in anderem Rechenzentrum	Multisite mit Zonen, Abgleich asynchron	Cross-Region Replication, asynchron
Betrieb	sehr einfach	3 oder 5 Master mit Raft, mehrere Volume-Server, Filer mit gemeinsamer Datenbank	aufwendig, eigenes Fachwissen nötig	kein eigener Betrieb
Kosten	Platte	Platte mal Zahl der Kopien	Platte mal Kopien, viel RAM und CPU	nach Menge und Abrufen
Geeignet für	Labor	kleine und mittlere Plattformen on-premises	große Plattformen on-premises	Cloud oder hybrid

AWS gibt für S3 eine Haltbarkeit von 99,999999999 % an und eine Verfügbarkeit von 99,99 % bei S3 Standard, vertraglich zugesichert sind 99,9 %. Kapitel 11 hat gezeigt, wie der Umzug funktioniert.

STOLPERFALLE

Ein Bucket, zwei Compactors: Datenverlust. Der Compactor von Thanos „must be deployed as a singleton against a bucket“, so die Doku. Zwei Compactors auf demselben Bucket bearbeiten dieselben Blöcke, erzeugen Überlappungen, und der Compactor hält an, bis du die Blöcke von Hand reparierst. Dasselbe Prinzip gilt für den Compactor von Loki und den Backend-Scheduler von Tempo. Teilen sich zwei Zentralen einen Bucket, darf nur eine davon kompaktieren.

15.9 Zwei Zentralen: Active/Passive und Active/Active

Ein hochverfügbarer Cluster übersteht den Ausfall eines Nodes, aber nicht den eines Rechenzentrums. Dafür brauchst du eine zweite Zentrale an einem anderen Standort.

Active/Passive

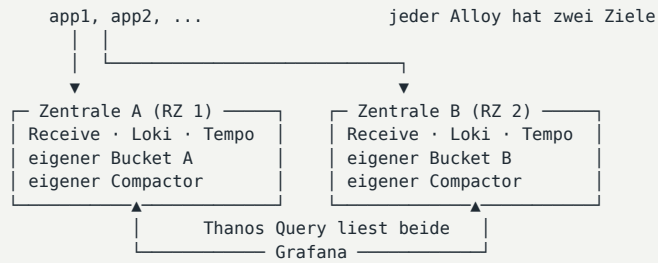
Eine Zentrale arbeitet, die zweite steht bereit. Die App-Cluster schreiben an einen Namen wie `receive.obs.example`, der im Normalfall auf Standort A zeigt.

Frage	Antwort
Umschalten	Per DNS mit kurzer TTL oder über einen globalen Load Balancer mit Gesundheitsprüfung. Wer umschaltet, ein Mensch oder ein Automat, muss vorher feststehen.
Daten der Vergangenheit	Liegen im S3 von Standort A. Damit B sie lesen kann, repliziert der Speicher asynchron nach B, oder B liest den Bucket von A mit, solange A erreichbar ist.
Compactor	Läuft nur am aktiven Standort. Nach dem Umschalten muss er in A sicher aus sein, bevor er in B startet.
Datenlücke	Was zwischen Ausfall und Umschalten anfällt, puffert Alloy. Danach landen die Daten in B.

Active/Active

Beide Zentralen arbeiten gleichzeitig, und jeder App-Cluster schreibt an **beide**. Alloy kann das: Ein `prometheus.remote_write` darf mehrere `endpoint`-Blöcke haben, `loki.write` ebenso, und für Traces leitet ein Empfänger an zwei Exporter weiter. Fällt ein Standort aus, hat der andere die Daten bereits.

DATEI Skizze: zwei Zentralen, jeder Cluster schreibt an beide



Jede Zentrale hat ihren eigenen Bucket und ihren eigenen Compactor, so kommen sie sich nie in die Quere. Eine Thanos Query kann beide Standorte als Endpunkte einbinden und fasst die doppelten Serien über ein Label wie `site` zusammen, genau wie die Replikas eines Standorts.

Active/Active kostet doppelten Speicher und doppelten Netzverkehr aus jedem App-Cluster. Dafür gibt es kein Umschalten und keine Lücke beim Ausfall eines Standorts. Ein Nachteil bleibt: Die Alarme. Beide Standorte werten dieselben Regeln aus. Damit nicht jeder Alarm doppelt ankommt, bilden die Alertmanager beider Standorte einen gemeinsamen Cluster (`--cluster.peer`, Port 9094 über TCP und UDP). Dann teilen sie Silences und wissen voneinander, welche Benachrichtigung schon verschickt ist.

15.10 Monitoring des Monitorings

Die wichtigste Regel: **Wer die Zentrale überwacht, darf nicht in der Zentrale laufen.** Fällt obs aus, kann obs dir nicht mehr Bescheid geben.

- **Dead Man's Switch.** Der Watchdog aus Kapitel 7 meldet sich jede Minute bei einem Dienst außerhalb. Bleibt das Signal aus, schlägt dieser Dienst Alarm, über einen eigenen Weg, etwa SMS oder eine App. Ohne Umleitung schickt kube-prometheus-stack den Watchdog übrigens an den Empfänger `null`, er würde also nie jemanden erreichen.
- **Prüfung von außen.** Ein externer Dienst ruft regelmäßig `https://grafana.obs.internal/api/health` und den Schreibpfad von Receive auf. Er bemerkt, wenn der Eingang ausfällt, auch wenn obs selbst noch läuft.
- **Gegenseitige Überwachung.** Bei zwei Standorten überwacht jeder den anderen. Fällt einer aus, alarmiert der andere über seinen eigenen Alertmanager.
- **Unabhängiger Kanal.** Die Benachrichtigung über den Ausfall der Zentrale darf nicht über einen Mailserver laufen, der an derselben Zentrale hängt.

Im Labor hast du gesehen, wie schnell das greift: `ClusterSchweigt` feuerte 8 Minuten nach Beginn des Ausfalls, und der Dead Man's Switch bei healthchecks.io meldet sich mit den Werten aus Kapitel 7 nach spätestens 10 Minuten.

15.11 Ausfälle üben: Disaster Recovery

Zwei Kennzahlen beschreiben, was ein Ausfall kosten darf:

- **RPO**, Recovery Point Objective: Wie viele Daten dürfen höchstens verloren gehen? Gemessen als Zeitspanne, etwa „höchstens 5 Minuten Logs".
- **RTO**, Recovery Time Objective: Wie lange darf es dauern, bis die Plattform wieder arbeitet? Etwa „Grafana nach 30 Minuten wieder erreichbar".

Beide legst du fest, bevor etwas passiert, und dann prüfst du sie mit Tests. Ein Test, den du nie gemacht hast, ist eine Hoffnung.

Test	So simulierst du ihn	Was du prüfst
Node fällt aus	`kubectl drain` oder VM hart abschalten	Schreiben geht weiter, Quorum hält, keine Lücke in den Daten
Receiver fällt aus	Einen Receive-Pod löschen, ein PVC verlieren	Query liefert weiter vollständige Daten, der neue Pod holt auf
Netzwerktrennung	Traefik auf 0 skalieren oder Firewall-Regel, wie im Labor	Wann feuert `ClusterSchweigt`, was liefert Alloy nach, wo bleibt eine Lücke
S3 fällt aus	SeaweedFS auf 0 skalieren, Zugangsdaten falsch setzen	Receive puffert lokal bis zur Retention, Alarm `ReceiveUploadFehler`, Abfragen alter Daten scheitern sichtbar
Zentrale fällt aus	ganzen Cluster stoppen	Dead Man's Switch meldet sich von außen, Umschalten auf Standort B gelingt innerhalb des RTO

Lücken kontrollieren. Nach jedem Test misst du, was fehlt. Die Abfragen dafür kennst du aus dem Messabschnitt am Anfang dieses Kapitels: Proben pro Minute in Thanos, Zeilen pro Minute in Loki, Traces pro Minute in Tempo. Eine Lücke, die du nur vermutest, kannst du nicht beheben.

15.12 Eine Referenzarchitektur

Für eine Plattform mit einigen Dutzend Clustern an einem Standort empfiehlt sich dieser Aufbau. Für einen zweiten Standort verdoppelst du ihn und schreibst Active/Active.

Baustein	Empfehlung
Kubernetes	3 Control-Plane-Nodes, 3 bis 5 Worker, Load Balancer vor API-Server und Ingress
Ingress	Traefik mit 3 Replikas, Anti-Affinity, PodDisruptionBudget
Metriken	Thanos Receive mit 2 Routern und 3 Ingestoren, `replication-factor=3`, `ketama`, PVC je Ingestor; Query mit 2 Replikas; Store mit 2 Replikas; Compactor genau einmal
Logs	Loki monolithisch mit 3 Instanzen, `replication_factor: 3`, WAL auf PVC; Compactor einmal
Traces	Tempo verteilt mit Kafka (3 Broker), sobald Traces geschäftskritisch sind; bis dahin monolithisch mit dem Wissen, dass ein Ausfall Traces kostet
Speicher	AWS S3 oder ein anderer verwalteter Objektspeicher; on-premises Ceph RGW oder SeaweedFS mit drei Mastern und Replikation `010`
Alarme	Alertmanager mit 3 Replikas im Cluster-Verbund, Ruler mit 2 Replikas

Baustein	Empfehlung
Grafana	2 Replikas mit externer PostgreSQL-Datenbank für Sitzungen und Einstellungen; Dashboards weiter aus dem Repo
Sammler	Alloy mit Speicherpfad auf `hostPath`, damit der WAL Neustarts übersteht
Außenwelt	Dead Man's Switch, externe Prüfung der Eingänge, unabhängiger Benachrichtigungskanal

15.13 Aufwand, Ressourcen und Kosten

Mehr Verfügbarkeit kostet Rechner, Speicher und vor allem Betriebszeit. Die Werte für das Labor stammen aus Kapitel 1, die für die Enterprise-Variante sind grobe Schätzungen für eine kleine Plattform mit wenigen App-Clustern, nicht gemessen.

	Labor	Enterprise, ein Standort (geschätzt)
Nodes für die Zentrale	1	6 (3 Control Plane, 3 Worker)
RAM der Zentrale	rund 8 GB	rund 60 bis 100 GB über alle Worker
Lokale Platte	rund 60 GB	rund 3 × 200 GB, weil Receive, Loki und Tempo je drei Kopien halten
Objektspeicher	eine Kopie	drei Kopien oder verwalteter Dienst
Komponenten im Dauerbetrieb	rund 15 Pods	rund 40 bis 60 Pods, mit Kafka mehr
Betriebsaufwand	ein Wochenende zum Aufbau	laufende Pflege: Updates, Tests, Bereitschaft

Die größte Position ist meist nicht die Hardware, sondern die Zeit. Jede zusätzliche Komponente will aktualisiert, überwacht und geübt werden. Fang deshalb dort an, wo der Verlust am meisten schmerzt: Bei den meisten Teams sind das Metriken und Alarme. Logs folgen, Traces zuletzt.

WAS DU GELERNT HAST

- Fällt die Zentrale aus, puffert Alloy Metriken im WAL, Logs und Traces nur kurz im Speicher. Was darüber hinausgeht, ist verloren.
- Im Labor liegt der Puffer von Alloy im Container und überlebt keinen Neustart. Für den Betrieb gehört er auf ein Volume.
- Hochverfügbarkeit entsteht aus Replikation mit Quorum, persistenten Volumes, Anti-Affinity und einem Load Balancer, nicht aus der Zahl der Pods allein.
- Späte Daten haben Grenzen: Thanos und Loki weisen Proben ab, die mehr als rund eine Stunde zurückliegen. Tempo nimmt späte Spans an.
- Compactor und vergleichbare Dienste laufen genau einmal pro Bucket, auch bei zwei Standorten.
- Wer die Zentrale überwacht, muss außerhalb von ihr laufen. RPO und RTO legst du vorher fest und prüfst sie mit Ausfalltests.

KAPITEL 16

Enterprise-Observability: Multi-Tenancy mit Thanos

Du weißt, wie eine Plattform die Metriken vieler Kunden mit beliebig vielen Clustern sicher trennt: beim Schreiben, beim Lesen und in Grafana. Und du weißt, wie du diese Trennung prüfst.

Dauer: 75 Minuten Lesen **Voraussetzung:** Kapitel 4 bis 7 und 13. Dieses Kapitel ändert nichts an deinem Labor.

Im Labor ist jeder Cluster ein eigener Mandant: `obs`, `app1` und `app2`. Das passt, solange dir alle drei Cluster gehören. In einem Unternehmen, das Kubernetes für andere betreibt, sieht die Welt anders aus. Ein Kunde hat mehrere Cluster, etwa für Produktion und Entwicklung, und er darf seine eigenen Daten sehen, aber nie die eines anderen Kunden. Der Betreiber wiederum will seine eigene Infrastruktur überwachen, ohne dass Kunden darin herumstöbern.

Dieses Kapitel spielt das an einem **ausgedachten** Cloud-Anbieter durch. Alle Namen sind erfunden:

Mandant	Cluster	Wer ist das
`kunde-a`	`production-eu`, `development-eu`	ein Kunde mit zwei Clustern
`kunde-b`	`production-de`, `staging-de`	ein zweiter Kunde
`platform`	`management`	der Betreiber selbst

WAS GETESTET IST UND WAS NICHT

Dieses Kapitel unterscheidet drei Arten von Aussagen. **Im Labor getestet** steht ausdrücklich dabei, mit dem Ergebnis. **Laut Quellcode oder Doku von Thanos v0.42.5** heißt: geprüft gegen die eingesetzte Version, aber nicht im Labor ausprobiert. Alles andere ist ein **Architekturbeispiel**, das du vor dem Einsatz selbst prüfen musst. Kein YAML in diesem Kapitel ist eine fertige Produktionskonfiguration.

16.1 Plattform-Monitoring und Kunden-Monitoring

Ein Betreiber hat zwei Arten von Metriken, und sie gehören in getrennte Welten:

	Plattform-Monitoring	Kunden-Monitoring
Fragt	Läuft unsere Infrastruktur? Knoten, Netze, Speicher, Control Planes	Läuft die Anwendung des Kunden? Seine Pods, Anfragen, Fehler
Wer liest	Betriebsteam des Anbieters	der Kunde, eventuell sein Dienstleister
Wer wird alarmiert	Bereitschaft des Anbieters	Bereitschaft des Kunden
Mandant	`platform`	ein Mandant pro Kunde
Dashboards	Plattformsicht über alle Cluster	nur die eigenen Cluster

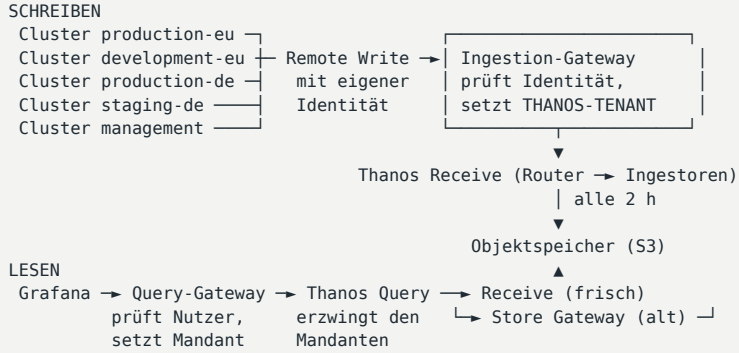
Die beiden Welten überschneiden sich: Der Anbieter muss auch in Kundenclustern Knoten und Kubelets überwachen, denn die gehören zu seiner Leistung. Diese Infrastruktur-Metriken schickt der Sammler im Kundencluster deshalb an den Mandanten `platform`, die Anwendungsmetriken an den Mandanten des Kunden. Ein Alloy kann das: zwei `prometheus.remote_write`-Komponenten mit unterschiedlichen Zugangsdaten, und jede Scrape-Komponente schickt an die passende.

WARUM DAS WICHTIG IST

Wer alles in einen Topf wirft, muss später jede einzelne Abfrage filtern, und ein vergessener Filter zeigt einem Kunden die Knoten aller anderen Kunden. Getrennte Mandanten machen die Grenze zur Voreinstellung, nicht zur Disziplin.

16.2 Architektur und Datenfluss

DATEI Datenfluss mit Gateways



Beide Wege haben ein Gateway, das die Identität prüft und den Mandanten festlegt. Thanos selbst prüft keine Identität. Es glaubt dem Header, der bei ihm ankommt.

Ein Punkt verdient Betonung: **Ein gemeinsamer Eingang zum Schreiben heißt nicht, dass alle alles lesen dürfen.** Im Labor schreiben alle Cluster an `receive.obs.internal`, und Grafana liest mit einer Datenquelle alle Mandanten. Das ist für ein Labor in Ordnung. In einer Plattform für mehrere Kunden sind Schreiben und Lesen zwei getrennte Türen mit getrennten Schlössern.

16.3 Thanos Receive: Mandant und Cluster sind zwei Dinge

Im Labor fallen zwei Begriffe zusammen, die du jetzt trennen musst:

- Die **Mandanten-ID** (`tenant_id`) sagt, **wem** die Daten gehören. Receive legt pro Mandant eine eigene TSDB an und hängt die ID als externes Label an jede Serie und an jeden Block in S3.
- Die **Cluster-ID** (`cluster`) sagt, **woher** die Daten kommen. Sie ist ein ganz normales Label, das der Sammler setzt.

Ein Mandant hat beliebig viele Cluster. Im Labor ist beides gleich, weil Kapitel 5 den Benutzernamen gleichzeitig als Clusternamen und als Mandanten verwendet. Für Kunden mit mehreren Clustern geht das nicht mehr.

Die Flags von Receive

Diese Flags hast du in Kapitel 4 schon gesetzt. Hier ihre genaue Bedeutung, geprüft im Quellcode von Thanos v0.42.5:

Flag	Standard	Bedeutung
`--receive.tenant-header`	`THANOS-TENANT`	Der HTTP-Header, aus dem Receive den Mandanten liest.
`--receive.tenant-label-name`	`tenant_id`	Unter diesem Label erscheint der Mandant an jeder Serie, als externes Label der TSDB.
`--receive.default-tenant-id`	`default-tenant`	Mandant für Anfragen ohne Header. Im Labor `unbekannt`, damit ein falsch eingerichteter Sender auffällt.
`--receive.tenant-certificate-field`	leer	Liest den Mandanten statt aus dem Header aus dem Client-Zertifikat: `organization`, `organizationalUnit` oder `commonName`. Der Header wird dann ignoriert. Ohne Zertifikat oder ohne das Feld scheitert die Anfrage.
`--receive.split-tenant-label-name`	leer	Ein Label in den Serien selbst bestimmt den Mandanten und wird danach entfernt. Hat Vorrang vor dem Header. Nur sinnvoll, wenn der Sender vertrauenswürdig ist.

Flag	Standard	Bedeutung
`--receive.replication-factor`	`1`	Wie oft jede Serie gespeichert wird. Quorum ist `RF/2+1`, bei drei Kopien also zwei (Kapitel 15).

Der **Hashring** kann Mandanten einzelnen Gruppen von Ingestoren zuweisen. Im Feld `tenants` stehen Mandanten-IDs, mit `"tenant_matcher_type": "glob"` auch Muster wie `kunde-*`. So bekommt ein großer Kunde eigene Ingestoren, und sein Datenaufkommen trifft die anderen nicht.

DATEI Skizze: Hashring mit eigener Gruppe für einen großen Kunden (nicht getestet)

```
[
  { "hashring": "gross",  "tenants": ["kunde-a"],
    "endpoints": ["ingest-a-0:10901", "ingest-a-1:10901", "ingest-a-2:10901"] },
  { "hashring": "standard",
    "endpoints": ["ingest-0:10901", "ingest-1:10901", "ingest-2:10901"] }
]
```

Der erste Eintrag nimmt nur `kunde-a`, der zweite ohne `tenants` alle übrigen. Die Reihenfolge zählt: Receive nimmt den ersten passenden Eintrag.

Zwei Cluster eines Kunden mit Alloy

In Kapitel 5 liest Alloy `CLUSTER_NAME`, `OBS_USER` und den Header aus demselben Benutzernamen. Für Kunden trennst du die Werte. Der Clustername steht in einer eigenen Variable, der Mandant ergibt sich aus der Anmeldung am Gateway, nicht aus der Konfiguration des Clusters.

DATEI Skizze: metrics.alloy für Kundencluster (Architekturbeispiel)

```
prometheus.remote_write "kunde" {
  external_labels = {
    // Woher: der Name dieses Clusters, z. B. production-eu
    cluster = sys.env("CLUSTER_NAME"),
  }
  endpoint {
    url = "https://ingest.plattform.example/api/v1/receive"
    // Wem: KEIN eigener THANOS-TENANT-Header. Den setzt das Gateway
    // aus der Identität. Ein Header hier wäre bestenfalls wirkungslos.
    basic_auth {
      username = sys.env("INGEST_USER")      // technische Identität des Clusters
      password = sys.env("INGEST_PASSWORD")
    }
    tls_config { ca_file = "/etc/plattform-ca/ca.crt" }
  }
}
```

Für die beiden Cluster von `kunde-a` sieht die Datei identisch aus. Nur die Umgebungsvariablen unterscheiden sich: `CLUSTER_NAME=production-eu` und `development-eu`, dazu je eine eigene Identität. Beide Identitäten ordnet das Gateway dem Mandanten `kunde-a` zu. Wird ein Cluster stillgelegt, sperrst du nur seine Identität.

Cluster	`CLUSTER_NAME`	Identität am Gateway	Mandant, vom Gateway gesetzt
production-eu	`production-eu`	`ka-production-eu`	`kunde-a`
development-eu	`development-eu`	`ka-development-eu`	`kunde-a`
production-de	`production-de`	`kb-production-de`	`kunde-b`
staging-de	`staging-de`	`kb-staging-de`	`kunde-b`
management	`management`	`plattform-management`	`platform`

16.4 Die Sicherheitsgrenze beim Schreiben

Thanos prüft keine Identität. Receive glaubt dem Header, der ankommt. Wer Receive erreicht, kann in jeden beliebigen Mandanten schreiben.

IM LABOR GETESTET

Ein Prometheus-Agent gab sich als Cluster `staging-de` aus, der `kunde-b` gehört. Er schickte direkt an Receive, ohne Gateway, mit dem Header `THANOS-TENANT: kunde-a`. Ergebnis: Seine Serien erschienen bei einer Abfrage von `kunde-a` neben dessen eigenen Clustern:

```
[["kunde-a","development-eu","1"],["kunde-a","production-eu","1"],["kunde-a","staging-de","1"]]
```

Der fremde Cluster hat sich in die Daten von `kunde-a` eingeschleust. Das ist nicht nur ein Datenschutzproblem: Er kann damit auch Alarme von `kunde-a` auslösen oder verdecken.

Die Grenze muss deshalb ein **Ingestion-Gateway** ziehen, das drei Dinge zuverlässig tut:

1. **Authentifizieren.** Jede Anfrage trägt eine Identität, die das Gateway prüft.
2. **Den Mandanten ableiten.** Aus der Identität, nicht aus der Anfrage. Die Zuordnung „Identität gehört zu Mandant“ steht beim Gateway, nicht beim Sender.
3. **Eingehende Mandanten-Header entfernen oder überschreiben.** Ein mitgeschickter `THANOS-TENANT` darf nie durchkommen.

Verfahren	Wie die Identität entsteht	Worauf du achtest
mTLS	Jeder Cluster hat ein Client-Zertifikat. Receive kann den Mandanten mit `--receive.tenant-certificate-field=organization` direkt aus dem Feld `O` lesen.	Eine eigene CA nur für Clients, kurze Laufzeiten, automatische Erneuerung, Sperrliste. Wer Zertifikate ausstellt, legt Mandanten fest.

Verfahren	Wie die Identität entsteht	Worauf du achtest
OIDC	Der Sammler holt sich ein Token von einem Identity Provider, das Gateway prüft es und liest den Mandanten aus einem Claim.	Gateway muss Signatur, Ablauf und Zielgruppe prüfen. Alloy unterstützt dafür `oauth2`.
Basic Auth mit Zuordnung	Benutzer und Passwort pro Cluster, das Gateway setzt den Mandanten aus dem Benutzer.	Einfach, aber Passwörter lassen sich kopieren. Rotation und eine Zuordnung Benutzer zu Mandant sind Pflicht.

STOLPERFALLE

Drei typische Lücken. Erstens: Das Gateway prüft die Anmeldung, reicht aber den Header des Clients unverändert durch. Dann schützt die Anmeldung nur vor Fremden, nicht vor Kunden untereinander. Genau so ist das Labor gebaut, Kapitel 5 weist darauf hin. Zweitens: Receive ist auch direkt erreichbar, an Traefik vorbei, etwa über einen `NodePort` oder aus einem anderen Namespace. Eine NetworkPolicy, die Port 19291 nur vom Gateway aus erlaubt, schließt diesen Weg. Drittens: Der Benutzername ist gleich dem Mandanten, und jemand legt versehentlich einen Benutzer `kunde-a` für einen Cluster von `kunde-b` an. Die Zuordnung braucht deshalb dieselbe Sorgfalt wie ein Passwort.

Ein Gateway mit Traefik

Für Basic Auth kann Traefik das Gateway selbst sein. Die Option `headerField` der Basic-Auth-Middleware schreibt den angemeldeten Benutzernamen in einen Header. Laut Quellcode von Traefik v3.7.1, der Version in k3s `v1.36.6`, löscht die Middleware dabei zuerst jeden gleichnamigen Header des Clients und setzt dann den Benutzernamen, und zwar nur nach erfolgreicher Anmeldung. Der Benutzername ist damit der Mandant.

DATEI `Middleware für ein Mandanten-Gateway (im Labor getestet)`

```yaml
apiVersion: traefik.io/v1alpha1
kind: Middleware
metadata: {name: gw-tenant, namespace: tenancy-test}
spec:
  basicAuth:
    secret: gw-users            # htpasswd: Benutzer = Mandant, z. B. kunde-a
    removeHeader: true          # Passwort nicht an Thanos weiterreichen
    headerField: THANOS-TENANT  # Mandant aus der Anmeldung, überschreibt den Client
```

Dieselbe Middleware sitzt vor dem Schreibpfad von Receive und vor dem Lesepfad von Query. Beim Schreiben braucht so jeder **Cluster** einen Benutzer, der wie sein Mandant heißt. Für mehrere Cluster pro Kunde teilen sich die Cluster eines Kunden diesen Benutzer, oder du nutzt mTLS mit

`--receive.tenant-certificate-field`, wo jedes Zertifikat eigen ist, aber denselben Mandanten im Feld `0` trägt.

IM LABOR GETESTET

Ein Prometheus-Agent meldete sich am Gateway als `kunde-b` an und versuchte zweierlei: Er schickte den Header `THANOS-TENANT: kunde-a` und trug zusätzlich das Label `tenant_id="kunde-a"` an allen Serien. Ergebnis der Abfragen danach:

```
als kunde-b:  [["kunde-b","production-de"],["kunde-b","staging-de-gw"]]
als kunde-a:  [["kunde-a","development-eu"],["kunde-a","production-eu"],["kunde-
a","staging-de"]]
```

Die Daten landeten bei `kunde-b`. Traefik hat den Header ersetzt, und Receive hat das mitgeschickte Label durch sein eigenes `tenant_id` überschrieben. Der Cluster `staging-de` bei `kunde-a` stammt übrigens noch aus dem Versuch **ohne** Gateway: Wer an Traefik vorbei direkt an Receive schreibt, kommt weiterhin durch.

Die letzte Zeile zeigt, warum zum Gateway immer eine NetworkPolicy gehört, die Receive nur für das Gateway erreichbar macht.

16.5 Die Sicherheitsgrenze beim Lesen

Auch Thanos Query glaubt dem Header. Seit einigen Versionen kann Query die Mandanten aber **erzwingen**. Die Flags in v0.42.5, geprüft im Quellcode:

Flag	Standard	Bedeutung
`--query.enforce-tenancy`	aus	Schaltet die Erzwingung ein. Antworten enthalten nur Serien, deren `tenant_id` zum Mandanten im Header passt.
`--query.tenant-header`	`THANOS-TENANT`	Header, aus dem Query den Mandanten liest.
`--query.tenant-label-name`	`tenant_id`	Label, auf das Query einschränkt.
`--query.default-tenant-id`	`default-tenant`	Mandant, wenn der Header fehlt.
`--query.tenant-certificate-field`	leer	Mandant aus dem Client-Zertifikat statt aus dem Header.

Wie es funktioniert: Query nutzt dieselbe Logik wie das Werkzeug `prom-label-proxy`. Es fügt in **jeden** Selektor einer PromQL-Abfrage den Matcher `tenant_id="<mandant>"` ein. Steht dort schon ein `tenant_id`-Matcher, ersetzt Query ihn stillschweigend.

IM LABOR GETESTET

Eine zweite Thanos Query mit `--query.enforce-tenancy` vor einem Test-Receive mit drei fiktiven Clustern. Jede Abfrage mit dem Header `THANOS-TENANT: kunde-a`:

Abfrage	Ergebnis
`count by (tenant_id, cluster) (up)`	nur `kunde-a`
`... (up{tenant_id="kunde-b"})`	nur `kunde-a`, der Selektor wurde ersetzt
`... (up{tenant_id=~".*"})`	nur `kunde-a`
`... (up) or ... (up{tenant_id="kunde-b"})`	nur `kunde-a`
`/api/v1/series`, `/api/v1/label/tenant_id/values`	nur `kunde-a`
ohne Header	leer, Standard-Mandant
Header `kunde-b`	**alles von `kunde-b`**

Die letzte Zeile ist die Lehre: Die Erzwingung ist nur so gut wie die Herkunft des Headers.

Was die Erzwingung nicht abdeckt. Laut Quellcode von v0.42.5 gilt sie für `query`, `query_range`, die `explain`-Varianten, `series`, `labels`, `label/<name>/values` und `status/tsdb`. Sie gilt **nicht** für `/api/v1/rules`, `/api/v1/alerts`, `/api/v1/targets`, `/api/v1/metadata`, `/api/v1/query_exemplars` und `/api/v1/stores`, und nicht für die gRPC-Schnittstelle von Query. Store Gateway und Receive erzwingen selbst nichts. Ein Gateway vor Query sollte deshalb nur die geschützten Pfade durchlassen, und Store Gateway und Receive dürfen von außen gar nicht erreichbar sein.

STOLPERFALLE

Geschachtelte Querier. Steht eine Query vor einer anderen, erzwingt laut Thanos-Doku nur die erste, die die Anfrage bekommt. Auch die Oberfläche von Query enthält ein freies Feld für den Mandanten. Sie gehört deshalb nicht in die Hände von Kunden. Die Doku formuliert es so: „there are no authentication mechanisms in Thanos, so anyone can set an arbitrary tenant in the HTTP header."

Das Query-Gateway

Vor die Query mit Erzwingung setzt du dieselbe Middleware. Der Ingress lässt nur den Pfad `/api/v1` durch, nicht die Oberfläche von Query.

IM LABOR GETESTET

Abfragen über das Gateway, angemeldet als `kunde-a`:

Versuch	Ergebnis
ohne zusätzlichen Header	nur `kunde-a`
mit `THANOS-TENANT: kunde-b`	nur `kunde-a`, Header ersetzt
mit `thanos-tenant: kunde-b` (klein)	nur `kunde-a`
ohne Anmeldung, falsches Passwort	`401`
Oberfläche `/graph`	`404`, nicht freigegeben
`/api/v1/rules`	**Antwort ohne Mandantenfilter**

Die letzte Zeile ist die bekannte Lücke: Alles unter `/api/v1/`, was Query nicht erzwingt, reicht ein Gateway mit Pfad-Präfix ungefiltert durch. Ein Gateway für Kunden lässt deshalb nur die erzwungenen Pfade zu, also `query`, `query_range`, `series`, `labels` und `label/*/values`.

STOLPERFALLE

Warnungen verraten andere Mandanten. Im Labor wurde der Receiver gestoppt und `kunde-a` fragte über das Gateway ab. Query antwortete mit `success` und leerem Ergebnis, aber mit einer Warnung, die alle Mandanten des ausgefallenen Receivers aufzählte: `tenant_id="kunde-a"`, `tenant_id="kunde-b"`, `tenant_id="unbekannt"`. Grafana zeigt solche Warnungen an. Die Namen anderer Kunden sind kein Datenleck im engeren Sinn, aber eine Information, die ein Kunde nicht bekommen sollte. Ein Gateway kann das Feld `warnings` aus Antworten entfernen, oder die Mandanten heißen nicht nach Kunden, sondern nach neutralen Kennungen.

16.6 Grafana für viele Kunden

In Kapitel 6 hast du eine Grafana-Instanz mit Datenquellen aus ConfigMaps gebaut, und die Loki-Datenquelle aus Kapitel 8 liest mit `obs|app1|app2` alle Mandanten. Für Kunden gibt es drei Modelle:

	Eine Instanz, gemeinsam	Eine Instanz, eine Organisation pro Kunde	Eine Instanz pro Kunde
Trennung	nur über Ordnerrechte	Organisationen sehen einander nicht: eigene Datenquellen, Dashboards, Nutzer, Alarme	vollständig, eigener Prozess

	Eine Instanz, gemeinsam	Eine Instanz, eine Organisation pro Kunde	Eine Instanz pro Kunde
Datenquellen	gemeinsam	pro Organisation, jede mit ihrem Mandanten	pro Instanz
Admin-Rechte	ein Admin sieht alles	Org-Admin sieht nur seine Organisation, der Server-Admin alles	Kunde kann Admin seiner Instanz sein
Betrieb	einfach	mittel, Provisioning pro Organisation	aufwendig, viele Instanzen
Geeignet für	ein Team, keine Kunden	viele Kunden mit Standardbedarf	Kunden mit eigenen Plugins oder Compliance-Vorgaben

STOLPERFALLE

Eine zentrale Thanos-Datenquelle für alle ist ein Datenleck. Wer eine Datenquelle in Grafana benutzen darf, kann über **Explore** jede Abfrage stellen, die diese Datenquelle erlaubt, unabhängig davon, welche Dashboards er sehen darf. Liest die Datenquelle alle Mandanten, liest der Nutzer alle Mandanten. Ein Filter `cluster=~"$cluster"` im Dashboard hilft nicht, denn der Nutzer schreibt in Explore einfach eine Abfrage ohne Filter. Mandantentrennung entsteht nur dort, wo der Nutzer sie nicht umgehen kann: im Gateway vor Thanos Query.

Für das Modell mit Organisationen sieht die Zuordnung so aus: Jede Organisation bekommt eine eigene Thanos-Datenquelle, die auf das Query-Gateway zeigt und sich dort mit einer eigenen technischen Identität anmeldet. Das Gateway kennt die Identität und setzt den Mandanten. Selbst wenn ein Org-Admin die Datenquelle bearbeitet und einen anderen Header einträgt, ändert das nichts, denn das Gateway überschreibt ihn.

Alarme für Kunden. Laufen die Alarmregeln eines Kunden im Thanos Ruler oder in Grafana Alerting, müssen sie ebenfalls auf seinen Mandanten beschränkt sein, und ihre Benachrichtigungen dürfen nur an seine Kontakte gehen. Am einfachsten ist ein eigener Alertmanager-Empfänger pro Mandant mit einer Route auf das Label `tenant_id`. Plattform-Alarme laufen getrennt davon an die Bereitschaft des Betreibers, wie in Kapitel 7.

16.7 Speicherung, Hochverfügbarkeit und die Grenzen einer gemeinsamen Plattform

Die Bausteine für eine ausfallsichere Speicherung beschreibt Kapitel 15: Receive mit Routern und Ingestoren, Hashring und `replication-factor=3`, WAL auf persistenten Volumes, hochverfügbarer

Objektspeicher, Store Gateway mit mehreren Replikas und ein einziger Compactor pro Bucket. Für viele Mandanten kommen Fragen dazu, die mit Ausfallsicherheit nichts zu tun haben, sondern mit Fairness.

Risiko	Was passiert	Gegenmittel
Noisy Neighbor	Ein Kunde schickt plötzlich zehnmal so viele Daten, Receive wird langsam für alle	Limits pro Mandant in Receive (`--receive.limits-config-file`, Kapitel 13)
Kardinalität	Ein Label mit Nutzer-IDs erzeugt Millionen Serien, der Speicher der Ingestoren läuft voll	Limit für Serien pro Anfrage, Beobachtung von `prometheus_tsdb_head_series` je Mandant
Teure Abfragen	Eine Abfrage über ein Jahr ohne Filter belastet Store Gateway und Query für alle	Query-Frontend mit Aufteilung und Cache, Zeitlimits für Abfragen, eigene Query-Instanzen für große Kunden
Retention	Kunden wollen unterschiedlich lange Aufbewahrung	Ein Compactor bearbeitet einen Bucket mit einer Retention. Unterschiedliche Retention heißt getrennte Buckets oder ein Compactor pro Bucket
Datenverlust	Ein Ausfall trifft alle Kunden gleichzeitig	Replikation, Puffer in den Sammlern, Ausfalltests, siehe Kapitel 15

STOLPERFALLE

Ein Bucket, ein Compactor. Auch bei vielen Mandanten in einem Bucket darf nur ein Compactor laufen. Wer pro Kunde einen eigenen Compactor will, etwa für eine eigene Retention, braucht auch einen eigenen Bucket oder muss die Compactors über `--selector.relabel-config` streng auf getrennte Blöcke aufteilen. Überlappen sich zwei, beschädigen sie die Daten.

16.8 Drei Architekturen im Vergleich

	A: eine gemeinsame Plattform	B: Thanos pro Kunde	C: hybrid
Aufbau	Ein Receive, ein Query, ein Bucket für alle Mandanten	Eigener Receive, Query, Bucket für jeden Kunden	Gemeinsame Plattform für die meisten, eigene Instanzen für Kunden mit besonderen Anforderungen

	A: eine gemeinsame Plattform	B: Thanos pro Kunde	C: hybrid
Isolation	logisch: Header, Labels, Gateways	physisch: eigene Prozesse, eigener Speicher	je nach Kunde
Sicherheit	hängt an korrekter Konfiguration der Gateways	ein Fehler betrifft nur einen Kunden	gemischt
Skalierung	ein großes System, gut ausgelastet	viele kleine Systeme, viel Leerlauf	gut
Kosten	niedrig pro Kunde	hoch pro Kunde	mittel
Compliance	schwer nachzuweisen, dass Daten nie gemischt werden	einfach: eigener Bucket, eigener Schlüssel, eigene Region	nach Bedarf
Betrieb und Updates	ein Update für alle, hohes Risiko pro Update	viele Updates, gut automatisierbar	zwei Betriebsmodelle

Die Empfehlung: Beginne mit **C**, wobei der gemeinsame Teil nach **A** gebaut ist. Die meisten Kunden teilen sich eine Plattform mit erzwungenen Mandanten beim Schreiben und Lesen und mit Limits pro Mandant. Kunden mit gesetzlichen Vorgaben, eigener Region oder sehr großen Datenmengen bekommen eine eigene Instanz aus derselben Vorlage. Weil beides aus demselben Git-Repo und denselben Helm-Werten entsteht, kostet die zweite Variante kaum zusätzliche Arbeit.

Die Grenze dieser Empfehlung: Die logische Trennung in A hängt davon ab, dass jedes Gateway richtig konfiguriert ist und niemand an ihm vorbei auf Receive oder Query zugreift. Diese Annahme musst du laufend prüfen, nicht einmalig. Die Testfälle am Ende dieses Kapitels gehören deshalb in eine automatische Prüfung.

16.9 GitOps mit Argo CD

Bei vielen Kunden und Clustern geht nichts mehr von Hand. Kapitel 13 hat GitOps als Ausblick erwähnt. Für eine Plattform mit Mandanten ist es Pflicht: Git ist die einzige Quelle der Wahrheit, und Argo CD bringt den Zustand aus Git in die Cluster.

DATEI Skizze: Aufbau eines Plattform-Repos (Architekturbeispiel)

```
plattform-repo/
├── basis/                Helm-Werte für Alloy, gleich für alle Cluster
├── zentrale/             Receive, Query, Gateways, Store, Compactor
├── mandanten/
```

```
│   ├── kunde-a.yaml       tenant: kunde-a, limits, retention, kontakte
│   └── kunde-b.yaml
└── cluster/
    ├── production-eu.yaml tenant: kunde-a, cluster: production-eu
    ├── development-eu.yaml
    ├── production-de.yaml tenant: kunde-b
    ├── staging-de.yaml
    └── management.yaml    tenant: platform
```

Eine Datei pro Cluster nennt Mandant und Clusternamen. Ein `ApplicationSet` in Argo CD liest das Verzeichnis `cluster/` mit dem Git-Generator und erzeugt für jede Datei eine Application, die Alloy mit genau diesen zwei Werten installiert. Ein neuer Cluster ist damit eine neue Datei.

Geheimnisse gehören nicht ins Repo. Die Zugangsdaten jedes Clusters für das Ingestion-Gateway liegen in einem Tresor, etwa Vault oder dem Secret Manager eines Cloud-Anbieters. Der **External Secrets Operator** holt sie von dort und legt sie als Kubernetes-Secret an. Im Repo steht nur der Verweis, nie der Wert.

Ein Onboarding-Ablauf, Schritt für Schritt:

1. Der neue Cluster `staging-de` ist erstellt und in Argo CD registriert.
2. Im Tresor entsteht eine technische Identität für ihn, etwa ein Client-Zertifikat oder ein OIDC-Client, und das Gateway lernt, dass diese Identität zu `kunde-b` gehört.
3. Ein Merge Request legt `cluster/staging-de.yaml` mit `tenant: kunde-b` an. Ein zweiter Blick prüft vor allem diese eine Zeile.
4. Argo CD installiert Alloy im neuen Cluster. External Secrets legt die Zugangsdaten an.
5. Die automatischen Testfälle aus dem nächsten Abschnitt laufen für `kunde-b` erneut: Der neue Cluster erscheint bei `kunde-b`, nirgends sonst.

16.10 Verifikation: Die Trennung beweisen

Eine Mandantentrennung, die nicht getestet ist, ist eine Behauptung. Die folgenden Testfälle gehören in eine automatische Prüfung, die nach jeder Änderung an Gateway, Receive oder Query und nach jedem Onboarding läuft. Die Spalte rechts zeigt das Ergebnis aus dem Labor mit Thanos v0.42.5 und Traefik v3.7.1.

Nr.	Test	Erwartung	Labor
1	`kunde-a` fragt `count by (cluster) (up)` über das Gateway	nur `production-eu` und `development-eu`	bestanden

Nr.	Test	Erwartung	Labor
2	`kunde-a` fragt mit `{tenant_id="kunde-b"}` oder per `=~".*"` oder per `or`	nur Daten von `kunde-a`	bestanden
3	`kunde-a` schickt beim Lesen `THANOS-TENANT: kunde-b`	Header wird ersetzt	bestanden
4	Cluster von `kunde-b` schickt beim Schreiben Header und Label `kunde-a`	Daten landen bei `kunde-b`	bestanden
5	Schreiben direkt an Receive, am Gateway vorbei	Verbindung abgelehnt (NetworkPolicy)	**nicht bestanden** ohne NetworkPolicy, siehe oben
6	Lesen direkt an Query, am Gateway vorbei, mit fremdem Header	Verbindung abgelehnt	**nicht bestanden** ohne NetworkPolicy: fremde Daten sichtbar
7	Abfrage ohne Anmeldung	`401`	bestanden
8	Abfrage ohne Mandanten-Header direkt an Query	nur Standard-Mandant, also leer	bestanden
9	`/api/v1/rules`, `/alerts`, `/metadata`, `/query_exemplars` über das Gateway	gesperrt	**nicht gesperrt** bei Pfad-Präfix `/api/v1`
10	Grafana: Nutzer von `kunde-a` schreibt in Explo-	nur Daten von `kunde-a`	Architekturbeispiel, nicht getestet

Nr.	Test	Erwartung	Labor
	re eine Abfrage ohne Filter		
11	Limit `series_limit: 50` für `kunde-b`	`413` nur für `kunde-b`, `kunde-a` unbeeinträchtigt	bestanden
12	Receiver fällt aus, `kunde-a` fragt ab	leeres Ergebnis mit Warnung, keine fremden Daten	bestanden, aber Warnung nennt fremde Mandanten

Die Tests 5, 6 und 9 zeigen, wo das Labor-Setup für Kunden nicht reicht. Sie sind kein Fehler von Thanos, sondern die Stellen, an denen Netzwerk und Gateway die Arbeit übernehmen müssen.

DATEI `Skizze: NetworkPolicy, die Receive nur für Traefik öffnet (nicht getestet)`

```
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata: {name: receive-nur-gateway, namespace: monitoring}
spec:
  podSelector:
    matchLabels: {app.kubernetes.io/name: thanos-receive}
  policyTypes: [Ingress]
  ingress:
    - from:   # Schreibpfad nur vom Gateway
        - namespaceSelector: {matchLabels: {kubernetes.io/metadata.name: kube-system}}
          podSelector: {matchLabels: {app.kubernetes.io/name: traefik}}
      ports: [{port: 19291}]
    - from:   # Store API nur für Thanos Query
        - podSelector: {matchLabels: {app.kubernetes.io/name: thanos-query}}
      ports: [{port: 10901}]
```

Achtung bei k3s: Das mitgelieferte Flannel setzt NetworkPolicies nicht selbst durch, das übernimmt der in k3s eingebaute Policy-Controller. Prüfe mit einem Test-Pod, ob die Policy wirklich greift, bevor du dich darauf verlässt. Prometheus auf obs schreibt im Labor direkt an Receive, er bräuchte eine weitere Regel.

PROBIER ES SELBST

Baue den Teststand selbst nach, in einem eigenen Namespace, damit dein Labor unberührt bleibt: ein Receive ohne S3, eine Query mit `--query.enforce-tenancy` und drei Prometheus-Agenten im Modus `--agent` als fiktive Cluster. Führe die Tests 1 bis 4 aus. Ergänze dann die NetworkPolicy und wiederhole Test 5. Was meldet der Agent, der direkt an Receive schreiben will, und woran erkennst du in Receive, dass nichts angekommen ist?

WAS DU GELERNT HAST

- Mandant und Cluster sind zwei Dinge: Ein Kunde ist ein Mandant mit beliebig vielen Clustern.
- Thanos glaubt dem Header. Ein Gateway muss die Identität prüfen und den Mandanten setzen, beim Schreiben wie beim Lesen.
- `--query.enforce-tenancy` ersetzt jeden `tenant_id`-Selektor, schützt aber nur die Abfrage-Endpunkte, nicht Regeln, Alarme, Metadaten oder die gRPC-Schnittstelle.
- Traefik `headerField` überschreibt einen mitgeschickten Header zuverlässig. Wer am Gateway vorbeikommt, umgeht alles. Dagegen hilft nur das Netzwerk.
- In Grafana entscheidet die Datenquelle über die Sichtbarkeit, nicht das Dashboard.
- Eine gemeinsame Plattform braucht Limits pro Mandant und automatische Tests der Trennung.

KAPITEL 17

Anhang

Versionen, Repo-Überblick, Spickzettel und Links zum schnellen Nachschlagen.

17.1 Versionen

Alle Angaben gelten für den Stand dieses Buches, 11. November 2026. Mit genau diesen Versionen lief der Test auf echten Clustern. Die Chart-Versionen stammen aus den offiziellen Helm-Repositories, die App-Versionen aus den Charts selbst.

Komponente	Chart	App bzw. Image
k3s	–	`v1.36.6+k3s1`
kube-prometheus-stack (`prometheus-community`)	`92.18.0`	Operator `v0.94.2`
davon Prometheus, Alertmanager, Grafana	–	`v3.15.1` , `v0.34.2` , `13.3.0`
Thanos	– (Manifeste)	`quay.io/thanos/thanos:v0.42.5`
SeaweedFS	– (Manifest)	`chrislusf/seaweedfs:4.52`
Loki (`grafana-community`)	`21.0.0`	`3.7.9`
Tempo (`grafana-community`)	`3.2.1`	`3.1.1`
Alloy (`grafana`)	`1.15.0`	`v1.21.3`
kube-state-metrics (`prometheus-community`)	`9.0.0`	`v2.20.0`
node-exporter (`prometheus-community`)	`5.0.0`	`v1.12.1`
Velero (`vmware-tanzu`)	`12.2.4`	`1.18.2`
Velero-Plugin für AWS	–	`velero/velero-plugin-for-aws:v1.14.4`
Blackbox-Exporter (`prometheus-community`)	`11.21.1`	`v0.29.1`
Pyroscope (`grafana` , optional)	`2.4.3`	`2.4.3`
Mailpit	– (Manifest)	`axllent/mailpit:v1.33.1`
HotROD (Demo-App)	– (Manifest)	`jaegertracing/example-hotrod:2.22.0`

17.2 Dein Repo am Ende

So sieht `observability/` aus, wenn du alle Kapitel durchgearbeitet hast. Dateien mit Geheimnissen fehlen absichtlich: Die privaten Schlüssel unter `tls/` schließt `.gitignore` aus, alle Passwörter liegen nur in Secrets.

```
observability/
├── k3s/            config-obs.yaml, config-app1.yaml, config-app2.yaml,
│                   coredns-custom.yaml                          (Kap. 2, 12)
├── tls/            openssl.cnf, ca.crt, obs.internal.crt        (Kap. 2)
├── seaweedfs/      seaweedfs.yaml, create-secrets.sh,
│                   create-buckets.sh                            (Kap. 3)
├── central/
│   ├── kps/        values.yaml (4), grafana.yaml (6), alerting.yaml (7)
│   ├── thanos/     receive.yaml, query.yaml, store.yaml, compact.yaml,
│   │               servicemonitor.yaml                          (Kap. 4)
│   ├── ingress/    middleware.yaml, receive.yaml, loki.yaml,
│   │               tempo.yaml                                   (Kap. 5, 8, 9)
│   ├── mailpit/    mailpit.yaml                                 (Kap. 7)
│   ├── loki/       values.yaml                                  (Kap. 8)
│   └── tempo/      values.yaml                                  (Kap. 9)
├── cluster/
│   ├── alloy/      values.yaml, metrics.alloy (5), logs.alloy (8),
│   │               traces.alloy (9)
│   └── exporters/  kube-state-metrics.yaml, node-exporter.yaml  (Kap. 5)
├── grafana/
│   └── datasources/ thanos.yaml, alertmanager.yaml, loki.yaml,
│                    tempo.yaml                                  (Kap. 6, 8, 9)
├── rules/          cluster.yaml, knoten.yaml, workloads.yaml,
│   │               thanos.yaml, runbooks.md                     (Kap. 7)
│   └── loki/       kustomization.yaml, log-alarme.yaml          (Kap. 8)
├── demo/           hotrod.yaml, last.yaml                       (Kap. 9)
├── dashboards/     kustomization.yaml                           (Kap. 10)
│   ├── plattform/  multi-cluster, node, workloads, alarme,
│   │               thanos-speicher (.json)
│   └── signale/    logs.json, services.json
├── aws/            iam-policy-{thanos,loki,tempo,backup}.json,
│                   lifecycle-{thanos,loki,tempo,backup}.json,
│                   bucket-policy-tls.json, encryption.json,
│                   thanos-objstore-aws.yml, loki-aws.yaml,
│                   tempo-aws.yaml                               (Kap. 11)
├── backup/         s3-ingress.yaml, k3s-etcd-s3.yaml,
│                   velero-values.yaml, velero-aws.yaml,
│                   backup.alloy, velero-rules.yaml              (Kap. 12)
└── extras/         blackbox-values.yaml, probes.yaml,
                    blackbox-rules.yaml, receive-limits.yaml,
                    slo-rules.yaml,
                    pyroscope-values.yaml,
                    pyroscope-datasource.yaml                    (Kap. 13)
```

17.3 Spickzettel PromQL

Frage	Abfrage
Welche Ziele sind unten?	`up == 0`
Welche Cluster liefern Daten?	`count by (cluster) (up)`
Anfragen pro Sekunde	`sum by (cluster) (rate(http_requests_total[5m]))`
Fehlerquote	`sum(rate(x_total{code=~"5.."}[5m])) / sum(rate(x_total[5m]))`
95. Perzentil aus Histogramm	`histogram_quantile(0.95, sum by (le) (rate(x_bucket[5m])))`
Speicher pro Pod	`sum by (pod) (container_memory_working_set_bytes{container!=""})`
Neustarts der letzten Stunde	`increase(kube_pod_container_status_restarts_total[1h]) > 0`
Freier Platz in Prozent	`100 * node_filesystem_avail_bytes / node_filesystem_size_bytes`
Größte Metriken	`topk(10, count by (__name__)({__name__=~".+"}))`
Serien pro Mandant	`sum by (tenant) (prometheus_tsdb_head_series{job="thanos-receive"})`
Verstummter Cluster	`group by (cluster) (up offset 1h) unless group by (cluster) (up)`

Merksätze: `rate` nur auf Zähler (`_total`), nie auf Gauges. Erst `rate`, dann `sum`. `by (...)` sagt, welche Labels übrig bleiben.

17.4 Spickzettel LogQL

Dieselben vier Schritte wie in Kapitel 8: Streams wählen, Zeilen filtern, zerlegen, zählen.

Frage	Abfrage
Alle Logs eines Namespaces	`{cluster="app1", namespace="demo"}`
Text suchen	`{cluster="app1"} |= "timeout"`
Regulärer Ausdruck, ohne Debug	`{cluster="app1"} |~ "(?i)error" != "level=debug"`
logfmt zerlegen und filtern	`{app="alloy"} | logfmt | level="error"`
JSON zerlegen	`{namespace="demo"} | json | status >= 500`
Strukturierte Metadaten	`{cluster=~"app1|app2"} | level="error"`
Fehler pro Namespace als Zeitreihe	`sum by (namespace) (count_over_time({cluster="app1"} |~ "(?i)error" [5m]))`
Nur ein Feld anzeigen	`... | logfmt | line_format "{{.msg}}"`

Merksatz: erst über Labels einschränken, dann über Text. Loki liest sonst alles im Zeitraum.

17.5 Spickzettel TraceQL

Attribute tragen ihren Bereich als Präfix: `resource.` für den Dienst, `span.` für den einzelnen Span. Dazu kommen eingebaute Felder wie `name`, `duration` und `status`.

Frage	Abfrage
Spans eines Dienstes	`{ resource.service.name = "frontend" }`
Langsame Spans	`{ resource.service.name = "customer" && duration > 250ms }`
Alle Fehler	`{ status = error }`
Fehler unterhalb eines Dienstes	`{ resource.service.name = "frontend" } >> { status = error }`
Bestimmter HTTP-Status	`{ span.http.response.status_code >= 500 }`
Traces mit vielen Spans	`{ resource.k8s.namespace.name = "demo" } | count() > 20`
Ein Cluster	`{ resource.cluster = "app1" }`

Das Attribut `resource.cluster` setzt `traces.alloy` aus Kapitel 9. Welche Attribute ein Span sonst noch trägt, zeigt Grafana beim Aufklappen.

17.6 Offizielle Dokumentation

Thema	Adresse
k3s	docs.k3s.io, etcd-Snapshots unter `/cli/etcd-snapshot`
Prometheus und PromQL	prometheus.io/docs
Prometheus-Operator	prometheus-operator.dev
kube-prometheus-stack	github.com/prometheus-community/helm-charts
Thanos	thanos.io, Receive unter `/tip/components/receive.md`
SeaweedFS	github.com/seaweedfs/seaweedfs/wiki
Loki und LogQL	grafana.com/docs/loki/latest
Tempo und TraceQL	grafana.com/docs/tempo/latest
Alloy	grafana.com/docs/alloy/latest
Grafana	grafana.com/docs/grafana/latest
Pyroscope	grafana.com/docs/pyroscope/latest
Blackbox-Exporter	github.com/prometheus/blackbox_exporter
Velero	velero.io/docs
Velero-Plugin für AWS	github.com/vmware-tanzu/velero-plugin-for-aws

Thema	Adresse
AWS S3	docs.aws.amazon.com/s3, Preise unter aws.amazon.com/s3/pricing
SLOs und Burn-Rate	sre.google/workbook/alerting-on-slos

KUBERNETES BEOBACHTEN

Drei Ubuntu-VMs, ein Laptop und eine Frage: Was passiert gerade in meinen Clustern, und was ist vor drei Wochen passiert?

Dieses Buch baut mit dir eine zentrale Observability-Plattform für mehrere k3s-Cluster auf. Metriken, Logs und Traces landen an einer Stelle, alte Daten liegen günstig in S3, und Alarme erreichen dich, bevor die Nutzer es merken. Jeder Befehl einzeln, mit einer Erklärung, warum er nötig ist und woran du erkennst, dass er gewirkt hat.

AUS DEM INHALT

Metriken	Prometheus und Thanos Receive für alle Cluster
Logs	Loki und Alloy mit einem Mandanten pro Cluster
Traces	Tempo und OpenTelemetry an einer Demo-App
Alarme	Alertmanager, Thanos Ruler und ein Dead Man's Switch
Speicher	S3 im Cluster mit SeaweedFS und auf AWS
Sicherheit	Backup und Restore mit etcd-Snapshots und Velero
Praxis	Fertige Dashboards, direkt aus dem Repo

Thomas Zachmann
Autor auf Leanpub



Hands-on-Handbuch · Stand: 11. November 2026